جزوهٔ درسی · دانشکدهٔ مهندسی کامپیوتر

ساختمان داده‌ها و الگوریتم‌ها

از تحلیل زمان اجرا، تقسیم و حل، الگوریتم‌های تصادفی و تحلیل سرشکنی تا لیست، پشته، صف، درخت، هرم و مرتب‌سازی خطی؛ با توضیح کامل هر مفهوم، اثبات‌ها، مثال‌های حل‌شده و پاسخ «چرا»هایی که معمولاً بی‌جواب می‌مانند.

۱ طراحی و تحلیل الگوریتم

۲ تقسیم و حل

۳ الگوریتم‌های تصادفی

۴ تحلیل سرشکنی

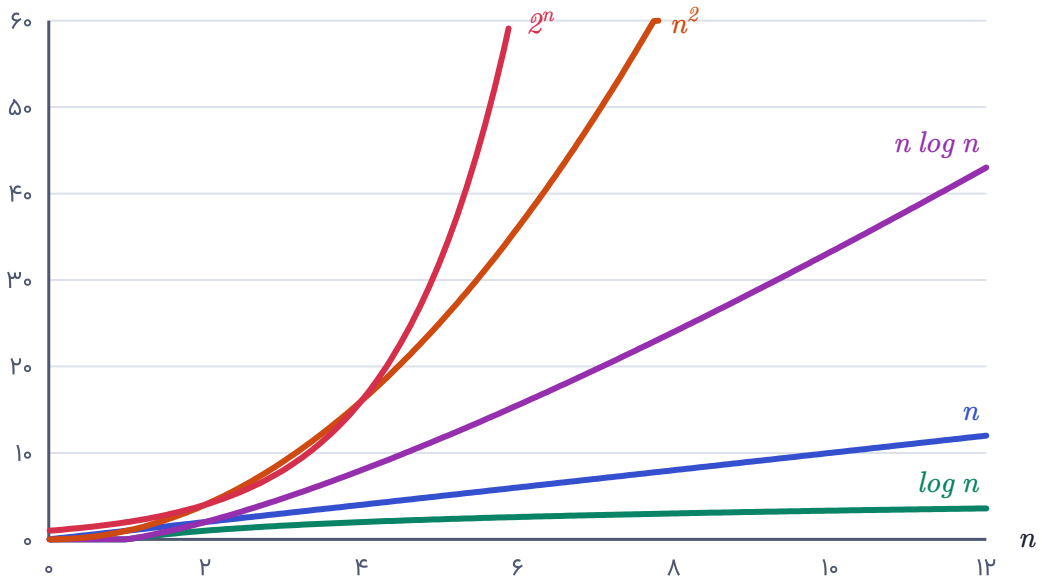
۵ لیست، پشته و صف

۶ درخت

۷ درخت عبارت

۸ هرم و صف اولویت

۹ مرتب‌سازی خطی



چرا تحلیل الگوریتم مهم است؟ با بزرگ‌شدن n، فاصلهٔ این منحنی‌ها آن‌قدر زیاد می‌شود که هیچ سخت‌افزار سریع‌تری جبرانش نمی‌کند.

پیش از شروع: این جزوه را چطور بخوانیم؟

اسلایدهای درس معمولاً فقط تیترها و فرمول‌ها را دارند و جای خالیِ «چرا این‌طور است؟» و «این را از کجا آوردیم؟» را به کلاس می‌سپارند. این جزوه همان جاهای خالی را پر می‌کند. ترتیب مطالب همان ترتیب اسلایدهاست، اما هر جا اسلاید از یک گام پرید، گام را کامل نوشته‌ایم. مطالبی هم که در اسلاید نبود ولی برای فهم عمیق‌تر یا امتحان لازم است، در کادرهای صورتیِ «تکمیلی» آمده‌اند تا مرز میان «اسلاید» و «افزوده» همیشه روشن باشد.

هر نوع کادر رنگ و معنای ثابتی دارد:

تعریف	تعریف دقیق یک مفهوم	**قضیه / لم**	حکمی که باید ثابت شود
اثبات	استدلال گام‌به‌گام	**مثال**	مثال حل‌شده با عدد
شهود	تصویر ذهنی پیش از فرمول	**چرا؟**	پاسخ به پرسش‌های رایج دانشجو
نکته	جزئیاتی که نباید از قلم بیفتد	**اشتباه رایج**	دام‌های امتحانی
تکمیلی	فراتر از اسلاید، برای فهم عمیق‌تر	**خودآزمایی**	سؤال تمرینی همراه پاسخ

نکته · قرارداد نمادها

در سراسر جزوه، $\log n$ بدون پایه یعنی $\log_2 n$ و $\ln n$ یعنی لگاریتم طبیعی. آرایه‌ها از اندیس ۱ شروع می‌شوند (مثل $A[1..n]$) مگر خلافش گفته شود. عبارت «زمان اجرا» یعنی تعداد عملیات پایه‌ای که الگوریتم انجام می‌دهد، نه ثانیه. شبه‌کدها چپ‌به‌راست نوشته شده‌اند و شمارهٔ خط دارند تا بتوانیم در متن به آن‌ها ارجاع دهیم.

فهرست مطالب

فصل ۱ طراحی و تحلیل الگوریتم‌ها

۱.۱ حل مسئله با الگوریتم

۱.۲ الگوهای طراحی و کاربردها

۱.۳ تحلیل الگوریتم چیست؟

۱.۴ چه چیزی زمان اجرا را تعیین می‌کند؟

۱.۵ چطور تحلیل کنیم؟ شمارش عملیات پایه

۱.۶ مسئلهٔ مرتب‌سازی و مرتب‌سازی درجی

۱.۷ انواع تحلیل: بدترین، میانگین، بهترین

۱.۸ تحلیل دقیق مرتب‌سازی درجی

۱.۹ مقایسهٔ توابع با نرخ رشد

۱.۱۰ نمادهای مجانبی Ω ،O و Θ

۱.۱۱ نکته‌های نمادگذاری و ویژگی‌ها

۱.۱۲ کران‌های مجانبی توابع رایج

۱.۱۳ زمان اجرای دستورهای مختلف

۱.۱۴ الگوریتم‌های چندجمله‌ای و اهمیت آن‌ها

۱.۱۵ مثال اول: اعداد فیبوناچی

۱.۱۶ مثال دوم: زیرآرایه با بیشترین مجموع

۱.۱۷ مروری بر زمان‌های اجرای رایج

۱.۱۸ خودآزمایی فصل ۱

فصل ۲ تقسیم و حل

۲.۱ ایدهٔ تقسیم و حل

۲.۲ مرتب‌سازی ادغامی

۲.۳ رابطهٔ بازگشتی و درخت بازگشت

۲.۴ درخت بازگشت در حالت کلی

۲.۵ قضیهٔ اصلی

۲.۶ اگر آرایه را به بیش از دو بخش تقسیم کنیم؟

۲.۷ زیرآرایهٔ بیشینه با تقسیم و حل

۲.۸ جمع و ضرب اعداد صحیح

۲.۹ ضرب کاراتسوبا

۲.۱۰ روش حدس و استقرا

۲.۱۱ خودآزمایی فصل ۲

فصل ۳ الگوریتم‌های تصادفی

۳.۱ تصادف در طراحی الگوریتم

۳.۲ فضای احتمال متناهی

۳.۳ متغیر تصادفی

۳.۴ امید ریاضی و خطی بودن آن

۳.۵ متغیر نشانگر و جایگشت تصادفی

۳.۶ میانهٔ تقریبی تصادفی

۳.۷ زمان اجرای الگوریتم‌های قطعی و تصادفی

۳.۸ مونت‌کارلو در برابر لاس‌وگاس

۳.۹ مسئلهٔ استخدام

۳.۱۰ انتخاب تصادفی

۳.۱۱ انتخاب قطعی: میانهٔ میانه‌ها

۳.۱۲ کوئیک‌سورت تصادفی

۳.۱۳ خودآزمایی فصل ۳

فصل ۴ **تحلیل سرشکنی**

۴.۱ سه نگاه به هزینه

۴.۲ مثال ۱: پشته با عمل multipop

۴.۳ مثال ۲: شمارندهٔ دودویی

۴.۴ روش‌های تحلیل سرشکنی

۴.۵ روش انبوهه

۴.۶ روش حساب‌داری

۴.۷ روش تابع پتانسیل

۴.۸ مثال تکمیلی: آرایهٔ پویا

۴.۹ مقایسهٔ سه روش

۴.۱۰ خودآزمایی فصل ۴

فصل ۵ **آرایه، لیست پیوندی، پشته و صف**

۵.۱ داده‌ساختار چیست؟

۵.۲ نوع دادهٔ انتزاعی

۵.۳ آرایه: حافظهٔ پیوسته و دسترسی مستقیم

۵.۴ لیست پیوندی: گره‌هایی که با اشاره‌گر به هم وصل‌اند

۵.۵ عملیات روی لیست پیوندی

۵.۶ لیست دوپیوندی و مقایسه با آرایه

۵.۷ معکوس کردن لیست پیوندی

۵.۸ پیاده‌سازی لیست پیوندی با آرایه

۵.۹ لیست دوری: شمارش گره‌ها با الگوریتم فلوید

۵.۱۰ لیست حلقوی و مسئلهٔ ژوزفوس

۵.۱۱ پشته: آخرین ورودی، اولین خروجی

۵.۱۲ کاربرد پشته: نزدیک‌ترین قیمت بزرگ‌تر بعدی

۵.۱۳ کاربرد پشته: قطار و جایگشت‌ها

۵.۱۴ صف: اولین ورودی، اولین خروجی

۵.۱۵ جمع‌بندی: رابط را به هزینه وصل کنید

۵.۱۶ خودآزمایی فصل ۵

فصل ۶ **درخت**

۶.۱ از لیست به درخت

۶.۲ درخت و جنگل

۶.۳ درخت جهت‌دار، ریشه و درخت ریشه‌دار

۶.۴ روابط: درخت به‌عنوان شجره‌نامه

۶.۵ درخت ترتیب‌دار

۶.۶ مسیر، اولاد و اجداد

۶.۷ عمق، ارتفاع و سطح

۶.۸ درخت‌های kتایی و انواع آن‌ها

۶.۹ ویژگی‌های پایه

۶.۱۰ پیاده‌سازی درخت با اشاره‌گر

۶.۱۱ پیاده‌سازی درخت با آرایه

۶.۱۲ تعریف بازگشتی و پیمایش درخت

۶.۱۳ مسئلهٔ ۱: بازسازی درخت از پیش‌ترتیب و پس‌ترتیب

۶.۱۴ درخت دودویی معادل: فرزند چپ، برادر راست

۶.۱۵ مسئلهٔ ۲ و ۳: پیمایش‌ها در درخت دودویی معادل

۶.۱۶ خودآزمایی فصل ۶

فصل ۷ **درخت عبارت**

۷.۱ مقدمه: درخت چه قدرتی به ما می‌دهد؟

۷.۲ عبارت، عملگر و عملوند

۷.۳ عملیات جبری: یگانی و دودویی

۷.۴ ترتیب عملیات

۷.۵ گرامر مستقل از متن (به زبان ساده)

۷.۶ درخت عبارت

۷.۷ مقداریابی بازگشتی

۷.۸ سه نگارش: میانوندی، پسوندی و پیشوندی

۷.۹ مسئلهٔ ۱: ساخت درخت از نگارش میانوندی با پرانتزبندی کامل

۷.۱۰ ساخت درخت از نگارش پیشوندی و پسوندی

۷.۱۱ مسئلهٔ ۲: ساخت درخت از نگارش میانوندی بدون پرانتز

۷.۱۲ خودآزمایی فصل ۷

فصل ۸ **هرم و صف اولویت**

۸.۱ از صف تا صف اولویت

۸.۲ پیاده‌سازی‌های ساده و مشکلشان

۸.۳ هرم

۸.۴ پیاده‌سازی هرم با آرایه

۸.۵ افزایش مقدار یک گره

۸.۶ افزودن یک گرهٔ جدید

۸.۷ حذف بزرگ‌ترین عنصر

۸.۸ ساخت هرم روی ورودی با اندازهٔ n

۸.۹ مسئلهٔ ۱: شمارش اعداد بزرگ‌تر از k

۸.۱۰ مسئلهٔ ۲: مرتب‌سازی با هرم

۸.۱۱ خودآزمایی فصل ۸

فصل ۹ **کران پایین مرتب‌سازی و مرتب‌سازی خطی**

۹.۱ مرور: تا این‌جا چطور مرتب کردیم؟

۹.۲ چقدر سریع می‌توان مرتب کرد؟

۹.۳ درخت تصمیم

۹.۴ کران پایین Ω(nlogn) برای مرتب‌سازی مقایسه‌ای

۹.۵ مرتب‌سازی شمارشی

۹.۶ پایداری

۹.۷ مرتب‌سازی مبنایی
۹.۸ جمع‌بندی مرتب‌سازی‌ها
۹.۹ خودآزمایی فصل ۹

پیوست پیوست: برگهٔ خلاصه، واژه‌نامه و منابع

پ.۱ فرمول‌های پرکاربرد
پ.۲ جدول الگوریتم‌های جزوه
پ.۳ واژه‌نامهٔ فارسی–انگلیسی
پ.۴ منابع

فصل ۱

طراحی و تحلیل الگوریتم‌ها

یاد می‌گیریم چطور سرعت یک الگوریتم را مستقل از کامپیوتر و زبان برنامه‌نویسی بسنجیم، و چرا یک ایدهٔ بهتر از هزار برابر سخت‌افزار سریع‌تر ارزش دارد.

مقدمه | اعداد فیبوناچی | زیرآرایه با بیشترین مجموع | مرتبهٔ رشد مجانبی | زمان‌های اجرای رایج

۱.۱ حل مسئله با الگوریتم

هر بار که با کامپیوتر مسئله‌ای را حل می‌کنیم، چهار مرحله را پشت سر می‌گذاریم، حتی اگر حواسمان به آن‌ها نباشد:

۱. **مسئله:** دقیقاً مشخص می‌کنیم ورودی چیست و خروجی باید چه ویژگی‌ای داشته باشد. مسئلهٔ مبهم، الگوریتم درست ندارد.

۲. **طراحی الگوریتم:** یک دستورالعمل گام‌به‌گام، متناهی و بی‌ابهام می‌نویسیم که برای **هر** ورودی مجاز، خروجی درست را تولید کند.

۳. **تحلیل الگوریتم:** ثابت می‌کنیم الگوریتم درست کار می‌کند و می‌سنجیم چقدر منابع (زمان و حافظه) مصرف می‌کند.

۴. **برنامه‌نویسی:** الگوریتم را به زبان یک ماشین واقعی ترجمه می‌کنیم.

تمرکز این درس روی مرحلهٔ دوم و سوم است. شاید بپرسید چرا مستقیم سراغ کدنویسی نرویم و بعد برنامه را اجرا نکنیم تا ببینیم سریع است یا نه؟ پاسخ در کادر زیر است.

چرا؟ · چرا قبل از کدنویسی تحلیل کنیم؟

اول، اجرای برنامه فقط دربارهٔ ورودی‌هایی که امتحان کرده‌ایم چیزی می‌گوید؛ ممکن است ورودی بدی وجود داشته باشد که آن را نیازموده‌ایم. دوم، نوشتن و اشکال‌زدایی برنامه گران است؛ اگر از روی کاغذ بفهمیم ایده‌ای برای ورودی‌های بزرگ چند هزار سال طول می‌کشد، از همان اول سراغش نمی‌رویم. سوم، تحلیل به ما می‌گوید **کجای** الگوریتم گلوگاه است و کجا ارزش بهینه‌سازی دارد. در این فصل با چند مثال می‌بینیم که تغییر ایده (نه تغییر سخت‌افزار) زمان را از «میلیون‌ها سال» به «چند میکروثانیه» می‌رساند.

۱.۲ الگوهای طراحی و کاربردها

الگوریتم‌ها هزاران شکل دارند، اما بیشترشان از چند **الگوی طراحی** (algorithmic paradigms) پیروی می‌کنند. الگو یعنی یک راهبرد کلی که برای دستهٔ بزرگی از مسائل جواب می‌دهد. شناختن این الگوها یعنی وقتی با مسئلهٔ تازه‌ای روبه‌رو می‌شوید، با دست خالی شروع نمی‌کنید:

الگو	ایدهٔ اصلی	نمونه
حریصانه (Greedy)	در هر گام بهترین انتخابِ همان لحظه را می‌کنیم و هرگز به عقب برنمی‌گردیم.	درخت پوشای کمینه
تقسیم و حل (Divide & Conquer)	مسئله را به چند زیرمسئلهٔ کوچک‌تر می‌شکنیم، هر کدام را بازگشتی حل می‌کنیم و جواب‌ها را ترکیب می‌کنیم.	مرتب‌سازی ادغامی (فصل ۲)
برنامه‌ریزی پویا (Dynamic Programming)	جواب زیرمسئله‌های هم‌پوشان را یک بار حساب و ذخیره می‌کنیم تا دوباره‌کاری نشود.	فیبوناچی با حلقه (همین فصل)
شارش شبکه (Network Flow)	مسئله را به عبور «جریان» در یک گراف با ظرفیت‌های محدود تبدیل می‌کنیم.	تطابق بیشینه
تصادفی (Randomized)	الگوریتم خودش سکه می‌اندازد تا از ورودی‌های بد فرار کند یا ساده‌تر شود.	کوئیک‌سورت تصادفی (فصل ۳)

دامنهٔ کاربرد الگوریتم‌ها تقریباً بی‌انتهاست: حافظهٔ نهان (caching)، کامپایلرها، پایگاه‌های داده، زمان‌بندی، شبکه، تحلیل داده، پردازش سیگنال، گرافیک کامپیوتری، محاسبات علمی، تحقیق در عملیات، هوش مصنوعی و زیست‌شناسی محاسباتی، تنها بخشی از آن‌اند. در همهٔ این حوزه‌ها، تفاوت میان الگوریتم خوب و بد، تفاوت میان «قابل استفاده» و «غیرقابل استفاده» است.

۱.۳ تحلیل الگوریتم چیست؟

تعریف · تحلیل الگوریتم

تحلیل به مجموعهٔ روش‌های ریاضی گفته می‌شود که با آن‌ها دو چیز را ثابت می‌کنیم: **درستی** (correctness) یعنی الگوریتم برای هر ورودی مجاز خروجی درست می‌دهد، و **کارایی** (efficiency) یعنی الگوریتم چه مقدار منبع (معمولاً زمان) مصرف می‌کند.

دربارهٔ کارایی یک الگوریتم A معمولاً دو پرسش داریم:

- **زمان اجرای A چقدر است؟** این را با یک تابع از اندازهٔ ورودی بیان می‌کنیم.
- **آیا A کاراترین الگوریتم ممکن برای این مسئله است؟** برای پاسخ باید یک **کران پایین** پیدا کنیم: ثابت کنیم **هیچ** الگوریتمی نمی‌تواند این مسئله را سریع‌تر از فلان مقدار حل کند. اگر زمان A با این کران پایین برابر باشد، A بهینه است. مثلاً در فصل ۲

می‌بینیم که جمع دو عدد n بیتی حداقل $\Omega(n)$ زمان می‌خواهد (چون باید همهٔ بیت‌ها را دست‌کم یک بار دید)، پس روش جمع دبستانی بهینه است.

۱.۴ چه چیزی زمان اجرا را تعیین می‌کند؟

زمان اجرای یک برنامه به عوامل زیادی بستگی دارد:

نکته · عوامل بیرونی

سخت‌افزار (سرعت پردازنده، حافظهٔ نهان) و **کامپایلر** (کیفیت بهینه‌سازی). این‌ها ربطی به ایدهٔ الگوریتم ندارند و معمولاً زمان را فقط در یک **ضریب ثابت** ضرب می‌کنند.

نکته · عوامل مربوط به ورودی

اندازهٔ ورودی (مثلاً تعداد عناصر آرایه) و **چیدمان ورودی** (مثلاً آرایه از قبل مرتب است یا برعکس). این‌ها همان چیزی‌اند که تحلیل ما باید دربارهٔ آن‌ها حرف بزند.

تعریف · تابع زمان اجرا

$T(n)$ یعنی زمان اجرای الگوریتم وقتی اندازهٔ ورودی n است. گاهی زمان به بیش از یک متغیر وابسته است؛ مثلاً برای گرافی با n رأس و m یال، زمان را با $T(n, m)$ نشان می‌دهیم.

نکته · «اندازهٔ ورودی» یعنی چه؟

اندازهٔ ورودی تعداد «واحد»های اطلاعاتی ورودی است و به مسئله بستگی دارد: در مرتب‌سازی تعداد عناصر، در ضرب دو عدد بزرگ **تعداد بیت‌ها** (نه خود عدد!)، و در گراف تعداد رأس‌ها و یال‌ها. این تمایز در فصل ۲ (ضرب اعداد صحیح) اهمیت پیدا می‌کند.

۱.۵ چطور تحلیل کنیم؟ شمارش عملیات پایه

برای اندازه‌گیری کارایی دو راه به ذهن می‌رسد:

اشتباه رایج · راه اول: زمان‌سنجی

برنامه را برای اندازه‌های مختلف ورودی اجرا کنیم و با ساعت زمان بگیریم. **خوب نیست**، چون عدد به‌دست‌آمده مخصوص همان کامپیوتر و همان کامپایلر است و دربارهٔ ورودی‌هایی که امتحان نکرده‌ایم چیزی نمی‌گوید.

مثال · راه دوم: شمارش عملیات

تعداد **عملیات پایه** (primitive operations) را بشماریم: جمع $(A = B + C)$، ضرب، افزایش $(A = A + 1)$، انتساب، مقایسه $(A < B)$، عملیات منطقی و ... . **خوب است**، ولی هر عمل پایه زمان متفاوتی دارد.

برای ساده شدن کار، راه دوم را با یک فرض انتخاب می‌کنیم:

تعریف · فرض هزینهٔ واحد

هر عمل پایه دقیقاً ۱ واحد زمان می‌برد. با این فرض، وابستگی به سرعت کامپیوتر حذف می‌شود؛ بدون آن، مقایسه و حتی وارسی ادعاهای ما ناممکن بود.

چرا؟ · این فرض که آشکارا غلط است! چرا مجاز است؟

درست است که ضرب از جمع کندتر است. اما اگر کندترین عمل پایه k برابر سریع‌ترین باشد، زمان واقعی حداکثر k برابر عدد شمرده‌شده است. k یک **ثابت** است و به n بستگی ندارد. در ادامه می‌بینیم که در تحلیل مجانبی ضرایب ثابت را عمداً نادیده می‌گیریم، پس این فرض هیچ چیزِ مهمی را خراب نمی‌کند. آنچه مهم است **شکل رشد** تعداد عملیات با n است.

این فرض فقط وقتی می‌شکند که اعداد بی‌اندازه بزرگ شوند (مثلاً جمع دو عدد هزاررقمی دیگر «یک عمل» نیست). در چنین جاهایی، مثل ضرب اعداد صحیح در فصل ۲، به‌جای عمل روی کل عدد، **عمل روی بیت‌ها** را می‌شماریم.

۱.۶ مسئلهٔ مرتب‌سازی و مرتب‌سازی درجی

تعریف · مسئلهٔ مرتب‌سازی

ورودی: دنبالهٔ $\langle a_1, a_2, \ldots, a_n \rangle$ از اعداد.

خروجی: جایگشتی $\langle a'_1, a'_2, \ldots, a'_n \rangle$ از همان اعداد به‌طوری که $a'_1 \le a'_2 \le \cdots \le a'_n$.

مثال: ورودی $6\ 3\ 9\ 4\ 2\ 8$ و خروجی $2\ 3\ 4\ 6\ 8\ 9$.

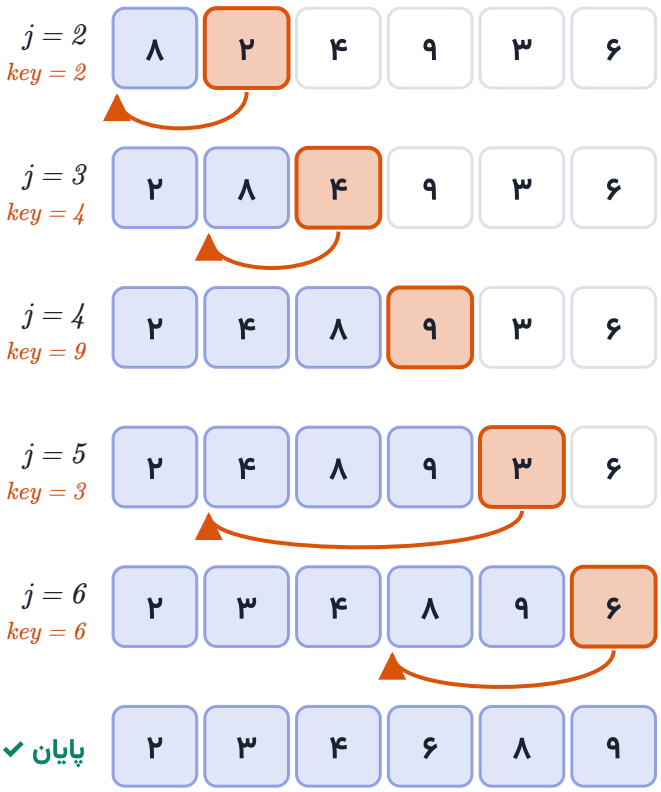
شهود · مرتب کردن ورق‌های بازی

مرتب‌سازی درجی (Insertion Sort) همان کاری است که موقع مرتب کردن ورق‌های بازی در دست انجام می‌دهید: ورق‌های دست چپ همیشه مرتب‌اند؛ ورق تازه را برمی‌دارید، از راست به چپ با ورق‌های دستتان مقایسه می‌کنید، ورق‌های بزرگ‌تر را یکی‌یکی کنار می‌زنید و ورق تازه را در جای درستش **درج** می‌کنید.

```
INSERTION-SORT(A, n) ▷ A[1..n]

1  for j = 2 to n
2      key = A[j]                      // ورق تازه
3      i = j - 1
4      while i > 0 and A[i] > key      // بزرگ‌ترها را یکی به راست ببر
5          A[i+1] = A[i]
6          i = i - 1
7      A[i+1] = key                    // ورق را در جای خالی بگذار
```

در هر دور حلقهٔ بیرونی، بخش $A[1..j-1]$ مرتب است و $A[j]$ (یعنی key) باید در آن درج شود. حلقهٔ `while` از انتهای بخش مرتب به سمت چپ حرکت می‌کند و هر عنصرِ بزرگ‌تر از key را یک خانه به راست می‌برد. وقتی به عنصری کوچک‌تر یا مساوی رسید (یا به ابتدای آرایه)، خانهٔ $i+1$ خالی شده است و key آنجا می‌نشیند.



شکل ۱-۱ اجرای مرتب‌سازی درجی روی 6 3 9 4 2 8. خانه‌های آبی بخش مرتب‌شده‌اند، خانهٔ نارنجی key است و پیکان نشان می‌دهد key کجا درج می‌شود. در گام $j=4$ عدد ۹ از همهٔ قبلی‌ها بزرگ‌تر است و جابه‌جا نمی‌شود.

چرا این الگوریتم درست است؟ ناوردای حلقه

برای اثبات درستی الگوریتم‌های حلقه‌ای از ابزاری به نام **ناوردای حلقه** (loop invariant) استفاده می‌کنیم: گزاره‌ای که در شروع هر دور حلقه برقرار است.

قضیه · ناوردای مرتب‌سازی درجی

در شروع هر دور حلقهٔ `for` با شمارندهٔ j، زیرآرایهٔ $A[1..j-1]$ شامل همان عناصری است که در ابتدا در $A[1..j-1]$ بودند، ولی به ترتیب صعودی.

آغاز: وقتی $j = 2$ است، $A[1..1]$ فقط یک عنصر دارد و آرایهٔ یک‌عنصری خودبه‌خود مرتب است.

نگهداری: فرض کنید ناوردا در شروع دور j برقرار باشد. حلقهٔ `while` عناصر $A[j-1], A[j-2], \ldots$ را که از key بزرگ‌ترند یک خانه به راست می‌برد و key را درست بعد از اولین عنصرِ نابزرگ‌تر قرار می‌دهد. پس $A[1..j]$ همان عناصر قبلی به‌اضافهٔ key است و مرتب است؛ یعنی ناوردا در شروع دور $j+1$ برقرار است.

پایان: حلقه وقتی تمام می‌شود که $j = n+1$. با جای‌گذاری در ناوردا، $A[1..n]$ مرتب است؛ یعنی کل آرایه مرتب شده است.

∎

نکته · ناوردای حلقه = استقرای ریاضی

«آغاز» همان پایهٔ استقراست، «نگهداری» همان گام استقرا، و «پایان» جایی است که از ناوردا نتیجهٔ دلخواه را بیرون می‌کشیم. تفاوتش با استقرای معمولی این است که استقرا تا بی‌نهایت ادامه دارد ولی حلقه یک جا متوقف می‌شود.

۱.۷ انواع تحلیل: بدترین، میانگین، بهترین

زمان مرتب‌سازی درجی فقط به n بستگی ندارد؛ به چیدمان ورودی هم وابسته است. اگر آرایه از قبل مرتب باشد حلقهٔ `while` هرگز اجرا نمی‌شود، اما اگر برعکس مرتب باشد هر عنصر باید تا ابتدای آرایه برود. پس باید مشخص کنیم «زمان اجرا» دقیقاً کدام عدد است:

بدترین حالت $T(n)$ = **بیشترین** زمان الگوریتم روی **هر** ورودی به اندازهٔ n. معمولاً همین را می‌خواهیم

حالت میانگین $T(n)$ = **امید ریاضی** زمان روی همهٔ ورودی‌های اندازهٔ n. نیازمند فرضی دربارهٔ **توزیع آماری** ورودی‌هاست. گاهی

بهترین حالت $T(n)$ = **کمترین** زمان روی ورودی‌های اندازهٔ n. تقریباً هرگز

چرا؟ · چرا بدترین حالت؟ و چرا بهترین حالت «تقلب» است؟

بدترین حالت یک **تضمین** است: مهم نیست ورودی چه باشد، الگوریتم از این کندتر نمی‌شود. برای سیستمی که باید همیشه پاسخ بدهد (مثلاً کنترل ترمز خودرو) همین تضمین ارزش دارد. به‌علاوه، در بسیاری از الگوریتم‌ها بدترین حالت زیاد پیش می‌آید (مثلاً جست‌وجوی چیزی که در پایگاه داده نیست).

حالت میانگین مفید است، اما «میانگین روی چه توزیعی؟» اگر فرض کنیم همهٔ ورودی‌ها هم‌احتمال‌اند و در عمل ورودی‌ها اغلب تقریباً مرتب باشند، نتیجهٔ ما بی‌معناست.

بهترین حالت تقریباً هیچ چیز نمی‌گوید: هر الگوریتم کُندی را می‌توان طوری دستکاری کرد که روی **یک** ورودی خاص سریع باشد (مثلاً اول بررسی کند «آیا ورودی همان آرایهٔ مرتبِ معروف است؟» و اگر بود فوراً جواب بدهد). این یعنی با بهترین حالت می‌شود سرِ هر کسی کلاه گذاشت.

۱.۸ تحلیل دقیق مرتب‌سازی درجی

حالا با فرض هزینهٔ واحد، تعداد اجرای هر خط را می‌شماریم. فرض کنید t_j تعداد دفعاتی باشد که **شرطِ** `while` برای مقدار j بررسی می‌شود (یک بار بیشتر از تعداد جابه‌جایی‌ها، چون آخرین بررسی شکست می‌خورد).

خط	دستور	هزینه	تعداد اجرا
۱	`for j = 2 to n`	c_1	n
۲	`key = A[j]`	c_2	$n-1$
۳	`i = j - 1`	c_3	$n-1$
۴	`while ...`	c_4	$\sum_{j=2}^{n} t_j$
۵	`A[i+1] = A[i]`	c_5	$\sum_{j=2}^{n} (t_j - 1)$
۶	`i = i - 1`	c_6	$\sum_{j=2}^{n} (t_j - 1)$
۷	`A[i+1] = key`	c_7	$n-1$

(خط `for` یک بار بیشتر از بدنه اجرا می‌شود، چون آخرین بار شرطش بررسی و رد می‌شود.) پس:

$$T(n) = c_1 n + (c_2 + c_3 + c_7)(n-1) + c_4 \sum_{j=2}^{n} t_j + (c_5 + c_6) \sum_{j=2}^{n} (t_j - 1).$$

بهترین حالت: آرایهٔ مرتب

اگر آرایه از قبل مرتب باشد، برای هر j اولین مقایسهٔ $A[j-1] > key$ نادرست است، پس $t_j = 1$ و جملهٔ $\sum (t_j - 1)$ صفر می‌شود:

$$T(n) = c_1 n + (c_2 + c_3 + c_4 + c_7)(n-1) = an + b \;=\; \Theta(n).$$

یعنی زمان **خطی** است. (همان $\sum_{j=2}^{n} 1$ که در اسلاید آمده.)

بدترین حالت: آرایهٔ معکوس

اگر آرایه نزولی باشد، key از همهٔ عناصر $A[1..j-1]$ کوچک‌تر است و باید با همه مقایسه شود، پس $t_j = j$. با فرمول جمع حسابی:

$$\sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1, \qquad \sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2}.$$

با جای‌گذاری، $T(n) = an^2 + bn + c$ برای ثابت‌های a, b, c به دست می‌آید، یعنی $T(n) = \Theta(n^2)$: زمان **درجهٔ دو**.

حالت میانگین: همهٔ جایگشت‌ها هم‌احتمال

اگر همهٔ $n!$ جایگشت ورودی هم‌احتمال باشند، key به‌طور متوسط از نیمی از عناصر $A[1..j-1]$ کوچک‌تر است؛ پس به‌طور متوسط $t_j \approx j/2$. جمع $\sum_{j=2}^{n} j/2 \approx n^2/4$ است که باز هم $\Theta(n^2)$ است. یعنی **حالت میانگین فقط حدود نصفِ بدترین حالت است و مرتبه‌اش عوض نمی‌شود.**

جمع‌بندی · مرتب‌سازی درجی

بدترین حالت $\Theta(n^2)$ · حالت میانگین $\Theta(n^2)$ · بهترین حالت $\Theta(n)$. برای nهای کوچک یا آرایه‌های تقریباً مرتب سریع است، ولی برای nهای بزرگ نه.

نکته · از ثابت‌ها خلاص شدیم!

در محاسبهٔ بالا ثابت‌های $c_1, \dots, c_7$ را نمی‌دانستیم، اما نتیجهٔ نهایی ($\Theta(n^2)$) به آن‌ها بستگی نداشت. این دقیقاً همان چیزی است که می‌خواستیم: نتیجه‌ای مستقل از سخت‌افزار. ابزار رسمی این «دور ریختن ثابت‌ها» نمادهای مجانبی‌اند که در ادامه می‌آیند.

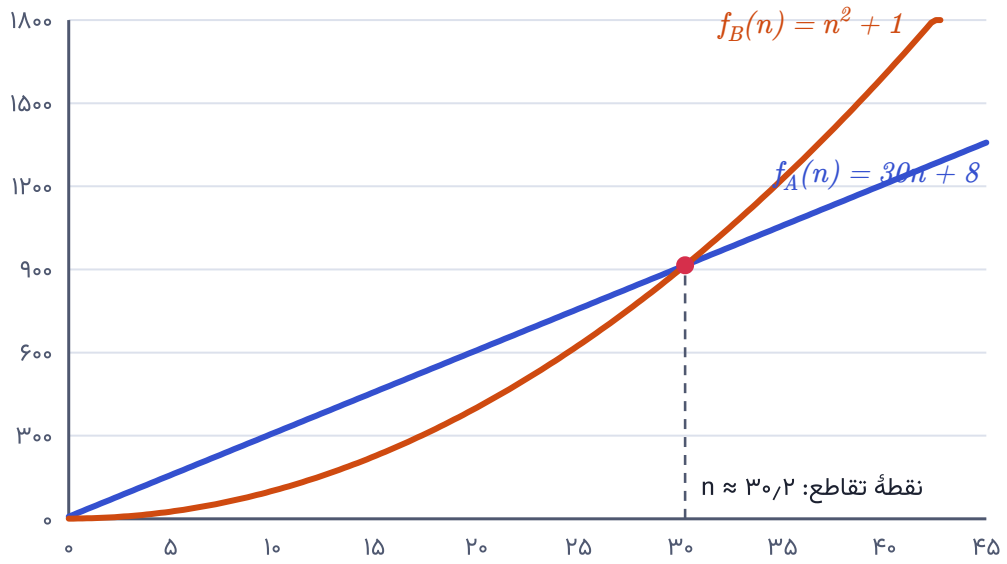
۱.۹ مقایسهٔ توابع با نرخ رشد

وقتی می‌گوییم «تابع f از g سریع‌تر رشد می‌کند»، منظورمان **رفتار مجانبی** است: برای nهای به‌اندازهٔ کافی بزرگ، $f(n)$ سرانجام از $g(n)$ بیشتر می‌شود و دیگر هرگز کمتر نمی‌شود، هر قدر هم که در ابتدا کوچک‌تر بوده باشد. به همین دلیل به این نوع مقایسه **تحلیل مجانبی** (asymptotic analysis) می‌گوییم.

مثال · کدام برنامه را انتخاب کنیم؟

فرض کنید سایتی برای پردازش داده‌های کاربران (مثلاً سوابق مالی) طراحی می‌کنید. برنامهٔ A برای پردازش n رکورد $f_A(n) = 30n + 8$ میکروثانیه و برنامهٔ B $f_B(n) = n^2 + 1$ میکروثانیه زمان می‌برد. اگر بدانید روزی میلیون‌ها کاربر خواهید داشت، کدام را انتخاب می‌کنید؟

برای nهای کوچک B سریع‌تر است (مثلاً $n = 10$: $f_A = 308$ ولی $f_B = 101$). اما $f_B(n) > f_A(n)$ یعنی $n^2 - 30n - 7 > 0$، که با حل معادلهٔ درجه دو برای $n \geq 31$ برقرار است. از آنجا به بعد B همیشه کندتر است و فاصله مدام بیشتر می‌شود: برای یک میلیون رکورد، A حدود ۳۰ ثانیه و B حدود 10^{12} میکروثانیه، یعنی بیش از ۱۱ روز طول می‌کشد. **پس A را انتخاب می‌کنیم.**



شکل ۱-۲ تابعی که سریع‌تر رشد می‌کند، دیر یا زود از دیگری بالاتر می‌رود. ضریب ۳۰ فقط محل تقاطع را جابه‌جا می‌کند، نه نتیجهٔ نهایی را.

می‌گوییم $f_A(n) = 30n + 8$ «از مرتبهٔ n» است و می‌نویسیم $O(n)$: حداکثر تقریباً متناسب با n رشد می‌کند. همین‌طور $f_B(n) = n^2 + 1$ از مرتبهٔ n^2 یعنی $O(n^2)$ است. به‌طور کلی، یک الگوریتم $O(n^2)$ برای ورودی‌های بزرگ از یک الگوریتم $O(n)$ کندتر است.

۱.۱۰ نمادهای مجانبی O، Ω و Θ

حالا همین ایده‌ها را دقیق می‌کنیم. در همهٔ تعریف‌ها f و g توابعی از اعداد طبیعی به اعداد نامنفی‌اند (زمان اجرا هرگز منفی نیست).

تعریف · کران بالا: O بزرگ

$f(n)$ از $O(g(n))$ است اگر ثابت‌های $c > 0$ و $n_0 \geq 0$ وجود داشته باشند به‌طوری که

$$\forall n \geq n_0 : \quad f(n) \leq c \cdot g(n).$$

تعریف · کران پایین: امگای بزرگ

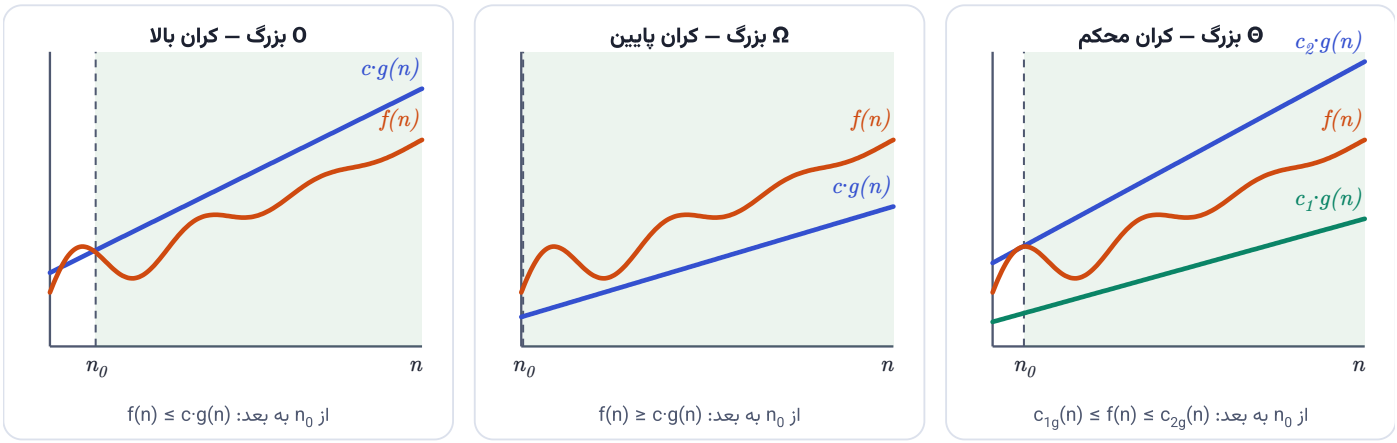
$f(n)$ از $\Omega(g(n))$ است اگر ثابت‌های $c > 0$ و $n_0 \geq 0$ وجود داشته باشند به‌طوری که

$$\forall n \geq n_0 : \quad f(n) \geq c \cdot g(n).$$

تعریف · کران محکم: تتای بزرگ

$f(n)$ از $\Theta(g(n))$ است اگر هم $O(g(n))$ باشد و هم $\Omega(g(n))$. معادلش: ثابت‌های $c_1, c_2 > 0$ و n_0 وجود دارند به‌طوری که

$$\forall n \geq n_0 : \quad c_1\, g(n) \leq f(n) \leq c_2\, g(n).$$



شکل ۱-۳ ناحیهٔ سبز جایی است که تعریف باید برقرار باشد. قبل از n_0 هر اتفاقی ممکن است بیفتد و برای ما مهم نیست.

چرا؟ · چرا c و چرا n_0؟

ثابت c همان ابزار «دور ریختن ضرایب» است: $30n$ و n باید هم‌مرتبه حساب شوند، پس اجازه می‌دهیم g را در هر ثابتی که لازم است ضرب کنیم. **آستانهٔ** n_0 ابزار «مجانبی بودن» است: رفتار تابع برای nهای کوچک اهمیت ندارد، پس می‌گوییم «از جایی به بعد». بدون n_0، مثلاً $n^2 = O(n^2 - 5)$ نادرست می‌شد، چون برای $n = 2$ سمت راست منفی است.

مثال · $f(n) = 32n^2 + 17n + 32$

ادعا: $f(n) = O(n^2)$. برای $n \geq 1$ داریم $17n \leq 17n^2$ و $32 \leq 32n^2$، پس

$$f(n) \leq 32n^2 + 17n^2 + 32n^2 = 81\, n^2.$$

یعنی $c = 81$ و $n_0 = 1$ کار می‌کند. **ادعا:** $f(n) = \Omega(n^2)$. برای همهٔ n داریم $f(n) \geq 32n^2$، پس $c = 32$ کافی است. در نتیجه $f(n) = \Theta(n^2)$.

از همین‌ها نتیجه می‌شود:

- $f(n)$ از $O(n^2)$، $O(n^3)$، $\Omega(n^2)$، $\Omega(n)$ و $\Theta(n^2)$ **است** (چون $n^2 \le n^3$ و $n^2 \ge n$ برای $n \ge 1$).
- $f(n)$ از $O(n)$، $\Omega(n^3)$، $\Theta(n)$ و $\Theta(n^3)$ **نیست**.

مثلاً چرا $f(n) \neq O(n)$؟ فرض کنید c و n_0 وجود داشته باشند که $32n^2 + 17n + 32 \le cn$. آنگاه $32n^2 \le cn$، یعنی $n \le c/32$ برای **همهٔ** $n \ge n_0$، که ممکن نیست چون n می‌تواند هر قدر بخواهد بزرگ شود. تناقض.

تکمیلی (فراتر از اسلاید) · آزمون حد و نمادهای o و ω کوچک

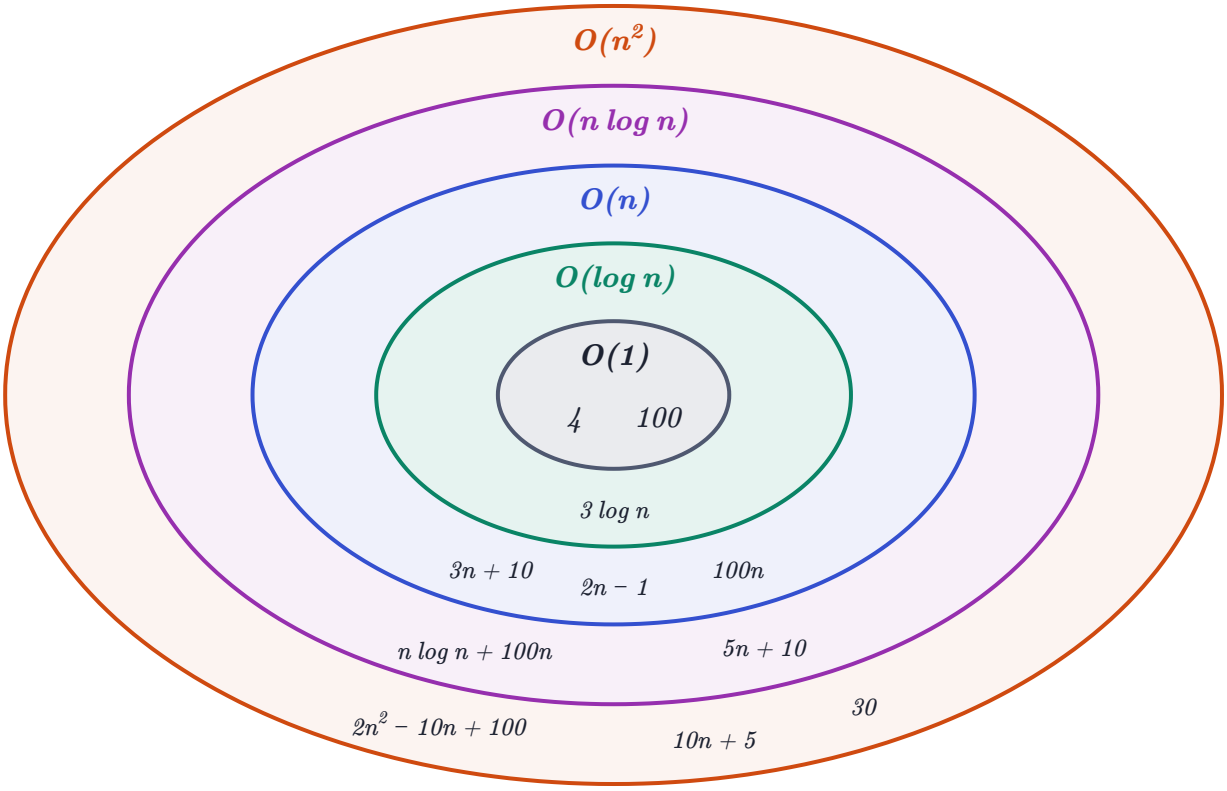
در عمل، ساده‌ترین راه مقایسهٔ دو تابع محاسبهٔ حد نسبت آن‌هاست. اگر $L = \lim_{n\to\infty} f(n)/g(n)$ وجود داشته باشد:

- اگر $0 < L < \infty$، آنگاه $f = \Theta(g)$.
- اگر $L = 0$، آنگاه $f = O(g)$ ولی $f \neq \Omega(g)$. در این حالت می‌نویسیم $f = o(g)$ (o کوچک: «اکیداً کندتر رشد می‌کند»).
- اگر $L = \infty$، آنگاه $f = \Omega(g)$ ولی $f \neq O(g)$. می‌نویسیم $f = \omega(g)$.

مثلاً $\lim \frac{32n^2+17n+32}{n^2} = 32$، پس $\Theta(n^2)$. برای حدهای سخت‌تر از قاعدهٔ هوپیتال استفاده می‌کنیم.

نماد O یک «مجموعه» است

دقیق‌ترین برداشت از $O(g(n))$ این است: **مجموعهٔ همهٔ توابعی که از g سریع‌تر رشد نمی‌کنند.** مثلاً $3n + 10$ شامل $O(n)$، $2n - 1$ و $100n$ است؛ ولی شامل ثابت‌هایی مثل 3 یا 100 هم هست، چون ثابت‌ها هم از n سریع‌تر رشد نمی‌کنند. به همین ترتیب $10n + 5$ و $100n$ و حتی عدد 30 عضو $O(n^2)$ هستند. نتیجه این‌که این مجموعه‌ها تودرتو هستند:



شکل ۱-۴ $O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2)$

. هر تابع داخل یک حلقه، عضو همهٔ حلقه‌های بیرونی آن هم هست.

اشتباه رایج · «O یعنی دقیقاً همین مرتبه»؟ نه!

O فقط **کران بالا** است. جملهٔ «زمان اجرای مرتب‌سازی درجی $O(n^3)$ است» **درست** است، فقط دقیق نیست. اگر بخواهید بگویید مرتبه دقیقاً همین است، باید از Θ استفاده کنید. در امتحان، وقتی سؤال «مرتبهٔ زمانی» را می‌خواهد، بهترین پاسخ Θ است.

۱.۱۱ نکته‌های نمادگذاری و ویژگی‌ها

سوءاستفادهٔ مجاز از علامت مساوی

همه می‌نویسیم $f(n) = O(g(n))$، اما این «مساوی» مساوی واقعی نیست؛ **یک‌طرفه** است. مثال اسلاید: اگر $f(n) = 5n^3$ و $g(n) = 3n^2$، آنگاه $f(n) = O(n^3)$ و $g(n) = O(n^3)$؛ اگر این‌ها مساوی واقعی بودند، با تعدی نتیجه می‌گرفتیم $f(n) = g(n)$، که غلط است. نمادگذاری دقیق‌تر $f(n) \in O(g(n))$ است، و $=$ را باید «عضوِ» خواند.

اشتباه رایج · جملهٔ بی‌معنا

«هر الگوریتم مرتب‌سازیِ مبتنی بر مقایسه **حداقل** $O(n \log n)$ مقایسه لازم دارد.» این جمله بی‌معناست، چون O کران بالاست و «حداقلِ یک کران بالا» هیچ اطلاعاتی نمی‌دهد (هر تابعی، حتی صفر، از «حداقل O» کوچک‌تر نیست!). شکل درست: «... به $\Omega(n \log n)$ مقایسه نیاز دارد.» **برای کران پایین از Ω استفاده کنید.**

ویژگی‌ها

قضیه · تعدی و جمع‌پذیری

تعدی: اگر $f = O(g)$ و $g = O(h)$ آنگاه $f = O(h)$. همین برای Ω و Θ هم برقرار است.

جمع‌پذیری: اگر $f = O(h)$ و $g = O(h)$ آنگاه $f + g = O(h)$. اگر $f = \Omega(h)$ و $g = \Omega(h)$ آنگاه $f + g = \Omega(h)$. و اگر $f = \Theta(h)$ و $g = O(h)$ آنگاه $f + g = \Theta(h)$.

تعدی: ثابت‌های c_1, n_1 و c_2, n_2 وجود دارند که برای $n \geq n_1$، $f(n) \leq c_1 g(n)$ و برای $n \geq n_2$، $g(n) \leq c_2 h(n)$. پس برای $n \geq \max(n_1, n_2)$ داریم $f(n) \leq c_1 c_2\, h(n)$. ثابت $c = c_1 c_2$ و آستانهٔ $n_0 = \max(n_1, n_2)$ کار می‌کنند.

جمع‌پذیری: برای $n \geq \max(n_1, n_2)$، $f(n) + g(n) \leq c_1 h(n) + c_2 h(n) = (c_1 + c_2)\, h(n)$.

حالت سوم: کران بالا از قسمت قبل می‌آید. برای کران پایین، چون $g \geq 0$، داریم $f + g \geq f \geq c\, h$.

∎

نکته · کاربرد عملی جمع‌پذیری

اگر الگوریتمی از دو مرحلهٔ پشت سر هم با زمان‌های $\Theta(n^2)$ و $O(n \log n)$ تشکیل شده باشد، کل زمان $\Theta(n^2)$ است: **کندترین مرحله تعیین‌کننده است.**

۱.۱۲ کران‌های مجانبی توابع رایج

قضیه · چندجمله‌ای‌ها

$a_0 + a_1 n + \cdots + a_d n^d$ با $a_d > 0$ از $\Theta(n^d)$ است. یعنی فقط جملهٔ با بالاترین توان مهم است.

کران بالا: برای $n \geq 1$ هر $|a_i| n^i \leq |a_i| n^d$، پس مجموع حداکثر $\left(\sum_i |a_i|\right) n^d$ است.

کران پایین: مجموع دست‌کم $a_d n^d - \sum_{i<d} |a_i| n^{d-1}$ است. اگر $S = \sum_{i<d} |a_i|$ و $n \geq 2S/a_d$، آنگاه $S\, n^{d-1} \leq \frac{a_d}{2} n^d$، پس مجموع دست‌کم $\frac{a_d}{2} n^d$ است.

■

زمان چندجمله‌ای: زمان اجرا $O(n^d)$ برای ثابتی مثل d که به اندازهٔ ورودی n بستگی ندارد.

قضیه · لگاریتم‌ها

(۱) برای هر دو ثابت $a, b > 1$: $O(\log_a n) = O(\log_b n)$. پس **می‌توان پایهٔ لگاریتم را ننوشت.**

(۲) برای هر $x > 0$: $\log n = O(n^x)$. یعنی لگاریتم از **هر** چندجمله‌ای، حتی $n^{0.001}$، کندتر رشد می‌کند.

(۱) با قاعدهٔ تغییر پایه، $\log_a n = \dfrac{\log_b n}{\log_b a}$. مخرج یک ثابت مثبت است، پس دو لگاریتم فقط در یک ضریب ثابت تفاوت دارند.

(۲) با هوپیتال: $\lim_{n\to\infty} \dfrac{\ln n}{n^x} = \lim_{n\to\infty} \dfrac{1/n}{x\, n^{x-1}} = \lim_{n\to\infty} \dfrac{1}{x\, n^x} = 0.$

■

قضیه · نمایی‌ها

برای هر $r > 1$ و هر $d > 0$: $n^d = O(r^n)$. یعنی **هر تابع نمایی از هر چندجمله‌ای سریع‌تر رشد می‌کند.**

نسبت جمله‌های متوالیِ $a_n = n^d / r^n$ برابر $\frac{a_{n+1}}{a_n} = \left(1 + \frac{1}{n}\right)^d \cdot \frac{1}{r}$ است که با بزرگ شدن n به $\frac{1}{r} < 1$ میل می‌کند. پس از جایی به بعد دنباله مثل یک دنبالهٔ هندسی با قدرنسبت کمتر از یک کوچک می‌شود و به صفر میل می‌کند؛ در نتیجه کران‌دار است و $n^d = O(r^n)$.

■

جمع‌بندی · نردبان رشد

$$1 \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n$$

علامت $f \prec g$ یعنی $f = o(g)$. هر چه به راست برویم، الگوریتم برای ورودی‌های بزرگ کندتر است.

۱.۱۳ زمان اجرای دستورهای مختلف

برای تحلیل سریع یک شبه‌کد، این قواعد را از ساختارهای کنترلی (شکل اسلاید ۲۸) به خاطر بسپارید:

ساختار	قاعده	زمان
دنباله‌ای از دستورها (بلوک)	زمان‌ها جمع می‌شوند؛ چون تعداد دستورها ثابت است، کندترین دستور غالب است.	$O(f_1 + \cdots + f_k) = O(\max_i f_i)$
شرط `if-else`	بررسی شرط $O(1)$ است و یکی از دو شاخه اجرا می‌شود؛ در بدترین حالت شاخهٔ کندتر.	$O(\max(f(n), g(n)))$
حلقهٔ `while`	اگر حداکثر $g(n)$ بار تکرار شود و هر بار بدنه $O(f(n))$ و آزمون شرط $O(1)$ باشد.	$O(g(n) \cdot f(n))$
حلقهٔ `for`	مقداردهی اولیه، آزمون و به‌روزرسانی هر کدام $O(1)$؛ بدنه $O(f(n))$؛ حداکثر $g(n)$ دور.	$O(g(n) \cdot f(n))$

اشتباه رایج · ضربِ «تعداد دور × بدترین بدنه» فقط کران بالاست

اگر هزینهٔ بدنه در دورهای مختلف تغییر کند، ضرب ممکن است کران شُل بدهد. برای کران دقیق باید هزینهٔ هر دور را جداگانه جمع بزنیم. مثال:

```
i = 1
while i ⩽ n
    for j = 1 to i
        x = x + 1
    i = 2 * i
```

حلقهٔ بیرونی $\log n + 1$ بار اجرا می‌شود و بدنه در بدترین دور n واحد کار دارد؛ قاعدهٔ ضرب $O(n \log n)$ می‌دهد. اما جمع دقیق $1 + 2 + 4 + \cdots + n = 2n - 1$ است، پس زمان واقعی $\Theta(n)$ است!

مثال · دو حلقهٔ تودرتوی وابسته

```
for i = 1 to n
    for j = i to n
        x = x + 1
```

دور بیرونی iام، $n - i + 1$ بار بدنه را اجرا می‌کند. جمع: $\sum_{i=1}^{n}(n - i + 1) = n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2} = \Theta(n^2)$. این‌جا قاعدهٔ ضرب هم $O(n^2)$ می‌داد و این بار کران دقیق است، چون نیمی از دورها دست‌کم $n/2$ کار دارند.

۱.۱۴ الگوریتم‌های چندجمله‌ای و اهمیت آن‌ها

جست‌وجوی جامع (brute force): برای بسیاری از مسائل غیربدیهی یک الگوریتم طبیعی وجود دارد که **همهٔ** جواب‌های ممکن را امتحان می‌کند. معمولاً تعداد جواب‌های ممکن 2^N یا بیشتر است (مثلاً همهٔ زیرمجموعه‌ها)، و این در عمل غیرقابل قبول است.

تعریف · الگوریتم چندجمله‌ای

الگوریتمی **چندجمله‌ای** (poly-time) است اگر ثابت‌های $c > 0$ و $d > 0$ وجود داشته باشند که روی **هر** ورودی به اندازهٔ N، زمان اجرایش حداکثر $c\,N^d$ گام باشد؛ یا به‌طور خلاصه زمانش $O(N^d)$ باشد.

چرا؟ · چرا «چندجمله‌ای» را معادل «کارا» می‌گیریم؟

ویژگی کلیدی این است: اگر اندازهٔ ورودی دو برابر شود، زمان الگوریتم $O(N^d)$ حداکثر در ضریب ثابت 2^d ضرب می‌شود. اما زمان الگوریتم 2^N **به توان دو** می‌رسد ($2^{2N} = (2^N)^2$). یعنی خرید کامپیوتری دو برابر سریع‌تر، به الگوریتم چندجمله‌ای اجازه می‌دهد ورودی‌هایی با اندازهٔ چند برابر را حل کند، ولی به الگوریتم نمایی فقط اجازه می‌دهد یک واحد به N اضافه کند!

تعریف: الگوریتمی را **عملی** (practical) می‌دانیم اگر زمانش چندجمله‌ای باشد. **توجیه:** در عمل جواب می‌دهد! گرچه $6.02 \times 10^{23} \times N^{20}$ از نظر فنی چندجمله‌ای است، در عمل بی‌فایده است؛ اما الگوریتم‌های چندجمله‌ای که مردم طراحی می‌کنند تقریباً همیشه ثابت‌ها و توان‌های کوچکی دارند.

نکته · استثناها

- بعضی الگوریتم‌های چندجمله‌ای ثابت یا توان بزرگی دارند و در عمل بی‌فایده‌اند.
- بعضی الگوریتم‌های نمایی (یا بدتر) به‌طور گسترده استفاده می‌شوند، چون ورودی‌های بدترین حالتشان در عمل نادرند. نمونهٔ معروف، روش سیمپلکس در برنامه‌ریزی خطی است.

چرا مهم است؟ جدول زمان‌ها

جدول زیر (برگرفته از کتاب کلاینبرگ و تاردوس) زمان اجرای الگوریتم‌هایی با مرتبه‌های مختلف را روی پردازنده‌ای با سرعت یک میلیون دستور در ثانیه نشان می‌دهد. «خیلی طولانی» یعنی بیش از 10^{25} سال.

	n	$n\log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n=10$	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	۴ ثانیه
$n=30$	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	۱۸ دقیقه	10^{25} سال
$n=50$	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	۱۱ دقیقه	۳۶ سال	خیلی طولانی
$n=100$	< ۱ ثانیه	< ۱ ثانیه	< ۱ ثانیه	۱ ثانیه	۱۲,۸۹۲ سال	10^{17} سال	خیلی طولانی
$n=1000$	< ۱ ثانیه	< ۱ ثانیه	۱ ثانیه	۱۸ دقیقه	خیلی طولانی	خیلی طولانی	خیلی طولانی
$n=10^4$	< ۱ ثانیه	< ۱ ثانیه	۲ دقیقه	۱۲ روز	خیلی طولانی	خیلی طولانی	خیلی طولانی
$n=10^5$	< ۱ ثانیه	۲ ثانیه	۳ ساعت	۳۲ سال	خیلی طولانی	خیلی طولانی	خیلی طولانی
$n=10^6$	۱ ثانیه	۲۰ ثانیه	۱۲ روز	۳۱,۷۱۰ سال	خیلی طولانی	خیلی طولانی	خیلی طولانی

شهود · از جدول چه بفهمیم؟

ستون‌های چندجمله‌ای «آرام» بدتر می‌شوند: از ۱۰ به یک میلیون، الگوریتم خطی از کمتر از ثانیه به ۱ ثانیه می‌رسد. ستون‌های نمایی «انفجاری» بدتر می‌شوند: الگوریتم 2^n با رفتن از $n=30$ به $n=50$ از ۱۸ دقیقه به ۳۶ سال می‌رسد. به همین دلیل مرز «چندجمله‌ای / نمایی» مهم‌ترین مرز در طراحی الگوریتم است.

1.15 مثال اول: اعداد فیبوناچی

تعریف · دنبالهٔ فیبوناچی

$$F_n = F_{n-1} + F_{n-2}, \qquad F_1 = F_2 = 1.$$

مسئله: برای n داده‌شده، F_n را حساب کنید. مثلاً ده جملهٔ اول: $1, 1, 2, 3, 5, 8, 13, 21, 34, 55$؛ پس $F_{10} = 55$.

با سه الگوریتم برای این مسئلهٔ ساده آشنا می‌شویم. هر سه درست‌اند، اما زمانشان زمین تا آسمان فرق می‌کند.

الگوریتم ۱: بازگشتی ساده

```
FIB1(n)
```

```
1  if n ⩽ 2
2      return 1
3  else
4      return FIB1(n-1) + FIB1(n-2)
```

این الگوریتم ترجمهٔ مستقیم تعریف است. زمانش در رابطهٔ بازگشتی زیر صدق می‌کند (c هزینهٔ ثابت هر فراخوانی است):

$$T(n) = T(n-1) + T(n-2) + c.$$

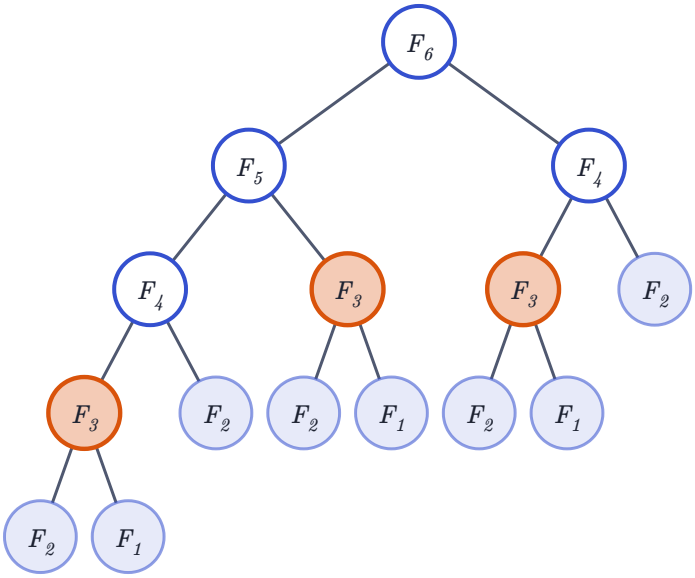
این رابطه تقریباً همان رابطهٔ فیبوناچی است! پس انتظار داریم $T(n)$ تقریباً مثل خود F_n رشد کند. در واقع با استقرا می‌توان نشان داد $T(n) \geq F_n$، و می‌دانیم $F_n \approx \varphi^n/\sqrt{5}$ که در آن $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ **نسبت طلایی** است. پس:

$$T(n) = \Theta(\varphi^n), \qquad \varphi = \frac{1+\sqrt{5}}{2}.$$

چرا؟ φ از کجا آمد؟

حدس می‌زنیم جواب رابطهٔ $F_n = F_{n-1} + F_{n-2}$ به شکل x^n باشد. با جای‌گذاری: $x^n = x^{n-1} + x^{n-2}$؛ با تقسیم بر x^{n-2} به $x^2 = x + 1$ می‌رسیم که ریشهٔ مثبتش $x = \frac{1+\sqrt{5}}{2} = \varphi$ است. جملهٔ ثابت c در رابطهٔ T فقط ضریب را عوض می‌کند، نه پایهٔ نمایی را.

یک استدلال سادهٔ دیگر برای نمایی بودن: چون T صعودی است، $T(n) \geq 2\,T(n-2)$؛ با تکرار این نابرابری $T(n) \geq 2^{n/2} \approx 1.41^n$ به دست می‌آید. از طرف دیگر $T(n) \leq 2\,T(n-1)$ نتیجه می‌دهد $T(n) = O(2^n)$. جواب دقیق φ^n بین این دو است.



شکل ۱-۵ درخت فراخوانی‌های $\mathrm{FIB1}(6)$: پانزده فراخوانی برای محاسبهٔ عدد ۸!
زیردرخت F_3 (نارنجی) سه بار از نو حساب می‌شود. علت کندی همین دوباره‌کاری است.

الگوریتم ۲: حلقهٔ ساده

```
FIB2(n)
```

```
1  a = b = 1
2  for i = 3 to n
3      b = a + b
4      a = b - a
5  return b
```

ایده: هر جمله را فقط یک بار حساب کن و دو جملهٔ آخر را نگه دار. **ناوردا:** پس از دور iام، $b = F_i$ و $a = F_{i-1}$. اگر پیش از دور i، $a = F_{i-2}$ و $b = F_{i-1}$ باشد، خط سوم b را $F_{i-2} + F_{i-1} = F_i$ می‌کند و خط چهارم a را $F_i - F_{i-2} = F_{i-1}$. (این ترفند تفریق، نیاز به متغیر کمکی را از بین می‌برد.) حلقه $n-2$ بار و هر بار $O(1)$ کار انجام می‌دهد، پس:

$$T(n) = O(n).$$

نکته · پیوند با برنامه‌ریزی پویا

تفاوت الگوریتم ۱ و ۲ دقیقاً ایدهٔ **برنامه‌ریزی پویا** است: الگوریتم ۱ زیرمسئله‌های تکراری را بارها حل می‌کند، الگوریتم ۲ هر زیرمسئله را یک بار حل و نتیجه را نگه می‌دارد.

الگوریتم ۳: توان ماتریس

از این مشاهده استفاده می‌کنیم که گام فیبوناچی یک ضرب ماتریسی است:

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} \quad \Longrightarrow \quad \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} \begin{pmatrix} F_2 \\ F_1 \end{pmatrix}.$$

(سطر اول: $F_{n+1} = F_n + F_{n-1}$، سطر دوم: $F_n = F_n$. با تکرار $n-1$ بار از (F_2, F_1) شروع می‌کنیم.) پس کافی است توان $n-1$ام ماتریس $M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ را حساب کنیم. ترفند کلیدی **به توان رساندن با مجذورسازی پیاپی** است: برای محاسبهٔ a^n فقط $a, a^2, a^4, a^8, \ldots$ را حساب می‌کنیم (هر کدام مربع قبلی) و آن‌هایی را که در نمایش دودویی n لازم‌اند در هم ضرب می‌کنیم.

مثال · محاسبهٔ a^{13}

$13 = (1101)_2 = 8 + 4 + 1$، پس $a^{13} = a^8 \cdot a^4 \cdot a^1$. سه مجذورسازی (a^2, a^4, a^8) و دو ضرب اضافه کافی است: ۵ ضرب به‌جای ۱۲ ضرب. در حالت کلی حداکثر $2\log_2 n$ ضرب لازم است.

POWER(M, k) ▷ M^k with O(log k) multiplications

```
if k = 0
    return I                        // identity matrix
H = POWER(M, floor(k/2))
H = H * H
if k is odd
    H = H * M
return H
```

هر ضرب ماتریس 2×2 فقط ۸ ضرب و ۴ جمع است، یعنی $O(1)$. تعداد فراخوانی‌ها $\log k$ است، پس:

$$T(n) = O(\log n).$$

مقایسهٔ زمان‌ها

n	۱۰	۲۰	۳۰	۴۰	۵۰	۱۰۰
الگوریتم ۱	۴۴µs	۵۳۰۰µs	۰٫۵ ثانیه	۶۴ ثانیه	۲٫۲ ساعت	**۷ میلیون سال**
الگوریتم ۲	۴µs	۷µs	۱۰µs	۱۴µs	۱۸µs	**۴۵µs**
الگوریتم ۳	۱۱µs	۱۲µs	۱۳µs	۱۵µs	۱۷µs	**۲۱µs**

نکته · «۷ میلیون سال» از کجا آمده؟

در جدول، با هر ۱۰ واحد افزایش n، زمان الگوریتم ۱ حدود $\varphi^{10} \approx 123$ برابر شده است (مثلاً از ۶۴ ثانیه به ۲٫۲ ساعت). اگر همین روند را از $n = 50$ تا $n = 100$ ادامه دهیم (یعنی ضرب در $\varphi^{50} \approx 2.8 \times 10^{10}$)، به حدود ۷ میلیون سال می‌رسیم. تخمین خام‌تر $\varphi^{100}/10^8$ ثانیه (با فرض 10^8 عمل در ثانیه و یک عمل برای هر فراخوانی) حدود ۲۵۰ هزار سال می‌دهد. عدد دقیق به هزینهٔ هر فراخوانی بستگی دارد، اما نتیجه یکی است: **الگوریتم ۱ برای $n = 100$ هرگز تمام نمی‌شود.**

دقت کنید که برای nهای کوچک (مثلاً ۱۰) الگوریتم ۳ از الگوریتم ۲ کندتر است، چون ضرب ماتریس ثابت بزرگ‌تری دارد. این همان پدیدهٔ «تقاطع» است که در مثال $30n + 8$ دیدیم.

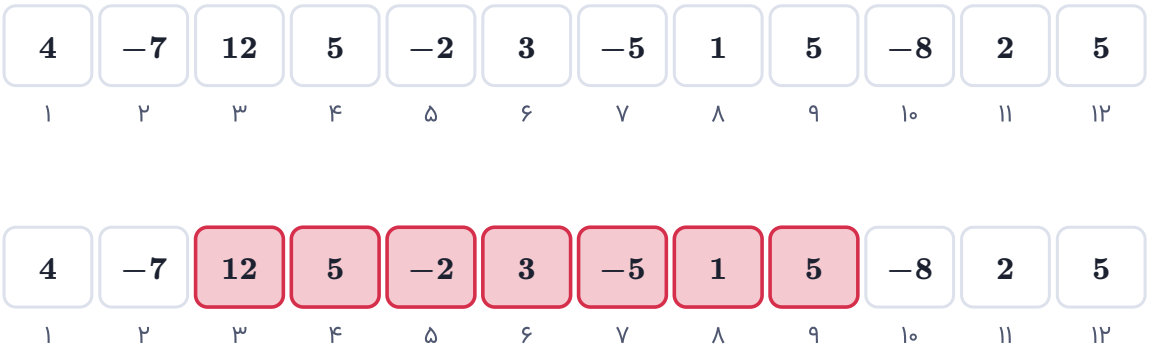
تکمیلی (فراتر از اسلاید) · فرض هزینهٔ واحد این‌جا کجا می‌لنگد؟

F_n تقریباً $0.694n$ بیت دارد (چون $\log_2 \varphi \approx 0.694$). برای nهای خیلی بزرگ، جمع و ضرب چنین اعدادی دیگر $O(1)$ نیست. با شمارش دقیق عملیات بیتی، الگوریتم ۲ در واقع $O(n^2)$ عمل بیتی انجام می‌دهد و الگوریتم ۳ به سرعت الگوریتم ضرب اعداد بزرگ (فصل ۲) وابسته می‌شود. اما ترتیب کلی «۱ خیلی کند، ۲ و ۳ سریع» تغییری نمی‌کند.

۱.۱۶ مثال دوم: زیرآرایه با بیشترین مجموع

تعریف · مسئلهٔ زیرآرایهٔ بیشینه

آرایهٔ یک‌بعدی $A[1..n]$ از اعداد (مثبت و منفی) داده شده است. زیرآرایهٔ **پیوسته‌ای** $A[i..j]$ را پیدا کنید که مجموع عناصرش بیشینه باشد. فرض می‌کنیم مجموع زیرآرایهٔ تهی صفر است؛ پس جواب هرگز منفی نیست.



شکل ۱-۶ جواب برای این آرایه زیرآرایهٔ $A[3..9]$ است: $12 + 5 - 2 + 3 - 5 + 1 + 5 = 19$. دقت کنید جواب عناصر منفی (-2 و -5) را هم در بر دارد، چون آن‌ها دو بخش پرسود را به هم وصل می‌کنند.

چرا؟ · چرا فرض زیرآرایهٔ تهی؟

اگر همهٔ اعداد منفی باشند، هر زیرآرایهٔ ناتهی مجموع منفی دارد. با این فرض، جواب در این حالت صفر (برداشتن هیچ چیز) است و دیگر لازم نیست حالت خاص تعریف کنیم. در شبه‌کدها این فرض با مقدار اولیهٔ `sol = 0` پیاده شده است.

الگوریتم ۱: جست‌وجوی جامع $O(n^3)$

MAX-SUB-1(A)

```
1  sol = 0
2  for i = 1 to n                  // start
3      for j = i to n              // end
4          sum = 0
5          for k = i to j
6              sum = sum + A[k]
7          if sum > sol then
8              sol = sum
9  return sol
```

همهٔ جفت‌های (i, j) را امتحان می‌کنیم و برای هر کدام مجموع $A[i..j]$ را از نو حساب می‌کنیم. تعداد اجرای خط جمع:

$$\sum_{i=1}^{n}\sum_{j=i}^{n}(j-i+1)=\sum_{L=1}^{n}L\,(n-L+1)=\frac{n(n+1)(n+2)}{6}=\Theta(n^3).$$

(در جمع وسط، $L=j-i+1$ طول زیرآرایه است و دقیقاً $n-L+1$ زیرآرایه به طول L وجود دارد.)

الگوریتم ۲: مجموع پیشوندی $O(n^2)$

شهود · کجا دوباره‌کاری می‌کنیم؟

الگوریتم ۱ مجموع $A[i..j]$ را از صفر حساب می‌کند، در حالی که مجموع $A[i..j-1]$ را همین الان داشت! اگر مجموع‌ها را یک بار پیش‌محاسبه کنیم، هر مجموع بازه‌ای در $O(1)$ به دست می‌آید.

مشاهده: فرض کنید $S[i]=A[1]+\cdots+A[i]$ (مجموع پیشوندی) و $S[0]=0$. آنگاه:

$$A[i]+A[i+1]+\cdots+A[j]=S[j]-S[i-1].$$

چون $S[j]$ مجموع تا j است و $S[i-1]$ همان بخش اضافهٔ اول را دارد که با تفریق حذف می‌شود.

PRE-PROCESSING ▷ O(n)

```
1  S[0] = 0
2  for i = 1 to n
3      S[i] = S[i-1] + A[i]
```

MAX-SUB-2(A) ▷ O(n^2)

```
1  sol = 0
2  for i = 1 to n
3      for j = i to n
4          if S[j] - S[i-1] > sol then
5              sol = S[j] - S[i-1]
6  return sol
```

پیش‌پردازش $O(n)$ و دو حلقهٔ تودرتو $\frac{n(n+1)}{2}$ بار بدنهٔ $O(1)$ را اجرا می‌کنند؛ پس $T(n) = O(n^2)$.

نکته · ادامهٔ داستان

در فصل ۲ همین مسئله را با تقسیم و حل در $O(n \log n)$ حل می‌کنیم. و در کادر زیر، الگوریتمی خطی می‌بینیم.

تکمیلی (فراتر از اسلاید) · الگوریتم کادان: $O(n)$

فرض کنید cur بیشترین مجموعِ زیرآرایه‌ای باشد که **دقیقاً** در خانهٔ فعلی تمام می‌شود. با آمدن $A[i]$، یا زیرآرایهٔ قبلی را ادامه می‌دهیم ($cur + A[i]$) یا اگر منفی شد، از نو شروع می‌کنیم (صفر). جواب بیشترین مقداری است که cur به خود می‌گیرد.

KADANE(A)

```
1  best = 0
2  cur = 0
3  for i = 1 to n
4      cur = max(0, cur + A[i])
5      best = max(best, cur)
6  return best
```

روی آرایهٔ مثال، مقادیر cur به ترتیب $4, 0, 12, 17, 15, 18, 13, 14, \mathbf{19}, 11, 13, 18$ است و جواب ۱۹ به دست می‌آید.

۱.۱۷ مروری بر زمان‌های اجرای رایج

برای هر مرتبهٔ زمانی رایج، یک الگوریتم نمونه می‌بینیم تا با «شکل» هر کدام آشنا شوید.

زمان خطی $O(n)$

زمان متناسب با اندازهٔ ورودی است. معمولاً یعنی ورودی را یک (یا چند) بار پیمایش کرده‌ایم و برای هر عنصر کار ثابتی انجام داده‌ایم.

محاسبهٔ بیشینه: بیشینهٔ n عدد $a_1, \ldots, a_n$.

```
MAX(a1..an)
```

```
1 max = a1
2 for i = 2 to n
3     if a[i] > max
4         max = a[i]
```

ادغام: دو لیست مرتب $A = a_1, \ldots, a_n$ و $B = b_1, \ldots, b_n$ را به یک لیست مرتب تبدیل کنید.

```
MERGE(A, B)
```

```
1 i = 1, j = 1
2 while both lists are nonempty
3     if a[i] ⩽ b[j]
4         append a[i] to output; i = i + 1
5     else
6         append b[j] to output; j = j + 1
7 append remainder of nonempty list to output
```

قضیه · زمان ادغام

ادغام دو لیست به اندازهٔ n زمان $O(n)$ می‌برد.

پس از هر مقایسه، دقیقاً یک عنصر به خروجی اضافه می‌شود. خروجی در نهایت $2n$ عنصر دارد، پس حداکثر $2n$ مقایسه انجام می‌شود و هر کدام $O(1)$ کار دارد.

■

زمان $O(n \log n)$

این زمان (که به آن «خطی‌لگاریتمی» هم می‌گویند) به‌طور طبیعی در الگوریتم‌های تقسیم و حل ظاهر می‌شود: $\log n$ سطح تقسیم و در هر سطح $O(n)$ کار.

- **مرتب‌سازی:** مرتب‌سازی ادغامی و هرمی (heapsort) با $O(n \log n)$ مقایسه مرتب می‌کنند.
- **بزرگ‌ترین بازهٔ خالی:** n برچسب زمانی $x_1, \ldots, x_n$ داریم که در آن‌ها نسخه‌هایی از یک فایل به سرور رسیده‌اند. طولانی‌ترین بازه‌ای که هیچ نسخه‌ای نرسیده کدام است؟ **راه‌حل** $O(n \log n)$**:** برچسب‌ها را مرتب کن ($O(n \log n)$)، سپس لیست مرتب را یک بار پیمایش کن و بیشترین فاصلهٔ دو برچسب متوالی را بیاب ($O(n)$).

زمان درجه دو $O(n^2)$

شکل معمول: **همهٔ جفت‌ها** را بررسی کن.

نزدیک‌ترین جفت نقاط: n نقطه $(x_1, y_1), \ldots, (x_n, y_n)$ در صفحه داده شده؛ نزدیک‌ترین دو نقطه را پیدا کنید. راه‌حل $O(n^2)$: همهٔ جفت‌ها را امتحان کن.

CLOSEST-PAIR-BRUTE

```
min = (x1 - x2)^2 + (y1 - y2)^2
for i = 1 to n
    for j = i+1 to n
        d = (x[i] - x[j])^2 + (y[i] - y[j])^2    // no square root needed
        if d < min
            min = d
```

چرا؟ · چرا جذر نگرفتیم؟

تابع جذر اکیداً صعودی است: $d_1 < d_2 \iff \sqrt{d_1} < \sqrt{d_2}$. پس جفتی که **مربع** فاصله‌اش کمینه است، همان جفتی است که فاصله‌اش کمینه است. با مقایسهٔ مربع‌ها از محاسبهٔ گران و غیردقیقِ جذر صرف‌نظر می‌کنیم.

نکته: به نظر می‌رسد $\Omega(n^2)$ اجتناب‌ناپذیر باشد، اما این توهمی بیش نیست: الگوریتمی تقسیم و حل برای این مسئله در $O(n \log n)$ وجود دارد.

زمان مکعبی $O(n^3)$

شکل معمول: **همهٔ سه‌تایی‌ها** را بررسی کن.

مجزا بودن مجموعه‌ها: n مجموعهٔ $S_1, \ldots, S_n$ داده شده که هر کدام زیرمجموعهٔ $\{1, 2, \ldots, n\}$ است. آیا جفتی از آن‌ها وجود دارد که اشتراکشان تهی باشد؟

SET-DISJOINTNESS

```
foreach set S_i
    foreach other set S_j
        foreach element p of S_i
            determine whether p also belongs to S_j
        if no element of S_i belongs to S_j
            report that S_i and S_j are disjoint
```

n^2 جفت مجموعه، هر کدام حداکثر n عنصر؛ اگر هر مجموعه با یک آرایهٔ بولی به طول n نمایش داده شود، «آیا $p \in S_j$؟» در $O(1)$ پاسخ داده می‌شود. پس زمان $O(n^3)$ است.

زمان چندجمله‌ای $O(n^k)$

مجموعهٔ مستقل به اندازهٔ k: گرافی داده شده؛ آیا k رأس وجود دارد که هیچ دوتایی‌شان با یال به هم وصل نباشند؟ (k یک ثابت است.)

INDEPENDENT-SET-k

```
foreach subset S of k nodes
    check whether S is an independent set
    if S is an independent set
        report S is an independent set
```

- بررسی مستقل بودن S: همهٔ $\binom{k}{2}$ جفت را چک کن، یعنی $O(k^2)$.
- تعداد زیرمجموعه‌های kعضوی: $\binom{n}{k} = \dfrac{n(n-1)(n-2)\cdots(n-k+1)}{k(k-1)(k-2)\cdots 1} \le \dfrac{n^k}{k!}$.
- جمع کل: $O(k^2\, n^k/k!) = O(n^k)$، چون k ثابت است.

نکته · چندجمله‌ای ولی غیرعملی

برای $k = 17$ این الگوریتم از نظر فنی چندجمله‌ای است ($O(n^{17})$)، اما برای گرافی با فقط ۱۰۰۰ رأس به حدود $10^{51}/17! \approx 3 \times 10^{36}$ زیرمجموعه نیاز دارد. این همان استثنایی است که در بخش الگوریتم‌های چندجمله‌ای گفتیم.

زمان نمایی

مجموعهٔ مستقل بیشینه: بزرگ‌ترین مجموعهٔ مستقل گراف را پیدا کنید. اکنون k ثابت نیست؛ باید همهٔ زیرمجموعه‌ها را امتحان کنیم.

```
MAX-INDEPENDENT-SET
1  S* = empty set
2  foreach subset S of nodes
3      check whether S is an independent set
4      if S is largest independent set seen so far
5          update S* = S
```

2^n زیرمجموعه و برای هر کدام بررسی $O(n^2)$: کل زمان $O(n^2\,2^n)$. (تا امروز هیچ الگوریتم چندجمله‌ای برای این مسئله شناخته نشده است.)

تکمیلی (فراتر از اسلاید) · زمان زیرخطی $O(\log n)$

الگوریتم‌هایی هم هستند که حتی کل ورودی را نمی‌خوانند. نمونهٔ کلاسیک **جست‌وجوی دودویی** در آرایهٔ مرتب است: با هر مقایسه نیمی از آرایه کنار گذاشته می‌شود، پس پس از $\log_2 n$ مقایسه یک خانه باقی می‌ماند. هر جا دیدید اندازهٔ مسئله در هر گام نصف (یا کسری ثابت) می‌شود، به $\log n$ فکر کنید.

جمع‌بندی · فصل ۱ در یک نگاه

- زمان اجرا = تعداد عملیات پایه، به‌عنوان تابعی از اندازهٔ ورودی؛ معمولاً در بدترین حالت.
- O کران بالا، Ω کران پایین، Θ کران محکم؛ همه با «ضریب ثابت» و «از جایی به بعد».
- چندجمله‌ای‌ها با بالاترین توان، لگاریتم‌ها بی‌پایه، و نمایی‌ها همیشه بالاتر از چندجمله‌ای‌ها.
- ایدهٔ بهتر بر سخت‌افزار بهتر غلبه می‌کند: فیبوناچی از $\Theta(\varphi^n)$ به $O(\log n)$، زیرآرایهٔ بیشینه از $O(n^3)$ به $O(n)$.

۱.۱۸ خودآزمایی فصل ۱

خودآزمایی

۱. آیا $2^{n+1} = O(2^n)$؟ آیا $2^{2n} = O(2^n)$؟

پاسخ: اولی بله: $2^{n+1} = 2 \cdot 2^n$، پس $c = 2$ کار می‌کند. دومی نه: $2^{2n}/2^n = 2^n$ به بی‌نهایت میل می‌کند و هیچ ثابت cای نمی‌تواند آن را کران‌دار کند.

۲. این توابع را به ترتیب رشد مرتب کنید: $n \log n$، $n^{1.5}$، $\log^2 n$، $2^{\log_2 n}$، $\sqrt{n}$، $n!$، 2^n، n^2.

پاسخ: $\log^2 n \prec \sqrt{n} \prec 2^{\log_2 n} = n \prec n \log n \prec n^{1.5} \prec n^2 \prec 2^n \prec n!$. دقت کنید $2^{\log_2 n}$ همان n است.

3. زمان این قطعه را با Θ بیان کنید: `for i = 1 to n: j = i; while j > 1: j = j / 2`

پاسخ: حلقهٔ داخلی برای هر i حدود $\log_2 i$ بار اجرا می‌شود. جمع: $\sum_{i=1}^{n} \log i = \log(n!) = \Theta(n \log n)$ (چون نیمهٔ بالایی جمله‌ها هر کدام دست‌کم $\log(n/2)$ هستند).

4. مرتب‌سازی درجی روی آرایهٔ $1\ 2\ 3\ 4\ 5\ 6$ چند بار خط `A[i+1] = A[i]` را اجرا می‌کند؟

پاسخ: در بدترین حالت برای هر j، $j-1$ جابه‌جایی داریم: $1+2+3+4+5 = 15$.

5. نشان دهید برای توابع نامنفی f, g: $\max(f,g) = \Theta(f+g)$.

پاسخ: $\max(f,g) \le f+g$ (پس $c_2 = 1$) و $f+g \le 2\max(f,g)$، یعنی $\max(f,g) \ge \frac{1}{2}(f+g)$ (پس $c_1 = \frac{1}{2}$).

6. چرا جملهٔ «زمان این الگوریتم دست‌کم $O(n^2)$ است» بی‌معناست؟

پاسخ: O کران بالاست. «دست‌کم یک کران بالا» هیچ قیدی ایجاد نمی‌کند. برای کران پایین باید گفت «$\Omega(n^2)$ است».

فصل ۲

تقسیم و حل

مسئلهٔ بزرگ را بشکن، تکه‌ها را بازگشتی حل کن و جواب‌ها را به هم بچسبان. این ایدهٔ ساده، مرتب‌سازی را از n^2 به $n \log n$ و ضرب اعداد را از n^2 به $n^{1.585}$ می‌رساند.

مرتب‌سازی ادغامی · رابطه‌های بازگشتی · قضیهٔ اصلی · زیرآرایهٔ بیشینه · ضرب اعداد صحیح · حدس و استقرا

۲.۱ ایدهٔ تقسیم و حل

تعریف · تقسیم و حل

الگوریتم تقسیم و حل (divide and conquer) سه گام دارد:

۱. **تقسیم (divide):** مسئله را به چند زیرمسئلهٔ کوچک‌تر از همان نوع بشکن.

۲. **حل (conquer):** هر زیرمسئله را به‌صورت **بازگشتی** حل کن. اگر زیرمسئله به‌اندازهٔ کافی کوچک بود، مستقیم حلش کن (حالت پایه).

۳. **ترکیب (combine):** جواب زیرمسئله‌ها را ترکیب کن تا جواب مسئلهٔ اصلی به دست آید.

رایج‌ترین شکل: مسئلهٔ اندازهٔ n را به دو بخش مساوی به اندازهٔ $n/2$ بشکن، هر دو را بازگشتی حل کن و دو جواب را در **زمان خطی** ترکیب کن. **نتیجه:** جایی که روش جامع n^2 زمان می‌برد، تقسیم و حل اغلب به $n \log n$ می‌رسد.

چرا؟ · چرا شکستن مسئله اصلاً سود دارد؟

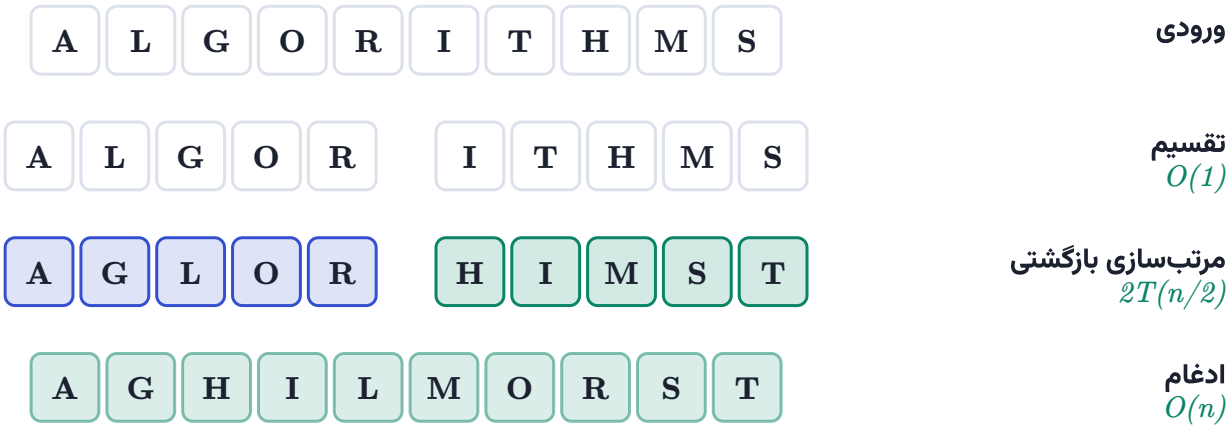
فرض کنید روش مستقیم n^2 کار می‌کند. اگر مسئله را به دو نیمه بشکنیم و هر نیمه را با **همان** روش مستقیم حل کنیم، هزینه $2 \cdot (n/2)^2 = n^2/2$ می‌شود: همین الان نصف شد! حالا اگر ترکیب ارزان (خطی) باشد و این کار را بازگشتی تکرار کنیم، صرفه‌جویی در هر سطح روی هم انباشته می‌شود. راز کار در این است که هزینهٔ روش مستقیم **فوق‌خطی** است، پس مسئله‌های کوچک «به نسبت» ارزان‌ترند.

۲.۲ مرتب‌سازی ادغامی

مرتب‌سازی: n عنصر داده شده؛ آن‌ها را به ترتیب صعودی بچینید. کاربردهای مرتب‌سازی بی‌شمارند: مرتب کردن فهرست نام‌ها، سازمان‌دهی کتابخانهٔ موسیقی، یافتن میانه، یافتن نزدیک‌ترین جفت، جست‌وجوی دودویی در پایگاه داده، یافتن تکراری‌ها در فهرست پستی و ... . بسیاری از مسائل وقتی داده مرتب باشد ساده می‌شوند.

الگوریتم · مرتب‌سازی ادغامی (Mergesort)

1. آرایه را به دو نیمه تقسیم کن.
2. هر نیمه را بازگشتی مرتب کن.
3. دو نیمهٔ مرتب را **ادغام** کن تا کل آرایه مرتب شود.



شکل ۲-۱ مرتب‌سازی ادغامی روی حروف ALGORITHMS. تقسیم فقط پیدا کردن وسط است ($O(1)$)، دو مرتب‌سازی بازگشتی $2T(n/2)$ و ادغام $O(n)$ زمان می‌برند.

ادغام: قلب الگوریتم

ادغام: دو لیست از پیش مرتب را به یک لیست مرتب تبدیل کن. چطور کارآمد ادغام کنیم؟ با تعداد **خطی** مقایسه و یک **آرایهٔ موقت**: دو اشاره‌گر روی ابتدای دو لیست می‌گذاریم. در هر گام، کوچک‌ترین عنصرِ باقی‌مانده حتماً یکی از دو عنصرِ زیر اشاره‌گرهاست (چون هر لیست مرتب است و کوچک‌ترین عنصرش اولش است). آن دو را مقایسه می‌کنیم، کوچک‌تر را به خروجی می‌فرستیم و اشاره‌گرش را جلو می‌بریم.

مثال · ادغام AGLOR و HIMST

step	L (i)	R (j)	compare	output so far
1	**A**GLOR	**H**IMST	A ≤ H	A
2	A**G**LOR	**H**IMST	G ≤ H	A G
3	AG**L**OR	**H**IMST	L > H	A G H
4	AG**L**OR	H**I**MST	L > I	A G H I
5	AG**L**OR	HI**M**ST	L ≤ M	A G H I L
6	AGL**O**R	HI**M**ST	O > M	A G H I L M
7	AGL**O**R	HIM**S**T	O ≤ S	A G H I L M O
8	AGLO**R**	HIM**S**T	R ≤ S	A G H I L M O R
9	(empty)	HIM**ST**	—	A G H I L M O R S T

هشت مقایسه برای ده عنصر. وقتی یک لیست تمام شد، بقیهٔ لیست دیگر بدون مقایسه به انتها چسبانده می‌شود.

```
MERGESORT(A[1..n])
```

```
1  if n = 1 then return A
2  m = floor(n/2)
3  L = MERGESORT(A[1..m])
4  R = MERGESORT(A[m+1..n])
5  i = 1, j = 1
6  for k = 1 to n
7      if i ⩽ m and j ⩽ n-m then          // both lists nonempty
8          if L[i] ⩽ R[j] then B[k] = L[i], i = i+1
9          else                B[k] = R[j], j = j+1
10     else if i > m then B[k] = R[j], j = j+1    // L is exhausted
11     else               B[k] = L[i], i = i+1    // R is exhausted
12 return B
```

نکته · دو جزئیات کوچک

- نیمهٔ راست $n - m = \lceil n/2 \rceil$ عنصر دارد، نه $\lfloor n/2 \rfloor$. در شبه‌کد اسلاید شرط $j \le n/2$ آمده که فقط وقتی n زوج است درست است؛ این‌جا $j \le n - m$ نوشته‌ایم تا برای n فرد هم درست باشد.
- استفاده از $\le$ (به‌جای $<$) در مقایسهٔ $L[i] \le R[j]$ باعث می‌شود از دو عنصر مساوی، آنکه در نیمهٔ چپ بوده زودتر بیاید. به این ویژگی **پایداری** (stability) می‌گویند.

۲.۳ رابطهٔ بازگشتی و درخت بازگشت

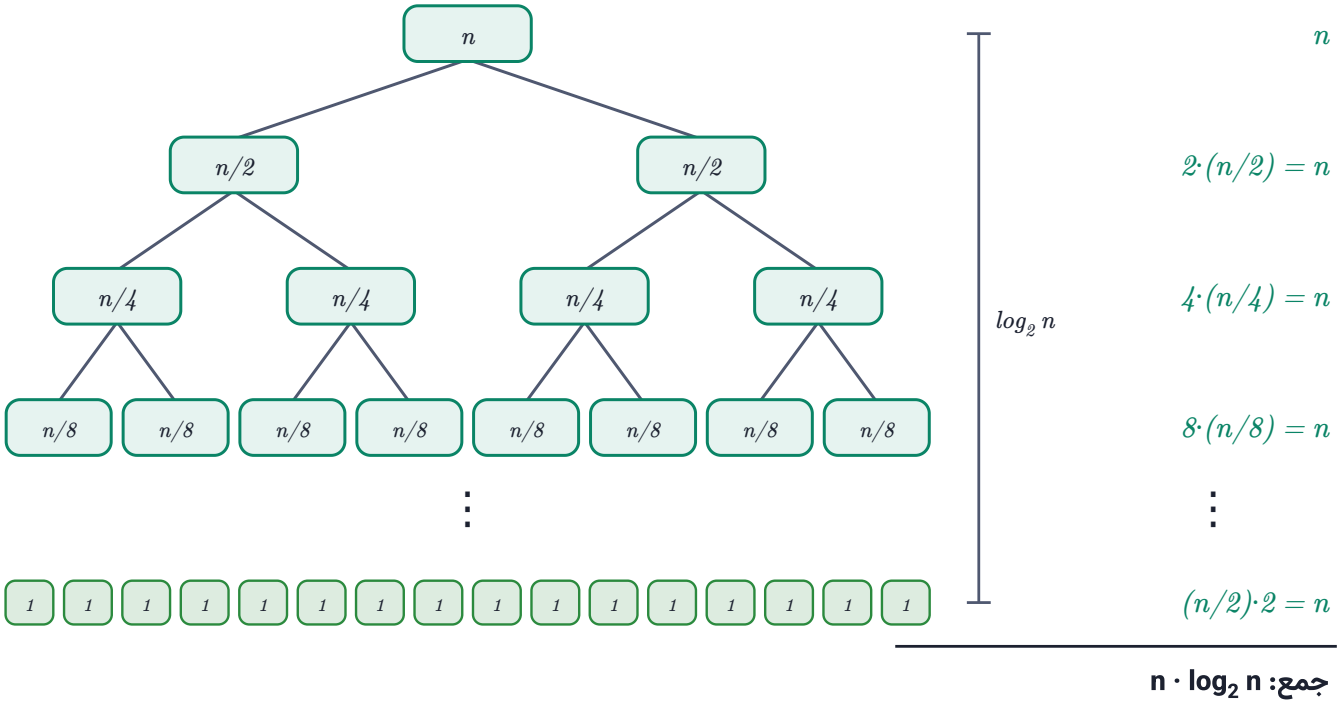
تعریف · رابطهٔ بازگشتی مرتب‌سازی ادغامی

$T(n)$ = تعداد مقایسه‌ها برای مرتب‌سازی ادغامی روی ورودی به اندازهٔ n.

$$T(n) = \begin{cases} 0 & n \le 1 \\ \underbrace{T(\lceil n/2 \rceil)}_{\text{نیمهٔ چپ}} + \underbrace{T(\lfloor n/2 \rfloor)}_{\text{نیمهٔ راست}} + \underbrace{n}_{\text{ادغام}} & \text{در غیر این صورت} \end{cases}$$

جواب: $T(n) = O(n \log_2 n)$.

این رابطه را با فرض اینکه n توانی از ۲ است حل می‌کنیم (تا کف و سقف حذف شوند): $T(n) = 2T(n/2) + n$ و $T(1) = 0$.
ابزار اول ما **درخت بازگشت** است: هر گره یک فراخوانی است و داخلش هزینه‌ای را می‌نویسیم که **خود آن فراخوانی** (بدون فرزندانش) انجام می‌دهد.



شکل ۲-۲ درخت بازگشت $T(n) = 2T(n/2) + n$. در سطح kام 2^k گره داریم که هر کدام $n/2^k$ کار می‌کنند؛ پس هر سطح دقیقاً n کار دارد.

اثبات · اثبات با درخت بازگشت

هزینهٔ هر سطح: سطح k (ریشه سطح ۰ است) 2^k زیرمسئله به اندازهٔ $n/2^k$ دارد و ادغام هر کدام $n/2^k$ کار می‌برد: $2^k \cdot \frac{n}{2^k} = n$.

تعداد سطح‌ها: اندازه در هر سطح نصف می‌شود و از n به ۱ می‌رسد؛ پس $\log_2 n$ سطح هزینه‌دار داریم (برگ‌ها، یعنی زیرمسئله‌های یک‌عنصری، هزینه‌ای ندارند).

جمع: $T(n) = \underbrace{n + n + \cdots + n}_{\log_2 n \text{ بار}} = n \log_2 n$.

∎

تکمیلی (فراتر از اسلاید) · اثبات دوم: تلسکوپی

طرفین $T(n) = 2T(n/2) + n$ را بر n تقسیم کنید:

$$\frac{T(n)}{n} = \frac{T(n/2)}{n/2} + 1 = \frac{T(n/4)}{n/4} + 1 + 1 = \cdots = \frac{T(1)}{1} + \underbrace{1 + \cdots + 1}_{\log_2 n} = \log_2 n.$$

پس $T(n) = n \log_2 n$. این ترفند (تقسیم بر n تا رابطه به شکل «جمع ساده» درآید) برای رابطه‌هایی با ضریب و تقسیم برابر خیلی کارآمد است.

نکته · اگر n توانی از ۲ نباشد؟

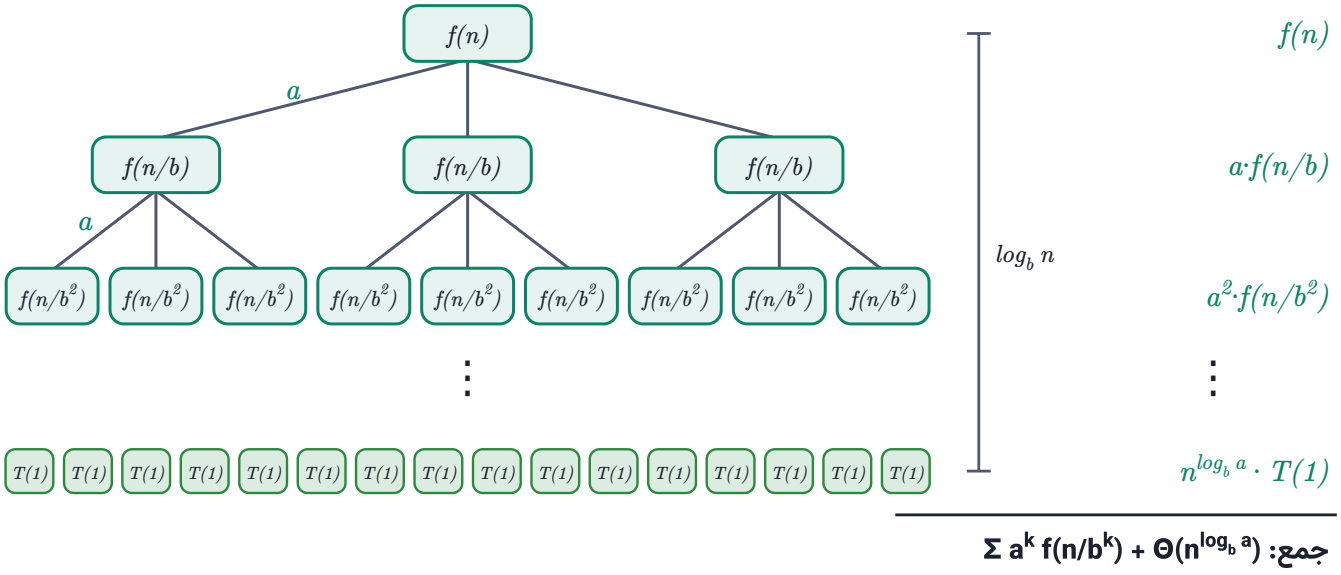
T صعودی است (آرایهٔ بزرگ‌تر مقایسهٔ کمتری نمی‌خواهد). اگر $2^{k-1} < n \le 2^k$، آنگاه $T(n) \le T(2^k) = k\,2^k \le 2n(\log_2 n + 1) = O(n \log n)$. یعنی گرد کردن به توان بعدی ۲ فقط ضریب ثابت را عوض می‌کند. به همین دلیل در تحلیل مجانبی معمولاً کف و سقف را نادیده می‌گیریم.

۲.۴ درخت بازگشت در حالت کلی

حالا رابطهٔ کلی تقسیم و حل را در نظر بگیرید:

$$T(n) = \begin{cases} 0 & n \le 1 \\ \underbrace{a\,T(n/b)}_{\text{زیرمسئله‌ها}} + \underbrace{f(n)}_{\text{تقسیم و ترکیب}} & \text{در غیر این صورت} \end{cases}$$

می‌توانید آن را تابع بازگشتی‌ای تصور کنید که خودش را a بار، هر بار روی ورودی به اندازهٔ n/b، صدا می‌زند و خروجی‌ها را در زمان $f(n)$ ترکیب می‌کند.



شکل ۲-۳ درخت بازگشت $T(n) = aT(n/b) + f(n)$. هر گره a فرزند دارد و اندازه در هر سطح بر b تقسیم می‌شود.

- **سطح kام** a^k گره دارد، هر کدام با ورودی n/b^k و هزینهٔ $f(n/b^k)$. هزینهٔ کل سطح: $a^k f(n/b^k)$.
- **عمق درخت** $\log_b n$ است، چون پس از $\log_b n$ بار تقسیم بر b به اندازهٔ ۱ می‌رسیم.
- **تعداد برگ‌ها** $a^{\log_b n} = n^{\log_b a}$ است.

چرا؟ · چرا $a^{\log_b n} = n^{\log_b a}$؟

از تعریف لگاریتم، $a = b^{\log_b a}$. پس

$$a^{\log_b n} = \left(b^{\log_b a}\right)^{\log_b n} = \left(b^{\log_b n}\right)^{\log_b a} = n^{\log_b a}.$$

(دو توان را جابه‌جا کردیم، چون $(x^p)^q = (x^q)^p$.) این اتحاد مهم است: تعداد برگ‌ها را به شکل یک **توان از** n درمی‌آورد تا بتوانیم با $f(n)$ مقایسه‌اش کنیم.

با جمع زدن سطح‌ها:

$$T(n) = \sum_{k=0}^{\log_b n - 1} a^k f\left(\frac{n}{b^k}\right) + \Theta\left(n^{\log_b a}\right).$$

۲.۵ قضیهٔ اصلی

جمع بالا را می‌شود یک بار برای همیشه حساب کرد. نتیجه، **قضیهٔ اصلی** (master theorem) است. ایدهٔ آن یک «کشمکش» است: کشمکش میان **تعداد برگ‌ها** ($n^{\log_b a}$) و **هزینهٔ ریشه** ($f(n)$). هر کدام بزرگ‌تر باشد، برنده است.

تعریف · به‌طور چندجمله‌ای بزرگ‌تر

$g(n)$ به‌طور چندجمله‌ای از $h(n)$ بزرگ‌تر است، اگر و فقط اگر برای عددی مثل $x > 0$: $\frac{g(n)}{h(n)} = \Omega(n^x)$.

مثلاً n^2 به‌طور چندجمله‌ای از $n^{1.5}$ بزرگ‌تر است (نسبت $n^{0.5}$)، ولی $n \log n$ به‌طور چندجمله‌ای از n بزرگ‌تر **نیست** (نسبت $\log n$ از هر n^x کندتر رشد می‌کند).

قضیه · قضیهٔ اصلی

برای $T(n) = aT(n/b) + f(n)$ با $a \geq 1$ و $b > 1$:

1. اگر $f(n) = \Theta(n^{\log_b a})$، آنگاه $T(n) = \Theta(n^{\log_b a} \log n)$. **مساوی**
2. اگر $f(n)$ به‌طور چندجمله‌ای از $n^{\log_b a}$ بزرگ‌تر باشد، آنگاه $T(n) = \Theta(f(n))$. **ریشه برنده**
3. اگر $n^{\log_b a}$ به‌طور چندجمله‌ای از $f(n)$ بزرگ‌تر باشد، آنگاه $T(n) = \Theta(n^{\log_b a})$. **برگ‌ها برنده**



شکل ۲-۴ هر نوار، هزینهٔ یک سطح درخت بازگشت است (ریشه بالا). وقتی هزینه‌ها یک دنبالهٔ هندسی‌اند، جمعشان توسط بزرگ‌ترین جمله تعیین می‌شود: یا ردیف برگ‌ها، یا ریشه. در حالت مساوی، همهٔ $\log n$ سطح سهم برابر دارند.

نکته · شرط نظم در حالت «ریشه برنده»

در صورت دقیق قضیه (کتاب CLRS)، حالت دوم یک شرط اضافه هم دارد: برای ثابتی مثل $c < 1$ و nهای بزرگ، $af(n/b) \le cf(n)$. یعنی هزینهٔ سطح‌ها واقعاً رو به پایین کم شود. برای همهٔ توابع چندجمله‌ای مثل $f(n) = n^d$ این شرط خودبه‌خود برقرار است، پس در عمل نگرانش نباشید.

اشتباه رایج · قضیهٔ اصلی همه چیز را پوشش نمی‌دهد

اگر $f(n)$ و $n^{\log_b a}$ «به‌طور چندجمله‌ای» فاصله نداشته باشند ولی برابر هم نباشند، هیچ حالتی اعمال نمی‌شود. مثال: $T(n) = 2T(n/2) + n\log n$. این‌جا $n^{\log_2 2} = n$ و $f(n)/n = \log n$ که چندجمله‌ای نیست. با درخت بازگشت: سطح k هزینهٔ $2^k \cdot \frac{n}{2^k}\log\frac{n}{2^k} = n(\log n - k)$ دارد، و جمع روی $k = 0..\log n - 1$ برابر $n \cdot \frac{\log n(\log n+1)}{2} = \Theta(n\log^2 n)$ است.

حالت خاص پرکاربرد: $f(n) = O(n^d)$

در بیشتر الگوریتم‌ها هزینهٔ ترکیب یک چندجمله‌ای است. در این حالت قضیه شکل ساده‌ای دارد:

قضیه · قضیهٔ اصلی برای $f(n) = O(n^d)$

$$T(n) = a\,T\!\left(\frac{n}{b}\right) + O(n^d) \quad\Longrightarrow\quad T(n) = \begin{cases} O(n^d \log n) & \log_b a = d \\ O(n^d) & \log_b a < d \\ O(n^{\log_b a}) & \log_b a > d \end{cases}$$

اثبات · چرا این سه حالت؟ (سری هندسی)

هزینهٔ سطح k حداکثر $a^k \cdot c\left(\frac{n}{b^k}\right)^d = c\,n^d\left(\frac{a}{b^d}\right)^k$ است. پس کل هزینه یک سری هندسی با قدرنسبت $r = a/b^d$ است:

$$T(n) \le c\,n^d \sum_{k=0}^{\log_b n} r^k.$$

- $r < 1$ (یعنی $a < b^d$، یعنی $\log_b a < d$): سری نزولی است و جمعش حداکثر $\frac{1}{1-r}$، یک ثابت. پس $T(n) = O(n^d)$: **ریشه غالب است.**
- $r = 1$ (یعنی $\log_b a = d$): همهٔ $\log_b n + 1$ جمله برابر ۱ هستند. پس $T(n) = O(n^d \log n)$.
- $r > 1$ (یعنی $\log_b a > d$): سری صعودی است و جمعش هم‌مرتبهٔ آخرین جمله است: $r^{\log_b n} = \frac{a^{\log_b n}}{(b^{\log_b n})^d} = \frac{n^{\log_b a}}{n^d}$. ضرب در n^d: $T(n) = O(n^{\log_b a})$: **برگ‌ها غالب‌اند.**

∎

مثال · چهار مثال اسلاید

رابطه	$a,\ b,\ d$	مقایسه	جواب
$T(n) = 4T(n/2) + O(n)$	$4, 2, 1$	$\log_2 4 = 2 > 1$	$O(n^2)$
$T(n) = 3T(n/2) + O(n)$	$3, 2, 1$	$\log_2 3 \approx 1.585 > 1$	$O(n^{\log_2 3}) \approx O(n^{1.6})$
$T(n) = 2T(n/2) + O(n)$	$2, 2, 1$	$\log_2 2 = 1 = d$	$O(n \log n)$
$T(n) = T(n/2) + O(n)$	$1, 2, 1$	$\log_2 1 = 0 < 1$	$O(n)$

چند مثال دیگر برای تمرین:

- جست‌وجوی دودویی $T(n) = T(n/2) + O(1)$: $a = 1, b = 2, d = 0$؛ $\log_2 1 = 0 = d$، پس $O(\log n)$.
- ضرب ماتریس استراسن $T(n) = 7T(n/2) + O(n^2)$: $\log_2 7 \approx 2.81 > 2$، پس $O(n^{2.81})$.
- $T(n) = 2T(n/4) + O(\sqrt{n})$: $\log_4 2 = \frac{1}{2} = d$، پس $O(\sqrt{n} \log n)$.

شهود · نسبت a/b

مجموع اندازهٔ ورودی‌هایی که به زیرمسئله‌ها داده می‌شود $a \cdot \frac{n}{b} = \frac{a}{b}\,n$ است. اگر $a/b > 1$، در هر سطح «حجم کار» بیشتر می‌شود و برگ‌ها سنگین‌اند؛ اگر کوچک‌تر باشد، حجم کار در هر سطح کم می‌شود. پس هر چه a/b کوچک‌تر باشد، زمان اجرا بهتر است. ترفند کاراتسوبا (پایین‌تر) دقیقاً همین است: کم کردن a از ۴ به ۳.

۲.۶ اگر آرایه را به بیش از دو بخش تقسیم کنیم؟

سؤال طبیعی: اگر نصف کردن خوب است، شاید سه‌تایی یا ده‌تایی کردن بهتر باشد؟ فرض کنید آرایه را به a بخش تقسیم کنیم. در مرحلهٔ ادغام، برای انتخاب هر عنصر خروجی باید **کمینهٔ** a **عدد** (عناصر جلوی a لیست) را پیدا کنیم که $a-1$ مقایسه است. اسلاید این را تقریباً a گرفته است:

$$T(n) = a\,T(n/a) + a\,n.$$

در هر سطح $a \cdot n$ کار و $\log_a n$ سطح داریم، پس

$$T(n) = a\,n \log_a n = \frac{a}{\ln a} \cdot n \ln n.$$

چرا؟ · کدام a بهترین است؟

باید ضریب را کمینه کنیم. با هزینهٔ **دقیق** ادغام ($a-1$ مقایسه برای هر عنصر)، ضریب $\frac{a-1}{\ln a}$ می‌شود: برای $a=2$ برابر 1.44، برای $a=3$ برابر 1.82 و برای $a=4$ برابر 2.16. این تابع برای $a \geq 2$ اکیداً صعودی است، پس $a=2$ **بهترین انتخاب است**، همان‌طور که اسلاید می‌گوید.

(اگر با تقریب خامِ a مقایسه حساب کنیم، ضریب $\frac{a}{\ln a}$ در $a = e \approx 2.7$ کمینه است و مقدارش برای $a=2$ و $a=3$ تقریباً برابر است؛ این تقریب برای مقایسهٔ دقیق مناسب نیست.)

تکمیلی (فراتر از اسلاید) · ادغام aتایی با هرم

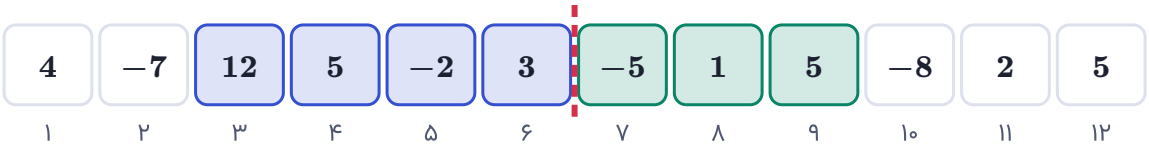
اگر کمینهٔ a عنصر را به‌جای جست‌وجوی خطی با یک هرم کمینه بیابیم، هر عنصر $O(\log a)$ هزینه دارد و کل زمان $n \log a \cdot \log_a n = n \log n$ می‌شود. یعنی با ساختار دادهٔ مناسب، تعداد بخش‌ها روی مرتبه اثری ندارد. این ایده در مرتب‌سازی فایل‌های بزرگ روی دیسک (مرتب‌سازی خارجی) کاربرد دارد.

۲.۷ زیرآرایهٔ بیشینه با تقسیم و حل

مسئلهٔ فصل ۱ را دوباره در نظر بگیرید: زیرآرایهٔ پیوسته‌ای با بیشترین مجموع. تا این‌جا الگوریتم $O(n^2)$ با مجموع پیشوندی داشتیم. حالا با تقسیم و حل به $O(n \log n)$ می‌رسیم.

راهبرد کلی: آرایه را به دو زیرآرایهٔ هم‌اندازه تقسیم کن. جواب بهینه یکی از این دو حالت را دارد:

- **حالت ۱:** جواب کاملاً درون یکی از دو نیمه است. آن را بازگشتی پیدا می‌کنیم.
- **حالت ۲:** جواب از خط وسط **عبور می‌کند**. این را مستقیم و در زمان خطی پیدا می‌کنیم.



شکل ۲-۵ خط قرمز مرز دو نیمه است. هر زیرآرایهٔ قطع‌کننده از یک **پسوند** نیمهٔ چپ (آبی) و یک **پیشوند** نیمهٔ راست (سبز) تشکیل شده است.

چرا؟ · چرا حالت ۲ در زمان خطی حل می‌شود؟

زیرآرایهٔ $A[i..j]$ که $i \le m < j$ (و m وسط است) را می‌توان به دو قسمت مستقل شکست: $A[i..m]$ که **حتماً به m ختم می‌شود** و $A[m+1..j]$ که **حتماً از $m+1$ شروع می‌شود**. مجموع کل = مجموع قسمت چپ + مجموع قسمت راست، و انتخاب i هیچ اثری روی انتخاب بهترین j ندارد. پس کافی است بهترین پسوند چپ ($n/2$ گزینه) و بهترین پیشوند راست ($n/2$ گزینه) را جداگانه پیدا کنیم و جمع بزنیم. به‌جای $n^2/4$ جفت، فقط n حالت بررسی می‌شود.

```
MCS(l, r) ▷ max contiguous sum in A[l..r], S = prefix sums
```

```
1  if l = r then return max(0, A[l])
2  m = floor((l + r) / 2)
3  sol = max(MCS(l, m), MCS(m+1, r))          // case 1
4  Lsol = 0
5  for i = m downto l                          // best suffix of left half
6      if S[m] - S[i-1] > Lsol then
7          Lsol = S[m] - S[i-1]
8  Rsol = 0
9  for i = m+1 to r                            // best prefix of right half
10     if S[i] - S[m] > Rsol then
11         Rsol = S[i] - S[m]
12 return max(sol, Lsol + Rsol)                // case 2
```

نکته · تصحیح یک اندیس

مجموع $A[m+1..i]$ برابر $S[i]-S[m]$ است (نه $S[i]-S[m-1]$ که در اسلاید آمده؛ آن عبارت $A[m]$ را هم می‌شمارد). پیشوند S یک بار در ابتدا با $O(n)$ ساخته می‌شود.

مثال · اجرا روی آرایهٔ مثال

نیمهٔ چپ $-2, 3, 4, -7, 12, 5$ و نیمهٔ راست $-8, 2, 5, -5, 1, 5$.

- **حالت ۱ (بازگشتی):** بهترین جواب نیمهٔ چپ $3 - 2 + 5 + 12 = 18$ و بهترین جواب نیمهٔ راست $5 + 2 = 7$ است.

- **پسوندهای چپ** (از خانهٔ ۶ به عقب): $3,\ 1,\ 6,\ \mathbf{18},\ 11,\ 15$. پس $Lsol = 18$.
- **پیشوندهای راست** (از خانهٔ ۷ به جلو): $-5,\ -4,\ \mathbf{1},\ -7,\ -5,\ 0$. پس $Rsol = 1$.
- **جواب:** $\max(18, 7, 18+1) = 19$ ✓.

زمان اجرا: دو فراخوانی بازگشتی روی نیمه‌ها و دو حلقهٔ خطی:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \implies T(n) = O(n \log n).$$

۲.۸ جمع و ضرب اعداد صحیح

حالا به مسئله‌ای می‌رسیم که فرض «هزینهٔ واحد» در آن معنا ندارد: کار با اعداد خیلی بزرگ (مثلاً هزاران رقمی در رمزنگاری). اندازهٔ ورودی تعداد **بیت‌ها** (n) است و «عمل پایه» یعنی یک عمل روی بیت‌ها.

جمع

جمع: دو عدد صحیح n بیتی a و b داده شده؛ $a+b$ را حساب کنید. روش دبستانی (ستونی با رقم نقلی) $\Theta(n)$ عمل بیتی انجام می‌دهد.

```
carry:   1 1 1 1 1 1 0 1
           1 1 0 1 0 1 0 1      (213)
       +   0 1 1 1 1 1 0 1      (125)
       ----------------------
         1 0 1 0 1 0 0 1 0      (338)
```

قضیه · بهینه بودن جمع دبستانی

هر الگوریتم جمع باید دست‌کم همهٔ $2n$ بیت ورودی را بخواند (تغییر هر بیت، جواب را تغییر می‌دهد)، پس $\Omega(n)$ زمان لازم است. روش دبستانی $O(n)$ است، پس **بهینه** است.

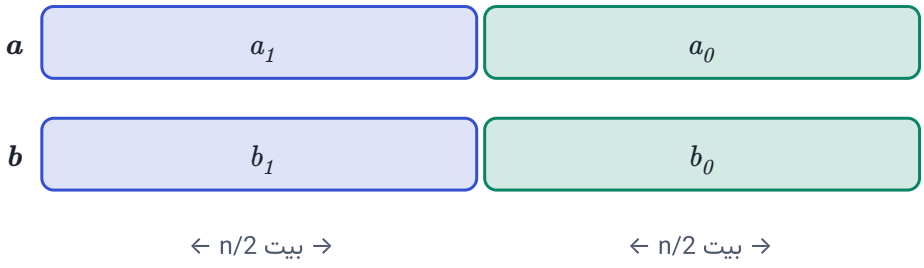
ضرب

ضرب: دو عدد صحیح n بیتی a و b داده شده؛ $a \times b$ را حساب کنید. روش دبستانی $\Theta(n^2)$ عمل بیتی انجام می‌دهد: n حاصل‌ضرب جزئی که هر کدام n بیت دارد، و جمع آن‌ها.

```
              1 1 0 1 0 1 0 1      (213)
            x 0 1 1 1 1 1 0 1      (125)
            ------------------
              1 1 0 1 0 1 0 1
            0 0 0 0 0 0 0 0
          1 1 0 1 0 1 0 1
        1 1 0 1 0 1 0 1
      1 1 0 1 0 1 0 1
    1 1 0 1 0 1 0 1
  1 1 0 1 0 1 0 1
0 0 0 0 0 0 0 0
----------------------------
0 1 1 0 1 0 0 0 0 0 0 0 0 0 0 1    (26625)
```

پرسش: آیا ضرب دبستانی هم بهینه است؟ برخلاف جمع، این‌جا کران پایین ساده فقط $\Omega(n)$ است و بین n و n^2 فاصلهٔ زیادی هست. تقسیم و حل نشان می‌دهد پاسخ **منفی** است.

گرم کردن: تقسیم و حل ساده



شکل ۲-۶ هر عدد n بیتی را به نیمهٔ پرارزش و نیمهٔ کم‌ارزش (هر کدام $n/2$ بیت) تقسیم می‌کنیم.

برای ضرب دو عدد n بیتی، هر کدام را دو نیم می‌کنیم:

$$a = 2^{n/2}\,a_1 + a_0, \qquad b = 2^{n/2}\,b_1 + b_0.$$

$$ab = \left(2^{n/2}a_1 + a_0\right)\left(2^{n/2}b_1 + b_0\right) = 2^{n}\,a_1b_1 + 2^{n/2}\,(a_1b_0 + a_0b_1) + a_0b_0.$$

ضرب در $2^{n/2}$ یا 2^n فقط یعنی **شیفت به چپ** (اضافه کردن صفر به انتها) که $O(n)$ است. پس چهار ضرب $n/2$ بیتی بازگشتی داریم، به‌علاوهٔ چند جمع و شیفت خطی:

$$T(n) = \underbrace{4T(n/2)}_{\text{فراخوانی‌های بازگشتی}} + \underbrace{\Theta(n)}_{\text{جمع و شیفت}} \implies T(n) = \Theta(n^2).$$

مثال · $a = 10001101,\ b = 11100001$

$a_1 = 1000 = 8$، $a_0 = 1101 = 13$، $b_1 = 1110 = 14$، $b_0 = 0001 = 1$. (یعنی $a = 141$، $b = 225$.)

$$ab = 2^8 \cdot (8 \cdot 14) + 2^4\,(8 \cdot 1 + 13 \cdot 14) + 13 \cdot 1 = 28672 + 3040 + 13 = 31725.\ \checkmark$$

چرا؟ · چرا تقسیم و حل این‌جا سودی نداشت؟

با قضیهٔ اصلی: $a = 4$، $b = 2$، $d = 1$ و $\log_2 4 = 2 > 1$؛ برگ‌ها برنده‌اند و $n^{\log_2 4} = n^2$. چهار زیرمسئله با نصف اندازه، دقیقاً همان کار ضرب دبستانی است که به شکل دیگری چیده شده. برای بهبود باید **تعداد زیرمسئله‌ها را کم کنیم.**

۲.۹ ضرب کاراتسوبا

شهود · ترفند گاوس

ما در واقع به a_1b_0 و a_0b_1 جداگانه نیازی نداریم؛ فقط **جمعشان** را می‌خواهیم. حاصل‌ضرب $(a_1 + a_0)(b_1 + b_0)$ را باز کنید: $a_1b_1 + a_1b_0 + a_0b_1 + a_0b_0$. همهٔ چهار جمله در آن هست! اگر a_1b_1 و a_0b_0 را (که به هر حال لازم داریم) کم کنیم، جملهٔ وسط با **یک** ضرب به دست می‌آید.

$$a_1b_0 + a_0b_1 = (a_1 + a_0)(b_1 + b_0) - a_1b_1 - a_0b_0.$$

پس با سه حاصل‌ضرب کار تمام است:

$$ab = 2^n \underbrace{a_1b_1}_{P_1} + 2^{n/2}\Big(\underbrace{(a_1 + a_0)(b_1 + b_0)}_{P_2} - \underbrace{a_1b_1}_{P_1} - \underbrace{a_0b_0}_{P_3}\Big) + \underbrace{a_0b_0}_{P_3}.$$

الگوریتم · ضرب کاراتسوبا

1. دو عدد $n/2$ بیتی را جمع کن: $a_1 + a_0$ و $b_1 + b_0$.
2. سه ضرب $n/2$ بیتی را بازگشتی انجام بده: P_1, P_2, P_3.
3. با جمع، تفریق و شیفت جواب را بساز.

```
KARATSUBA(a, b, n)
1  if n = 1 then return a * b
2  split a = a1 * 2^(n/2) + a0  and  b = b1 * 2^(n/2) + b0
3  P1 = KARATSUBA(a1, b1, n/2)
4  P3 = KARATSUBA(a0, b0, n/2)
5  P2 = KARATSUBA(a1 + a0, b1 + b0, n/2 + 1)
6  return P1 * 2^n + (P2 - P1 - P3) * 2^(n/2) + P3      // shifts and adds: O(n)
```

زمان اجرا:

$$T(n) \le \underbrace{T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + T(1 + \lceil n/2 \rceil)}_{\text{فراخوانی‌های بازگشتی}} + \underbrace{\Theta(n)}_{\text{جمع، تفریق، شیفت}} \implies T(n) = O(n^{\log_2 3}) = O(n^{1.585}).$$

چرا؟ · آن «۱+» در $T(1+\lceil n/2 \rceil)$ از کجا آمد و چرا مهم نیست؟

جمع دو عدد $n/2$ بیتی ممکن است یک رقم نقلی تولید کند، پس $a_1 + a_0$ تا $n/2 + 1$ بیت دارد. این یک بیت اضافه در هر سطح، اندازه را فقط به اندازهٔ یک ثابت تغییر می‌دهد و مرتبهٔ جواب را عوض نمی‌کند (می‌توان نشان داد $T(n) \le 3T(n/2+1) + cn$ همان جواب $O(n^{\log_2 3})$ را دارد). با قضیهٔ اصلی: $a = 3, b = 2, d = 1$ و $\log_2 3 \approx 1.585 > 1$.

مثال · کاراتسوبا روی همان مثال

$P_1 = 8 \cdot 14 = 112$، $P_3 = 13 \cdot 1 = 13$، $P_2 = (8+13)(14+1) = 21 \cdot 15 = 315$.

جملهٔ وسط: $P_2 - P_1 - P_3 = 315 - 112 - 13 = 190$ (درست برابر $8 \cdot 1 + 13 \cdot 14 = 190$).

$$ab = 112 \cdot 2^8 + 190 \cdot 2^4 + 13 = 28672 + 3040 + 13 = 31725. \ \checkmark$$

همین ایده در مبنای ده هم کار می‌کند. برای 1234×5678 با تقسیم به دو رقمی‌ها: $P_1 = 12 \cdot 56 = 672$، $P_3 = 34 \cdot 78 = 2652$، $P_2 = 46 \cdot 134 = 6164$ و جملهٔ وسط $6164 - 672 - 2652 = 2840$:

$$1234 \times 5678 = 672 \cdot 10^4 + 2840 \cdot 10^2 + 2652 = 7006652.$$

n (تعداد بیت)	دبستانی n^2	کاراتسوبا $n^{1.585}$	نسبت
10^3	10^6	$\approx 5.7 \times 10^4$	≈ 18
10^6	10^{12}	$\approx 3.2 \times 10^9$	≈ 310

تکمیلی (فراتر از اسلاید) · ادامهٔ داستان ضرب

کتابخانه‌های اعداد بزرگ (مثلاً در پایتون و GMP) برای اعداد متوسط از کاراتسوبا استفاده می‌کنند؛ برای اعداد کوچک، روش دبستانی به‌خاطر ثابت کمترش سریع‌تر است. الگوریتم‌های سریع‌تری هم وجود دارند: تقسیم به سه بخش (Toom-Cook)، روش‌های مبتنی بر تبدیل فوریهٔ سریع (Schönhage–Strassen) و در سال ۲۰۱۹ الگوریتم هاروی و فان‌درهوون با زمان $O(n \log n)$.

۲.۱۰ روش حدس و استقرا

قضیهٔ اصلی فقط رابطه‌هایی به شکل $aT(n/b) + f(n)$ را حل می‌کند. برای رابطه‌های دیگر (مثلاً دو زیرمسئلهٔ نابرابر) ابزار عمومی‌تری لازم است: **حدس و استقرا** (که به آن «روش جای‌گذاری» یا substitution method هم می‌گویند).

۱ **حدس:** جوابی برای رابطه حدس بزن (با باز کردن رابطه، درخت بازگشت، یا تجربه).

۲ **اثبات:** با استقرای ریاضی ثابت کن حدست درست است.

۳ **پاک‌نویس:** وانمود کن هرگز گام ۱ را انجام نداده‌ای و فقط قضیه و اثباتِ گام ۲ را تمیز بنویس.

مثال اول: $T(n) = 2T(n/2) + n$

با $T(0) = 0$ و $T(1) = 1$. قضیهٔ اصلی می‌گوید $T(n) = O(n \log n)$. این را با حدس و استقرا ثابت می‌کنیم (فرض: n توانی از ۲).

گام ۱: حدس با باز کردن رابطه

$$
\begin{aligned}
T(n) &= 2\,T(n/2) + n \\
&= 2\left(2\,T(n/4) + \tfrac{n}{2}\right) + n = 4\,T(n/4) + 2n && \text{باز کردن و ساده‌سازی} \\
&= 4\left(2\,T(n/8) + \tfrac{n}{4}\right) + 2n = 8\,T(n/8) + 3n && \text{باز کردن دوباره} \\
&\;\;\vdots \\
&= 2^t\,T(n/2^t) + t \cdot n. && \text{حدس الگو}
\end{aligned}
$$

با قرار دادن $t = \log n$ (تا $n/2^t = 1$ شود): $T(n) = n\,T(1) + n \log n = n(\log n + 1)$.

گام ۲: اثبات حدس با استقرا

فرض استقرا: $T(n) = n(\log n + 1)$.

پایه ($n = 1$): $T(1) = 1 = 1 \cdot (\log 1 + 1)$ ✓.

گام: فرض کنید حکم برای همهٔ $1 \le n < k$ برقرار است. آنگاه

$$T(k) = 2\,T(k/2) + k = 2 \cdot \frac{k}{2}\left(\log \frac{k}{2} + 1\right) + k = k\,(\log k - 1 + 1) + k = k(\log k + 1).$$

(خط اول تعریف T، تساوی دوم فرض استقرا برای $k/2 < k$، و بقیه ساده‌سازی است.) پس حکم برای $n = k$ هم برقرار است.

نتیجه: برای همهٔ $n \geq 1$ (توان ۲)، $T(n) = n(\log n + 1) = O(n \log n)$.

■

مثال دوم: وقتی ثابت را نمی‌دانیم

$$T(n) = 2\,T(n/2) + 32\,n, \qquad T(2) = 2.$$

حدس: $O(n \log n)$. اما حدس دقیقی دربارهٔ **شکل کامل** جواب نداریم؛ ضریب پیشرو چیست؟ آیا باز هم می‌توانیم گام ۲ را انجام دهیم؟ اول یک اثبات **غلط** ببینیم:

اشتباه رایج · این اثبات کجایش غلط است؟

فرض استقرا: $T(n) = O(n \log n)$. **پایه:** $T(2) = 2 = O(1) = O(2 \log 2)$.

گام: فرض کنید برای $n < k$، $T(n) = O(n \log n)$. آنگاه $T(k) = 2T(k/2) + 32k = 2 \cdot O\left(\frac{k}{2} \log \frac{k}{2}\right) + 32k =$ $O(k \log k)$. پس حکم برای k برقرار است.

با **همین استدلال** می‌توان «ثابت کرد» $T(n) = O(n)$: فرض کن $T(n) = O(n)$ برای $n < k$؛ آنگاه $T(k) = 2 \cdot O(k/2) + 32k = O(k)$. اما این **نادرست** است!

مشکل: جملهٔ «$T(n) = O(n)$ برای $n < k$» بی‌معناست. O دربارهٔ رفتار تابع **وقتی n به بی‌نهایت می‌رود** حرف می‌زند و برای تعداد متناهی n همیشه درست است. تعریف O یک ثابت c واحد برای **همهٔ** nها می‌خواهد. در اثبات غلط، اگر $T(n) \leq cn$ را فرض کنیم، به $T(k) \leq (c + 32)\,k$ می‌رسیم: ثابت در هر گام بزرگ‌تر می‌شود، و پس از $\log n$ گام به $(c + 32 \log n)\,n$ می‌رسد که دیگر ثابت نیست.

جمع‌بندی · قانون طلایی

در فرض استقرا هرگز O ننویسید. ثابت را **صریح** بنویسید ($T(n) \leq C \cdot g(n)$) و ثابت کنید **همان** C در گام استقرا حفظ می‌شود.

گام ۲ درست: ثابت را باز بگذار و در آخر پیدایش کن

حدس: $T(n) \leq C \cdot n \log n$ برای ثابتی مثل C که هنوز نمی‌دانیم.

پایه: $T(2) = 2 \le C \cdot 2\log 2 = 2C$، که برای $C \ge 1$ برقرار است.

گام: فرض کنید برای $n < k$ برقرار باشد. آنگاه

$$\begin{aligned} T(k) &= 2\,T(k/2) + 32k \;\le\; 2C\,\frac{k}{2}\log\frac{k}{2} + 32k \\ &= Ck(\log k - 1) + 32k \;=\; Ck\log k + (32 - C)\,k \\ &\le Ck\log k \qquad C \ge 32 \text{ به شرط} \end{aligned}$$

پس با $C = 32$ (که شرط پایه را هم دارد) حکم برقرار است.

∎

گام ۳: پاک‌نویس

$$.T(n) = O(n\log n)$$

فرض استقرا: $T(n) \le 32\,n\log n$. **پایه:** $T(2) = 2 \le 32 \cdot 2\log 2$. ✓

گام: برای $n < k$ فرض را داریم. $T(k) = 2T(k/2) + 32k \le 2 \cdot 32 \cdot \frac{k}{2}\log\frac{k}{2} + 32k = 32k(\log k - 1) + 32k = 32\,k\log k$.

نتیجه: برای همهٔ $n \ge 2$، $T(n) \le 32\,n\log n$. طبق تعریف O با $n_0 = 2$ و $c = 32$، $T(n) = O(n\log n)$.

∎

مثال سوم: دو زیرمسئلهٔ نابرابر

$$T(n) \le T\!\left(\frac{n}{5}\right) + T\!\left(\frac{7n}{10}\right) + n \quad (n > 10), \qquad T(n) = 1 \quad (1 \le n \le 10).$$

این رابطه (که در فصل ۳، الگوریتم انتخاب قطعی، دوباره می‌بینیم) شکل $aT(n/b)$ ندارد، پس قضیهٔ اصلی کاربرد ندارد. حدس می‌زنیم $O(n)$ باشد (چون $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$؛ کل کار در هر سطح کم می‌شود).

اثبات · گام ۲: C را در طول اثبات پیدا می‌کنیم

فرض استقرا: $T(n) \le C\,n$. **پایه:** $T(n) = 1 \le Cn$ برای $1 \le n \le 10$؛ کافی است $C \ge 1$.

گام: برای $k > 10$ و فرض برقرار برای $1 \le n < k$:

$$T(k) \le k + C \cdot \frac{k}{5} + C \cdot \frac{7k}{10} = k + \frac{9C}{10}\,k \;\overset{?}{\le}\; C\,k.$$

نامساوی آخر یعنی $k \le \frac{C}{10}k$، یعنی $C \ge 10$. پس $C = 10$ کار می‌کند: $T(k) \le k + 2k + 7k = 10k$.

نتیجه: برای همهٔ $n \ge 1$، $T(n) \le 10n$؛ یعنی $T(n) = O(n)$ با $n_0 = 1$ و $c = 10$.

■

نکته · کف و سقف

در عمل $n/5$ و $7n/10$ باید گرد شوند. چون T صعودی است و ما فقط نامساوی $\le$ می‌خواهیم، با کمی دقت (و احتمالاً تغییر جزئی ثابت‌ها) همین اثبات کار می‌کند. در این درس کف و سقف را نادیده می‌گیریم مگر خلافش خواسته شود.

حدس نادرست

$$T(n) \le T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n \quad (n > 3), \qquad T(n) = 1 \quad (1 \le n \le 3).$$

دوباره $O(n)$ را حدس می‌زنیم و فرض استقرا را $T(n) \le Cn$ می‌گیریم:

$$T(k) \le k + C \cdot \frac{k}{3} + C \cdot \frac{2k}{3} = k + C\,k \overset{?}{\le} C\,k.$$

این نامساوی برای **هیچ** Cای برقرار نیست (همیشه k اضافه داریم). نتیجه: یا حدس ما غلط است، یا باید اثبات را تغییر دهیم. این‌جا **حدس غلط است**: چون $\frac{1}{3} + \frac{2}{3} = 1$، هر سطح درخت بازگشت کل n کار دارد و عمق درخت $\Theta(\log n)$ است، پس جواب $\Theta(n \log n)$ است.

تکمیلی (فراتر از اسلاید) · اثبات $O(n \log n)$ برای همین رابطه

فرض استقرا: $T(n) \le C\,n \log n$. آنگاه

$$T(k) \le k + C\frac{k}{3}\log\frac{k}{3} + C\frac{2k}{3}\log\frac{2k}{3} = Ck\log k + k\Big(1 - C\underbrace{\big(\tfrac{1}{3}\log 3 + \tfrac{2}{3}\log\tfrac{3}{2}\big)}_{\approx 0.918}\Big).$$

عبارت داخل پرانتز بزرگ برای $C \ge 1/0.918 \approx 1.09$ نامثبت است، پس با $C = 2$ (و بررسی پایه) حکم ثابت می‌شود.

قضیه · رابطه‌های دو شاخه‌ای

$$T(n) \le T(an) + T(bn) + O(n) \implies T(n) = \begin{cases} O(n) & a + b < 1 \\ O(n \log n) & a + b = 1 \end{cases}$$

شهود با درخت بازگشت: اگر ریشه cn کار کند، دو فرزندش $c(an)+c(bn)=(a+b)\,cn$ کار می‌کنند، نوه‌ها $(a+b)^2cn$ و ... . اگر $a+b<1$، این یک سری هندسی نزولی است با جمع حداکثر $\frac{cn}{1-(a+b)}=O(n)$. اگر $a+b=1$، هر سطح حداکثر cn کار دارد و عمق درخت $O(\log n)$ است.

اثبات استقرایی حالت اول: با فرض $T(n)\le Cn$: $T(k)\le Cak+Cbk+ck=Ck(a+b)+ck\le Ck$ اگر $C\ge\frac{c}{1-a-b}$.

حالت دوم: مثل اثبات کادر قبل؛ با فرض $T(n)\le Cn\log n$ به شرط $C\ge c/H$ می‌رسیم که $H=a\log\frac{1}{a}+b\log\frac{1}{b}>0$.

■

مثال آخر: تقویت فرض استقرا

$$T(n)\le 2\,T\!\left(\frac{n}{2}\right)+1\quad(n\ge 2),\qquad T(1)=1.$$

حدس: $O(n)$. فرض استقرا: $T(n)\le Cn$. پایه: $1\le C$، پس $C\ge 1$. گام:

$$T(k)\le 2\,T(k/2)+1\le 2C\cdot\frac{k}{2}+1=Ck+1\overset{?}{\le}Ck.$$

برای هیچ Cای برقرار نیست! آیا حدس غلط است؟ **نه!** با باز کردن رابطه می‌بینیم $T(n)=1+2+4+\cdots+n=2n-1$ که واقعاً $O(n)$ است. مشکل در حدس نیست؛ **فرض استقرا ضعیف است**: آن «+۱» اضافه در هر گام جمع می‌شود و فرض Cn جایی برای جذبش ندارد.

اثبات · فرض قوی‌تر: یک جملهٔ مرتبه‌پایین کم کن

فرض استقرا: $T(n)\le Cn+B$ (که B می‌تواند منفی باشد). **پایه:** $1=T(1)\le C+B$.

گام:

$$T(k)\le 2\left(C\frac{k}{2}+B\right)+1=Ck+2B+1\overset{?}{\le}Ck+B\iff B\le -1.$$

با $B=-1$، پایه $C\ge 2$ می‌خواهد. پس **حدس جدید** $T(n)\le 2n-1$ با استقرا ثابت می‌شود.

■

شهود · تناقضِ ظاهری

$T(n) \le 2n$ را **نمی‌توان** مستقیم با استقرا ثابت کرد، اما $T(n) \le 2n - 1$ را که **قوی‌تر** است، می‌توان! دلیلش این است که در گام استقرا، فرض استقرا هم قوی‌تر شده است: وقتی بیشتر فرض می‌کنیم، می‌توانیم بیشتر نتیجه بگیریم. هر وقت گام استقرا «تقریباً» درست درآمد و فقط یک جملهٔ مرتبه‌پایین اضافه داشت، فرض را با کم کردن یک جملهٔ مرتبه‌پایین تقویت کنید.

جمع‌بندی · فصل ۲ در یک نگاه

- تقسیم و حل = تقسیم + حل بازگشتی + ترکیب. زمانش با یک رابطهٔ بازگشتی توصیف می‌شود.
- درخت بازگشت: هزینهٔ هر سطح را حساب و سطح‌ها را جمع کن. برگ‌ها $n^{\log_b a}$ تا هستند.
- قضیهٔ اصلی: مقایسهٔ $f(n)$ با $n^{\log_b a}$. در حالت n^d: مقایسهٔ d با $\log_b a$.
- مرتب‌سازی ادغامی و زیرآرایهٔ بیشینه $O(n \log n)$؛ ضرب کاراتسوبا $O(n^{1.585})$ با کم کردن تعداد زیرمسئله‌ها از ۴ به ۳.
- حدس و استقرا: ثابت‌ها را صریح بنویس، C را در طول اثبات پیدا کن، و اگر لازم شد فرض را با کم کردن جملهٔ مرتبه‌پایین تقویت کن.

۲.۱۱ خودآزمایی فصل ۲

خودآزمایی

۱. $T(n) = 9T(n/3) + n^2$ را حل کنید.

پاسخ: $a = 9, b = 3, d = 2$؛ $\log_3 9 = 2 = d$، پس $\Theta(n^2 \log n)$.

۲. $T(n) = 2T(n/2) + n^2$ را حل کنید.

پاسخ: $\log_2 2 = 1 < 2 = d$، ریشه برنده است: $\Theta(n^2)$.

۳. $T(n) = 16T(n/4) + n$ را حل کنید.

پاسخ: $\log_4 16 = 2 > 1$، برگ‌ها برنده‌اند: $\Theta(n^2)$.

۴. اگر در ضرب تقسیم و حل بتوانیم با ۵ ضرب یک‌سوم‌اندازه کار را انجام دهیم ($T(n) = 5T(n/3) + O(n)$)، زمان چه می‌شود؟ از کاراتسوبا بهتر است؟

پاسخ: $\log_3 5 \approx 1.465 < 1.585$، پس $O(n^{1.465})$ که از کاراتسوبا بهتر است. (این ایدهٔ الگوریتم Toom-3 است.)

5. چرا اثبات «$T(n) = O(n)$ برای $T(n) = 2T(n/2) + n$» با فرض استقرای $T(n) = O(n)$ غلط است؟

پاسخ: چون ثابت پنهان در O در هر گام عوض می‌شود ($c \to c+1$). استقرا باید با ثابت صریح انجام شود: $T(n) \le cn$ نتیجه می‌دهد $T(k) \le ck + k$ که از ck بیشتر است.

6. در الگوریتم تقسیم و حل زیرآرایهٔ بیشینه، اگر به‌جای مجموع پیشوندی، مجموع‌های حالت ۲ را هر بار از نو با جمع زدن حساب کنیم، زمان چه می‌شود؟

پاسخ: اگر پسوندها را با یک جمع تجمعی از وسط به چپ (و پیشوندها از وسط به راست) حساب کنیم، باز هم $O(n)$ برای هر فراخوانی است و جواب $O(n \log n)$ می‌ماند. اما اگر هر مجموع را مستقل از صفر حساب کنیم، حالت ۲ $O(n^2)$ و کل زمان $O(n^2)$ می‌شود.

فصل ۳

الگوریتم‌های تصادفی

الگوریتمی که سکه می‌اندازد! می‌بینیم چطور کمی تصادف، الگوریتم را از دست ورودی‌های بد نجات می‌دهد و چطور با امید ریاضی و متغیرهای نشانگر زمانش را تحلیل کنیم.

مبانی احتمال · متغیر تصادفی و امید · میانهٔ تقریبی · مسئلهٔ استخدام · انتخاب · کوئیک‌سورت

۳.۱ تصادف در طراحی الگوریتم

تا این‌جا با الگوهای حریصانه، تقسیم و حل، برنامه‌ریزی پویا و شارش شبکه آشنا شدیم. الگوی پنجم **تصادفی‌سازی** است.

تعریف · الگوریتم تصادفی

الگوریتم تصادفی (randomized algorithm) الگوریتمی است که رفتارش نه فقط به ورودی، بلکه به **انتخاب‌های تصادفی**‌ای که خود الگوریتم در حین اجرا انجام می‌دهد هم بستگی دارد. پس روی یک ورودی ثابت، دو اجرای مختلف ممکن است مسیر متفاوتی بروند.

نکته · فرض مولد عدد تصادفی

فرض می‌کنیم تابع $\text{Random}(a, b)$ را داریم که برای دو عدد صحیح $a < b$، عدد صحیح r با $a \le r \le b$ را **به‌طور یکنواخت تصادفی** برمی‌گرداند (یعنی هر کدام از $b - a + 1$ مقدار با احتمال برابر). فرض می‌کنیم این تابع در $O(1)$ اجرا می‌شود؛ یا دقیق‌تر، پرتاب یک سکهٔ سالم یک واحد زمان می‌برد.

چرا؟ · چرا تصادف؟

تصادف می‌تواند به **ساده‌ترین**، **سریع‌ترین** یا حتی **تنها** الگوریتم شناخته‌شده برای یک مسئله منجر شود. نمونه‌ها: الگوریتم‌های گراف، کوئیک‌سورت، درهم‌سازی (hashing)، توازن بار و رمزنگاری.

شهود اصلی: در الگوریتم قطعی، یک «دشمن» که الگوریتم را می‌شناسد همیشه می‌تواند بدترین ورودی را بسازد (مثلاً آرایهٔ مرتب برای کوئیک‌سورتی که همیشه اولین عنصر را محور می‌گیرد). اما اگر الگوریتم انتخاب‌هایش را با سکه انجام دهد، دشمن نمی‌داند الگوریتم چه خواهد کرد و نمی‌تواند ورودیِ همیشه-بد بسازد. بدشانسی ممکن است، ولی بسیار نادر است.

۳.۲ فضای احتمال متناهی

برای تحلیل الگوریتم‌های تصادفی به زبان احتمال نیاز داریم. فقط فضاهای **متناهی** لازم‌اند و همه چیز با جمع ساده بیان می‌شود.

آزمایش فرایندی که یکی از نتایج ممکنِ مشخصی را تولید می‌کند (مثل پرتاب تاس).

فضای نمونه مجموعهٔ همهٔ نتایج ممکن آزمایش؛ با Ω نشان می‌دهیم.

پیشامد هر زیرمجموعه از فضای نمونه (مثل «عدد زوج آمد» = $\{2,4,6\}$).

پیشامد ساده هر عضو تکی $\omega \in \Omega$.

تعریف · فضای احتمال متناهی

زوج (Ω, P) که Ω یک مجموعهٔ متناهی است و P یک اندازهٔ جمع‌پذیر روی زیرمجموعه‌های Ω با $P[\Omega] = 1$. کافی است P را روی پیشامدهای ساده مشخص کنیم: تابعی $P : \Omega \to [0,1]$ با $\sum_{\omega \in \Omega} P[\omega] = 1$. آنگاه احتمال هر پیشامد A برابر است با

$$P[A] = \sum_{a \in A} P[a].$$

تعریف · توزیع یکنواخت

ساده‌ترین مثال: $P[A] = \dfrac{|A|}{|\Omega|}$ برای هر $A \subseteq \Omega$. یعنی همهٔ نتایج هم‌احتمال‌اند، مثل پرتاب تاس سالم.

مثال · چند فضای احتمال

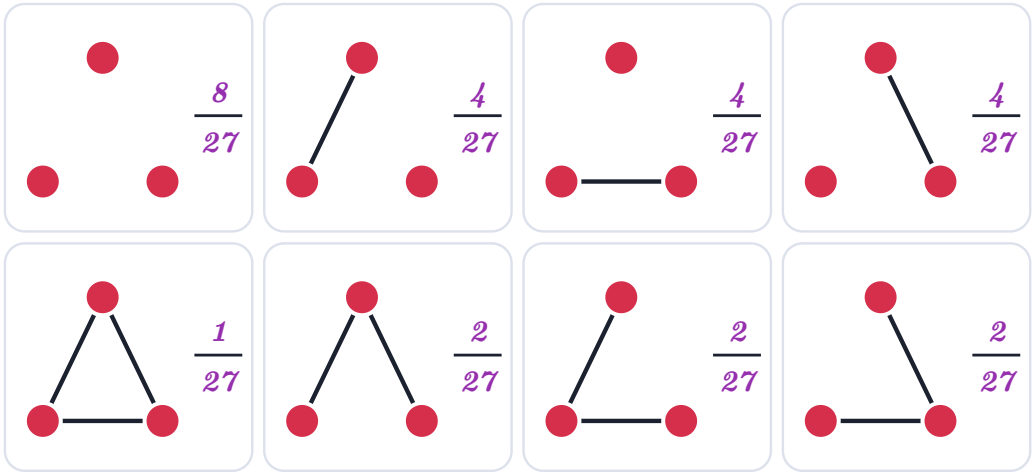
- **یک سکهٔ سالم:** $\Omega = \{H, T\}$، $P[H] = P[T] = \frac{1}{2}$. ($H$ شیر، T خط)
- **دو بار پرتاب سکه:** $\Omega = \{(H,H),(H,T),(T,H),(T,T)\}$ و هر کدام $\frac{1}{4}$.
- **یک تاس سالم:** $\Omega = \{1,\dots,6\}$ و $P[1] = \cdots = P[6] = \frac{1}{6}$.
- **تاس و سپس سکه:** $\Omega = \{(1,H),\dots,(6,H),(1,T),\dots,(6,T)\}$ و $P[(i,H)] = P[(i,T)] = \frac{1}{12}$.

فضای احتمال $G(n,p)$: گراف تصادفی

آزمایش: گرافی با n رأس می‌سازیم که در آن هر یال (از $\binom{n}{2}$ یال ممکن)، مستقل از بقیه، با احتمال p وجود دارد. فضای نمونه همهٔ گراف‌های ممکن روی n رأسِ ثابت است. احتمال یک گراف مشخص با m یال:

$$P[\text{گراف با } m \text{ یال}] = p^m\,(1-p)^{\binom{n}{2}-m}.$$

(m یالِ موجود هر کدام با احتمال p آمده‌اند و $\binom{n}{2} - m$ یالِ غایب هر کدام با احتمال $1-p$ نیامده‌اند.)



شکل ۳-۱ هشت عضو فضای نمونهٔ $G(3, \frac{1}{3})$. گراف بی‌یال $(\frac{2}{3})^3 = \frac{8}{27}$، هر گراف یک‌یالی $\frac{1}{3}(\frac{2}{3})^2 = \frac{4}{27}$، هر گراف دویالی $\frac{2}{27}$ و مثلث $\frac{1}{27}$. جمع: $8 + 3 \cdot 4 + 3 \cdot 2 + 1 = 27$. ✓

چطور این آزمایش را انجام دهیم (یک گراف تصادفی بسازیم)؟

- **روش ۱:** برای هر یال یک سکهٔ اریب بینداز که با احتمال p شیر بیاید؛ اگر شیر آمد یال را بگذار.
- **روش ۲:** همهٔ اعضای فضای نمونه را با احتمالشان فهرست کن. سپس عددی صحیح را به‌طور یکنواخت از $[1, 27]$ انتخاب کن: اگر بین ۱ تا ۸ بود گراف بی‌یال، اگر بین ۹ تا ۱۲ بود اولین گراف یک‌یالی، و به همین ترتیب.

نکته · روش ۲ همیشه ممکن است، ولی ...

روش ۲ برای هر فضای متناهی کار می‌کند، اما فهرست کردن کل فضا معمولاً نمایی است ($G(n,p)$ دارای $2^{\binom{n}{2}}$ عضو است). روش ۱ فقط $\binom{n}{2}$ سکه می‌خواهد. هنر طراحی، یافتن روش‌هایی مثل روش ۱ است.

مجموعه‌های مستقل تصادفی

گراف G داده شده است. فضای احتمالی را در نظر بگیرید که فضای نمونه‌اش مجموعه‌های مستقل G (زیرمجموعه‌ای از رأس‌ها که هیچ یالی بین دو رأسش نیست) و توزیعش یکنواخت است. یک روش ساده برای نمونه‌برداری: همهٔ مجموعه‌های مستقل را تولید و از ۱ تا m شماره‌گذاری کن، سپس عدد k را یکنواخت از $[1, m]$ بردار و مجموعهٔ شمارهٔ k را گزارش کن. این روش کارا نیست، چون m می‌تواند نمایی باشد. **روش سریع‌تری هست؟**

تکمیلی (فراتر از اسلاید) · پاسخ کوتاه

در حالت کلی این مسئله دشوار است: حتی **شمردن** مجموعه‌های مستقل یک گراف مسئله‌ای سخت است. برای خانواده‌هایی از گراف‌ها (مثلاً با بیشینهٔ درجهٔ کم) روش‌های نمونه‌برداری تقریبی مبتنی بر زنجیرهٔ مارکوف وجود دارد. این مثال نشان می‌دهد که «تعریف یک فضای احتمال» و «نمونه‌برداری کارا از آن» دو چیز کاملاً متفاوت‌اند.

۳.۳ متغیر تصادفی

تعریف · متغیر تصادفی

متغیر تصادفی X **تابعی** از فضای نمونه به اعداد حقیقی است: $X : \Omega \to \mathbb{R}$. به هر نتیجهٔ ممکنِ آزمایش یک عدد نسبت می‌دهد.

اشتباه رایج · نام گمراه‌کننده

متغیر تصادفی یک **تابع** است؛ نه متغیر است و نه تصادفی! آنچه تصادفی است ورودی آن (نتیجهٔ آزمایش) است. چون ورودی تصادفی است، خروجی هم عددی تصادفی می‌شود.

توزیع متغیر تصادفی X:

$$P[X = r] = \sum_{\omega:\ X(\omega)=r} P[\omega].$$

اگر پیشامد $A_r = \{\omega : X(\omega) = r\}$ را تعریف کنیم، آنگاه $P[X = r] = P[A_r]$. یعنی «$X = r$» نام کوتاه یک پیشامد است.

مثال · تعداد شیرها در سه پرتاب

سکهٔ سالمی سه بار پرتاب می‌شود. $X(\omega)$ تعداد شیرها در نتیجهٔ ω است:

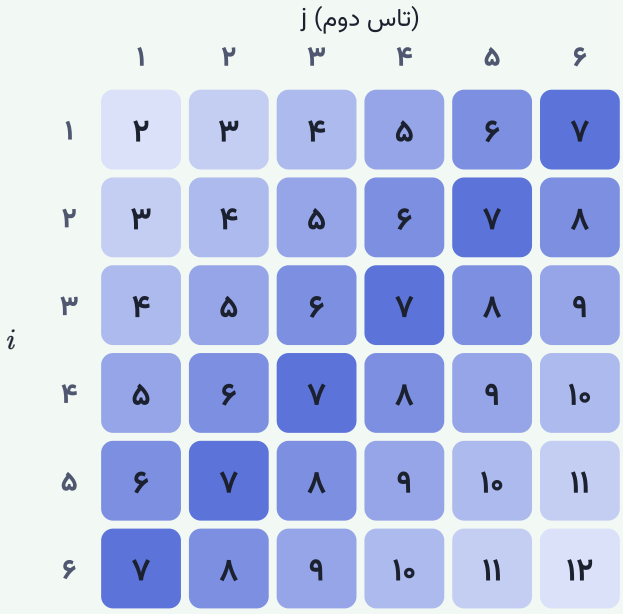
$X(HHH) = 3$؛ $X(HHT) = X(HTH) = X(THH) = 2$؛ $X(TTH) = X(THT) = X(HTT) = 1$؛ $X(TTT) = 0$.

هر کدام از ۸ نتیجه احتمال $\frac{1}{8}$ دارد، پس:

$$P[X=0] = \tfrac{1}{8},\quad P[X=1] = \tfrac{3}{8},\quad P[X=2] = \tfrac{3}{8},\quad P[X=3] = \tfrac{1}{8}.$$

مثال · مجموع دو تاس

X مجموع اعداد دو تاس است. مقادیر X برای ۳۶ نتیجهٔ ممکن (i, j) در جدول زیر آمده است. خانه‌های هم‌رنگ روی قطرهای فرعی هم‌مجموع‌اند.



شکل ۳-۲ عدد ۷ روی بلندترین قطر (۶ خانه) است، پس محتمل‌ترین مجموع است: $P[X=7]=\frac{6}{36}$. در حالت کلی $P[X=k]=\frac{6-|k-7|}{36}$ برای $2\le k\le 12$.

۳.۴ امید ریاضی و خطی بودن آن

تعریف · امید ریاضی

امید ریاضی (expected value) متغیر تصادفی X روی فضای نمونهٔ Ω:

$$E[X]=\sum_{\omega\in\Omega}P[\omega]\,X(\omega)=\sum_{r}P[X=r]\cdot r.$$

تساوی دوم با دسته‌بندی نتیجه‌ها بر اساس مقدار X به دست می‌آید. امید ریاضی یعنی میانگین وزن‌دار مقادیر، با وزن احتمال.

مثال · امید تعداد یال‌های گراف تصادفی در $G(3,\frac{1}{3})$

فرض کنید $X(\omega)$ تعداد یال‌های گراف ω باشد. با فرمول اول، روی ۸ گراف جمع می‌زنیم:

$$E[X]=\tfrac{8}{27}\cdot 0+\tfrac{4}{27}\cdot 1+\tfrac{4}{27}\cdot 1+\tfrac{4}{27}\cdot 1+\tfrac{2}{27}\cdot 2+\tfrac{2}{27}\cdot 2+\tfrac{2}{27}\cdot 2+\tfrac{1}{27}\cdot 3.$$

با گروه‌بندی بر اساس مقدار (فرمول دوم):

$$E[X] = \underbrace{\tfrac{8}{27}}_{P[X=0]} \cdot 0 + \underbrace{\tfrac{12}{27}}_{P[X=1]} \cdot 1 + \underbrace{\tfrac{6}{27}}_{P[X=2]} \cdot 2 + \underbrace{\tfrac{1}{27}}_{P[X=3]} \cdot 3 = \frac{12+12+3}{27} = 1.$$

قضیه · خطی بودن امید ریاضی

اگر $X_1, \dots, X_n$ متغیرهای تصادفی روی Ω و a, b دو عدد حقیقی باشند:

$$\text{(i)}\ \ E\Big[\sum_{i=1}^{n} X_i\Big] = \sum_{i=1}^{n} E[X_i], \qquad \text{(ii)}\ \ E[aX+b] = a\,E[X] + b.$$

(i) برای دو متغیر (و با استقرا برای n متغیر):

$$E[X_1 + X_2] = \sum_{\omega} P[\omega]\big(X_1(\omega) + X_2(\omega)\big) = \sum_{\omega} P[\omega]X_1(\omega) + \sum_{\omega} P[\omega]X_2(\omega) = E[X_1] + E[X_2].$$

$$E[aX+b] = \textstyle\sum_{\omega} P[\omega]\big(aX(\omega)+b\big) = a\sum_{\omega} P[\omega]X(\omega) + b\underbrace{\sum_{\omega} P[\omega]}_{=1} = aE[X] + b. \text{ (ii)}$$

∎

اشتباه رایج · به استقلال نیازی نیست!

در اثبات بالا هیچ جا از استقلال X_1 و X_2 استفاده نکردیم. **خطی بودن امید برای هر متغیرهای تصادفی‌ای برقرار است، حتی کاملاً وابسته.** همین ویژگی آن را به قدرتمندترین ابزار تحلیل الگوریتم‌های تصادفی تبدیل می‌کند. (در مقابل، $E[XY] = E[X]E[Y]$ فقط برای متغیرهای مستقل درست است.)

مثال · امید مجموع دو تاس

راه سخت: ۳۶ نتیجه را با مقدار X و احتمالشان فهرست کن و جمع بزن.

راه ساده: $X_1(i,j) = i$ (عدد تاس اول) و $X_2(i,j) = j$ (عدد تاس دوم) را تعریف کن. آنگاه $X = X_1 + X_2$ و

$$E[X_1] = E[X_2] = \frac{1+2+3+4+5+6}{6} = \frac{7}{2} \quad\Longrightarrow\quad E[X] = E[X_1] + E[X_2] = 7.$$

همین روش برای مثال گراف: $X = \sum_e X_e$ که $X_e = 1$ اگر یال e باشد. $E[X_e] = p$، پس $E[X] = \binom{n}{2}p = 3 \cdot \frac{1}{3} = 1$.
بدون فهرست کردن ۸ گراف!

۳.۵ متغیر نشانگر و جایگشت تصادفی

تعریف · متغیر تصادفی نشانگر

برای پیشامد A، متغیر نشانگر (indicator random variable) I_A را تعریف می‌کنیم:

$$I_A(\omega) = \begin{cases} 1 & \omega \in A \\ 0 & \omega \notin A \end{cases} \qquad \Longrightarrow \qquad E[I_A] = 1 \cdot P[A] + 0 \cdot P[\bar{A}] = P[A].$$

شهود · دستور پخت تحلیل با متغیر نشانگر

1. کمیتی را که می‌خواهی (مثلاً تعداد مقایسه‌ها) به صورت **جمع** چند متغیر نشانگر بنویس: $X = \sum_i X_i$.
2. برای هر i، فقط **احتمال** پیشامد مربوط را حساب کن: $E[X_i] = P[A_i]$.
3. با خطی بودن امید جمع بزن: $E[X] = \sum_i P[A_i]$.

این دستور را در ادامه برای نقاط ثابت جایگشت، مسئلهٔ استخدام و کوئیک‌سورت به کار می‌بریم.

تولید جایگشت تصادفی

مسئله: برای n داده‌شده، جایگشت تصادفی σ از $1, 2, \ldots, n$ را با **توزیع یکنواخت** تولید کنید (هر کدام از $n!$ جایگشت با احتمال $1/n!$).

راه ساده (ناکارا): همهٔ $n!$ جایگشت را تولید و شماره‌گذاری کن، سپس عدد k را یکنواخت انتخاب کن. **راه کاراتر:** $\sigma(1)$ (چپ‌ترین عنصر) را یکنواخت از $\{1, \ldots, n\}$ انتخاب کن، سپس $\sigma(2)$ را یکنواخت از $\{1, \ldots, n\} - \{\sigma(1)\}$، و به همین ترتیب. چطور این را برنامه‌نویسی کنیم؟ با جابه‌جایی درجا:

```
RANDOM-PERMUTATION(A[1..n]) ▷ Fisher-Yates shuffle
```

```
1  for i = 1 to n
2      r = Random(i, n)
3      swap(A[i], A[r])
```

در دور iام، خانه‌های $A[i..n]$ عناصرِ «هنوز انتخاب‌نشده» هستند. یکی از آن‌ها را یکنواخت برمی‌داریم و در خانهٔ i می‌گذاریم. این دقیقاً همان «راه کاراتر» است و $O(n)$ زمان می‌برد.

اثبات · چرا یکنواخت است؟

جایگشت مشخص π را در نظر بگیرید. برای اینکه خروجی π شود، در دور ۱ باید عنصر درست از بین n عنصر انتخاب شود (احتمال $\frac{1}{n}$)، در دور ۲ از بین $n-1$ عنصر باقی‌مانده (احتمال $\frac{1}{n-1}$)، و ... . این انتخاب‌ها مستقل‌اند، پس

$$P[\pi = \text{خروجی}] = \frac{1}{n} \cdot \frac{1}{n-1} \cdots \frac{1}{1} = \frac{1}{n!}.$$

■

خودآزمایی · پرسش اسلاید

اگر $\text{Random}(i, n)$ را با $\text{Random}(1, n)$ جایگزین کنیم، الگوریتم هنوز درست کار می‌کند؟

پاسخ: خیر. در آن صورت الگوریتم n انتخاب مستقل از n گزینه دارد، پس n^n مسیر اجرای هم‌احتمال وجود دارد. برای یکنواخت بودن، هر جایگشت باید از تعداد برابری از این مسیرها حاصل شود، یعنی $n!$ باید n^n را بشمارد. برای $n = 3$: 27 مسیر و 6 جایگشت، اما $27/6$ صحیح نیست. پس بعضی جایگشت‌ها محتمل‌ترند. (بررسی دقیق نشان می‌دهد $\langle 1,3,2\rangle$، $\langle 2,1,3\rangle$ و $\langle 2,3,1\rangle$ هر کدام $\frac{5}{27}$ و بقیه $\frac{4}{27}$ احتمال دارند.)

امید تعداد نقاط ثابت

مثال · چند عدد سر جای خودشان می‌مانند؟

σ جایگشتی تصادفی (یکنواخت) است و X تعداد iهایی است که $\sigma(i) = i$. نشان دهید $E[X] = 1$.

حل: پیشامد A_i را «$\sigma(i) = i$» و X_i را متغیر نشانگرش بگیرید. آنگاه $X = X_1 + \cdots + X_n$. چون $\sigma(i)$ با احتمال برابر هر یک از n مقدار را می‌گیرد، $E[X_i] = P[A_i] = \frac{1}{n}$. با خطی بودن امید:

$$E[X] = E[X_1] + \cdots + E[X_n] = \underbrace{\frac{1}{n} + \cdots + \frac{1}{n}}_{n} = 1.$$

جالب است که جواب به n بستگی ندارد. توجه کنید X_iها مستقل نیستند (اگر $n-1$ عدد سر جایشان باشند، آخری هم هست)، اما خطی بودن امید به استقلال نیازی ندارد.

۳.۶ میانهٔ تقریبی تصادفی

ورودی مجموعهٔ S از n عدد. برای سادگی فرض می‌کنیم همه متمایزند.

رتبه رتبهٔ عدد x در S برابر ۱ به‌اضافهٔ تعداد عناصری از S است که از x کوچک‌ترند. (کوچک‌ترین عنصر رتبهٔ ۱ دارد.)

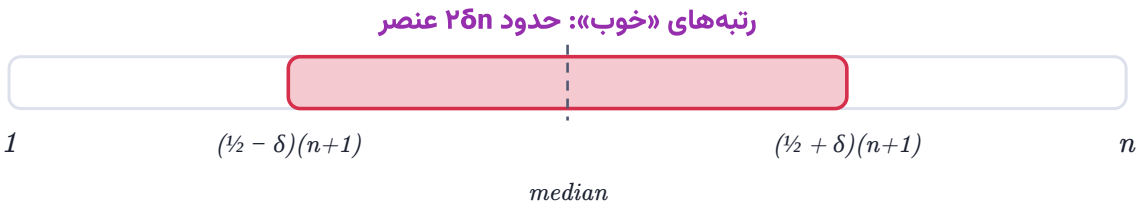
میانه عنصری با رتبهٔ $\lfloor (n+1)/2 \rfloor$.

تعریف · میانهٔ δ-تقریبی

برای ثابت دادهشدهٔ $0 \le \delta \le \frac{1}{2}$، عنصری با رتبهٔ k که

$$\left(\tfrac{1}{2} - \delta\right)(n+1) \;\le\; k \;\le\; \left(\tfrac{1}{2} + \delta\right)(n+1).$$

یعنی عنصری که رتبه‌اش حداکثر $\delta(n+1)$ از وسط فاصله دارد. با $\delta = 0$ همان میانهٔ دقیق و با $\delta = \frac{1}{2}$ هر عنصری قبول است.



شکل ۳-۳ عناصر به ترتیب رتبه. بازهٔ قرمز رتبه‌های قابل قبول است که حدود 2δ کل عناصر را در بر می‌گیرد. شکل برای $\delta = \frac{1}{4}$ کشیده شده است.

مسئله: یک میانهٔ δ-تقریبی گزارش کنید.

الگوریتم ۱: یک تیر در تاریکی

```
APPROX-MEDIAN-1(S, δ)
```

```
1  r = Random(1, n)
2  x* = S[r]
3  k = 1
4  for i = 1 to n                          // compute rank of x*
5      if S[i] < x* then
6          k = k + 1
7  if (1/2 - δ)(n+1) ⩽ k ⩽ (1/2 + δ)(n+1) then
8      return x*
9  else
10     return "error"
```

زمان اجرا: $O(n)$ (یک پیمایش برای محاسبهٔ رتبه). **احتمال موفقیت:** x^* یکنواخت انتخاب شده و رتبه‌اش هم یکنواخت در $\{1, \dots, n\}$ است. تعداد رتبه‌های خوب حدود $2\delta(n+1)$ است، پس:

$$P[\text{موفقیت}] = \frac{\left(\frac{1}{2} + \delta\right)(n+1) - \left(\frac{1}{2} - \delta\right)(n+1)}{n} \approx 2\delta.$$

برای $\delta = \frac{1}{4}$ احتمال موفقیت حدود $\frac{1}{2}$ است. برای $\delta = \frac{1}{10}$ (یعنی دنبال عنصری نزدیک‌تر به میانه هستیم) احتمال موفقیت به حدود $\frac{1}{5}$ کاهش می‌یابد. هر چه دقت بیشتری بخواهیم، شانس کمتری داریم.

الگوریتم ۲: چند بار امتحان کن

```
APPROX-MEDIAN-2(S, δ, c)
```

```
1  j = 1
2  repeat
3      result = APPROX-MEDIAN-1(S, δ)
4      j = j + 1
5  until (result ≠ error) or (j = c + 1)
6  return result
```

حداکثر c بار الگوریتم ۱ را صدا می‌زنیم. **زمان اجرا:** $O(cn)$. **احتمال موفقیت:** الگوریتم فقط وقتی شکست می‌خورد که هر c تلاش مستقل شکست بخورند، و هر تلاش با احتمال $1 - 2\delta$ شکست می‌خورد:

$$P[\text{موفقیت}] = 1 - (1 - 2\delta)^c.$$

مثال · تقویت احتمال موفقیت

- $\delta = \frac{1}{4},\ c = 10$: $1 - (\frac{1}{2})^{10} = 1 - \frac{1}{1024} \approx 99.9\%$.
- $\delta = \frac{1}{10},\ c = 10$: $1 - (0.8)^{10} \approx 1 - 0.107 = 89.3\%$.

احتمال شکست به‌صورت **نمایی** با c کم می‌شود. این ترفند، «تقویت» (amplification) نام دارد: یک الگوریتم با شانس موفقیت متوسط را چند بار تکرار کن تا شانس شکست ناچیز شود.

الگوریتم ۳: تا موفقیت ادامه بده

```
APPROX-MEDIAN-3(S, δ)
```

```
1  repeat
2      result = APPROX-MEDIAN-1(S, δ)
3  until result ≠ error
4  return result
```

احتمال موفقیت: ۱ (هرگز جواب غلط نمی‌دهد). اما زمان اجرا دیگر قطعی نیست و یک متغیر تصادفی است. امید آن:

$$E[\text{زمان}] = E[\text{تعداد فراخوانی‌ها}] \cdot O(n) = \frac{1}{2\delta} \cdot O(n) = O\left(\frac{n}{\delta}\right).$$

چرا؟ · چرا امید تعداد تلاش‌ها $1/(2\delta)$ است؟

اگر هر تلاش مستقلاً با احتمال p موفق شود، تعداد تلاش‌ها تا اولین موفقیت Y یک متغیر **هندسی** است. با احتمال p همان تلاش اول موفق است؛ وگرنه یک تلاش هدر رفته و دوباره در همان وضعیت اولیه‌ایم. پس $E[Y] = p \cdot 1 + (1-p)(1+E[Y])$، که با حل آن $E[Y] = \frac{1}{p}$ به دست می‌آید. با $p = 2\delta$: $E[Y] = \frac{1}{2\delta}$. مثلاً اگر شانس موفقیت $\frac{1}{2}$ باشد، به‌طور میانگین ۲ تلاش لازم است، و اگر $\frac{1}{5}$ باشد، ۵ تلاش.

نکته · «امید زمان اجرا» یعنی چه؟

وقتی از «امید زمان اجرا» حرف می‌زنیم، منظورمان **امید زمان اجرا در بدترین حالت** است: برای ورودی‌های مختلف ممکن است امید زمان متفاوت باشد، و ما بدترینش را گزارش می‌کنیم. (در APPROX-MEDIAN-3 این امید برای همهٔ ورودی‌ها یکی است.)

۳.۷ زمان اجرای الگوریتم‌های قطعی و تصادفی

تعریف · الگوریتم قطعی

$T(X)$ زمان روی ورودی X است (یک عدد ثابت).

- $T_{\text{worst}}(n) = \max_{|X|=n} T(X)$
- $T_{\text{best}}(n) = \min_{|X|=n} T(X)$
- $T_{\text{avg}}(n) = \sum_{|x|=n} T(x) \cdot P[X = x]$

تعریف · الگوریتم تصادفی

$T(X)$ روی ورودی ثابت X یک **متغیر تصادفی** است (به سکه‌ها بستگی دارد).

- $T_{\text{worst-exp}}(n) = \max_{|X|=n} E[T(X)]$

اشتباه رایج · «حالت میانگین» با «امید زمان» فرق دارد

حالت میانگین میانگین‌گیری روی **ورودی‌ها** است و به فرضی دربارهٔ توزیع ورودی نیاز دارد؛ اگر ورودی‌های واقعی بدتر از فرض ما باشند، تحلیل بی‌اعتبار است. **امید زمان** میانگین‌گیری روی **سکه‌های خود الگوریتم** است و برای **هر** ورودی برقرار است. ما توزیع سکه‌ها را کنترل می‌کنیم، نه توزیع ورودی را. به همین دلیل تضمینِ الگوریتم تصادفی محکم‌تر است.

۳.۸ مونت‌کارلو در برابر لاس‌وگاس

تعریف · الگوریتم مونت‌کارلو

تضمین می‌شود در زمان چندجمله‌ای اجرا شود؛ **احتمالاً** جواب درست می‌دهد. مثال: APPROX-MEDIAN-1.

تعریف · الگوریتم لاس‌وگاس

تضمین می‌شود جواب درست بدهد؛ **احتمالاً** در زمان چندجمله‌ای اجرا می‌شود. مثال: APPROX-MEDIAN-3.

	زمان اجرا	درستی
لاس‌وگاس	احتمالاتی	**قطعی**
مونت‌کارلو	**قطعی**	احتمالاتی

نکته: APPROX-MEDIAN-2 ترکیبی است: انتخاب‌های تصادفی هم روی زمان و هم روی درستی اثر دارند. گاهی به چنین الگوریتمی هم مونت‌کارلو می‌گویند (چون زمانش حداکثر $O(cn)$ تضمین شده است).

شهود · نام‌ها از کجا آمده‌اند؟

هر دو نام شهرهای معروف قمارند. یادآوری ساده: در **لاس‌وگاس** همیشه با جواب درست بیرون می‌آیی ولی معلوم نیست چقدر طول بکشد؛ در **مونت‌کارلو** زمانت محدود است ولی ممکن است ببازی.

تکمیلی (فراتر از اسلاید) · تبدیل این دو به هم

- **لاس‌وگاس ← مونت‌کارلو:** الگوریتم لاس‌وگاس را پس از مثلاً دو برابر امید زمانش متوقف کن. طبق نامساوی مارکوف، احتمال اینکه تا آن موقع تمام نشده باشد حداکثر $\frac{1}{2}$ است.
- **مونت‌کارلو ← لاس‌وگاس:** اگر بتوانیم درستی جواب را سریع بررسی کنیم، الگوریتم مونت‌کارلو را تا رسیدن به جواب درست تکرار می‌کنیم. دقیقاً همان کاری که APPROX-MEDIAN-3 کرد (بررسی رتبه در خود الگوریتم ۱ انجام می‌شد).

۳.۹ مسئلهٔ استخدام

تعریف · صورت مسئله

برای پروژه‌ای به بهترین دستیار ممکن نیاز دارید. یک بنگاه کاریابی قول داده هر روز یک متقاضی برای مصاحبه بفرستد. چون واقعاً بهترین دستیار را می‌خواهید، این راهبرد را انتخاب می‌کنید: **هر وقت متقاضی‌ای که مصاحبه کردید از دستیار فعلی بهتر بود، دستیار فعلی را اخراج و متقاضی جدید را استخدام کنید.** هر تعویض دستیار هزینهٔ f دارد. هزینهٔ کل چقدر است؟

```
HIRE-ASSISTANT(candidates 1..n)
```

```
1  current = nil
2  for i = 1 to n
3      interview candidate i
4      if candidate i is better than current then
5          current = candidate i              // pay f
```

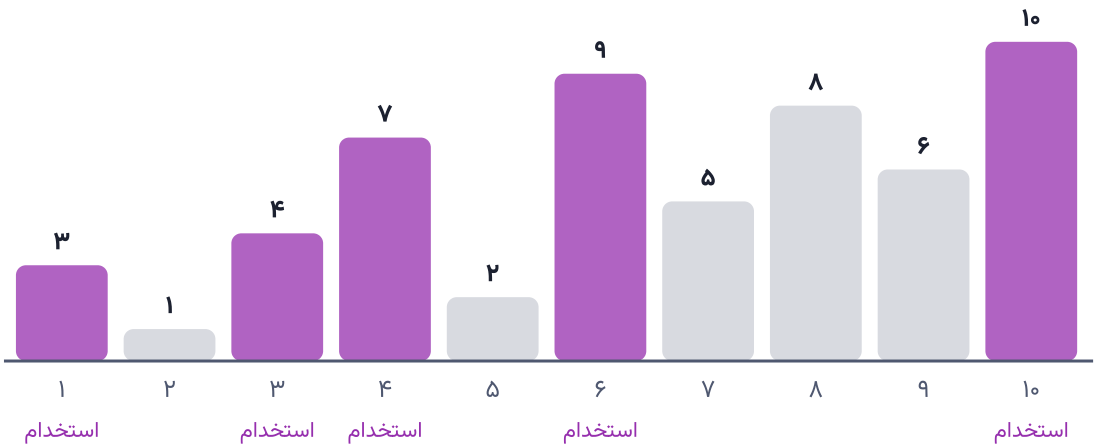
بدترین حالت: اگر متقاضی‌ها به ترتیب **صعودی** کیفیت بیایند، هر کدام از قبلی بهتر است و هر n نفر استخدام می‌شوند: هزینهٔ $f \cdot n$. اگر بنگاه کاریابی بدجنس باشد، دقیقاً همین ترتیب را می‌فرستد!

الگوریتم تصادفی

```
RANDOMIZED-HIRE-ASSISTANT(candidates 1..n)
```

```
1  compute a random permutation of the candidates
2  current = nil
3  for i = 1 to n
4      interview candidate i
5      if candidate i is better than current then
6          current = candidate i              // pay f
```

تنها تغییر: پیش از شروع، ترتیب متقاضی‌ها را با یک جایگشت تصادفی به هم می‌ریزیم. هزینه در بدترین حالت هنوز $f \cdot n$ است (ممکن است جایگشت تصادفی بدشانسانه صعودی دربیاید)، اما این اتفاق احتمال $\frac{1}{n!}$ دارد. **امید هزینه** بسیار کمتر است.



شکل ۳-۴ یک ترتیب تصادفی از ۱۰ متقاضی (ارتفاع = کیفیت). فقط «رکوردشکن‌ها» (ستون‌های رنگی) استخدام می‌شوند: ۵ استخدام به‌جای ۱۰.

اثبات · تحلیل با متغیر نشانگر

X_i را متغیر نشانگری بگیرید که ۱ است اگر و فقط اگر متقاضی iام از همهٔ متقاضی‌های $1, 2, \dots, i-1$ بهتر باشد (یعنی استخدام شود).

احتمال: در جایگشت تصادفی، i متقاضی اول یک زیرمجموعهٔ تصادفی‌اند که ترتیبشان هم تصادفی است. بهترینِ این i نفر با احتمال برابر در هر یک از i جایگاه قرار دارد، پس احتمال اینکه دقیقاً در جایگاه iام (آخرین) باشد $\frac{1}{i}$ است: $P[X_i = 1] = \frac{1}{i}$.

$$E[\text{هزینه}] = E\Big[\sum_{i} f \cdot X_i\Big] = f \sum_{i=1}^{n} E[X_i] = f \sum_{i=1}^{n} P[X_i = 1] = f \sum_{i=1}^{n} \frac{1}{i} \approx f \ln n.$$

∎

چرا؟ · چرا $\sum 1/i \approx \ln n$؟

جمع $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$ را **عدد هارمونیک** می‌نامند. با مقایسه با انتگرال (هر جملهٔ $\frac{1}{i}$ مساحت مستطیلی است که زیر یا روی منحنی $\frac{1}{x}$ قرار دارد):

$$\ln(n+1) = \int_{1}^{n+1} \frac{dx}{x} \;\le\; H_n \;\le\; 1 + \int_{1}^{n} \frac{dx}{x} = 1 + \ln n.$$

پس $H_n = \ln n + O(1)$. برای ۱۰۰۰ متقاضی، به‌جای ۱۰۰۰ استخدام فقط حدود ۷٫۵ استخدام انتظار داریم!

۳.۱۰ انتخاب تصادفی

تعریف · مسئلهٔ انتخاب

مجموعهٔ S از n عنصر متمایز و عدد صحیح i داده شده؛ عنصری با رتبهٔ i را در S پیدا کنید. ($i=1$: کمینه، $i=n$: بیشینه، $i=\lfloor (n+1)/2 \rfloor$: میانه.)

راه بدیهی: مرتب کن و خانهٔ iام را بردار: $O(n \log n)$. می‌خواهیم در $O(n)$ انجامش دهیم.

```
SELECTION(S, i) ▷ randomized (quickselect)
1  if |S| = 1 then return the only element of S
2  choose a splitter a_j in S uniformly at random
3  foreach a in S
4      if a < a_j then put a in S-
5      else if a > a_j then put a in S+
6  k = |S-|
7  if k = i-1 then return a_j                 // the splitter has rank i
8  else if k > i-1 then return SELECTION(S-, i)
9  else return SELECTION(S+, i-k-1)
```

ایده: یک عنصر تصادفی a_j («جداکننده» یا «محور») انتخاب کن و بقیه را به کوچک‌ترها (S^-) و بزرگ‌ترها (S^+) تقسیم کن. رتبهٔ a_j دقیقاً $k+1$ است. اگر همان i بود، تمام؛ اگر i کوچک‌تر بود جواب در S^- است؛ وگرنه جواب در S^+ است، ولی چون $k+1$ عنصر کوچک‌تر را کنار گذاشته‌ایم، در S^+ دنبال رتبهٔ $i-k-1$ می‌گردیم. برخلاف کوئیک‌سورت، فقط روی **یک** طرف بازگشت می‌کنیم.

تحلیل

- **بهترین حالت:** اگر جداکننده همیشه میانه باشد، $T(n)=O(n)+T(n/2)$، پس $\Theta(n)$ مقایسه.
- **بدترین حالت:** اگر جداکننده همیشه کوچک‌ترین عنصر باشد، $T(n)=O(n)+T(n-1)$، پس $\Theta(n^2)$ مقایسه.

تصادفی‌سازی با انتخاب تصادفی جداکننده، از بدترین حالت محافظت می‌کند.

شهود · جداکنندهٔ «به‌اندازهٔ کافی خوب»

لازم نیست جداکننده دقیقاً میانه باشد. اگر همیشه عنصری را انتخاب کنیم که از ۲۵٪ عناصر بزرگ‌تر و از ۲۵٪ عناصر کوچک‌تر است، اندازهٔ مسئله در هر گام حداقل به $\frac{3}{4}$ کاهش می‌یابد: $T(n) \le cn + T(3n/4)$، که با سری هندسی $cn(1+\frac{3}{4}+\frac{9}{16}+\dots) = 4cn = O(n)$ می‌شود. و نیمی از عناصر (رتبه‌های بین $n/4$ و $3n/4$) چنین ویژگی‌ای دارند!

تحلیل دقیق: هر عنصر با احتمال $\frac{1}{n}$ جداکننده است. اگر جداکننده رتبهٔ j داشته باشد، در بدترین حالت روی طرف بزرگ‌تر، با اندازهٔ $\max(j-1, n-j)$، بازگشت می‌کنیم:

$$T_{\exp}(n) \le O(n) + \sum_{j=1}^{n} \frac{1}{n}\, T_{\exp}\big(\max(j-1, n-j)\big).$$

می‌توان نشان داد $T_{\exp}(n) = O(n)$. اما یک روش ساده‌تر هم هست:

اثبات · روش ساده‌تر

با احتمال $\frac{1}{2}$ جداکننده «خوب» است (رتبه‌اش بین $n/4$ و $3n/4$)، و در این صورت روی حداکثر $\frac{3n}{4}$ عنصر بازگشت می‌کنیم. با احتمال $\frac{1}{2}$ «بد» است و در بدترین حالت روی $n-1$ عنصر بازگشت می‌کنیم. چون T صعودی است:

$$T_{\exp}(n) \le cn + \tfrac{1}{2}\,T_{\exp}(3n/4) + \tfrac{1}{2}\,T_{\exp}(n-1) \;\le\; cn + \tfrac{1}{2}\,T_{\exp}(3n/4) + \tfrac{1}{2}\,T_{\exp}(n).$$

$\frac{1}{2}T_{\exp}(n)$ را به سمت چپ ببرید و در ۲ ضرب کنید:

$$T_{\exp}(n) \le 2cn + T_{\exp}(3n/4) \le 2cn\left(1 + \tfrac{3}{4} + \left(\tfrac{3}{4}\right)^2 + \dots\right) = 2cn \cdot 4 = 8cn = O(n).$$

(برای دقت کامل، همین را با استقرا روی فرض $T_{\exp}(n) \le 8cn$ ثابت کنید: $cn + \frac{1}{2}\cdot 8c\frac{3n}{4} + \frac{1}{2}\cdot 8cn = cn + 3cn + 4cn = 8cn$.)

■

۳.۱۱ انتخاب قطعی: میانهٔ میانه‌ها

آیا می‌توان بدون تصادف هم در $O(n)$ انتخاب کرد؟ بله! ترفند این است که جداکنندهٔ خوب را به‌جای شانس، با یک بازگشت دیگر **تضمین** کنیم.

```
SELECT(S, i) ▷ deterministic, median of medians
```

```
1  if |S| = 1 then return the only element of S
2  divide S into groups of 5:  S[5t+1..5t+5]
3  foreach group t
4      B[t] = median of that group                  // O(1) per group
5  x = SELECT(B, ceil(|B| / 2))                     // median of the medians
6  foreach a in S
7      if a < x then put a in S-
8      else if a > x then put a in S+
9  k = |S-|
10 if k = i-1 then return x
11 else if k > i-1 then return SELECT(S-, i)
12 else return SELECT(S+, i-k-1)
```

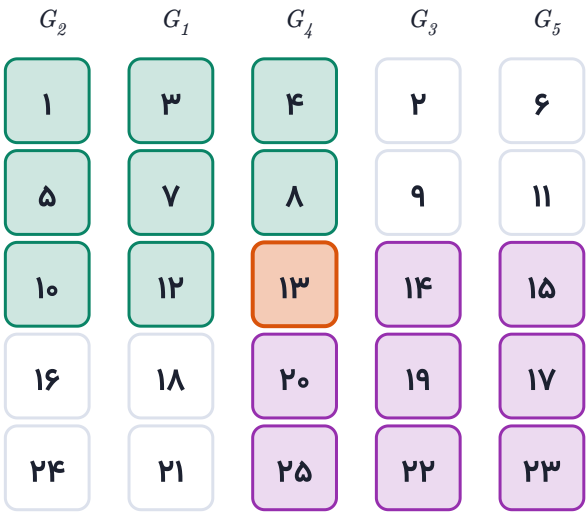
تنها تفاوت با نسخهٔ تصادفی، روش انتخاب جداکننده است: x میانهٔ میانه‌های گروه‌های پنج‌تایی است. ادعا: x همیشه جداکنندهٔ «به‌اندازهٔ کافی خوب» است.

اثبات با تصویر

G_1	G_2	G_3	G_4	G_5
۱۲	۵	۲۲	۸	۱۷
۳	۲۴	۹	۲۵	۶
۲۱	۱	۱۴	۱۳	۲۳
۷	۱۶	۲	۲۰	۱۱
۱۸	۱۰	۱۹	۴	۱۵

شکل ۳-۵۲۵ عدد در ۵ گروه پنج‌تایی ($m = \lceil n/5 \rceil = 5$ گروه).

حالا در ذهنمان (الگوریتم واقعاً این کار را نمی‌کند) هر گروه را مرتب می‌کنیم تا میانه‌اش در ردیف وسط قرار گیرد، و سپس گروه‌ها را بر اساس میانه‌شان از چپ به راست مرتب می‌کنیم. میانه‌ها $12, 10, 14, 13, 15$ هستند و میانهٔ آن‌ها $x = 13$ است: محور ما.



شکل ۳-۶ نارنجی: محور (۱۳). سبز: عناصری که **قطعاً** از محور کوچک‌ترند (بالا و چپِ محور). بنفش: عناصری که **قطعاً** بزرگ‌ترند (پایین و راستِ محور).

چرا؟ · چرا ناحیهٔ سبز حتماً از محور کوچک‌تر است؟

ستون‌های سمت چپِ ستونِ محور، میانه‌هایی کوچک‌تر از x دارند. در هر یک از این ستون‌ها، سه عنصرِ بالایی (میانه و دو عنصر بالای آن) از میانهٔ آن ستون کوچک‌تر یا مساوی‌اند، پس از x کوچک‌ترند. در ستون خود محور هم دو عنصر بالای x از آن کوچک‌ترند. این‌ها ناحیهٔ «بالا-چپ» را می‌سازند. ناحیهٔ «پایین-راست» به همین ترتیب بزرگ‌تر است.

چند عنصر قطعاً از محور کوچک‌ترند؟ حدود نیمی از m ستون ($\lceil m/2 \rceil$ ستون، شامل ستون محور) سمت چپ قرار دارند و هر کدام ۳ عنصر در ناحیهٔ سبز دارند؛ البته یکی از این ستون‌ها خود ستون محور است (که محور را نباید شمرد) و ممکن است یکی هم گروه «باقی‌مانده» با کمتر از ۵ عنصر باشد. پس دست‌کم

$$3\left(\left\lceil \frac{m}{2} \right\rceil - 2\right)$$

عنصر از محور کوچک‌ترند. در نتیجه حداکثر این تعداد عنصر از محور بزرگ‌ترند (با $m = \lceil n/5 \rceil \geq n/5$):

$$|S^+| \leq n - 1 - 3\left(\left\lceil \frac{m}{2} \right\rceil - 2\right) \leq n + 5 - \frac{3m}{2} \leq n + 5 - \frac{3n}{10} = \frac{7n}{10} + 5.$$

لم · اندازهٔ دو طرف

اگر S^- و S^+ مانند الگوریتم SELECT بالا باشند، آنگاه $|S^-| \leq \frac{7n}{10} + 5$ و $|S^+| \leq \frac{7n}{10} + 5$. (اثبات طرف دیگر دقیقاً مشابه است.)

زمان اجرا: گروه‌بندی و یافتن میانهٔ گروه‌ها $O(n)$، بازگشت برای میانهٔ میانه‌ها $T(n/5)$، تقسیم $O(n)$، و بازگشت روی یک طرف حداکثر $T(7n/10 + 5)$:

$$T(n) \le T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10} + 5\right) + O(n) \implies T(n) = O(n).$$

این دقیقاً «مثال سوم» روش حدس و استقرا در فصل ۲ است: $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$.

چرا؟ · چرا گروه‌های پنج‌تایی؟ سه‌تایی نمی‌شد؟

با گروه‌های سه‌تایی، تقریباً $\frac{n}{3}$ عنصر قطعاً کوچک‌ترند، پس بازگشت دوم روی حداکثر حدود $\frac{2n}{3}$ عنصر است و رابطه $T(n) \le T(n/3) + T(2n/3) + O(n)$ می‌شود. حالا $\frac{1}{3} + \frac{2}{3} = 1$ و جواب $O(n \log n)$ است: دیگر خطی نیست! با گروه‌های پنج‌تایی جمع کسرها $\frac{9}{10}$ است که اکیداً کمتر از ۱ است. گروه‌های هفت‌تایی و بزرگ‌تر هم کار می‌کنند، اما ثابت‌ها بدتر می‌شوند.

۳.۱۲ کوئیک‌سورت تصادفی

```
RANDOMIZED-QUICKSORT(S)
```

```
1  if |S| = 0 then return
2  choose a splitter a_i in S uniformly at random
3  foreach a in S
4      if a < a_i then put a in S-
5      else if a > a_i then put a in S+
6  RANDOMIZED-QUICKSORT(S-)
7  output a_i
8  RANDOMIZED-QUICKSORT(S+)
```

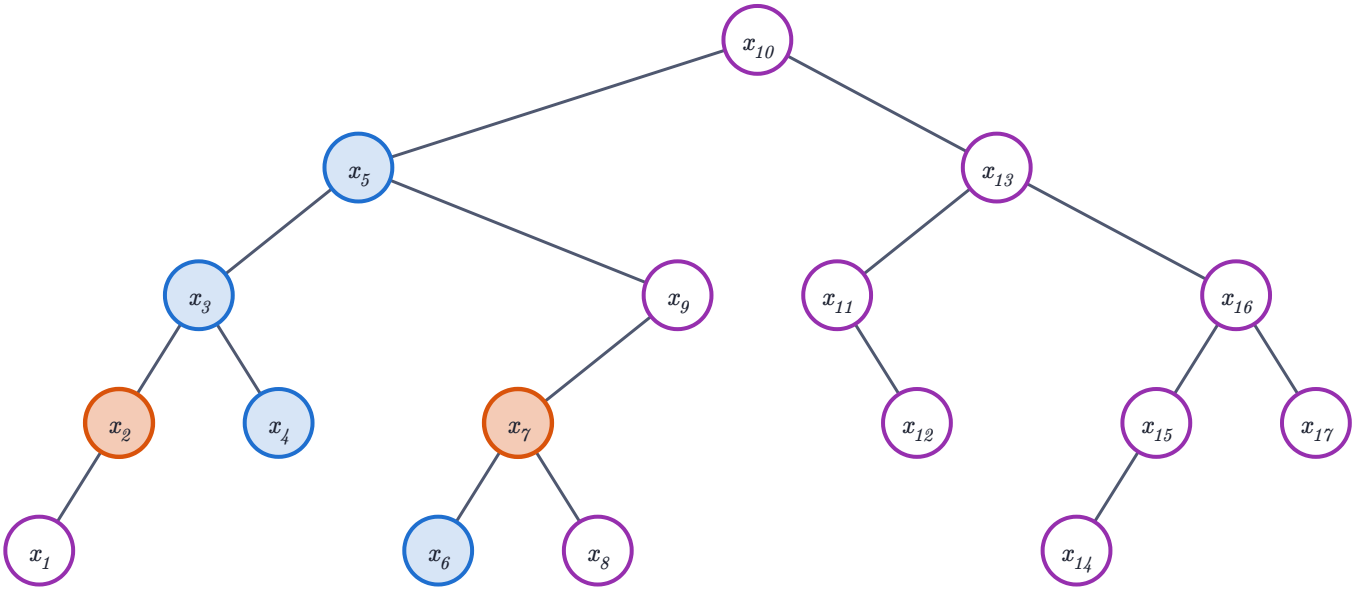
همان ایدهٔ انتخاب تصادفی، با این تفاوت که روی **هر دو** طرف بازگشت می‌کنیم.

- **بهترین حالت:** جداکننده همیشه میانه: $T(n) = 2T(n/2) + O(n)$، پس $\Theta(n \log n)$ مقایسه.
- **بدترین حالت:** جداکننده همیشه کوچک‌ترین: $T(n) = T(n-1) + O(n)$، پس $\Theta(n^2)$ مقایسه.
- **شهود:** اگر همیشه جداکننده‌ای بین صدک ۲۵ و ۷۵ انتخاب کنیم، $T(n) \le T(n/4) + T(3n/4) + O(n)$ که طبق قضیهٔ دو شاخه‌ای فصل ۲، $\Theta(n \log n)$ است.

نمادگذاری: عناصر را به ترتیب مرتب‌شده نام‌گذاری می‌کنیم: $x_1 < x_2 < \cdots < x_n$. (توجه: x_i یعنی iامین کوچک‌ترین عنصر، نه عنصر خانهٔ iام ورودی.)

نمایش جداکننده‌ها با درخت جست‌وجوی دودویی

اجرای کوئیک‌سورت را می‌توان با یک درخت نشان داد: ریشه اولین جداکننده است؛ زیردرخت چپ اجرای بازگشتی روی S^- و زیردرخت راست اجرای روی S^+. این درخت یک **درخت جست‌وجوی دودویی** است (همهٔ عناصر زیردرخت چپ کوچک‌ترند).



شکل ۳-۷ درخت جداکننده‌ها برای ۱۷ عنصر که اولین جداکنندهٔ تصادفی x_{10} بوده است. عناصر به ترتیب از چپ به راست چیده شده‌اند. x_2 و x_7 (نارنجی) هرگز با هم مقایسه نمی‌شوند: عناصر آبی (x_3 تا x_6) بین آن دو قرار دارند و یکی از آن‌ها، x_5، پیش از هر دو به‌عنوان جداکننده انتخاب شده و آن‌ها را از هم جدا کرده است.

نکته · مشاهدهٔ کلیدی

هر عنصر فقط با **اجداد و نوادگانش** در این درخت مقایسه می‌شود: وقتی a جداکننده است، با همهٔ عناصر زیردرختش مقایسه می‌شود و پس از آن دیگر در هیچ مقایسه‌ای شرکت نمی‌کند. مثلاً x_2 و x_7 فقط وقتی مقایسه می‌شوند که پایین‌ترین جد مشترکشان (lca) خود x_2 یا x_7 باشد؛ اگر جد مشترک x_3, x_4, x_5 یا x_6 باشد، مقایسه نمی‌شوند.

قضیه · احتمال مقایسه

$$P\big[x_i \text{ و } x_j \text{ مقایسه می‌شوند}\big] = \frac{2}{j-i+1} \qquad (i<j).$$

- مجموعهٔ $S_{ij} = \{x_i, x_{i+1}, \dots, x_j\}$ را در نظر بگیرید ($j-i+1$ عنصر).
- تا وقتی جداکنندهٔ انتخاب‌شده عضو S_{ij} نباشد، همهٔ اعضای S_{ij} کنار هم و در یک زیرمسئله می‌مانند (یا همه از جداکننده کوچک‌ترند یا همه بزرگ‌تر) و هیچ مقایسه‌ای بین x_i و x_j انجام نمی‌شود.
- این ادامه دارد تا وقتی که بالاخره یکی از عناصر S_{ij} به‌عنوان جداکننده انتخاب شود.
- اگر آن عنصر x_i یا x_j باشد، این دو با هم مقایسه می‌شوند. وگرنه (یک x_k با $i<k<j$)، x_i به S^- و x_j به S^+ می‌رود و **هرگز** مقایسه نمی‌شوند.
- چون جداکننده یکنواخت انتخاب می‌شود، هر عضو S_{ij} احتمال برابر دارد که اولین جداکننده از S_{ij} باشد. ۲ حالت از $j-i+1$ حالت مطلوب است، پس احتمال $\frac{2}{j-i+1}$ است.

■

قضیه · امید تعداد مقایسه‌های کوئیک‌سورت تصادفی

امید تعداد مقایسه‌ها $O(n \log n)$ است.

$X_{ij} = 1$ اگر x_i و x_j مقایسه شوند و ۰ در غیر این صورت. هر جفت حداکثر یک بار مقایسه می‌شود (پس از مقایسه، جداکننده کنار می‌رود)، پس $X = \sum_{i<j} X_{ij}$ تعداد کل مقایسه‌هاست و

$$E[X] = \sum_{1 \le i < j \le n} E[X_{ij}] = \sum_{1 \le i < j \le n} \frac{2}{j - i + 1} = 2 \sum_{i=1}^{n} \sum_{d=2}^{n-i+1} \frac{1}{d} \;\le\; 2n \sum_{d=2}^{n} \frac{1}{d} \;\le\; 2n \int_{1}^{n} \frac{dx}{x} = 2n \ln n.$$

(در جمع وسط، $d = j - i + 1$ را جایگزین کردیم؛ برای هر i، d از ۲ تا $n - i + 1$ تغییر می‌کند و ما آن را با n کران زدیم.) پس $E[X] \le 2n \ln n \approx 1.39\, n \log_2 n = O(n \log n)$.

■

نکته · تمرکز حول امید

امید کم به‌تنهایی کافی نیست؛ باید بدانیم احتمال اینکه زمان خیلی بیشتر از امید شود کم است. با محاسبهٔ واریانس و **نامساوی چبیشف** ($P[|X - \mu| \ge k\sigma] \le \frac{1}{k^2}$) می‌توان نشان داد مثلاً برای n = یک میلیون، احتمال اینکه کوئیک‌سورت تصادفی کمتر از $4n \ln n$ مقایسه انجام دهد دست‌کم 99.94% است.

رویکرد دیگر: محور با میانهٔ تقریبی

به‌جای اینکه محور را یکنواخت تصادفی برداریم، می‌توانیم اصرار کنیم که محور «خوب» باشد. راه ساده: با APPROX-MEDIAN-3 یک میانهٔ $\frac{1}{4}$-تقریبی پیدا کنیم (امید زمان $O(n/\delta) = O(n)$). چنین محوری هر دو طرف را حداکثر $\frac{3}{4}n$ می‌کند. امید زمان:

$$E[\text{زمان APPROX-MEDIAN-3}] + E[\text{فراخوانی‌های بازگشتی}] = O(n) + E[\text{فراخوانی‌های بازگشتی}]$$

$$T_{\exp}(n) \le O(n) + T_{\exp}(3n/4) + T_{\exp}(n/4) \quad \Longrightarrow \quad T_{\exp}(n) = O(n \log n).$$

(دوباره قضیهٔ دو شاخه‌ای فصل ۲ با $\frac{3}{4} + \frac{1}{4} = 1$.)

جمع‌بندی · فصل ۳ در یک نگاه

- متغیر تصادفی یک تابع است؛ امید ریاضی میانگین وزن‌دار آن.
- خطی بودن امید بدون نیاز به استقلال؛ با متغیر نشانگر: $E[I_A] = P[A]$.
- مونت‌کارلو: زمان قطعی، درستی احتمالاتی. لاس‌وگاس: درستی قطعی، زمان احتمالاتی. تکرار، احتمال شکست را نمایی کم می‌کند.
- استخدام تصادفی: امید هزینه $f H_n \approx f \ln n$ به‌جای fn.
- انتخاب: تصادفی با امید $O(n)$؛ قطعی با میانهٔ میانه‌ها $O(n)$ در بدترین حالت.
- کوئیک‌سورت تصادفی: $P[\text{مقایسه } x_i, x_j] = \frac{2}{j-i+1}$، و امید مقایسه‌ها $\leq 2n \ln n$.

۳.۱۳ خودآزمایی فصل ۳

خودآزمایی

1. سکه‌ای سالم را ۱۰ بار پرتاب می‌کنیم. امید تعداد دفعاتی که دو پرتاب متوالی هر دو شیر باشند چقدر است؟

پاسخ: برای $i = 1..9$، X_i نشانگر «پرتاب i و $i+1$ هر دو شیر» است: $E[X_i] = \frac{1}{4}$. با خطی بودن امید: $9 \cdot \frac{1}{4} = 2.25$. ($X_i$ها مستقل نیستند، ولی مهم نیست.)

2. در کوئیک‌سورت تصادفی، احتمال اینکه کوچک‌ترین و بزرگ‌ترین عنصر با هم مقایسه شوند چقدر است؟

پاسخ: $i = 1, j = n$: احتمال $\frac{2}{n}$. فقط وقتی مقایسه می‌شوند که یکی از این دو، اولین جداکنندهٔ کل آرایه باشد.

3. در APPROX-MEDIAN-2 با $\delta = \frac{1}{4}$، حداقل چند تکرار لازم است تا احتمال شکست کمتر از یک در میلیون شود؟

پاسخ: $(\frac{1}{2})^c < 10^{-6}$ یعنی $c > 6 \log_2 10 \approx 19.9$؛ پس $c = 20$.

4. چرا الگوریتم انتخاب قطعی با گروه‌های سه‌تایی خطی نیست؟

پاسخ: رابطه $T(n) \leq T(n/3) + T(2n/3) + O(n)$ می‌شود که جمع کسرهایش ۱ است و جواب $\Theta(n \log n)$ دارد.

5. آیا الگوریتم کوئیک‌سورت تصادفی مونت‌کارلو است یا لاس‌وگاس؟

پاسخ: لاس‌وگاس: خروجی همیشه درست مرتب است و فقط زمان اجرا تصادفی است.

فصل ۴

تحلیل سرشکنی

گاهی یک عمل گران است، ولی گران بودنش آن‌قدر نادر است که اگر هزینه را میان همهٔ عمل‌ها «سرشکن» کنیم، سهم هر عمل ناچیز می‌شود. این فصل سه روش برای اثبات دقیق همین شهود می‌آموزد.

پشته با multipop | شمارندهٔ دودویی | روش انبوهه | روش حساب‌داری | روش پتانسیل

۴.۱ سه نگاه به هزینه

تا این‌جا دو نوع تحلیل دیدیم:

- **تحلیل بدترین حالت:** بیشترین هزینهٔ **یک** عمل روی بدترین ورودی.
- **تحلیل حالت میانگین:** امید هزینه، با فرضی دربارهٔ توزیع ورودی.

تحلیل سرشکنی (amortized analysis) نوع سومی است که برای **داده‌ساختارها** طراحی شده است:

تعریف · تحلیل سرشکنی

محیط: یک داده‌ساختار و **دنباله‌ای از عملیات** روی آن.

هزینهٔ سرشکن‌شدهٔ هر عمل: متوسط هزینهٔ آن عمل در **بدترین** دنباله. به زبان دقیق: اگر $T(n)$ بیشترین هزینهٔ کل روی همهٔ دنباله‌های n عملی باشد، هزینهٔ سرشکن‌شده $T(n)/n$ است.

شهود · قلک

فرض کنید هر روز ناهار ۱ واحد خرج دارد، اما هر چند وقت یک بار باید یک کفش تازه بخرید که گران است. اگر هر روز ۲ واحد کنار بگذارید (۱ برای ناهار، ۱ در قلک)، روز خرید کفش قلک پر است و هیچ‌وقت کم نمی‌آورید. «هزینهٔ سرشکن‌شدهٔ هر روز» ۲ است، با اینکه بعضی روزها خرج واقعی خیلی بیشتر است. تحلیل سرشکنی دقیقاً همین حساب‌وکتاب است.

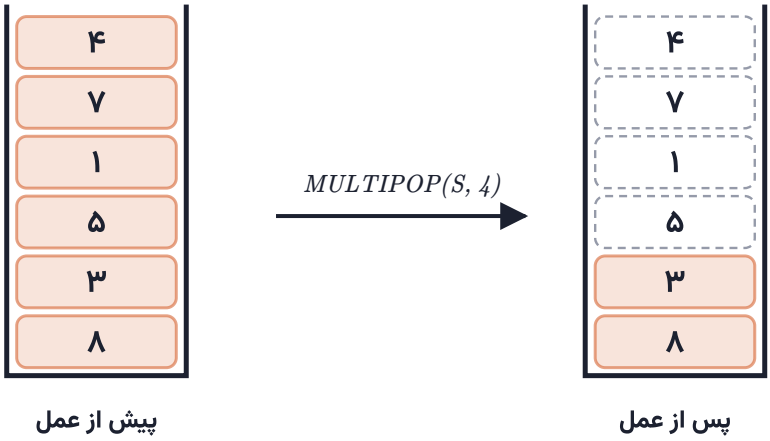
اشتباه رایج · سرشکن ≠ میانگین

در تحلیل سرشکنی **هیچ احتمالی** در کار نیست. ما میانگین را روی عملیاتِ **یک** دنباله می‌گیریم و آن دنباله را بدترین دنبالهٔ ممکن فرض می‌کنیم. پس نتیجه یک **تضمین قطعی** است: «هر دنبالهٔ n عملی حداکثر فلان‌قدر هزینه دارد». اما یک عمل **تکی** ممکن است هنوز گران باشد؛ برای سیستم‌های بی‌درنگ که هر پاسخ باید سریع باشد، این تفاوت مهم است.

۴.۲ مثال ۱: پشته با عمل multipop

پشته علاوه بر PUSH(S, x) و POP(S)، عمل سومی هم دارد: MULTIPOP(S, k) که k عنصر بالای پشته را برمی‌دارد (یا اگر کمتر از k عنصر بود، همه را).

```
MULTIPOP(S, k)
1  while not STACK-EMPTY(S) and k ≠ 0
2      POP(S)
3      k = k - 1
```



شکل ۴-۱ MULTIPOP$(S, 4)$ چهار عنصر بالایی را برمی‌دارد و ۴ واحد هزینه دارد.

هزینه‌ها در بدترین حالت

PUSH و POP $\Theta(1)$

MULTIPOP $\Theta(\min\{k, \text{SIZE}(S)\})$؛ در بدترین حالت $\Theta(n)$ چون پشته ممکن است n عنصر داشته باشد.

پس اگر دنباله‌ای از n عمل داشته باشیم و هر کدام در بدترین حالت $O(n)$ باشد، جمع کل در بدترین حالت $O(n^2)$ به دست می‌آید. این کران درست است، اما **شُل** است: نمی‌شود n عمل MULTIPOP داشت که هر کدام n عنصر بردارند، چون عنصری که برداشته شد دیگر نیست!

جمع‌بندی · هدف

نشان می‌دهیم هزینهٔ سرشکن‌شدهٔ هر عمل $O(1)$ است؛ یعنی هر دنبالهٔ n عملی روی پشتهٔ خالی، $O(n)$ هزینه دارد.

۴.۳ مثال ۲: شمارندهٔ دودویی

یک شمارندهٔ دودویی k بیتی با آرایهٔ $A[0..k-1]$ داریم ($A[0]$ کم‌ارزش‌ترین بیت است). مقدار شمارنده $\sum_{i=0}^{k-1} A[i] \cdot 2^i$ است و از صفر شروع می‌شود. تنها عمل، **افزایش** یک واحد است:

```
INCREMENT(A)
```

```
1  i = 0
2  while i < length[A] and A[i] = 1
3      A[i] = 0                    // carry: flip 1 → 0
4      i = i + 1
5  if i < length[A]
6      then A[i] = 1               // flip 0 → 1
```

همان جمع دبستانی با ۱ است: از راست، یک‌ها را صفر می‌کنیم (رقم نقلی جلو می‌رود) تا به اولین صفر برسیم و آن را یک کنیم. **هزینه** = تعداد بیت‌هایی که عوض می‌شوند.

هزینهٔ شمارنده در بدترین حالت

هر عمل افزایش در بدترین حالت $O(k)$ است (مثلاً از $0111\ldots1$ به $1000\ldots0$ همهٔ k بیت عوض می‌شوند). پس n عمل افزایش $O(nk)$. اما این‌جا هم کران شل است:

مقدار	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	هزینهٔ این افزایش	هزینهٔ کل	۲n
۰	0	0	0	0	0	0	0	0	—	۰	۰
۱	0	0	0	0	0	0	0	1	۱	۱	۲
۲	0	0	0	0	0	0	1	0	۲	۳	۴
۳	0	0	0	0	0	0	1	1	۱	۴	۶
۴	0	0	0	0	0	1	0	0	۳	۷	۸
۵	0	0	0	0	0	1	0	1	۱	۸	۱۰
۶	0	0	0	0	0	1	1	0	۲	۱۰	۱۲
۷	0	0	0	0	0	1	1	1	۱	۱۱	۱۴
۸	0	0	0	0	1	0	0	0	۴	۱۵	۱۶
۹	0	0	0	0	1	0	0	1	۱	۱۶	۱۸
۱۰	0	0	0	0	1	0	1	0	۲	۱۸	۲۰
۱۱	0	0	0	0	1	0	1	1	۱	۱۹	۲۲
۱۲	0	0	0	0	1	1	0	0	۳	۲۲	۲۴
۱۳	0	0	0	0	1	1	0	1	۱	۲۳	۲۶
۱۴	0	0	0	0	1	1	1	0	۲	۲۵	۲۸
۱۵	0	0	0	0	1	1	1	1	۱	۲۶	۳۰
۱۶	0	0	0	1	0	0	0	0	۵	۳۱	۳۲

جدول بالا ۱۶ افزایش اول یک شمارندهٔ ۸ بیتی را نشان می‌دهد. بیت‌های زرد در همان افزایش عوض شده‌اند. افزایش‌های گران (مثل ۷ ← ۸ با ۴ تغییر، یا ۱۵ ← ۱۶ با ۵ تغییر) نادرند و **هزینهٔ کل همیشه کمتر از $2n$ است** (ستون آخر را با ستون «هزینهٔ کل» مقایسه کنید).

جمع‌بندی · هدف

نشان می‌دهیم هزینهٔ هر عمل افزایش به‌صورت سرشکن‌شده $O(1)$ است.

۴.۴ روش‌های تحلیل سرشکنی

روش	ایده
انبوهه (aggregate)	هزینهٔ کل n عمل را مستقیم حساب کن و بر n تقسیم کن. همهٔ عمل‌ها هزینهٔ سرشکن یکسان می‌گیرند.
حساب‌داری (accounting)	برای هر عمل مبلغی «پرداخت» کن؛ مازاد را به‌صورت اعتبار روی اشیای داده‌ساختار ذخیره کن تا عمل‌های گران بعدی را بپردازد.
تابع پتانسیل (potential)	حالت کلی‌تر روش قبل: به‌جای اعتبار روی اشیا، یک «انرژی» برای کل داده‌ساختار تعریف کن.

۴.۵ روش انبوهه

ساده‌ترین روش: یک کران بالا $T(n)$ برای هزینهٔ کل هر دنبالهٔ n عملی پیدا کن. آنگاه هزینهٔ سرشکن هر عمل $T(n)/n$ است.

تحلیل پشته با روش انبوهه

اگر n عمل (PUSH، POP یا MULTIPOP) روی پشتهٔ ابتدا خالی داشته باشیم:

- حداکثر تعداد عناصر پشته n است (حداکثر n PUSH).
- هر عنصر داخل پشته دقیقاً **یک بار** PUSH و حداکثر **یک بار** POP می‌شود.
- یک عنصر یا مستقیماً POP می‌شود یا در یک MULTIPOP.
- هر عمل PUSH و POP هزینهٔ $O(1)$ دارد.

پس تعداد کل POPها (شامل POPهای داخل MULTIPOP) حداکثر برابر تعداد PUSHهاست که حداکثر n است. **جمع هزینه‌ها حداکثر** $2n$ است و هزینهٔ سرشکن‌شدهٔ هر عمل $\frac{2n}{n} = O(1)$.

■

چرا؟ · ترفند اصلی چه بود؟

به‌جای اینکه بپرسیم «هر **عمل** چقدر خرج دارد؟» پرسیدیم «هر **عنصر** در طول عمرش چقدر خرج ایجاد می‌کند؟». هر عنصر حداکثر ۲ واحد (یک ورود، یک خروج). تغییر زاویهٔ شمارش، قلب روش انبوهه است.

تحلیل شمارنده با روش انبوهه

دوباره زاویهٔ شمارش را عوض می‌کنیم: به‌جای شمردن تغییرات در هر افزایش، تغییرات **هر بیت** را در کل دنباله می‌شماریم.

- هر افزایش حداکثر **یک** بیت را ۱ می‌کند و تعدادی را از ۱ به ۰ تغییر می‌دهد.
- بیت ۰ با **هر** افزایش عوض می‌شود (n بار).
- بیت ۱ **یکی در میان** عوض می‌شود ($\lfloor n/2 \rfloor$ بار).
- بیت ۲ هر **۴** بار یک دفعه عوض می‌شود ($\lfloor n/4 \rfloor$ بار).
- ...
- بیت i پس از هر 2^i افزایش عوض می‌شود (در مجموع $\lfloor n/2^i \rfloor$ بار).

پس در مجموع:

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n.$$

یعنی هزینهٔ سرشکن‌شدهٔ هر عمل $O(1)$ است (حداکثر ۲).

■

چرا؟ · چرا بیت i دقیقاً هر 2^i افزایش یک بار عوض می‌شود؟

بیت i وقتی عوض می‌شود که همهٔ بیت‌های 0 تا $i-1$ پیش از افزایش ۱ باشند (رقم نقلی تا بیت i برسد). این یعنی مقدار شمارنده به پیمانهٔ 2^i برابر $2^i - 1$ باشد، که در هر 2^i مقدار متوالی دقیقاً یک بار رخ می‌دهد. در جدول ببینید: ستون $A[0]$ هر بار زرد است، $A[1]$ یکی در میان، $A[2]$ هر چهار بار.

۴.۶ روش حساب‌داری

در این روش به هر عمل یک **هزینهٔ سرشکن** $\hat{c}$ (مبلغی که «می‌پردازیم») نسبت می‌دهیم که ممکن است با هزینهٔ واقعی c فرق داشته باشد. وقتی $\hat{c} > c$، مازاد را به‌صورت **اعتبار** روی اشیای مشخصی از داده‌ساختار ذخیره می‌کنیم؛ وقتی $\hat{c} < c$، کسری را از اعتبار ذخیره‌شده برمی‌داریم.

نکته · تنها شرط

اعتبار کل **هرگز نباید منفی شود** (نمی‌توان از مخزن خالی برداشت کرد). معادلش: برای هر دنباله و هر پیشوند آن، $\sum \hat{c}_i \geq \sum c_i$. اگر این برقرار باشد، جمع هزینه‌های سرشکن یک کران بالا برای جمع هزینه‌های واقعی است.

تحلیل پشته به روش حساب‌داری

- برای هر PUSH، **۲ ریال** خرج می‌کنیم.
- **۱ ریال** صرف خود عمل PUSH می‌شود.
- **۱ ریال** را روی عنصرِ داخل پشته می‌گذاریم.
- POPها همه **مجانی** خواهند بود: هزینهٔ برداشتن هر عنصر از ریالی پرداخت می‌شود که رویش گذاشته‌ایم. MULTIPOP هم چند POP است و مجانی است.

اعتبار کل همیشه برابر تعداد عناصر پشته است، پس هرگز منفی نمی‌شود. کلاً حداکثر $2n$ ریال هزینه شده است، پس هزینهٔ سرشکن‌شده $O(1)$ است.

■

تحلیل شمارنده به روش حساب‌داری

- برای هر بار تبدیل یک بیت از ۰ به ۱، **۲ ریال** هزینه می‌کنیم.
- **۱ ریال** صرف خود تبدیل ۰ به ۱ می‌شود.
- **۱ ریال** را روی همان بیت ۱ قرار می‌دهیم.
- هزینهٔ تبدیل ۱ به ۰ **مجانی** خواهد بود: ریالِ روی آن بیت را خرج می‌کنیم.

ناوردا: روی هر بیت ۱ دقیقاً یک ریال هست، پس اعتبار کل = تعداد یک‌ها ≥ 0. هر افزایش حداکثر **یک** بیت را از ۰ به ۱ می‌کند (حلقهٔ `while` فقط یک‌ها را صفر می‌کند که مجانی است)، پس هزینهٔ سرشکن هر افزایش حداکثر ۲ ریال است. n افزایش حداکثر $2n$ هزینه دارد.

■

شهود · اعتبار روی «اشیا»

در روش حساب‌داری اعتبار به چیزهای مشخصی چسبیده است: به عناصر پشته، یا به بیت‌های ۱. این کمک می‌کند به سؤال «کار گران بعدی را چه کسی می‌پردازد؟» دقیق جواب دهیم: همان شیئی که کار گران رویش انجام می‌شود.

۴.۷ روش تابع پتانسیل

به‌جای اینکه اعتبار را روی اشیای جداگانه نگه داریم، آن را به‌صورت یک عدد برای **کل** داده‌ساختار در نظر می‌گیریم؛ مثل انرژی پتانسیل در فیزیک.

D_0 داده‌ساختار اولیه

D_i داده‌ساختار پس از عمل iام

c_i هزینهٔ واقعی عمل iام

$\Phi(D_i)$ تابع پتانسیل که D_i را به یک عدد حقیقی نگاشت می‌کند

$$D_0 \xrightarrow{\text{عمل ۱}} D_1 \xrightarrow{\text{عمل ۲}} D_2 \cdots \xrightarrow{\text{عمل } n} D_n$$

تعریف · هزینهٔ سرشکن در روش پتانسیل

هزینهٔ سرشکن‌شدهٔ عمل iام:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

یعنی هزینهٔ واقعی به‌اضافهٔ **تغییر پتانسیل**. عملی که پتانسیل را بالا می‌برد گران‌تر از هزینهٔ واقعی‌اش «حساب» می‌شود (ذخیره می‌کند)، و عملی که پتانسیل را پایین می‌آورد ارزان‌تر (از ذخیره خرج می‌کند).

با جمع زدن روی همهٔ عمل‌ها، جمله‌های پتانسیل به‌صورت **تلسکوپی** حذف می‌شوند:

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} \big(c_i + \Phi(D_i) - \Phi(D_{i-1})\big) = \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0).$$

(در جمع، $\Phi(D_1)$ یک بار با علامت مثبت و یک بار با علامت منفی می‌آید، و همین‌طور بقیه؛ فقط اولی و آخری می‌مانند.) اگر $\Phi(D_0) = 0$ و $\Phi(D_i) \geq 0$ برای همهٔ i:

$$\sum_{i=1}^{n} \hat{c}_i \;\geq\; \sum_{i=1}^{n} c_i.$$

یعنی $\sum \hat{c}_i$ همان **کران بالایی** برای $\sum c_i$ است که می‌خواهیم به دست آوریم.

نکته · چرا $\Phi(D_i) \geq 0$ برای همهٔ i، نه فقط $i = n$؟

چون نمی‌دانیم دنباله کجا تمام می‌شود. برای اینکه کران برای هر n (هر پیشوند دنباله) برقرار باشد، باید در هر لحظه پتانسیل از مقدار اولیه کمتر نشود.

تناظر با روش حساب‌داری

$\Phi(D_i)$ مقدار پول موجود در مخزن پس از عمل i.

$\hat{c}_i$ مقدار پولی که برای عمل i پرداخت می‌کنیم.

c_i مقدار هزینهٔ صرف‌شده برای عمل i.

- اگر $c_i < \hat{c}_i$، مابه‌التفاوت به مخزن اضافه می‌شود: $\Phi(D_i) = \Phi(D_{i-1}) + \hat{c}_i - c_i$.
- اگر $c_i > \hat{c}_i$، برای انجام عمل i لازم است به اندازهٔ $c_i - \hat{c}_i$ از مخزن برداریم تا بتوانیم این عمل را انجام دهیم. پس پول مخزن $\Phi(D_i) = \Phi(D_{i-1}) - (c_i - \hat{c}_i)$ می‌شود.

هر دو حالت همان رابطهٔ $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$ است. شرط «مخزن منفی نشود» همان شرط $\Phi(D_i) \geq 0$ است.

تحلیل پشته با روش پتانسیل

تعریف: $\Phi(D_i)$ = تعداد عناصر موجود در پشته. در ابتدا $\Phi(D_0) = 0$ و همیشه $\Phi \geq 0$.

عمل PUSH: یک عنصر اضافه می‌شود: $\Phi(D_i) - \Phi(D_{i-1}) = 1$. هزینهٔ واقعی $c_i = 1$. پس $\hat{c}_i = 1 + 1 = 2$.

عمل POP: یک عنصر کم می‌شود: $\Phi(D_i) - \Phi(D_{i-1}) = -1$. هزینهٔ واقعی $c_i = 1$. پس $\hat{c}_i = 1 - 1 = 0$.

عمل MULTIPOP: اگر $k' = \min(k, s)$ عنصر برداشته شود، $c_i = k'$ و $\Delta\Phi = -k'$، پس $\hat{c}_i = 0$. (MULTIPOP هم تعدادی POP است، پس هزینهٔ سرشکنش صفر است.)

$$\sum_{i=1}^{n} c_i \;\leq\; \sum_{i=1}^{n} \hat{c}_i \;\leq\; 2n.$$

پس هزینهٔ سرشکن‌شدهٔ هر عمل $O(1)$ است.

■

تحلیل شمارنده به روش پتانسیل

تعریف تابع پتانسیل: $\Phi(D_i) = b_i$ = تعداد یک‌های A پس از عمل i. داریم $\Phi(D_0) = 0$ و $\Phi(D_i) \geq 0$.

عمل «افزایش»: فرض کنید افزایش iام، t_i بیت را از ۱ به صفر و حداکثر یک بیت صفر را به ۱ تبدیل کند. پس

$$\left.\begin{aligned} \hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ \Phi(D_i) - \Phi(D_{i-1}) &\leq -t_i + 1 \\ c_i &\leq t_i + 1 \end{aligned}\right\} \implies \hat{c}_i \leq (t_i + 1) + (1 - t_i) = 2.$$

پس هزینهٔ سرشکن‌شدهٔ هر عمل افزایش $O(1)$ است و n افزایش حداکثر $2n$ هزینه دارد.

■

نکته · چرا «حداکثر» یک بیت ۰ ← ۱؟

اگر همهٔ k بیت ۱ باشند (سرریز)، حلقه همه را صفر می‌کند و شرط $i < \text{length}[A]$ نادرست است، پس هیچ بیتی ۱ نمی‌شود. در این حالت $c_i = t_i = k$ و $\Delta\Phi = -k$، پس $\hat{c}_i = 0 \leq 2$. نامساوی‌ها هر دو حالت را پوشش می‌دهند.

تکمیلی (فراتر از اسلاید) · شمارنده‌ای که از صفر شروع نمی‌شود

اگر شمارنده با b_0 یک شروع شود، $\Phi(D_0) = b_0 \neq 0$. از تساوی تلسکوپی: $\sum c_i = \sum \hat{c}_i - \Phi(D_n) + \Phi(D_0) \leq 2n -$ $b_n + b_0 \leq 2n + k$. پس اگر $n = \Omega(k)$ (حداقل به تعداد بیت‌ها افزایش انجام دهیم)، هزینهٔ کل باز هم $O(n)$ است. این نشان می‌دهد روش پتانسیل حتی وقتی $\Phi(D_0) \neq 0$ هم مفید است.

چرا؟ · تابع پتانسیل را از کجا بیاوریم؟

قاعدهٔ سرانگشتی: پتانسیل باید «میزان بی‌نظمیِ ذخیره‌شده» را اندازه بگیرد، یعنی چیزی که بعداً عمل‌های گران را ایجاد می‌کند. در پشته، عناصر موجود همان چیزی‌اند که بعداً MULTIPOP را گران می‌کنند؛ در شمارنده، یک‌ها همان چیزی‌اند که بعداً رقم نقلی را طولانی می‌کنند. عمل گران باید پتانسیل را به همان اندازه‌ای که هزینه دارد پایین بیاورد. اغلب، اعتبارهای روش حساب‌داری را جمع بزنید؛ پتانسیل به دست می‌آید.

۴.۸ مثال تکمیلی: آرایهٔ پویا

تکمیلی (فراتر از اسلاید) · چرا این مثال؟

این مثال در اسلایدها نیامده، اما مهم‌ترین کاربرد عملی تحلیل سرشکنی است (`vector` در C++، `ArrayList` در جاوا و `list` در پایتون) و سؤال محبوب امتحان‌هاست. هر سه روش را روی آن تمرین می‌کنیم.

آرایه‌ای با ظرفیت size داریم که num خانه‌اش پر است. **درج:** اگر جا هست، عنصر را در خانهٔ بعدی بگذار (هزینهٔ ۱). اگر آرایه پر است، آرایه‌ای با **دو برابر** ظرفیت بساز، همهٔ num عنصر را کپی کن و بعد درج کن (هزینهٔ $\text{num} + 1$). از آرایهٔ خالی با ظرفیت ۱ شروع می‌کنیم.

بدترین حالت یک درج $O(n)$ است، پس کران سادهٔ n درج $O(n^2)$ است. اما:

اثبات · انبوهه

درج iام هزینهٔ i دارد اگر $i - 1$ توانی از ۲ باشد (آرایه پر است)، و ۱ در غیر این صورت. پس

$$\sum_{i=1}^{n} c_i \leq n + \sum_{j=0}^{\lfloor \log n \rfloor} 2^j < n + 2n = 3n.$$

هزینهٔ سرشکن هر درج حداکثر ۳ است.

∎

اثبات · حساب‌داری

برای هر درج **۳ ریال** بپرداز: ۱ ریال برای درج خودش، ۱ ریال روی خودش برای وقتی که در گسترش بعدی کپی می‌شود، و ۱ ریال روی یکی از عناصر **قدیمی** (نیمهٔ اول آرایه) که ریالش را در گسترش قبلی خرج کرده است. وقتی آرایه با ظرفیت s پر می‌شود، $s/2$ عنصر جدید هر کدام ۲ ریال ذخیره کرده‌اند، یعنی s ریال: دقیقاً به اندازهٔ کپی همهٔ s عنصر.

∎

اثبات · پتانسیل

$\Phi = 2 \cdot \text{num} - \text{size}$ (پس از اولین درج، آرایه همیشه دست‌کم نیمه‌پر است، پس $\Phi \geq 0$).

- **درج بدون گسترش:** $c = 1$، num یکی زیاد می‌شود: $\Delta\Phi = 2$. پس $\hat{c} = 3$.
- **درج با گسترش** ($\text{num} = \text{size} = s$): $c = s + 1$. پیش از عمل $\Phi = 2s - s = s$، پس از آن $\Phi = 2(s+1) - 2s = 2$. پس $\hat{c} = (s+1) + (2 - s) = 3$.

∎

چرا؟ · چرا دو برابر؟ چرا نه «ظرفیت + ۱۰۰»؟

اگر هر بار فقط c خانه اضافه کنیم، هر c درج یک کپی کامل داریم و هزینهٔ کپی‌ها $c + 2c + 3c + \cdots + n \approx \frac{n^2}{2c}$ است: سرشکن $\Theta(n/c) = \Theta(n)$. کلید کار **رشد ضربی** است: با ضریب رشد $r > 1$، هزینهٔ کپی‌ها یک سری هندسی است و سرشکن $O(1)$ می‌ماند (مثلاً با $r = 1.5$، حداکثر حدود $3n$ کپی).

۴.۹ مقایسهٔ سه روش

	انبوهه	حساب‌داری	پتانسیل
چه چیزی را می‌شماریم؟	هزینهٔ کل دنباله	اعتبار روی تک‌تک اشیا	یک عدد برای کل ساختار
هزینهٔ سرشکن	برای همهٔ عمل‌ها یکسان	برای هر نوع عمل جدا	برای هر نوع عمل جدا
شرط درستی	–	اعتبار هرگز منفی نشود	$\Phi(D_i) \geq \Phi(D_0)$
پشته	$\leq 2n$	PUSH: ۲، POP: ۰	Φ = اندازهٔ پشته
شمارنده	$\sum \lfloor n/2^i \rfloor < 2n$	۰ ← ۱، ۲ : ۱ ← ۰ : ۰	Φ = تعداد یک‌ها
مناسب برای	ساختارهای ساده	شهود «چه کسی می‌پردازد»	ساختارهای پیچیده (هرم فیبوناچی، درخت اسپلی)

جمع‌بندی · فصل ۴ در یک نگاه

- هزینهٔ سرشکن = هزینهٔ کل بدترین دنبالهٔ n عملی تقسیم بر n. تضمین قطعی است، بدون احتمال.
- جمع زدن «بدترین هزینهٔ هر عمل» اغلب کران شل می‌دهد ($O(n^2)$ و $O(nk)$ به‌جای $O(n)$).
- انبوهه: زاویهٔ شمارش را عوض کن (هر عنصر حداکثر یک بار POP می‌شود؛ بیت i هر 2^i بار یک بار عوض می‌شود).
- حساب‌داری: عمل ارزان بیشتر بپردازد و اعتبار روی اشیا بماند تا عمل گران را بپردازد.
- پتانسیل: $\hat{c}_i = c_i + \Delta\Phi$؛ با $\Phi(D_0) = 0$ و $\Phi \geq 0$، $\sum \hat{c}_i$ کران بالای $\sum c_i$ است.

۴.۱۰ خودآزمایی فصل ۴

خودآزمایی

1. اگر پشته عمل MULTIPUSH(S, k) هم داشته باشد (k عنصر را درج کند)، آیا هزینهٔ سرشکن همهٔ عمل‌ها هنوز $O(1)$ است؟

پاسخ: خیر. خود هر MULTIPUSH k هزینهٔ واقعی دارد؛ دنباله‌ای از n عمل MULTIPUSH هزینهٔ nk دارد، پس سرشکن $\Theta(k)$ است.

2. اگر شمارنده عمل DECREMENT هم داشته باشد، آیا هزینهٔ سرشکن $O(1)$ می‌ماند؟

پاسخ: خیر. با شروع از $0111\ldots1$، تناوب INCREMENT و DECREMENT هر بار همهٔ k بیت را عوض می‌کند؛ n عمل هزینهٔ nk دارند.

3. با پتانسیل $\Phi = 2\times$ (تعداد یک‌ها) برای شمارنده، هزینهٔ سرشکن افزایش چقدر می‌شود؟

پاسخ: $\hat{c} \leq (t+1) + 2(1-t) = 3 - t \leq 3$. باز هم $O(1)$؛ تابع پتانسیل یکتا نیست.

4. در دنباله‌ای از n عمل، عمل iام اگر i توانی از ۲ باشد هزینهٔ i و وگرنه هزینهٔ ۱ دارد. هزینهٔ سرشکن هر عمل چقدر است؟

پاسخ: انبوهه: $\sum c_i \leq n + (1 + 2 + 4 + \cdots + 2^{\lfloor \log n \rfloor}) < n + 2n = 3n$؛ پس سرشکن $O(1)$.

5. آیا هزینهٔ سرشکن $O(1)$ تضمین می‌کند که هر عمل تکی $O(1)$ باشد؟

پاسخ: خیر. فقط جمع n عمل $O(n)$ است؛ یک عمل تکی (مثل گسترش آرایهٔ پویا) می‌تواند $\Theta(n)$ باشد.

فصل ۵

آرایه، لیست پیوندی، پشته و صف

از اینجا وارد دنیای داده‌ساختارها می‌شویم: اینکه داده را **چطور بچینیم** تا کارهایی که لازم داریم سریع انجام شود. با ساده‌ترین و پرکاربردترین داده‌ساختارها شروع می‌کنیم.

نوع دادهٔ انتزاعی · آرایه · لیست پیوندی · پشته · صف حلقوی · کاربردهای الگوریتمی

۵.۱ داده‌ساختار چیست؟

تعریف · داده‌ساختار

داده‌ساختار (data structure) روشی برای سازمان‌دهی و نمایش داده است، طوری که عملیات مورد نیاز روی آن **کارآمد** انجام شود. به بیان دقیق‌تر: نمایشی از داده‌ها **به‌همراه روابط میان عناصر آن**. نمونه‌ها: آرایه، لیست پیوندی، پشته، صف، درخت، جدول درهم‌سازی و گراف.

انتخاب داده‌ساختار مستقیماً تعیین می‌کند که **جست‌وجو، درج، حذف و به‌روزرسانی** با چه سرعتی انجام شوند. یک داده‌ساختار برای همهٔ کارها بهترین نیست؛ هر کدام بعضی عملیات را ارزان و بعضی را گران می‌کند. هنر طراح این است که بداند کدام عملیات برایش مهم‌تر است.

شهود · فروشگاه

فروشگاهی را تصور کنید که همهٔ اجناس را وسط سالن روی هم ریخته است. برای پیدا کردن یک بستهٔ نمک باید **تک‌تک** اجناس را نگاه کنید. حالا فروشگاهی را تصور کنید که هر جنس را در قفسهٔ مربوط به دسته‌اش گذاشته است: مستقیم به قفسهٔ «ادویه‌ها» می‌روید.

نحوهٔ چیدن اجناس، داده‌ساختار است (روی هم ریختن یا قفسه‌بندی). **روش پیدا کردن جنس، الگوریتم است** (بررسی تک‌تک اقلام یا جست‌وجو بر اساس دسته). الگوریتم دوم فقط به این دلیل ممکن است که داده‌ساختار مناسبی انتخاب شده است.

الگوریتم و داده‌ساختار با هم کار می‌کنند

نکته · الگوریتم

عملیات و گام‌ها؛ مثلاً «یک کلید را جست‌وجو کن».

نکته · داده‌ساختار

نمایش داده‌ها؛ مثلاً آرایه یا جدول درهم‌سازی.

مثال: جست‌وجو در یک آرایهٔ مرتب‌نشده $\Theta(n)$ زمان می‌برد، چون ممکن است مجبور شویم همهٔ خانه‌ها را ببینیم. اما یک جدول درهم‌سازی خوب، جست‌وجو را با **امید** $\Theta(1)$ انجام می‌دهد. مسئله یکی است؛ فقط داده‌ساختار عوض شده است. **تحلیل الگوریتم** (فصل ۱) دقیقاً هزینهٔ زمانی و حافظه‌ای همین انتخاب‌ها را می‌سنجد.

۵.۲ نوع دادهٔ انتزاعی

تعریف · نوع دادهٔ انتزاعی (ADT)

یک نوع دادهٔ انتزاعی (Abstract Data Type) مشخص می‌کند که یک ساختار **«چه کاری»** انجام می‌دهد، نه **«چگونه»**:

- چه **مقادیری** را می‌تواند نگه دارد؛
- چه **عملیاتی** در اختیار کاربر قرار می‌دهد؛
- **معنا و قواعد** هر عمل چیست؛
- و جزئیات نمایش داخلی را **پنهان** می‌کند.

مثال · نوع انتزاعی پشته

عملیات پشته (Stack):

- push(x): x را روی پشته (بالای آن) بگذار.
- pop(): عنصر بالای پشته را بردار و برگردان.
- top(): عنصر بالا را بدون برداشتن ببین.
- isEmpty(): آیا پشته خالی است؟

```
client code: uses only the interface
```

```
S.push(10)
S.push(20)
x = S.pop()          // returns 20
```

کد کاربر لازم نیست بداند S با آرایه پیاده شده یا با لیست پیوندی. اگر هر دو پیاده‌سازی از قواعد یکسانی پیروی کنند، می‌توان یکی را با دیگری عوض کرد و کد کاربر **بدون تغییر** کار خواهد کرد.

شهود · فرمان و پدال خودرو

رابط خودرو (فرمان، پدال گاز و ترمز) یک «نوع انتزاعی» است. راننده نمی‌داند موتور بنزینی است یا برقی؛ فقط می‌داند پدال گاز یعنی «سریع‌تر شو». سازنده می‌تواند موتور را عوض کند بدون اینکه راننده رانندگی را از نو یاد بگیرد.

سه سطح: نوع انتزاعی، انتخاب مدل، پیاده‌سازی

۱. **نوع انتزاعی:** رابط (مثلاً «درج، حذف، جست‌وجو») به‌همراه معنا و تضمین‌های آن.

۲. **انتخاب مدل:** کدام داده‌ساختار؟ آرایه، لیست، درخت، جدول درهم‌سازی و ... .

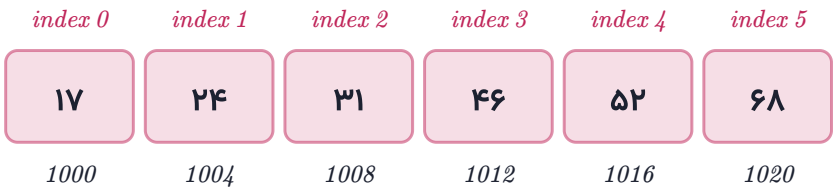
۳. **پیاده‌سازی:** جزئیات واقعی: اندیس‌ها، اشاره‌گرها، چیدمان حافظه و کد.

اشتباه رایج · نوع انتزاعی به‌تنهایی کارایی را مشخص نمی‌کند

«پشته» فقط می‌گوید push و pop چه کار می‌کنند، نه اینکه چقدر طول می‌کشند. زمان اجرا به **پیاده‌سازی** بستگی دارد. برای همین در این فصل همیشه دو سؤال می‌پرسیم: «چه کاری؟» و «با چه هزینه‌ای؟».

۵.۳ آرایه: حافظهٔ پیوسته و دسترسی مستقیم

آرایه (array) عناصر را در **خانه‌های پشت سر هم** حافظه ذخیره می‌کند.



نشانی هر خانه = ۱۰۰۰ + ۴ × i (اندازهٔ هر عنصر ۴ بایت)

شکل ۵-۱ شش عدد در خانه‌های پیوستهٔ حافظه. اگر آرایه از نشانی ۱۰۰۰ شروع شود و هر عنصر ۴ بایت باشد، خانهٔ i در نشانی $1000 + 4i$ است.

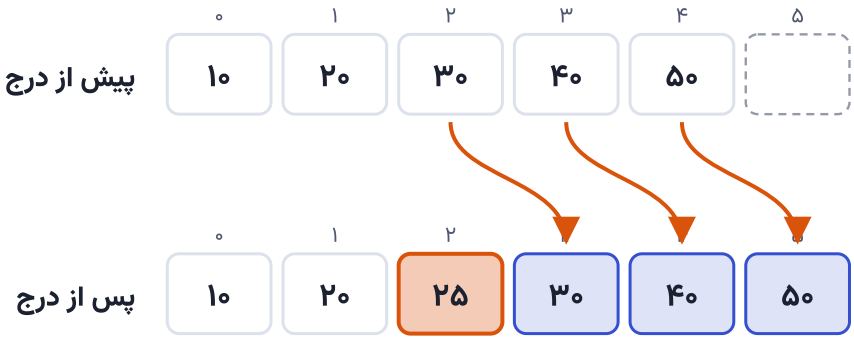
چرا؟ · چرا دسترسی به $arr[i]$ در $O(1)$ است؟

$$\text{Address}(arr[i]) = \text{base} + i \times \text{element_size}.$$

نشانی خانه با یک ضرب و یک جمع **مستقیم محاسبه** می‌شود؛ لازم نیست هیچ خانهٔ دیگری را بخوانیم. این محاسبه برای $i = 0$ و $i = 10^9$ به یک اندازه وقت می‌گیرد، پس زمانش به n بستگی ندارد. این شرط فقط به این دلیل برقرار است که همهٔ عناصر **هم‌اندازه** و **پشت سر هم** هستند.

درج در وسط آرایه

فرض کنید آرایه جای خالی دارد و می‌خواهیم ۲۵ را در اندیس ۲ درج کنیم. عناصر اندیس ۲ به بعد باید یک خانه به راست بروند تا جا باز شود.



شکل ۵-۲ برای درج ۲۵ در اندیس ۲، سه عنصر ۳۰، ۴۰ و ۵۰ (آبی) یک خانه جابه‌جا می‌شوند.

```
 n">
for j = n-1 downto i          // shift the suffix right, starting from the end
    A[j+1] = A[j]
A[i] = x
n = n + 1
```

چرا؟ · چرا جابه‌جایی را از انتها شروع می‌کنیم؟

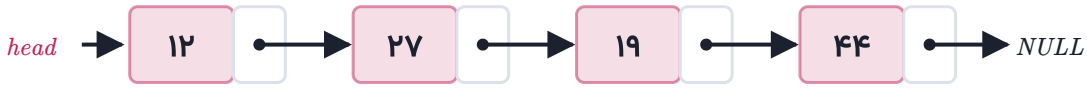
اگر از اندیس i شروع کنیم، $A[i+1] = A[i]$ مقدار قبلی $A[i+1]$ را **بازنویسی** می‌کند و آن را از دست می‌دهیم. با شروع از انتها، هر خانه قبل از اینکه رویش نوشته شود، به خانهٔ بعدی منتقل شده است.

در بدترین حالت ($i = 0$) همهٔ n عنصر جابه‌جا می‌شوند، پس زمان $\Theta(n)$ است. اگر آرایه پر باشد، باید آرایهٔ بزرگ‌تری گرفت و همه را کپی کرد (همان آرایهٔ پویای فصل ۴).

عمل	هزینهٔ معمول	دلیل
خواندن / نوشتن $A[i]$	$\Theta(1)$	محاسبهٔ مستقیم نشانی
جست‌وجو در آرایهٔ نامرتب	$\Theta(n)$	ممکن است همهٔ عناصر بررسی شوند
درج در انتها (با جای خالی)	$\Theta(1)$	فقط یک خانه نوشته می‌شود
درج در اندیس i	$\Theta(n)$	پسوند به راست منتقل می‌شود
حذف از اندیس i	$\Theta(n)$	پسوند به چپ منتقل می‌شود
پیمایش همهٔ عناصر	$\Theta(n)$	هر عنصر یک بار

۵.۴ لیست پیوندی: گره‌هایی که با اشاره‌گر به هم وصل‌اند

مشکل آرایه این بود که عناصر باید **کنار هم** باشند، پس درج در وسط یعنی جابه‌جا کردن بقیه. ایدهٔ لیست پیوندی (linked list) این است: **عناصر لازم نیست کنار هم باشند**؛ کافی است هر عنصر بداند عنصر بعدی کجاست.



شکل ۵-۳ هر گره دو بخش دارد: مقدار (رنگی) و اشاره‌گر به گرهٔ بعدی (نقطه). اشاره‌گر آخرین گره NULL است، یعنی «گرهٔ بعدی وجود ندارد».

تعریف · گره

هر گره (node) دو چیز نگه می‌دارد: (۱) یک **مقدار** (data یا key) و (۲) یک **اشاره‌گر به گرهٔ بعدی** (next).

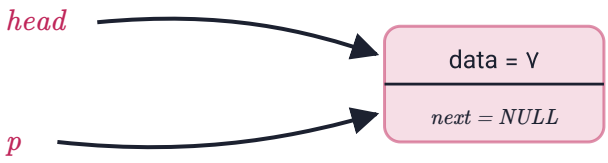
اشتباه رایج · پیامد کلیدی

دسترسی با اندیس در زمان ثابت **ممکن نیست**. برای رسیدن به گرهٔ iام باید از ابتدا i پیوند را دنبال کنیم، چون نشانی آن را هیچ‌جا نداریم.

اشاره‌گر چیست؟

اشاره‌گر (pointer) متغیری است که **نشانی** یک گره را نگه می‌دارد؛ مثل کاغذی که آدرس یک خانه رویش نوشته شده، نه خود خانه.

```
head = null          // empty list: head points to nothing
p = new Node(7)      // create a node; p holds its address
head = p             // now head and p point to the same node
```



شکل ۵-۴ $head$ و p دو متغیر جدا هستند که هر دو نشانی **یک** گره را نگه می‌دارند.

- $head$ نشانی اولین گره را نگه می‌دارد.
- p و $head$ می‌توانند به یک گره اشاره کنند.
- **تغییر p به‌طور خودکار $head$ را تغییر نمی‌دهد**؛ اگر بنویسیم $p = \text{null}$، گره و $head$ دست‌نخورده می‌مانند. فقط کاغذِ p پاک شده است.
- یک اشاره‌گر می‌تواند NULL باشد (به هیچ گره‌ای اشاره نکند). پیش از خواندن $p.data$ یا $p.next$ باید مطمئن شویم p به گرهٔ معتبری اشاره می‌کند؛ وگرنه برنامه خطا می‌دهد.

singly linked-list node

```
Node:
    data             // the value stored in this node
    next             // address of the next node (null = end of list)

head = null          // the list is represented by one pointer: head
```

کل لیست با **یک** اشاره‌گر نمایش داده می‌شود: $head$. لیست خالی یعنی $head = \text{null}$.

۵.۵ عملیات روی لیست پیوندی

عمل ۱: پیمایش و چاپ

```
PRINT-LIST(head)
```

```
1  cur = head
2  while cur ≠ null
3      output cur.data
4      cur = cur.next          // advance, or the loop never ends
```

cur از $head$ شروع می‌کند. هر دور حلقه یک گره را ملاقات می‌کند و سپس به گرهٔ بعدی می‌رود. برای لیست $12 \to 27 \to 19$، خروجی $12, 27, 19$ است و پس از گرهٔ ۱۹، cur برابر null می‌شود و حلقه تمام می‌شود. **زمان:** $\Theta(n)$ · **حافظهٔ اضافه:** $\Theta(1)$ (فقط یک متغیر cur).

اشتباه رایج · فراموش کردن جلو بردن اشاره‌گر

اگر خط `cur = cur.next` را فراموش کنید، cur همیشه به گرهٔ اول اشاره می‌کند و حلقه هرگز تمام نمی‌شود.

عمل ۲: جست‌وجوی یک کلید

```
FIND(head, key)
```

```
1  cur = head
2  while cur ≠ null
3      if cur.data = key
4          return cur          // return the node's address
5      cur = cur.next
6  return null                 // not found
```

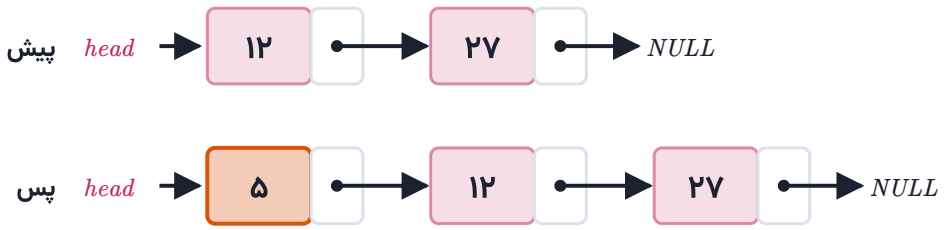
مثال find(19) روی $19 \to 27 \to 12$: بررسی ۱۲، بررسی ۲۷، بررسی ۱۹ و برگرداندن نشانی آن. **بدترین حالت:** $\Theta(n)$، چون لیست پیوندی دسترسی مستقیم به وسط ندارد (حتی اگر لیست مرتب باشد، جست‌وجوی دودویی ممکن نیست؛ چرا؟ چون رسیدن به «وسط» خودش $\Theta(n)$ است).

عمل ۳: درج در ابتدای لیست

برای درج گرهٔ جدید x در ابتدا: گرهٔ جدید را به head قدیمی وصل کن، سپس head را به‌روز کن.

```
INSERT-FRONT(head, x)
```

```
1  p = new Node(x)
2  p.next = head
3  return p                    // p is the new head
```



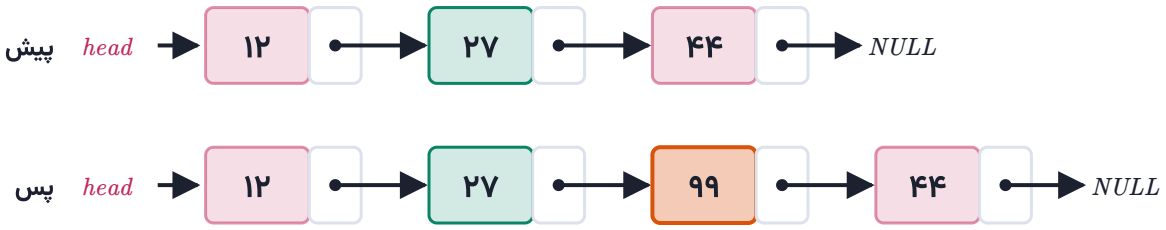
شکل ۵-۵ درج ۵ در ابتدا: فقط دو انتساب، مستقل از طول لیست.

زمان: $\Theta(1)$. این کد برای لیست خالی هم درست است: اگر $head = \text{null}$، آنگاه $p.next$ هم null می‌شود و p تنها گرهٔ لیست است.

عمل ۴: درج بعد از یک گرهٔ معلوم

```
INSERT-AFTER(p, x)
```

```
1  if p = null then return
2  q = new Node(x)
3  q.next = p.next             // 1) new node points to p's old successor
4  p.next = q                  // 2) p points to the new node
```



شکل ۵-۶ درج ۹۹ بعد از گرهٔ ۲۷ (سبز).

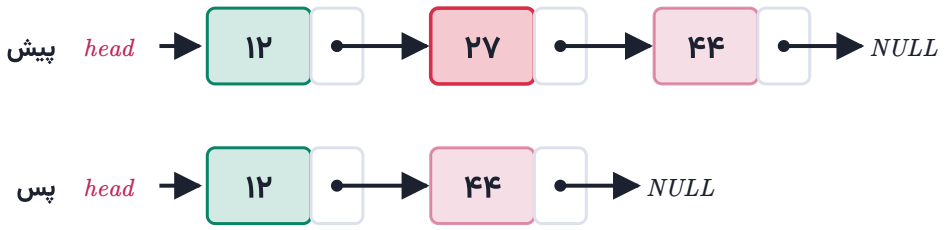
چرا؟ · چرا ترتیب دو خط آخر مهم است؟

اگر اول `p.next = q` را اجرا کنیم، تنها جایی که نشانی گرهٔ ۴۴ (و بقیهٔ لیست) نگه داشته شده بود، بازنویسی می‌شود. حالا `q.next = p.next` یعنی `q.next = q` : گرهٔ جدید به خودش اشاره می‌کند و بقیهٔ لیست برای همیشه گم شده است. **قاعده:** پیش از بازنویسی یک اشاره‌گر، مطمئن شوید چیزی که به آن اشاره می‌کرد را جای دیگری نگه داشته‌اید.

عمل ۵: حذف گرهٔ بعد از p

DELETE-AFTER(p)

```
1  if p = null or p.next = null then return
2  victim = p.next              // save the node to remove
3  p.next = victim.next         // link p directly to victim's successor
4  delete victim                // free its memory (in C++)
```



شکل ۵-۷ حذف ۲۷ (قرمز) با دور زدن آن: ۱۲ مستقیماً به ۴۴ وصل می‌شود.

اگر p از قبل معلوم باشد، حذف $\Theta(1)$ است. اما اگر اول باید p را **پیدا** کنیم، پیدا کردنش ممکن است $\Theta(n)$ طول بکشد.

دام رایج در تحلیل: گره کجاست؟

مثال · اشاره‌گر به p داده شده

درج بعد از p: دو اشاره‌گر عوض می‌شود. **زمان $\Theta(1)$.** گرهٔ هدف از قبل معلوم است.

اشتباه رایج · فقط مقدار یا اندیس داده شده

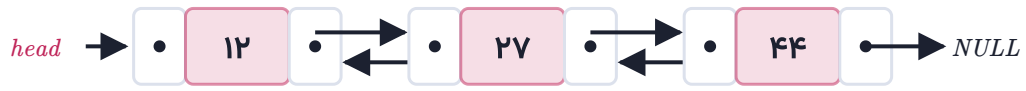
اول باید گرهٔ هدف را با پیمایش پیدا کنیم: ممکن است $\Theta(n)$ گره بررسی شود، سپس اشاره‌گرها عوض شوند. **کل زمان $\Theta(n)$ در بدترین حالت.**

جمع‌بندی · قاعده

هنگام تحلیل زمان یک عمل، همیشه بگویید **چه اطلاعاتی به عمل داده شده است.** «درج در لیست پیوندی $O(1)$ است» فقط وقتی درست است که محل درج از قبل معلوم باشد.

۵.۶ لیست دوپیوندی و مقایسه با آرایه

در لیست دوپیوندی (doubly linked list) هر گره علاوه بر $next$، اشاره‌گری به گرهٔ **قبلی** ($prev$) هم دارد.



شکل ۵-۸ هر گره سه بخش دارد: prev، مقدار و next.

مثال · مزایا

می‌توان به جلو **و** عقب حرکت کرد؛ و مهم‌تر، می‌توان یک گرهٔ معلوم را **بدون جست‌وجوی گرهٔ قبلی‌اش** حذف کرد.

اشتباه رایج · هزینه‌ها

یک اشاره‌گر اضافه در هر گره و به‌روزرسانی‌های بیشتر؛ اشتباه در یکی از این به‌روزرسانی‌ها پیوندها را ناسازگار می‌کند.

تکمیلی (فراتر از اسلاید) · حذف گرهٔ x از لیست دوپیوندی

```
DELETE(head, x) ▷ x is a pointer to the node
```

```
1  if x.prev ≠ null then x.prev.next = x.next
2  else head = x.next                    // x was the first node
3  if x.next ≠ null then x.next.prev = x.prev
```

در لیست یک‌طرفه برای حذف x باید گرهٔ قبل از آن را داشته باشیم (چون $next$ آن باید عوض شود) و پیدا کردنش $\Theta(n)$ است. در لیست دوپیوندی $x.prev$ همان گرهٔ قبلی است، پس حذف $\Theta(1)$ می‌شود.

آرایه یا لیست پیوندی؟

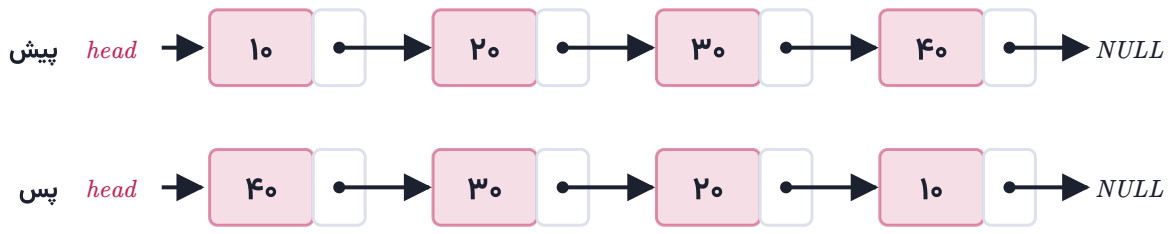
پرسش / عمل	آرایه / آرایهٔ پویا	لیست پیوندی
دسترسی به عنصر iام	$\Theta(1)$	$\Theta(n)$
جست‌وجو در دادهٔ نامرتب	$\Theta(n)$	$\Theta(n)$
درج در ابتدا	$\Theta(n)$	$\Theta(1)$
درج بعد از گرهٔ معلوم	$\Theta(n)$ **جابه‌جایی**	$\Theta(1)$
محلی بودن حافظه	**معمولاً عالی**	گره‌ها ممکن است پراکنده باشند
حافظهٔ اضافه	معمولاً کم برای هر عنصر	یک یا دو اشاره‌گر برای هر گره

نکته · «محلی بودن حافظه» یعنی چه؟

پردازنده‌ها داده را به‌صورت بلوک‌های پشت سر هم از حافظه به حافظهٔ نهان (cache) می‌آورند. وقتی خانهٔ $A[i]$ خوانده می‌شود، چند خانهٔ بعدی هم مجانی می‌آیند. در آرایه این خانه‌ها همان عناصر بعدی‌اند؛ در لیست پیوندی گرهٔ بعدی ممکن است جای دوری از حافظه باشد. برای همین، با وجود مرتبهٔ یکسان، پیمایش آرایه در عمل چند برابر سریع‌تر است.

۵.۷ معکوس کردن لیست پیوندی

می‌خواهیم جهت همهٔ پیوندها را برعکس کنیم، **بدون** ساختن گرهٔ جدید (درجا).



شکل ۵-۹ سر قدیمی (۱۰) دم جدید می‌شود و دم قدیمی (۴۰) سر جدید.

REVERSE(head)

```
prev = null
current = head
while current ≠ null
    next = current.next      // save the rest of the list first!
    current.next = prev      // flip this link
    prev = current           // advance prev
    current = next           // advance current
return prev                  // the new head
```

شهود · ناوردا

در شروع هر دور: $prev$ سرِ بخشِ **معکوس‌شده** است و $current$ سرِ بخشِ **هنوز دست‌نخورده**. هر دور یک گره را از بخش دوم برمی‌دارد و به سر بخش اول می‌چسباند. وقتی بخش دوم تمام شد ($current = \text{null}$)، $prev$ سرِ کل لیست معکوس است.

iteration	reversed part (from prev)	remaining part (from current)
start	null	10 → 20 → 30 → 40
1	10 → null	20 → 30 → 40
2	20 → 10 → null	30 → 40
3	30 → 20 → 10 → null	40
4	40 → 30 → 20 → 10 → null	null

زمان: $\Theta(n)$ (هر گره دقیقاً یک بار) · **حافظهٔ اضافه:** $\Theta(1)$ (سه اشاره‌گر).

اشتباه رایج · چرا $next$ را اول ذخیره می‌کنیم؟

خط `current.next = prev` تنها پیوند به بقیهٔ لیست را پاک می‌کند. اگر قبلش آن را در $next$ نگه نداشته باشیم، بقیهٔ لیست گم می‌شود. این همان درس عمل ۴ است.

۵.۸ پیاده‌سازی لیست پیوندی با آرایه

«اشاره‌گر» لازم نیست حتماً نشانی حافظه باشد؛ می‌تواند **اندیس یک آرایه** باشد. دو آرایهٔ موازی می‌گیریم: $Data[\,]$ برای مقدار و $Next[\,]$ برای اندیس گرهٔ بعدی.

Index	0	1	2	3
Data	10	30	20	–
Next	2	null	1	–

با $head = 0$، پیمایش منطقی این است: $0{:}\,10 \to 2{:}\,20 \to 1{:}\,30$ (چون $Next[0] = 2$ و $Next[2] = 1$ و $Next[1] = \text{null}$). یعنی لیست $10, 20, 30$ است، با اینکه در آرایه ترتیب فیزیکی $10, 30, 20$ است.

مثال · درج ۱۵ بین ۱۰ و ۲۰ با خانهٔ آزاد ۳

```
Data[3] = 15
Next[3] = 2        // 15 → 20
Next[0] = 3        // 10 → 15
```

ترتیب منطقی جدید: $10 \to 15 \to 20 \to 30$. هیچ عنصری جابه‌جا نشد؛ فقط دو «اشاره‌گر» عوض شد.

حذف ۲۰: گرهٔ قبلی‌اش اکنون اندیس ۳ (یعنی ۱۵) است، پس $Next[3] = 1$. حالا خانهٔ ۲ آزاد است و ترتیب منطقی $10 \to 15 \to 30$ می‌شود.

جمع‌بندی · ایدهٔ کلیدی

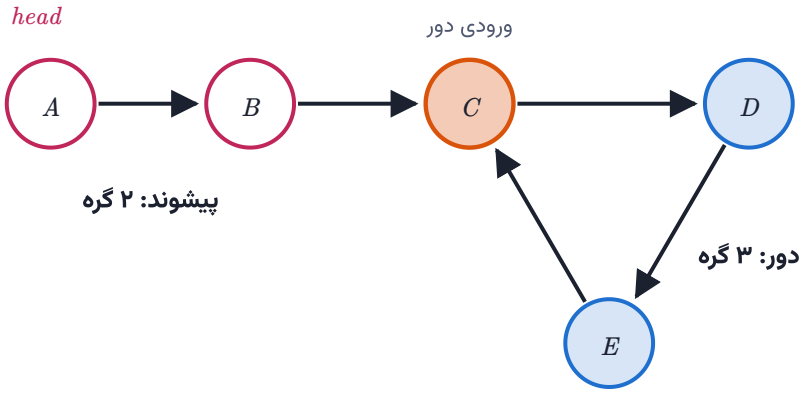
ترتیب منطقی را آرایهٔ $Next$ تعیین می‌کند، نه جای فیزیکی عناصر در آرایه.

تکمیلی (فراتر از اسلاید) · مدیریت خانه‌های آزاد

خانه‌های آزاد را هم می‌توان با همان آرایهٔ $Next$ در یک «لیست آزاد» (free list) به هم زنجیر کرد. برای گرفتن خانهٔ جدید، اولین عضو لیست آزاد را برمی‌داریم و پس از حذف، خانه را به سر لیست آزاد برمی‌گردانیم. هر دو $O(1)$ هستند. این کار همان کاری است که `new` و `delete` پشت صحنه انجام می‌دهند.

۵.۹ لیست دوری: شمارش گره‌ها با الگوریتم فلوید

گاهی (عمداً یا به‌خاطر یک باگ) پیوند آخرین گره به جای null، به یکی از گره‌های قبلی برمی‌گردد. ساختار حاصل یک **پیشوند** غیردوری دارد و بعد وارد یک **دور** می‌شود. مسئله: تعداد گره‌های متمایز را بشمارید.



شکل ۵-۱۰ پیشوند A, B (۲ گره) و دور $C \to D \to E \to C$ (۳ گره). تعداد گره‌های متمایز: $2 + 3 = 5$.

چرا؟ · چرا پیمایش معمولی کار نمی‌کند؟

در ساختار دوری هیچ پیوند null وجود ندارد، پس حلقهٔ `while cur ≠ null` هرگز تمام نمی‌شود. می‌توان نشانی همهٔ گره‌های دیده‌شده را در یک مجموعه نگه داشت، ولی این $O(n)$ حافظهٔ اضافه می‌خواهد. الگوریتم فلوید با $O(1)$ حافظه کار می‌کند.

الگوریتم · فلوید (لاک‌پشت و خرگوش) + دو شمارش

1. با دو اشاره‌گر $slow$ (یک قدم در هر گام) و $fast$ (دو قدم در هر گام) از $head$ راه بیفت تا به هم برسند (نقطهٔ ملاقات).
2. یک اشاره‌گر را به $head$ برگردان و دیگری را در نقطهٔ ملاقات نگه دار؛ هر دو را **یک قدم یک قدم** جلو ببر. جایی که به هم می‌رسند، **ورودی دور** است.
3. گره‌های قبل از ورودی را بشمار (طول پیشوند).
4. از ورودی یک دور کامل بزن و گره‌های دور را بشمار.
5. جواب: طول پیشوند + طول دور.

```
COUNT-NODES(head) ▷ assumes the list has a cycle
```

```
1  slow = head, fast = head                  // step 1: find a meeting point
2  repeat
3      slow = slow.next
4      fast = fast.next.next
5  until slow = fast
6  p1 = head, p2 = slow                      // step 2: find the cycle entry
7  while p1 ≠ p2
8      p1 = p1.next, p2 = p2.next
9  entry = p1
10 prefix = 0, p = head                      // step 3: count the prefix
11 while p ≠ entry
12     prefix = prefix + 1, p = p.next
13 cycle = 1, p = entry.next                 // step 4: count the cycle
14 while p ≠ entry
15     cycle = cycle + 1, p = p.next
16 return prefix + cycle                     // step 5
```

اثبات · چرا الگوریتم فلوید درست است؟

فرض کنید طول پیشوند μ و طول دور λ باشد (در مثال، $\mu = 2$ و $\lambda = 3$).

(الف) حتماً به هم می‌رسند: پس از μ گام، $slow$ وارد دور می‌شود و $fast$ هم جایی درون دور است. از آن پس در هر گام، $fast$ یک قدم بیشتر از $slow$ جلو می‌رود؛ پس فاصلهٔ $fast$ تا رسیدن به $slow$ (در جهت حرکت) هر گام یکی کم می‌شود و حداکثر پس از λ گام صفر می‌شود.

(ب) زمان ملاقات مضرب λ است: اگر ملاقات در گام t رخ دهد، $slow$ مسافت t و $fast$ مسافت $2t$ را پیموده و هر دو در یک گره‌اند. پس $fast$ دقیقاً چند دور کامل بیشتر زده است: $2t - t = t$ مضربی از λ است.

(ج) گام ۲ به ورودی دور می‌رسد: p_1 از $head$ و p_2 از نقطهٔ ملاقات (یعنی t قدم پس از $head$) هم‌زمان یک‌قدمی جلو می‌روند. پس از μ قدم، p_1 در ورودی دور است و p_2 در نقطه‌ای $t + \mu$ قدم پس از $head$. چون t مضرب λ است و جای گره‌های درون دور هر λ قدم تکرار می‌شود، $t + \mu$ قدم همان جایی است که μ قدم: **ورودی دور**. قبل از آن هم نمی‌توانند به هم برسند، چون p_1 هنوز روی پیشوند است و p_2 درون دور.

■

مثال · اجرا روی مثال شکل

گره‌ها را به ترتیب پیمایش شماره کنید: $A{=}0, B{=}1, C{=}2, D{=}3, E{=}4$ و پس از آن دوباره C.

- گام ۱: $slow = B$، $fast = C$. گام ۲: $slow = C$، $fast = E$. گام ۳: $slow = D$، $fast = D$ ← **ملاقات در** D با $t = 3$ (مضرب $\lambda = 3$ ✓).

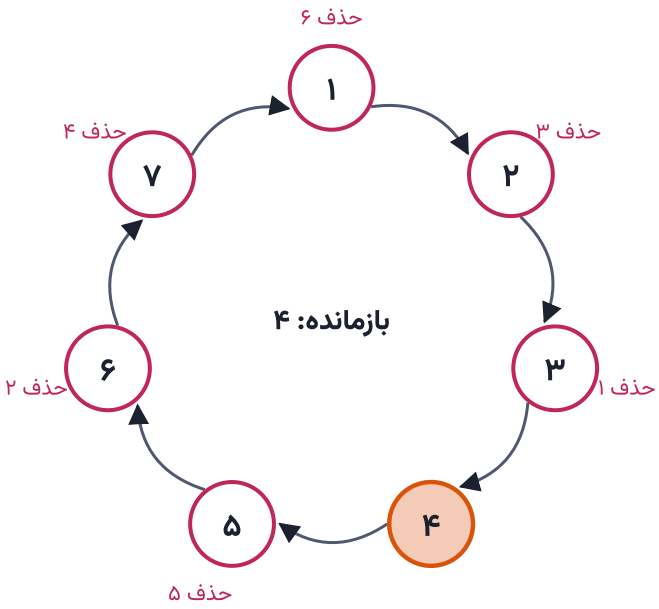
- $p_1 = C, p_2 = C \leftarrow p_1 = B, p_2 = E \leftarrow p_1 = A, p_2 = D$: **ورودی دور** C پس از $\mu = 2$ قدم.
- پیشوند: A, B یعنی ۲. دور: $C \to D \to E \to C$ یعنی ۳. جواب: 5.

زمان: $O(n)$ · **حافظهٔ اضافه:** $O(1)$. اگر لیست **کاملاً** حلقوی باشد (دم به سر برگردد)، کافی است از $head$ یک دور کامل بزنیم و بشماریم.

۵.۱۰ لیست حلقوی و مسئلهٔ ژوزفوس

تعریف · مسئلهٔ ژوزفوس

n نفر در یک دایره ایستاده‌اند. از نفر ۱ شروع می‌کنیم و ۱، ۲، ...، k می‌شماریم؛ نفرِ kام حذف می‌شود و شمارش از نفر بعدی از نو شروع می‌شود. آخرین نفر باقی‌مانده کیست؟



شکل ۵-۱۱ برای $n = 7$ و $k = 3$: ترتیب حذف $3, 6, 2, 7, 5, 1$ و بازمانده نفر 4.

لیست **حلقوی** (circular linked list) که در آن دم به سر وصل است، دقیقاً مدل همین دایره است: «نفر بعدی» همیشه $next$ است، حتی برای آخرین نفر.

```
JOSEPHUS(n, k)
```

```
build a circular list 1..n
current = node 1
previous = node n
while more than one node remains
    repeat k-1 times                    // walk k-1 steps
        previous = current
        current = current.next
    previous.next = current.next        // remove current
    delete current
    current = previous.next             // counting restarts here
return current.value
```

مثال · چند گام اول

شروع: $current = 1$. دو قدم: $current = 3$ ← حذف ۳؛ $current = 4$. دو قدم: $current = 6$ ← حذف ۶؛ $current = 7$. دو قدم (۷ ← ۱ ← ۲): حذف ۲؛ $current = 4$. به همین ترتیب ۷، ۵ و ۱ حذف می‌شوند و ۴ می‌ماند.

زمان: $n - 1$ حذف و برای هر کدام $k - 1$ قدم: $O(nk)$. برای k ثابت، $O(n)$. **حافظه:** $O(n)$.

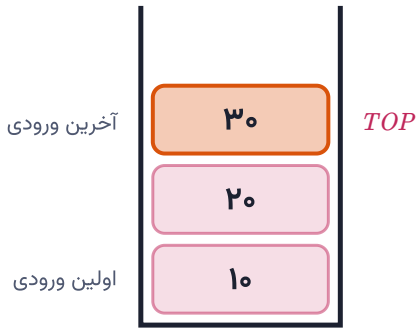
تکمیلی (فراتر از اسلاید) · فرمول بازگشتی

اگر افراد را از ۰ شماره کنیم و $J(n)$ شمارهٔ بازمانده باشد: $J(1) = 0$ و $J(n) = (J(n-1) + k) \bmod n$. (پس از اولین حذف، مسئله همان مسئلهٔ $n - 1$ نفره است که شماره‌هایش k واحد جابه‌جا شده.) برای $n = 7, k = 3$: $0, 1, 1, 0, 3, 0, 3$؛ یعنی شمارهٔ ۳ از صفر، که همان نفر ۴ است. این روش بدون شبیه‌سازی $O(n)$ زمان می‌برد.

۵.۱۱ پشته: آخرین ورودی، اولین خروجی

تعریف · پشته (Last In, First Out :LIFO)

عملیات: push(x) (افزودن x به بالا)، pop() (برداشتن عنصر بالا)، top() (دیدن عنصر بالا)، isEmpty() (آیا خالی است؟). **فقط عنصر بالا مستقیماً در دسترس است.**



شکل ۵-۱۲ پس از $\text{push}(10), \text{push}(20), \text{push}(30)$، اولین pop() عدد ۳۰ را برمی‌گرداند، سپس ۲۰ و سپس ۱۰.

تکمیلی (فراتر از اسلاید) · دو پیاده‌سازی $O(1)$

```
array-based stack (top = index of the top item, -1 if empty)
```

```
1  PUSH(x):  if top = capacity-1 then overflow
2            top = top + 1;  A[top] = x
3  POP():    if top = -1 then underflow
4            x = A[top];  top = top - 1;  return x
```

با لیست پیوندی: push همان «درج در ابتدا» و pop همان «حذف از ابتدا» است. هر دو $O(1)$. (چرا در ابتدا و نه انتها؟ چون حذف از انتهای لیست یک‌طرفه گرهٔ قبلی را لازم دارد.)

پشته کجا ظاهر می‌شود؟

کاربرد	چرا پشته؟
فراخوانی توابع	تابعی که تابع دیگری را صدا می‌زند معلق می‌شود. «پشتهٔ فراخوانی» محل بازگشت و متغیرهای محلی هر تابع را نگه می‌دارد.
برگرداندن (undo) و عقب‌گرد	آخرین کار، اولین کاری است که برگردانده می‌شود. جست‌وجوی عمق‌اول (DFS) هم صریح یا ضمنی (با بازگشت) از پشته استفاده می‌کند.
پشتهٔ یکنوا	پشته‌ای از مقادیر یا اندیس‌ها به ترتیب مرتب، برای مسائل «بزرگ‌تر/کوچک‌تر بعدی» (بخش بعد).

شهود · قدرت واقعی پشته

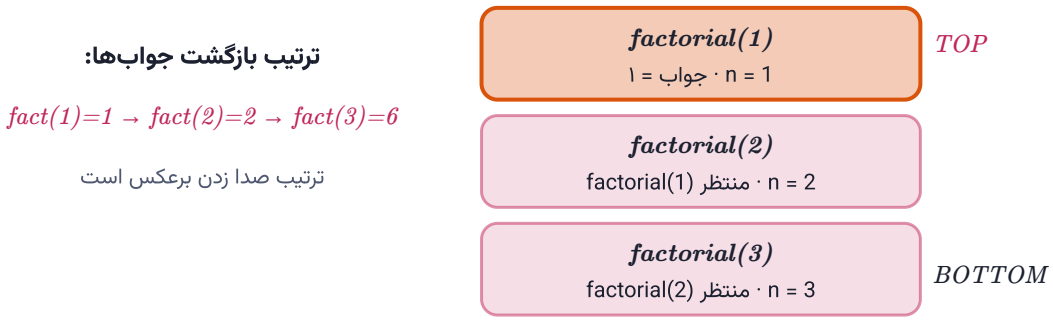
قدرت پشته فقط در عملیاتش نیست؛ در این است که ساختار **«تازه‌ترین کارِ هنوز حل‌نشده»** را دقیقاً مدل می‌کند. هر جا کارهایی تودرتو دارید (پرانتزها، فراخوانی‌ها، زیرمسئله‌ها) که آخرین شروع‌شده باید اول تمام شود، به پشته فکر کنید.

بازگشت از پشته استفاده می‌کند، حتی اگر آن را نبینید

```
FACTORIAL(n)
```

```
1  if n ⩽ 1 then return 1
2  return n * FACTORIAL(n-1)
```

factorial(3) تابع factorial(2) را صدا می‌زند و آن هم factorial(1) را، تا به حالت پایه برسیم. هر فراخوانی باید به خاطر بسپارد **کجا ادامه دهد** و **کدام** n مال اوست. این اطلاعات در یک «قاب» (frame) روی پشتهٔ زمان اجرا گذاشته می‌شود.



شکل ۵-۱۳ در عمیق‌ترین لحظه سه قاب روی پشته است. فراخوانی‌ها به ترتیب ۳، ۲، ۱ عمیق می‌شوند و جواب‌ها به ترتیب عکس برمی‌گردند: $6 \to 2 \to 1$.

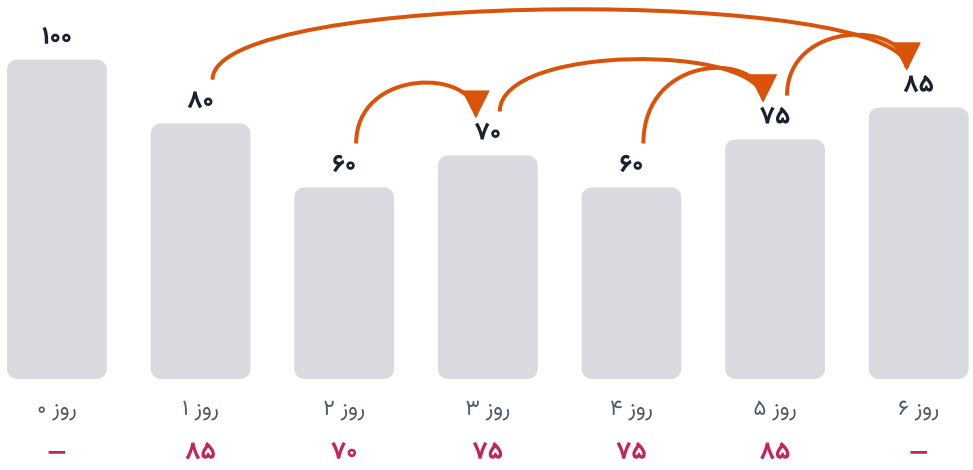
نکته · سرریز پشته

حافظهٔ پشتهٔ فراخوانی محدود است. بازگشتی که خیلی عمیق شود (مثلاً factorial(10^6) یا بازگشت بی‌پایان به‌خاطر حالت پایهٔ غلط) پشته را پر می‌کند و برنامه با خطای stack overflow متوقف می‌شود.

۵.۱۲ کاربرد پشته: نزدیک‌ترین قیمت بزرگ‌تر بعدی

تعریف · مسئله

قیمت‌های روزانهٔ یک سهم $P[0..n-1]$ داده شده است. برای هر روز i، قیمتِ **اولین** روز بعدی $j > i$ را پیدا کنید که $P[j] > P[i]$. اگر چنین روزی نبود، جواب -1 (یا –).



شکل ۵-۱۴ $P = 100, 80, 60, 70, 60, 75, 85$. هر پیکان از یک روز به نزدیک‌ترین روز بعدی با قیمت بیشتر می‌رود. ردیف پایین جواب است.

اشتباه رایج · «اولین» نه «بیشترین»

«اولین روز بعدی» یعنی **نزدیک‌ترین** اندیس سمت راست با قیمت **اکیداً** بیشتر، نه بیشترین قیمت آینده. مثلاً برای روز ۲ (قیمت ۶۰) جواب ۷۰ است، نه ۸۵.

الگوریتم ساده: $\Theta(n^2)$

NEXT-GREATER-BRUTE(P)

```
for i = 0 to n-1
    next[i] = -1
    for j = i+1 to n-1
        if P[j] > P[i] then
            next[i] = P[j]
            break
```

برای هر i ممکن است تقریباً همهٔ اندیس‌های بعدی را بررسی کنیم. در بدترین حالت (قیمت‌های نزولی، که هیچ‌کدام جواب ندارند) تعداد مقایسه‌ها $(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}=\Theta(n^2)$ است. این الگوریتم ساده و درست است، اما **اسکن‌های تکراری کاری را دوباره انجام می‌دهند** که پشته می‌تواند از آن پرهیز کند.

ایدهٔ پشته

- روزهایی را که هنوز جوابشان معلوم نیست («حل‌نشده») روی یک پشته نگه دار.
- وقتی قیمت جدیدی می‌رسد، **همهٔ** قیمت‌های کوچک‌ترِ بالای پشته را حل می‌کند: آن‌ها را pop کن و جوابشان را قیمت جدید بگذار.
- سپس اندیس روز جاری را push کن (جواب خودش هنوز معلوم نیست).

مثال · قیمت جاری ۷۵

اگر بالای پشته روز ۴ با قیمت ۶۰ باشد، ۷۵ اولین قیمت بعدی بزرگ‌تر از ۶۰ است: روز ۴ را pop کن و جوابش را ۷۵ بگذار. ادامه بده: روز ۳ (۷۰) هم کوچک‌تر است، pop و جواب ۷۵. وقتی بالای پشته $75 \geq$ شد (روز ۱ با ۸۰) یا پشته خالی شد، متوقف شو.

NEXT-GREATER(P) ▷ monotonic stack, Θ(n)

```
n = length(P)
ans = array of n values, all -1
create empty stack st                    // stores indices
for i = 0 to n-1
    while st is not empty and P[st.top] < P[i]
        ans[st.pop()] = P[i]             // P[i] resolves this day
    st.push(i)
return ans
```

i	P[i]	popped (answer = P[i])	stack after (indices: prices)
0	100	–	[0:100]
1	80	–	[0:100, 1:80]
2	60	–	[0:100, 1:80, 2:60]
3	70	**2**	[0:100, 1:80, 3:70]
4	60	–	[0:100, 1:80, 3:70, 4:60]
5	75	**4, 3**	[0:100, 1:80, 5:75]
6	85	**5, 1**	[0:100, 6:85]

در پایان روزهای ۰ و ۶ روی پشته مانده‌اند، پس جوابشان -1 می‌ماند. جواب نهایی: $-1, 85, 70, 75, 75, 85, -1$ ✓.

نکته · جزئیات مهم

- **اندیس ذخیره کن، نه فقط قیمت:** باید بدانیم جواب را در کدام خانهٔ ans بنویسیم.
- از $<$ برای «اکیداً بزرگ‌تر» استفاده کن؛ $\leq$ فقط اگر مسئله «بزرگ‌تر یا مساوی» بخواهد.
- اندیس‌هایی که حل نشده می‌مانند، -1 باقی می‌مانند.
- **ناوردا:** قیمت‌های روی پشته از پایین به بالا نزولی‌اند (به همین دلیل «پشتهٔ یکنوا»). چرا؟ هر عنصری که push می‌شود، قبلش همهٔ عناصر کوچک‌تر از خودش را از بالای پشته برداشته است.

اثبات · چرا $\Theta(n)$؟ (شمارش سرشکنی)

حلقهٔ `while` ممکن است در **یک** دور خیلی اجرا شود (مثلاً روز ۶ دو pop داشت)، پس ضرب «n دور × بدترین while» کران $O(n^2)$ می‌دهد. اما مثل پشتهٔ multipop در فصل ۴ (روش انبوهه)، از دید هر **اندیس** حساب کنید: هر اندیس دقیقاً یک بار push و حداکثر یک بار pop می‌شود. پس در کل اجرا حداکثر n push و n pop داریم و بقیهٔ بررسی‌ها هم $O(n)$ هستند:

$$\text{کل کار} = O(n) + O(n) = O(n).$$

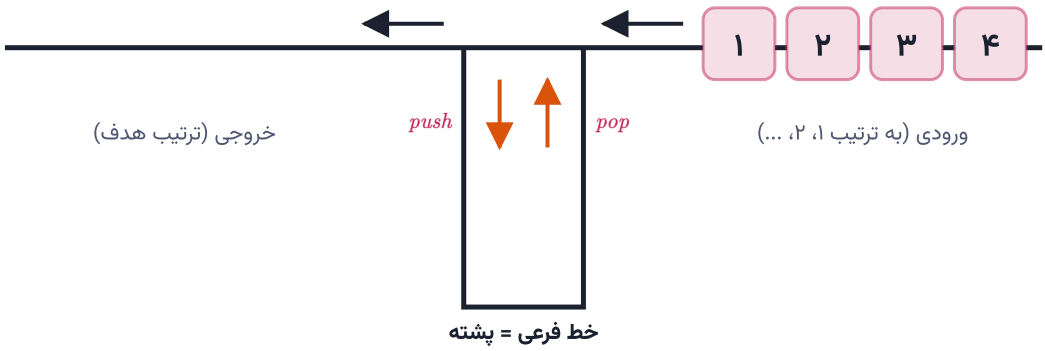
از طرف دیگر هر الگوریتمی باید همهٔ قیمت‌ها را دست‌کم یک بار بخواند، پس $\Omega(n)$ هم هست. در نتیجه $\Theta(n)$.

∎

۵.۱۳ کاربرد پشته: قطار و جایگشت‌ها

تعریف · مسئلهٔ خط فرعی

واگن‌ها به ترتیب $1, 2, \ldots, n$ وارد می‌شوند. یک خط فرعی (siding) مثل پشته رفتار می‌کند: واگنی که آخر وارد شده، اول خارج می‌شود. آیا می‌توان ترتیب خروجی دلخواه π را تولید کرد؟



شکل ۵-۱۵ هر واگن ورودی باید وارد خط فرعی شود (push) و بعداً از آن خارج شود (pop) و به خروجی برود.

مثال · هدف $2, 1, 4, 3$: ممکن

push ۱، push ۲؛ push ۳، push ۴؛ pop ۱. pop ۲؛ pop ۳، pop ۴. ✓

اشتباه رایج · هدف $3, 1, 2$: ناممکن

برای خروج ۳ باید ۱، ۲، ۳ را push کنیم و ۳ را pop کنیم. حالا ۲ روی ۱ است، پس ۱ نمی‌تواند خروجی بعدی باشد. ×

نکتهٔ کلیدی: محدودیت ترتیب پشته؛ واگنی که آخر وارد شده باید پیش از همهٔ واگن‌های زیرش خارج شود.

شرط لازم و کافی: شبیه‌سازی حریصانه

```
CAN-PRODUCE(π) ▷ π[1..n] is the target order
next_in = 1                    // next car waiting at the input
create empty stack S
for t = 1 to n                 // we want to output π[t]
    while (S is empty or S.top ≠ π[t]) and next_in ⩽ n
        S.push(next_in)
        next_in = next_in + 1
    if S.top = π[t] then S.pop()
    else return false          // input exhausted, top cannot match
return true
```

اثبات · چرا این شبیه‌سازی هم لازم است و هم کافی؟

فرض کنید خروجی بعدیِ مورد نیاز $x = \pi[t]$ باشد. فقط سه حالت ممکن است و در هر کدام **تنها یک کار** مجاز است:

- x **بالای پشته است:** باید همین حالا pop شود. هر push دیگری آن را زیر واگن دیگری دفن می‌کند که باید پیش از آن خارج شود، ولی خروجی بعدی باید x باشد.
- x **هنوز وارد نشده:** تنها راه رساندن x به بالای پشته، push کردن واگن‌های ورودی تا رسیدن به x است.
- x **در پشته است ولی بالا نیست:** واگن‌های بالای آن باید زودتر خارج شوند، ولی خروجی بعدی باید x باشد: **ناممکن**.

چون در هر گام انتخاب اجباری است، اگر شبیه‌سازی گیر کند هیچ راه دیگری هم وجود ندارد، و اگر تا آخر برود، خودش یک دنبالهٔ معتبر push/pop ساخته است. هر واگن یک بار push و یک بار pop می‌شود، پس زمان $\Theta(n)$ است.

∎

تکمیلی (فراتر از اسلاید) · چند جایگشت ممکن است؟

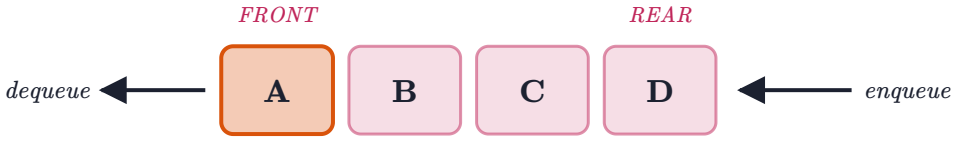
یک جایگشت دقیقاً وقتی تولیدشدنی است که الگوی «۳۱۲» نداشته باشد: سه موقعیت $i < j < k$ با $\pi_j < \pi_k < \pi_i$ (یعنی بزرگی، بعد کوچکی، بعد متوسطی). تعداد جایگشت‌های تولیدشدنی عدد کاتالان $C_n = \frac{1}{n+1}\binom{2n}{n}$ است. برای $n = 3$: از ۶ جایگشت، ۵ تا ممکن‌اند و فقط $3, 1, 2$ ناممکن است.

۵.۱۴ صف: اولین ورودی، اولین خروجی

تعریف · صف (FIFO: First In, First Out)

صف نوعی لیست است که فقط اجازهٔ **افزودن به انتها** و **حذف از ابتدا** را می‌دهد. عملیات:

- enqueue(x): درج x در انتهای صف (عقب).
- dequeue(): حذف و برگرداندن عنصر ابتدای صف (جلو).
- front(): برگرداندن عنصر ابتدای صف بدون حذف.
- isEmpty(): آیا صف خالی است؟ و size(): تعداد عناصر.



ورود از عقب، خروج از جلو: اولین ورودی، اولین خروجی

شکل ۵-۱۶ dequeue() عنصر A را برمی‌دارد؛ enqueue(E) عنصر E را پس از D می‌گذارد. مثل صف نانوایی.

صف با آرایه: چرا جلو و عقب مهم‌اند؟

اشتباه رایج · روش ساده: جابه‌جا کردن

صف همیشه از خانهٔ ۰ شروع شود. پس از حذف اولین عنصر، همهٔ عناصر باقی‌مانده یک خانه به چپ می‌روند. هر dequeue $\Theta(n)$ می‌شود و هدف یک صف کارا از بین می‌رود.

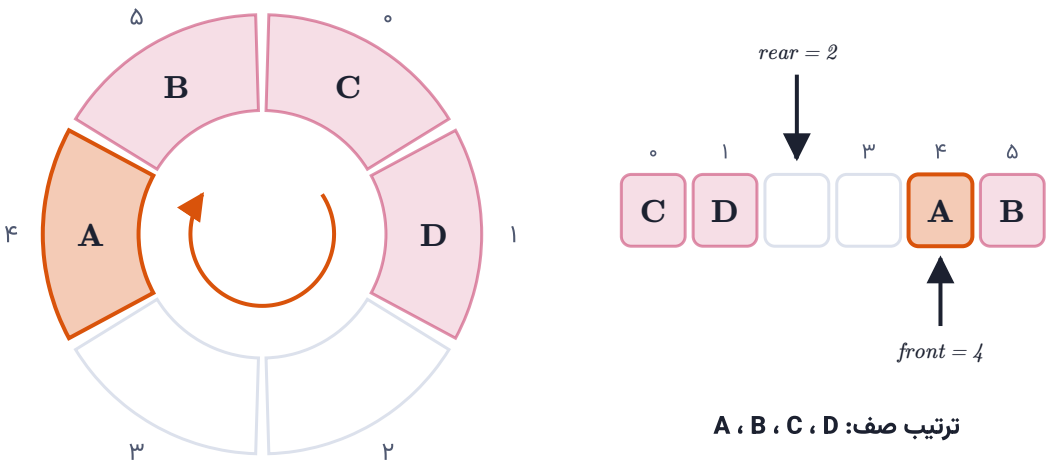
مثال · روش آرایهٔ حلقوی

به‌جای جابه‌جا کردن **عناصر**، **اندیس‌های** جلو و عقب را جابه‌جا کن. هر دو عمل $\Theta(1)$ می‌شوند.

صف حلقوی: اندیس‌ها دور می‌زنند

اگر فقط اندیس‌ها را جلو ببریم، دیر یا زود به انتهای آرایه می‌رسیم، در حالی که خانه‌های ابتدایی (که عناصرشان dequeue شده‌اند) خالی‌اند. راه حل: آرایه را **حلقوی** فرض کنیم؛ بعد از آخرین خانه، دوباره خانهٔ ۰ می‌آید:

$$\text{next}(i) = (i + 1) \bmod C.$$



شکل ۵-۱۷ آرایه‌ای با ظرفیت ۶. اندیس $front = 4$ به عنصری اشاره می‌کند که نوبت حذف آن است و اندیس $rear = 2$ جای درج بعدی است. صف از خانهٔ ۴ شروع می‌شود، تا ۵ می‌رود و سپس به ۰ و ۱ دور می‌زند.

CIRCULAR QUEUE with a count (capacity C)

```
isEmpty:  return count = 0
isFull:   return count = C

enqueue(x):
    if isFull then report overflow
    A[rear] = x
    rear = (rear + 1) mod C
    count = count + 1

dequeue():
    if isEmpty then report underflow
    x = A[front]
    front = (front + 1) mod C
    count = count - 1
    return x
```

$front$ اندیس عنصر بعدی برای حذف.

$rear$ اندیسی که عنصر بعدی در آن درج می‌شود.

$count$ تعداد عناصر ذخیره‌شده. خالی اگر و فقط اگر $count = 0$؛ پر اگر و فقط اگر $count = C$.

پر و خالی را چطور از هم تشخیص دهیم؟

چرا؟ · مشکل

اگر فقط $front$ و $rear$ را داشته باشیم، در صف **خالی** داریم $front = rear$. اما اگر C عنصر درج کنیم، دقیقاً یک دور می‌زند و دوباره $front = rear$ می‌شود: صف **پر** هم همین وضعیت را دارد! پس به اطلاعات بیشتری نیاز داریم.

مثال · روش الف: نگه داشتن count

$count = 0$ یعنی خالی و $count = C$ یعنی پر. ساده است و صف می‌تواند از **همهٔ** C خانه استفاده کند.

مثال · روش ب: یک خانه را خالی بگذار

خالی: $front = rear$. پر: $(rear + 1) \bmod C = front$. ظرفیت قابل استفاده $C - 1$ است؛ آن یک خانهٔ خالی ابهام پر/خالی را از بین می‌برد.

اشتباه رایج · دو قرارداد را با هم قاطی نکنید

ناوردایی که انتخاب می‌کنید، آزمون‌های درست را تعیین می‌کند. پیاده‌سازی‌ای که count نگه می‌دارد ولی پر بودن را با $(rear + 1) \bmod C = front$ می‌آزماید، یک خانه را بی‌دلیل هدر می‌دهد یا بدتر، خطا دارد.

مثال · ردگیری صف حلقوی با $C = 5$

operation	front	rear	count	array (indices 0..4)
initial	0	0	0	[_, _, _, _, _]
enqueue(A)	0	1	1	[A, _, _, _, _]
enqueue(B)	0	2	2	[A, B, _, _, _]
dequeue() → A	1	2	1	[(A), B, _, _, _]
enqueue(C)	1	3	2	[(A), B, C, _, _]
enqueue(D), enqueue(E)	1	**0**	4	[(A), B, C, D, E]

دقت کنید $rear$ پس از آخرین درج از ۴ به ۰ **دور زد**. A هنوز فیزیکی در خانهٔ ۰ هست، اما چون $front = 1$ است، دیگر جزو صف نیست و درج بعدی رویش نوشته می‌شود. هیچ عنصری جابه‌جا نشد.

هزینهٔ پیاده‌سازی‌های صف

پیاده‌سازی	enqueue	dequeue	نکتهٔ اصلی
آرایه + جابه‌جایی	$\Theta(1)$ در انتها	$\Theta(n)$	ساده، ولی حذف عناصر را جابه‌جا می‌کند
آرایهٔ حلقوی	$\Theta(1)$	$\Theta(1)$	ظرفیت ثابت یا سیاست بزرگ کردن (مثل آرایهٔ پویا)
لیست پیوندی با head و tail	$\Theta(1)$	$\Theta(1)$	مدیریت اشاره‌گرها؛ حافظهٔ اضافه برای هر گره

چرا؟ · چرا در لیست پیوندی درج در tail و حذف از head؟

درج بعد از tail و حذف از head هر دو فقط به اشاره‌گرِ موجود نیاز دارند: $\Theta(1)$. اما اگر برعکس کار کنیم، «حذف از tail» گرهٔ قبل از tail را لازم دارد که در لیست یک‌طرفه پیدا کردنش $\Theta(n)$ است.

۵.۱۵ جمع‌بندی: رابط را به هزینه وصل کنید

ساختار	قوت	محدودیت / احتیاط
نوع انتزاعی	قرارداد روشن، مستقل از نمایش	به‌تنهایی کارایی را مشخص نمی‌کند
آرایه	اندیس‌دهی $\Theta(1)$؛ محلی بودن عالی حافظه	درج/حذف وسط ممکن است $\Theta(n)$ جابه‌جایی بخواهد
لیست پیوندی یک‌طرفه	درج بعد از گرهٔ معلوم $\Theta(1)$	جست‌وجو / رسیدن به موقعیت i هزینهٔ $\Theta(n)$
پشته	LIFO؛ مناسب کارهای تودرتو و حل‌نشده	فقط عنصر بالا مستقیماً در دسترس است
صف حلقوی	enqueue و dequeue در $\Theta(1)$	ناوردای پر/خالی باید با دقت انتخاب شود

جمع‌بندی · پرسش درست

پرسش درست این نیست که «کدام داده‌ساختار بهترین است؟» بلکه این است که **«کدام عملیات باید کارا باشند؟»**

۵.۱۶ خودآزمایی فصل ۵

خودآزمایی

1. در پیاده‌سازی آرایه‌ای بخش «پیاده‌سازی لیست با آرایه»، پس از درج ۱۵ و حذف ۲۰، پیمایش از head چه خروجی‌ای می‌دهد؟ مقادیر $Next$ را بنویسید.

پاسخ: $Next[0] = 3$، $Next[3] = 1$، $Next[1] = \text{null}$ (خانهٔ ۲ آزاد است). خروجی: $10, 15, 30$.

2. آیا ترتیب خروجی $4, 3, 5, 1, 2$ با خط فرعی ممکن است؟

پاسخ: pop ۴، push ۱..۴؛ ۳ بالاست، pop ۳؛ pop ۵، push ۵؛ حالا بالای پشته ۲ است ولی ۱ لازم است و ورودی تمام شده: **ناممکن** (الگوی ۳۱۲: ۴، ۱، ۲).

3. جواب «بزرگ‌تر بعدی» را برای $2, 7, 3, 5, 4, 6, 8$ بنویسید.

پاسخ: $7, 8, 5, 6, 6, 8, -1$.

4. صف حلقوی با $C = 4$ و روش «یک خانه خالی» حداکثر چند عنصر نگه می‌دارد؟

پاسخ: $C - 1 = 3$ عنصر.

5. صفی با دو پشته بسازید. هزینهٔ سرشکن هر عمل چقدر است؟

پاسخ: enqueue: push در پشتهٔ ورودی. dequeue: اگر پشتهٔ خروجی خالی بود، همهٔ عناصر پشتهٔ ورودی را pop و در پشتهٔ خروجی push کن (ترتیبشان برعکس می‌شود)، سپس از پشتهٔ خروجی pop کن. هر عنصر حداکثر دو بار push و دو بار pop می‌شود، پس هزینهٔ سرشکن $O(1)$ است (روش انبوههٔ فصل ۴)، هرچند یک dequeue تکی ممکن است $\Theta(n)$ باشد.

6. چرا در الگوریتم فلوید لازم است $fast$ دقیقاً دو قدم برود؟ اگر سه قدم برود چه؟

پاسخ: با سرعت ۲، فاصلهٔ $fast$ تا $slow$ در هر گام دقیقاً یکی کم می‌شود، پس هرگز از روی $slow$ نمی‌پرد و حتماً به آن می‌رسد. با سرعت ۳ فاصله هر گام **دو** واحد (به پیمانهٔ λ) کم می‌شود؛ اگر طول دور زوج و فاصلهٔ اولیه فرد باشد، فاصله همیشه فرد می‌ماند و **هرگز به هم نمی‌رسند**. مثلاً $\lambda = 2$ و فاصلهٔ ۱.

فصل ۶

درخت

لیست پیوندی هر عنصر را به **یک** عنصر بعدی وصل می‌کرد. اگر هر عنصر بتواند چند عنصر بعدی داشته باشد، به درخت می‌رسیم: ساختاری برای داده‌های سلسله‌مراتبی، از شجره‌نامه تا پوشه‌های کامپیوتر.

۶

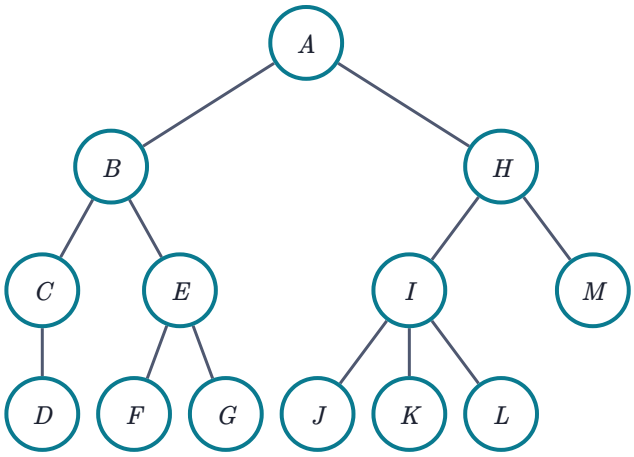
تعریف‌ها و واژه‌ها | انواع درخت | پیاده‌سازی | پیمایش‌ها | بازسازی درخت | درخت دودویی معادل

۶.۱ از لیست به درخت

یادآوری: در لیست پیوندی هر عنصر دو مؤلفه دارد:

1. **کلید:** مقداری که می‌خواهیم نگه داریم (می‌تواند یک یا چند مؤلفه باشد، مثلاً نام و شماره).
2. **بعدی:** اشاره‌گری به عنصر بعدی لیست (**تنها یک** عنصر).

حالا محدودیت دوم را برداریم: **اگر هر عنصر بتواند دو یا چند عنصر بعدی داشته باشد**، ساختار حاصل دیگر یک زنجیر نیست؛ شاخه شاخه می‌شود. این ساختار **درخت** است.



شکل ۶-۱ درخت نمونهٔ این فصل. A دو «بعدی» دارد (B و H)، I سه تا (J، K و L) و D هیچ.

شهود · درخت را کجا دیده‌اید؟

شجره‌نامهٔ خانوادگی، ساختار پوشه‌ها و فایل‌ها در کامپیوتر، نمودار سازمانی یک شرکت، فهرست مطالب همین جزوه (فصل ← بخش ← زیربخش). در همهٔ این‌ها هر چیز یک «والد» دارد و ممکن است چند «فرزند» داشته باشد.

۶.۲ درخت و جنگل

در نظریهٔ گراف، درخت را بدون هیچ ریشه یا جهتی تعریف می‌کنند:

تعریف · درخت

گرافی **همبند** و **بدون دور**.

تعریف · جنگل

گرافی **بدون دور** (نه لزوماً همبند). به بیان دیگر: **جنگل = چند درخت** کنار هم.

- **همبند** یعنی از هر رأس به هر رأس دیگر مسیری وجود دارد؛ گراف «یک‌تکه» است.
- **دور** یعنی مسیری که از یک رأس شروع شود و بدون تکرار یال به همان رأس برگردد.

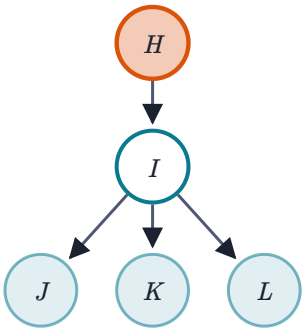
چرا؟ · چرا «بدون دور» مهم است؟

در گرافی همبند و بدون دور، بین هر دو رأس **دقیقاً یک** مسیر ساده وجود دارد: حداقل یکی چون همبند است، و حداکثر یکی چون اگر دو مسیر متفاوت بود، کنار هم یک دور می‌ساختند. همین «یکتایی مسیر» است که مفاهیمی مثل «عمق» و «پدر» را خوش‌تعریف می‌کند.

۶.۳ درخت جهت‌دار، ریشه و درخت ریشه‌دار

تعریف · درخت جهت‌دار و ریشه

درخت جهت‌دار: درختی که یال‌هایش جهت دارند. **ریشه:** گره‌ای که **هیچ یال ورودی** ندارد.



شکل ۶-۲H (نارنجی) ریشه است، چون هیچ پیکانی به آن وارد نمی‌شود. J، K و L (سبز) هیچ یال خروجی ندارند.

تعریف · درخت ریشه‌دار

درختی جهت‌دار که با نظمی بر حسب ریشه نمایش داده شده است: ریشه بالا و همهٔ یال‌ها رو به پایین (از ریشه دور می‌شوند). در این حالت معمولاً پیکان‌ها را نمی‌کشیم، چون جهت از روی شکل معلوم است.

از این به بعد، «درخت» یعنی درخت ریشه‌دار. با رسم ریشه در بالا، هر گره‌ای را می‌توان با یک مسیر یکتا از ریشه رو به پایین رسید (مثل مسیر A ← B ← E ← F در شکل اول).

۶.۴ روابط: درخت به‌عنوان شجره‌نامه

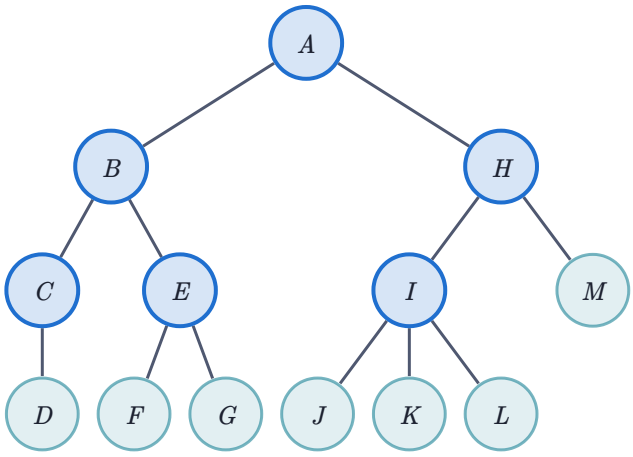
پدر و فرزند برای هر یال جهت‌دار $u \to v$، گره u **پدر** و گره v **فرزند** است. در شکل بالا: H پدر I است و I فرزند H. همچنین I پدر J، K و L است و آن‌ها فرزندان I.

برادر گره‌هایی که پدر یکسان دارند. J، K و L برادرند.

درجه درجهٔ (خروجی) هر رأس = **تعداد فرزندان** آن. $\deg(H) = 1$ و $\deg(I) = 3$.

برگ گرهٔ **بدون فرزند** (درجهٔ صفر).

گرهٔ داخلی گره‌ای که برگ **نیست** (دست‌کم یک فرزند دارد).



شکل ۶-۳ برگ‌ها (سبز): D، F، G، J، K، L و M. گره‌های داخلی (آبی): A، B، C، E، H و I.

نکته · درجه در درخت و در گراف

در نظریهٔ گراف، درجهٔ رأس تعداد **همهٔ** یال‌های متصل به آن است (شامل یال به پدر). در درخت ریشه‌دار، «درجه» معمولاً یعنی **تعداد فرزندان**. در این درس تعریف دوم را به کار می‌بریم.

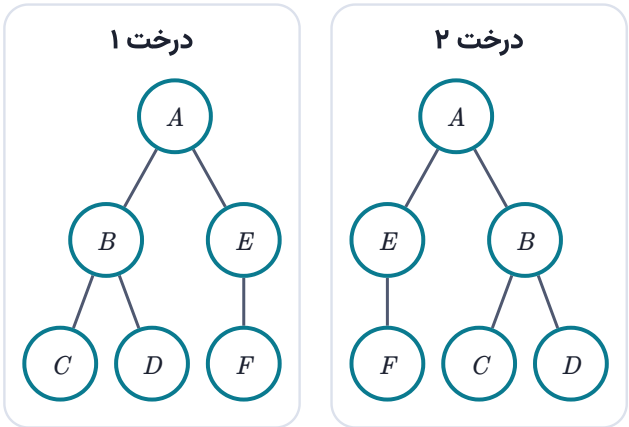
۶.۵ درخت ترتیب‌دار

تعریف · درخت بدون ترتیب

درختی که **ترتیب** فرزندان یک گره اهمیت ندارد. فقط مهم است چه کسی فرزند چه کسی است.

تعریف · درخت ترتیب‌دار

درختی که ترتیب فرزندان اهمیت دارد: «فرزند اول»، «فرزند دوم» و ... معنا دارند.



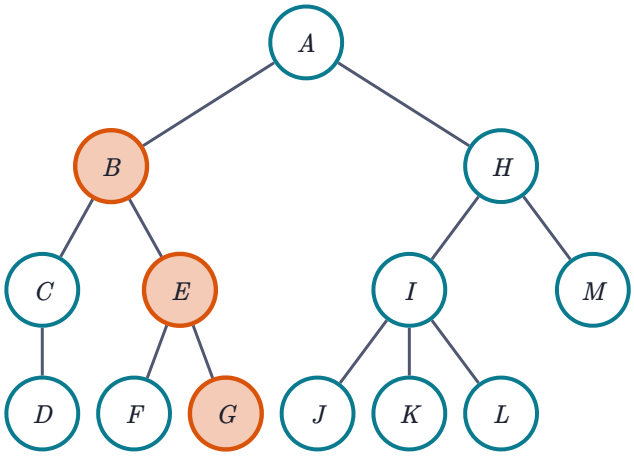
شکل ۶-۴ به‌عنوان درخت **بدون ترتیب** این دو یکی‌اند (روابط پدر و فرزندی یکسان است). به‌عنوان درخت **ترتیب‌دار** متفاوت‌اند، چون فرزند اول A در یکی B و در دیگری E است.

پیمایش‌ها (بخش‌های بعد) و درخت دودویی همیشه ترتیب‌دار در نظر گرفته می‌شوند، چون «اول زیردرخت اول را ببین» فقط در درخت ترتیب‌دار معنا دارد.

۶.۶ مسیر، اولاد و اجداد

تعریف · مسیر و طول مسیر

مسیر: دنباله‌ای به شکل $(a_0, a_1, \dots, a_k)$ به‌طوری که هر a_{i+1} فرزند a_i باشد. **طول مسیر:** تعداد **یال‌های** مسیر، یعنی k (نه تعداد گره‌ها).



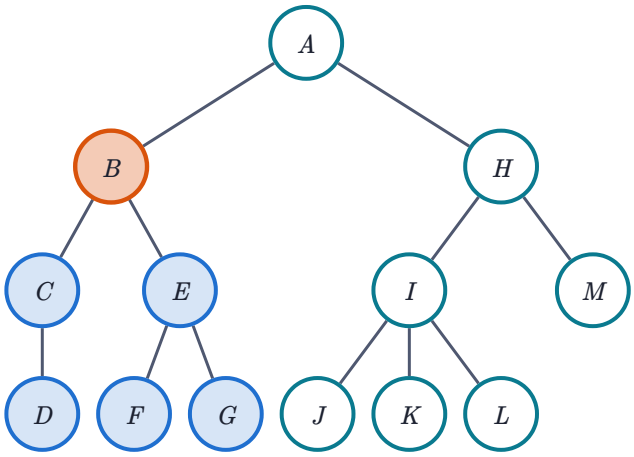
شکل ۶-۵ مسیر (B, E, G) با طول ۲: دو یال B–E و E–G.

تعریف · اولاد

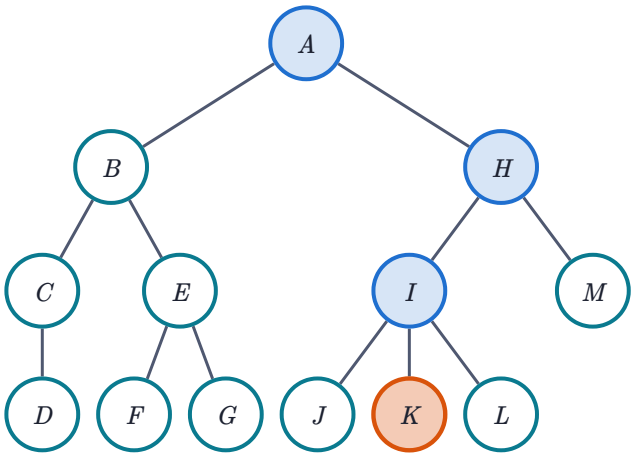
اولاد گرهٔ v: گره‌هایی که مسیری از v به آن‌ها وجود دارد. **زیردرخت گرهٔ v:** درختی شامل v و تمام اولاد آن، با v به‌عنوان ریشه.

تعریف · اجداد

اجداد گرهٔ v: تمام گره‌هایی که مسیری از آن‌ها به v وجود دارد. یعنی پدر، پدرِ پدر، ... تا ریشه.



شکل ۶-۶ اولاد B (آبی): C، D، E، F و G. همراه با خود B، زیردرخت B را می‌سازند.



شکل ۶-۷ اجداد K (آبی): H، I و A. مسیر از هر کدام از این‌ها به K می‌رسد.

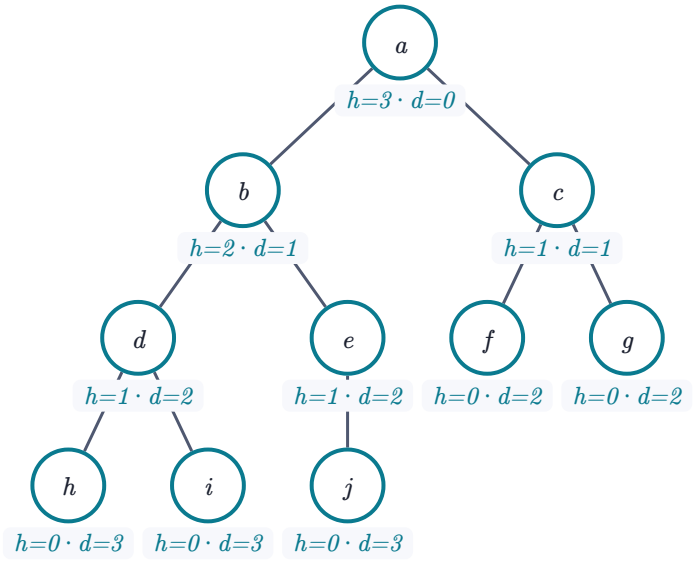
نکته · آیا v جدِ خودش است؟

چون مسیری به طول صفر (v) از v به خودش وجود دارد، با این تعریف هر گره جد و فرزندزادهٔ خودش هم هست. اگر بخواهیم خود v را کنار بگذاریم، می‌گوییم «جد **سره**» یا «اولاد سره» (proper). در شکل‌های بالا خود گره را جدا رنگ کرده‌ایم.

۶.۷ عمق، ارتفاع و سطح

تعریف · عمق و ارتفاع

- **عمق (سطح) گره:** طول مسیر از ریشه تا آن گره. چون در درخت فقط یک مسیر از ریشه به هر گره وجود دارد، این عدد خوش‌تعریف است. در شکل اول: عمق E برابر ۲ و عمق L برابر ۳ است.
- **ارتفاع گره:** طول مسیر از آن گره تا **دورترین برگ** زیردرختش.
- **ارتفاع درخت:** ارتفاع ریشه؛ یعنی بیشترین عمق یک برگ.



شکل ۶-۸ برای هر گره h ارتفاع و d عمق است. ارتفاع درخت = ارتفاع ریشه = ۳. برگ‌ها ارتفاع صفر دارند و ریشه عمق صفر.

شهود · عمق از بالا، ارتفاع از پایین

عمق را از ریشه رو به پایین می‌شماریم (ریشه صفر). ارتفاع را از برگ‌ها رو به بالا (برگ صفر). ارتفاع هر گره داخلی یکی بیشتر از بیشترین ارتفاع فرزندانش است:

$$h(v) = \begin{cases} 0 & \text{اگر } v \text{ برگ باشد} \\ 1 + \max\limits_{c \in \mathrm{ch}(v)} h(c) & \text{در غیر این صورت} \end{cases}$$

که در آن $\mathrm{ch}(v)$ مجموعهٔ فرزندان v است.

اشتباه رایج · «سطح» از ۰ یا از ۱؟

بعضی کتاب‌ها (و شکل اسلاید) سطح ریشه را ۱ می‌گیرند، یعنی سطح = عمق + ۱. بعضی سطح را همان عمق می‌دانند. در امتحان به قرارداد صورت سؤال دقت کنید؛ «عمق» و «ارتفاع» در همهٔ کتاب‌ها یکسان‌اند.

۶.۸ درخت‌های kتایی و انواع آن‌ها

درخت kتایی درختی که هر گره **حداکثر** k فرزند دارد.

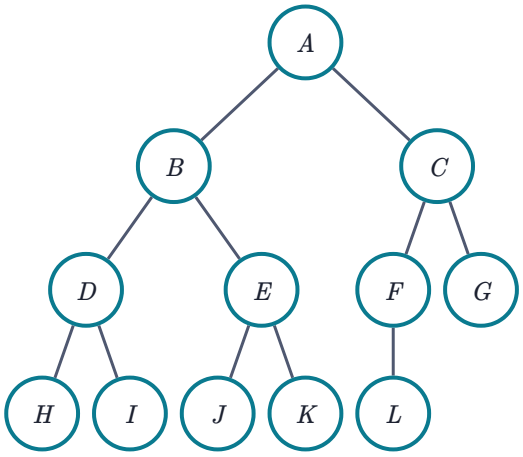
درخت دودویی نام دیگر درخت ۲تایی. در درخت دودویی، دو فرزند هر گره نام دارند: **فرزند چپ** و **فرزند راست**.

درخت kتایی کامل همهٔ سطح‌ها به‌جز سطح آخر کاملاً پرند و سطح آخر **از چپ به راست** پر شده است (بدون جای خالی در وسط).

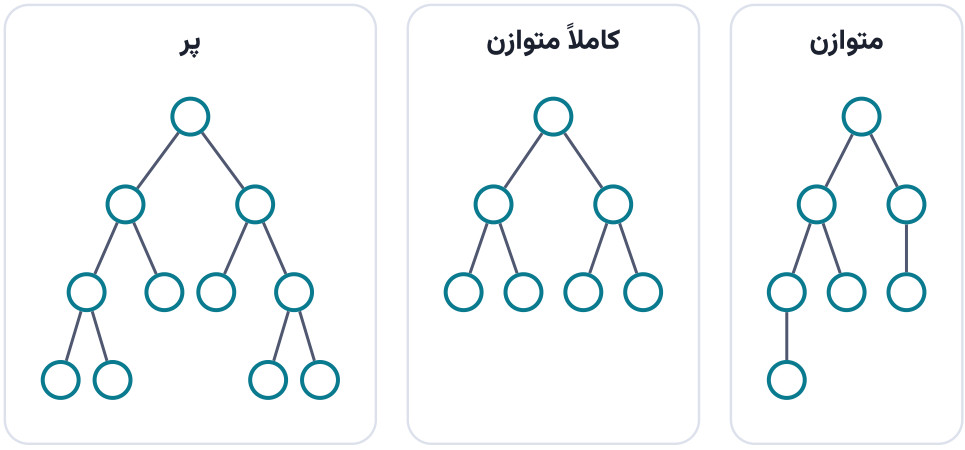
درخت kتایی پر هر گرهٔ غیربرگ **دقیقاً** k فرزند دارد.

درخت متوازن عمق برگ‌ها حداکثر ۱ واحد با هم اختلاف دارد.

درخت کاملاً متوازن عمق **همهٔ** برگ‌ها برابر است.



شکل ۶-۹ درخت دودویی کامل: سطح‌های ۰ تا ۲ پرند و سطح آخر (H تا L) از چپ پر شده است. اگر L فرزند G بود (به‌جای F)، دیگر کامل نبود.



شکل ۶-۱۰ «پر»: هر گرهٔ داخلی دو فرزند دارد. «کاملاً متوازن»: همهٔ برگ‌ها در یک عمق‌اند (و پر هم هست). «متوازن»: عمق برگ‌ها ۲ یا ۳ است، پس حداکثر ۱ اختلاف دارند.

اشتباه رایج · نام‌ها در کتاب‌ها فرق دارند

در منابع انگلیسی full، complete و perfect گاهی جابه‌جا به کار می‌روند و «متوازن» هم تعریف‌های متفاوتی دارد (مثلاً اختلاف ارتفاع دو زیردرخت هر گره حداکثر ۱). در این درس تعریف‌های بالا ملاک است.

۶.۹ ویژگی‌های پایه

قضیه · تعداد یال‌ها

درختی با n رأس دقیقاً $n-1$ یال دارد.

اثبات · با استقرا روی n

پایه ($n=1$): یک رأس و صفر یال. ✓

گام: فرض کنید حکم برای همهٔ درخت‌های $n-1$ رأسی درست باشد و T درختی با $n \geq 2$ رأس باشد.

- T **یک برگ دارد** (رأسی با فقط یک یال): طولانی‌ترین مسیر ساده در T را در نظر بگیرید. اگر انتهای آن یال دیگری داشت، یا آن یال به رأسی بیرون از مسیر می‌رفت (و مسیر طولانی‌تر می‌شد) یا به رأسی روی مسیر (و دور می‌ساخت). هر دو ناممکن‌اند.
- آن برگ و تنها یالش را حذف کنید. چیزی که می‌ماند هنوز همبند است (برگ روی هیچ مسیری بین دو رأس دیگر نبود) و بدون دور؛ پس درختی با $n-1$ رأس است و طبق فرض استقرا $n-2$ یال دارد.
- با برگرداندن برگ: $n-2+1 = n-1$ یال. ✓

∎

شهود · اثبات یک‌خطی برای درخت ریشه‌دار

هر گره به‌جز ریشه دقیقاً **یک پدر** دارد و هر یال دقیقاً یک فرزند را به پدرش وصل می‌کند. پس تعداد یال‌ها = تعداد گره‌های غیرریشه = $n-1$.

تکمیلی (فراتر از اسلاید) · ارتفاع درخت دودویی کامل

درخت دودویی به ارتفاع h حداکثر $1+2+4+\cdots+2^h = 2^{h+1}-1$ گره دارد (سطح i حداکثر 2^i گره). در درخت دودویی **کامل** با ارتفاع h، سطح‌های ۰ تا $h-1$ پرند (2^h-1 گره) و سطح آخر بین ۱ و 2^h گره دارد؛ پس

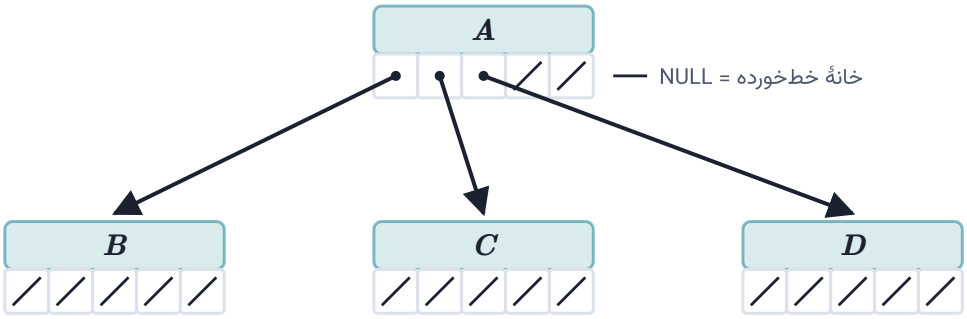
$$2^h \leq n \leq 2^{h+1}-1 \quad \Longrightarrow \quad h = \lfloor \log_2 n \rfloor .$$

این نتیجه در فصل ۸ (هرم) اساسی است: همهٔ عملیات هرم متناسب با ارتفاع‌اند، یعنی $O(\log n)$.

۶.۱۰ پیاده‌سازی درخت با اشاره‌گر

همان ایدهٔ گرهٔ لیست را گسترش می‌دهیم. برای درخت kتایی، هر عنصر $k+1$ مؤلفه دارد:

1. **کلید:** مقداری که می‌خواهیم نگه داریم.
2. k **متغیر فرزند:** اشاره‌گرهایی به فرزندان (می‌توان آن‌ها را به‌صورت آرایه یا لیست نگه داشت).



شکل ۶-۱۱ گرهٔ A با ۵ خانهٔ اشاره‌گر برای یک درخت ۵تایی. سه خانه به B، C و D اشاره می‌کنند و دو خانهٔ دیگر NULL هستند (خط‌خورده). برگ‌های B، C و D هم ۵ خانه دارند که همه NULL است؛ این همان هدررفت حافظه است.

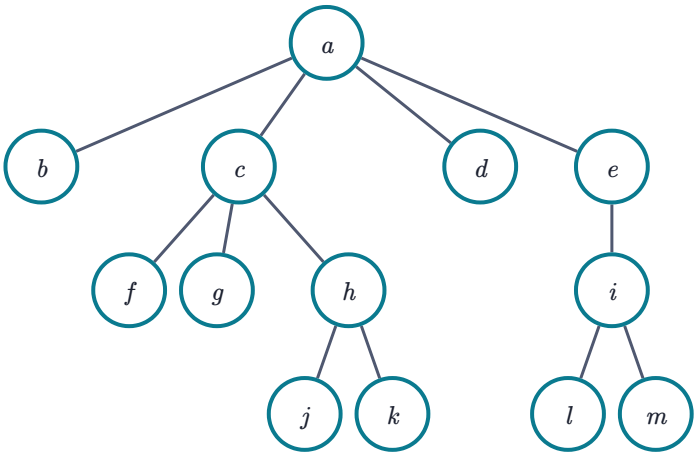
چرا؟ · ایراد این روش: هدر رفتن حافظه

n گره در کل nk خانهٔ اشاره‌گر دارند، ولی درخت فقط $n-1$ یال دارد. پس $nk-(n-1)=n(k-1)+1$ اشاره‌گر **NULL** است. برای $k=10$، بیش از ۹۰٪ خانه‌ها خالی‌اند. راه حل این مشکل «درخت دودویی معادل» است که در آخر فصل می‌بینیم.

۶.۱۱ پیاده‌سازی درخت با آرایه

روش ۱: هر گره، اندیس پدرش را نگه می‌دارد

هر گره دو متغیر نگه می‌دارد: (۱) کلید، (۲) اندیس پدر خود (ریشه پدر ندارد و ۰ می‌گیرد).



شکل ۶-۱۲ درخت نمونه با ۱۳ گره که در جدول زیر ذخیره شده است.

Index	1	2	3	4	5	6	7	8	9	10	11	12	13
Key	a	b	c	d	e	f	g	h	i	j	k	l	m
P	0	1	1	1	1	3	3	3	5	8	8	9	9

مثلاً $P[8] = 3$ یعنی پدرِ h (خانهٔ ۸) گرهٔ c (خانهٔ ۳) است. رفتن به پدر (و در نتیجه به ریشه) سریع است: $O(1)$ برای هر قدم.

چرا؟ · مشکل این روش چیست؟

دسترسی سریع به فرزندان ممکن نیست. برای پیدا کردن فرزندان c باید کل آرایه را بگردیم و ببینیم P کدام خانه‌ها ۳ است: $\Theta(n)$ برای هر گره. پیمایش‌هایی که از ریشه رو به پایین می‌روند (مثل پیمایش‌های بخش بعد) با این نمایش بسیار کند می‌شوند.

روش ۲: اندیس پدر و اندیس فرزندان

اگر بخواهیم به فرزندان هم دسترسی داشته باشیم، برای درخت kتایی هر گره $k + 2$ متغیر نگه می‌دارد: (۱) کلید، (۲) اندیس پدر، (۳) k اندیس فرزندان.

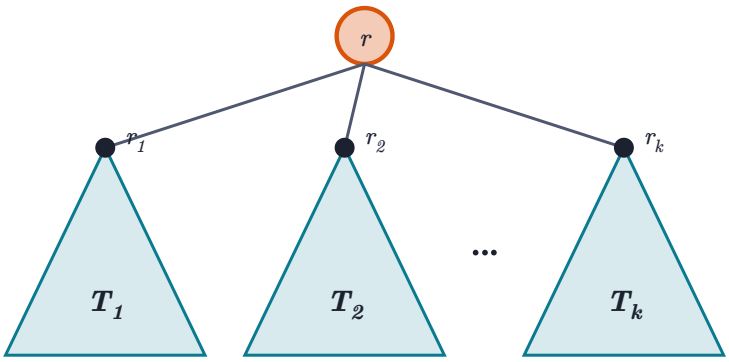
Index	1	2	3	4	5	6	7	8	9	10	11	12	13
Key	a	b	c	d	e	f	g	h	i	j	k	l	m
P	0	1	1	1	1	3	3	3	5	8	8	9	9
Ch 1	2		6		9			10	12				
Ch 2	3		7					11	13				
Ch 3	4		8										
Ch 4	5												

حالا فرزندان هر گره در $O(k)$ در دسترس‌اند، ولی همان مشکل هدر رفتن حافظه (این بار خانه‌های خالی جدول) برمی‌گردد.

۶.۱۲ تعریف بازگشتی و پیمایش درخت

تعریف · تعریف بازگشتی درخت

درخت T یا تهی است، یا از این‌ها تشکیل می‌شود: یک **ریشه** r و **زیردرخت‌های** $T_1, \dots, T_k$ که هر کدام خودشان درخت‌اند و ریشه‌هایشان $r_1, \dots, r_k$ فرزندان r هستند.



شکل ۶-۱۳ هر زیردرخت خودش یک درخت کامل است. پس هر الگوریتمی روی درخت را می‌توان بازگشتی نوشت: «برای ریشه کاری بکن و بقیه را به زیردرخت‌ها بسپار».

پیمایش (traversal) یعنی خواندن همهٔ عناصر درخت، هر کدام دقیقاً یک بار. سه روش استاندارد داریم که فقط در **زمان دیدن ریشه** با هم فرق دارند:

$$\text{preorder}(T) = r,\ \text{preorder}(T_1),\ \text{preorder}(T_2),\ \ldots,\ \text{preorder}(T_k)$$
$$\text{inorder}(T) = \text{inorder}(T_1),\ r,\ \text{inorder}(T_2),\ \ldots,\ \text{inorder}(T_k)$$
$$\text{postorder}(T) = \text{postorder}(T_1),\ \text{postorder}(T_2),\ \ldots,\ \text{postorder}(T_k),\ r$$

پیش‌ترتیب (preorder) ریشه **پیش از** زیردرخت‌ها.

میان‌ترتیب (inorder) ریشه **میان** زیردرخت اول و بقیه.

پس‌ترتیب (postorder) ریشه **پس از** همهٔ زیردرخت‌ها.

PREORDER(v) / INORDER(v) / POSTORDER(v)

```
PREORDER(v):
    if v = null then return
    visit(v)
    for each child c of v (in order)
        PREORDER(c)

INORDER(v):
    if v = null then return
    INORDER(first child of v)
    visit(v)
    for each remaining child c of v (in order)
        INORDER(c)

POSTORDER(v):
    if v = null then return
    for each child c of v (in order)
        POSTORDER(c)
    visit(v)
```

مثال · سه پیمایش روی درخت نمونه (ریشهٔ a)

Preorder $a, b, c, f, g, h, j, k, d, e, i, l, m$

Inorder $b, a, f, c, g, j, h, k, d, l, i, m, e$

Postorder $b, f, g, j, k, h, c, d, l, m, i, e, a$

میان‌ترتیب را گام‌به‌گام بسازیم:

$$\text{inorder}(a) = \text{inorder}(b),\ a,\ \text{inorder}(c),\ \text{inorder}(d),\ \text{inorder}(e)$$

$\text{inorder}(b) = b$ (برگ). $\text{inorder}(c) = f, c, g$، $\text{inorder}(h) = f, c, g, j, h, k$. $\text{inorder}(d) = d$. $\text{inorder}(e) = e, l, i, m, e$ (فقط یک فرزند دارد، پس ریشه پس از آن می‌آید). کنار هم:

✓ $b, a, f, c, g, j, h, k, d, l, i, m, e$.

شهود · قدم زدن دور درخت

تصور کنید از بالای ریشه شروع کرده و دور درخت راه می‌روید، طوری که دست چپتان همیشه به درخت بچسبد. به هر گره چند بار می‌رسید. **پیش‌ترتیب** = گره‌ها به ترتیب **اولین** رسیدن. **پس‌ترتیب** = به ترتیب **آخرین** رسیدن (وقتی برای همیشه ترکش می‌کنید). **میان‌ترتیب** در درخت دودویی = وقتی از زیر گره (بین فرزند چپ و راست) رد می‌شوید.

چرا؟ · چرا زمان پیمایش $\Theta(n)$ است؟

هر گره دقیقاً یک بار «ملاقات» می‌شود و تابع برای هر گره یک بار صدا زده می‌شود. کار درون هر فراخوانی (بدون فراخوانی‌های بازگشتی) $O(1+\deg(v))$ است. جمع: $\sum_v (1+\deg(v)) = n + (n-1) = \Theta(n)$، چون جمع درجه‌ها همان تعداد یال‌هاست.

۶.۱۳ مسئلهٔ ۱: بازسازی درخت از پیش‌ترتیب و پس‌ترتیب

تعریف · صورت مسئله

اگر پیمایش preorder و postorder یک درخت داده شده باشد، آیا می‌توان پیمایش inorder آن را پیدا کرد؟

Preorder `MNHCRSKWTGDXIYAJPOEZVBULQF`

Postorder `CWTKSGRHDNAOEPJYZIBQLFUVXM`

راهبرد: درخت را **می‌سازیم**؛ وقتی درخت را داشته باشیم، inorder با تعریف به دست می‌آید. کلید ساختن، باز کردن یک مرحله از تعریف دو پیمایش است:

$$\begin{aligned}\text{preorder}(T) &= r,\ \ r_1, \text{preorder}(T_1 - r_1),\ \ r_2, \text{preorder}(T_2 - r_2),\ \ \ldots,\ \ r_k, \text{preorder}(T_k - r_k)\\ \text{postorder}(T) &= \text{postorder}(T_1 - r_1), r_1,\ \ \text{postorder}(T_2 - r_2), r_2,\ \ \ldots,\ \ \text{postorder}(T_k - r_k), r_k,\ \ r\end{aligned}$$

(منظور از $T_i - r_i$ زیردرخت T_i بدون ریشه‌اش است.) از این دو تساوی سه مشاهده بیرون می‌آید:

۱. **پیدا کردن r:** ریشه **اولین** حرف preorder و **آخرین** حرف postorder است: $r = M$.

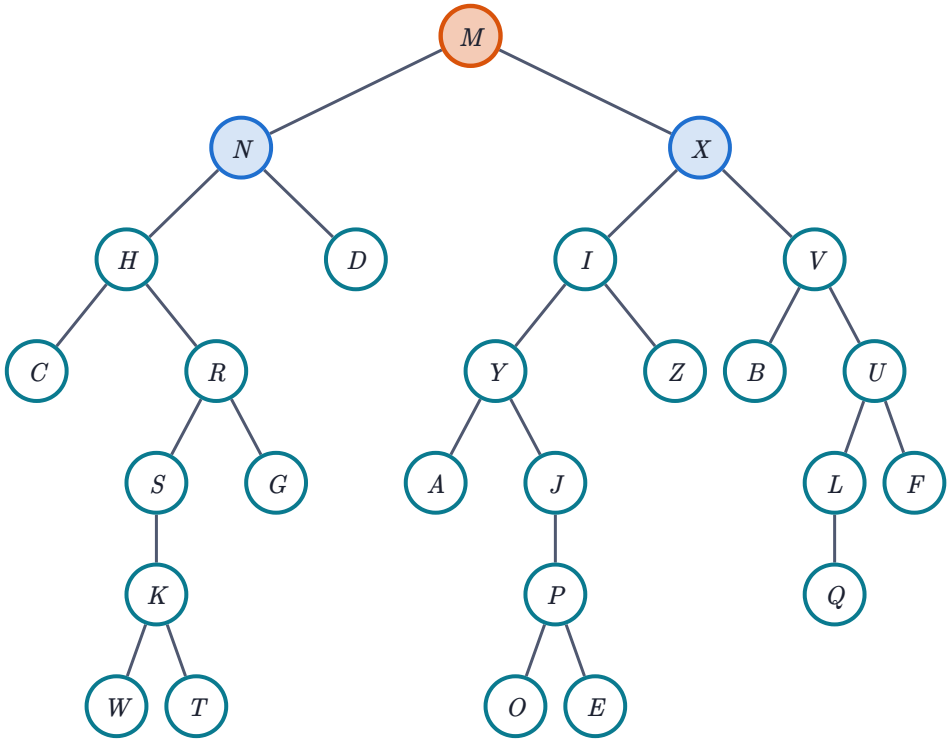
۲. **پیدا کردن r_1:** ریشهٔ زیردرخت اول، **دومین** حرف preorder است: $r_1 = N$.

۳. **پیدا کردن T_1:** در postorder، زیردرخت T_1 دقیقاً جایی **تمام می‌شود** که r_1 آمده است. پس T_1 در postorder برابر `CWTKSGRHDN` است (۱۰ حرف). در preorder هم همان ۱۰ حرفِ پس از M: `NHCRSKWTGD`.

۴ **پیدا کردن r_2 و T_2:** حرف بعدی preorder ریشهٔ زیردرخت دوم است: $r_2 = X$. در postorder، T_2 از ادامه تا X است: `AOEPJYZIBQLFUVX` (۱۵ حرف). دیگر حرفی جز M نمانده، پس M فقط دو فرزند N و X دارد.

۵ همین کار را **بازگشتی** برای هر زیردرخت تکرار می‌کنیم.

preorder	postorder	root	children (subtrees)
MNHCRSKWTGDXIYAJPOEZVBULQF	CWTKSGRHDNAOEPJYZIBQLFUVXM	M	N, X
NHCRSKWTGD	CWTKSGRHDN	N	H, D
HCRSKWTG	CWTKSGRH	H	C, R
RSKWTG	WTKSGR	R	S, G
SKWT	WTKS	S	K
KWT	WTK	K	W, T
XIYAJPOEZVBULQF	AOEPJYZIBQLFUVX	X	I, V
IYAJPOEZ	AOEPJYZI	I	Y, Z
YAJPOE	AOEPJY	Y	A, J
JPOE	OEPJ	J	P
POE	OEP	P	O, E
VBULQF	BQLFUV	V	B, U
ULQF	QLFU	U	L, F
LQ	QL	L	Q



شکل ۶-۱۴ درخت بازسازی‌شده. ریشه M (نارنجی) و ریشه‌های دو زیردرخت اول N و X (آبی).

حالا inorder را با تعریف حساب می‌کنیم (زیردرخت اول، ریشه، بقیهٔ زیردرخت‌ها):

مثال · جواب

```
Inorder: CHWKTSRGNDMAYOPEJIZXBVQLUF
```

مثلاً $\text{inorder}(N) = \text{inorder}(H), N, \text{inorder}(D)$ و $\text{inorder}(H) = C, H, \text{inorder}(R)$ و $\text{inorder}(R) = \text{inorder}(S), R, G = W, K, T, S, R, G$. پس $\text{inorder}(N) = C, H, W, K, T, S, R, G, N, D$.

چرا؟ · چرا جواب یکتاست؟

در هر گام، تقسیم پیمایش‌ها به «ریشه» و «زیردرخت‌ها» **اجباری** بود: ریشه و مرز هر زیردرخت را دقیقاً از روی دو رشته خواندیم و هیچ انتخابی نکردیم. پس فقط یک درخت ترتیب‌دار با این دو پیمایش وجود دارد، و inorder آن یکتاست.

اشتباه رایج · در درخت دودویی این روش کار نمی‌کند

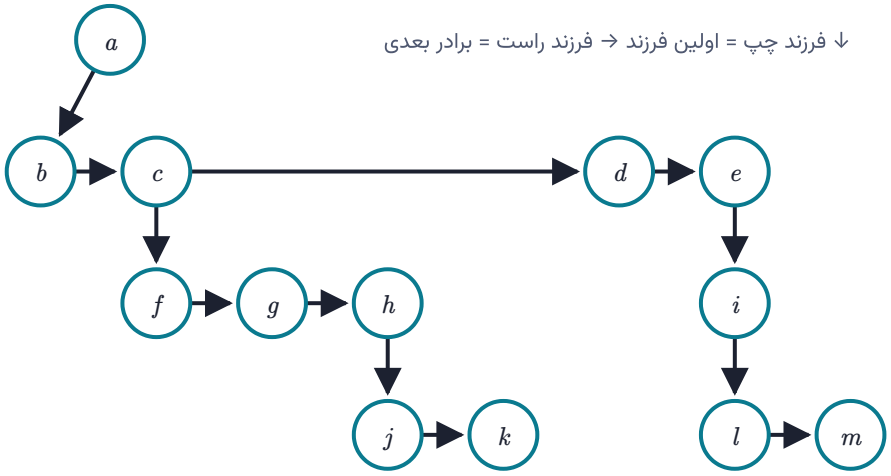
در درخت **دودویی**، فرزند تنها می‌تواند «چپ» یا «راست» باشد. دو درخت «A با فرزند چپ B» و «A با فرزند راست B» هر دو preorder برابر AB و postorder برابر BA دارند، ولی inorder یکی BA و دیگری AB است. پس برای درخت دودویی، preorder و postorder با هم کافی نیستند؛ ترکیب «preorder + inorder» یا «postorder + inorder» کافی است.

۶.۱۴ درخت دودویی معادل: فرزند چپ، برادر راست

دیدیم پیاده‌سازی درخت kتایی با k اشاره‌گر حافظه هدر می‌دهد. ترفند زیبایی وجود دارد: **هر درخت kتایی را می‌توان با یک درخت دودویی نمایش داد.** هر گره فقط دو اشاره‌گر دارد:

فرزند چپ اشاره به **اولین فرزند** گره در درخت اصلی.

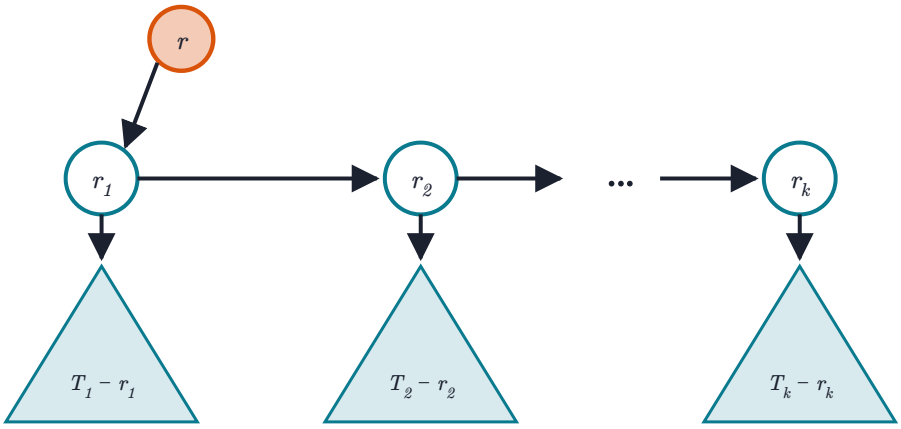
فرزند راست اشاره به **برادر بعدی** گره (فرزند بعدی پدرش).



شکل ۶-۱۵ درخت دودویی معادل درخت نمونه. پیکان‌های رو به پایین «فرزند چپ» و پیکان‌های افقی «فرزند راست» (برادر بعدی) هستند. مثلاً c سه فرزند f، g و h داشت: حالا c فقط به f اشاره می‌کند و f به g و g به h.

چرا؟ · چرا این نمایش اطلاعاتی از دست نمی‌دهد؟

فرزندان هر گره را می‌توان با شروع از «اولین فرزند» و دنبال کردن زنجیرهٔ «برادر بعدی» به ترتیب به دست آورد. پس درخت اصلی کاملاً قابل بازسازی است. هزینه: هر گره دقیقاً ۲ اشاره‌گر دارد، یعنی $2n$ اشاره‌گر که فقط $n+1$ تا NULL هستند (به‌جای $n(k-1)+1$).



شکل ۶-۱۶ شکل کلی: ریشهٔ r به اولین فرزندش r_1 اشاره می‌کند، r_1 به برادرش r_2، r_2 به برادر بعدی و همین‌طور تا r_k. زیر هر r_i درخت دودویی معادلِ باقی‌ماندهٔ زیردرخت T_i آویزان است.

۶.۱۵ مسئلهٔ ۲ و ۳: پیمایش‌ها در درخت دودویی معادل

مسئلهٔ ۲: آیا preorder یک درخت با preorder درخت دودویی معادلش برابر است؟

مثال · مثال

preorder درخت نمونه: `abcfghjkdeilm` . preorder درخت دودویی معادلش (ریشه، زیردرخت چپ، زیردرخت راست): `abcfghjkdeilm` . **برابرند!**

قضیه · پاسخ مسئلهٔ ۲

بله؛ برای هر درخت T: $\text{preorder}_{\text{دودویی}}(\text{LCRS}(T)) = \text{preorder}(T)$.

حکم کلی‌تری را برای یک **جنگل** ثابت می‌کنیم (دنباله‌ای از درخت‌ها $T_1, \dots, T_k$ که ریشه‌هایشان برادرند). درخت دودویی معادل جنگل، ریشه‌اش r_1 است؛ زیردرخت چپش معادلِ جنگلِ فرزندان r_1 (یعنی $T_1 - r_1$) و زیردرخت راستش معادلِ جنگلِ $T_2, \dots, T_k$ است. با استقرا روی تعداد گره‌ها:

$$\begin{aligned}\text{pre}_{\text{دودویی}} &= r_1,\ \underbrace{\text{pre}_{\text{دودویی}}(\text{چپ})}_{=\ \text{preorder}(T_1 - r_1)},\ \underbrace{\text{pre}_{\text{دودویی}}(\text{راست})}_{=\ \text{preorder}(T_2),\dots,\text{preorder}(T_k)} \\ &= \text{preorder}(T_1),\ \text{preorder}(T_2),\ \dots,\ \text{preorder}(T_k).\end{aligned}$$

(تساوی‌های زیر آکولادها فرض استقراست، چون آن جنگل‌ها گرهٔ کمتری دارند.) درخت T هم جنگلی تک‌درختی است، پس حکم برقرار است. همان مراحل اسلاید: r، سپس $\text{preorder}(T')$ برای جنگل فرزندان، که به $r_1, \text{preorder}(T_1 - r_1), \text{preorder}(T'')$ باز می‌شود و الی آخر.

∎

مسئلهٔ ۳: آیا postorder یک درخت با postorder درخت دودویی معادلش برابر است؟

مثال · مثال نقض

postorder درخت نمونه: `bfgjkhcdlmiea` . postorder درخت دودویی معادل (چپ، راست، ریشه): `kjhgfmliedcba` . **برابر نیستند.**

علت: در postorder دودویی، گرهٔ b باید پس از زیردرخت راستش (یعنی همهٔ برادرهای بعدی‌اش c، d، e و اولادشان) بیاید، در حالی که در درخت اصلی b هیچ ربطی به برادرهایش ندارد و همان اول ظاهر می‌شود.

تکمیلی (فراتر از اسلاید) · پس postorder به چه چیزی تبدیل می‌شود؟

به **inorder** درخت دودویی معادل! inorder دودویی (چپ، ریشه، راست) درخت مثال: `bfgjkhcdlmiea` که دقیقاً postorder درخت اصلی است. اثبات مثل مسئلهٔ ۲:

$$\text{in}_{\text{دودویی}} = \underbrace{\text{in}(\text{چپ})}_{\text{postorder}(T_1 - r_1)}, r_1, \underbrace{\text{in}(\text{راست})}_{\text{postorder}(T_2), \ldots} = \text{postorder}(T_1), \text{postorder}(T_2), \ldots$$

چون $\text{postorder}(T_1) = \text{postorder}(T_1 - r_1), r_1$.

جمع‌بندی · فصل ۶ در یک نگاه

- درخت = گراف همبند بدون دور؛ بین هر دو رأس دقیقاً یک مسیر. n رأس، $n-1$ یال.
- عمق از ریشه، ارتفاع از برگ؛ ارتفاع درخت دودویی کامل $\lfloor \log_2 n \rfloor$.
- پیاده‌سازی: k اشاره‌گر (هدر رفتن حافظه)، آرایهٔ پدر (کندی رسیدن به فرزندان)، یا درخت دودویی معادل (دو اشاره‌گر).
- پیش‌ترتیب، میان‌ترتیب، پس‌ترتیب: فقط زمان دیدن ریشه فرق دارد؛ همه $\Theta(n)$.
- پیش‌ترتیب + پس‌ترتیب یک درخت ترتیب‌دار را یکتا مشخص می‌کنند (ولی درخت دودویی را نه).
- در تبدیل به درخت دودویی معادل: preorder ← preorder و inorder ← postorder.

۶.۱۶ خودآزمایی فصل ۶

خودآزمایی

1. در درخت نمونهٔ اول فصل (ریشهٔ A)، عمق G، ارتفاع B و ارتفاع درخت چقدر است؟

پاسخ: عمق (A–B–E–G) ۳ = G. ارتفاع (B–E–F) ۲ = B. ارتفاع درخت = ۳.

2. درخت دودویی با preorder برابر `ABDECF` و inorder برابر `DBEAFC` را بسازید و postorder آن را بنویسید.

پاسخ: ریشه A (اول preorder). در inorder، چپ A: DBE و راست: FC. زیردرخت چپ: preorder BDE ← ریشه B، فرزند چپ D و راست E. زیردرخت راست: preorder CF و inorder FC ← ریشه C با فرزند چپ F. postorder: `DEBFCA`.

3. درخت دودویی به ارتفاع h حداقل و حداکثر چند گره دارد؟

پاسخ: حداقل $h+1$ (یک زنجیر) و حداکثر $2^{h+1}-1$ (همهٔ سطح‌ها پر).

4. در پیاده‌سازی k اشاره‌گری درختی با n گره، چند اشاره‌گر NULL است؟ در درخت دودویی معادل چطور؟

پاسخ: $nk-(n-1)$. در درخت دودویی معادل $2n-(n-1)=n+1$.

5. ثابت کنید هر درخت (بدون ریشه) با $n \geq 2$ رأس دست‌کم دو برگ دارد.

پاسخ: دو سر طولانی‌ترین مسیر ساده هر دو برگ‌اند (همان استدلال اثبات «$n-1$ یال»): اگر یکی از آن‌ها یال دیگری داشت، یا مسیر طولانی‌تر می‌شد یا دوری پدید می‌آمد.

فصل ۷

درخت عبارت

ماشین‌حساب چطور می‌فهمد ۸ - ۵ × (۴ + ۲) / ۳ یعنی چه؟ در این فصل می‌بینیم درخت چه قدرتی به ما می‌دهد: هر عبارت ریاضی را به یک درخت تبدیل می‌کنیم و مقدارش را با یک تابع بازگشتی ساده حساب می‌کنیم.

عملگر و عملوند | گرامر عبارت‌ها | درخت عبارت | میانوندی، پیشوندی، پسوندی | ساخت درخت

۷.۱ مقدمه: درخت چه قدرتی به ما می‌دهد؟

تا این‌جا لیست را گسترش دادیم و درخت را تعریف کردیم. اما درخت چه مسئله‌ای را حل می‌کند که لیست نمی‌کند؟ این مسئله را در نظر بگیرید:

تعریف · صورت مسئله

الگوریتمی ارائه دهید که مقدار یک عبارت ریاضی مثل این را حساب کند:

$$8 - 5 \times (4 + 2)/3$$

شما در ذهنتان بلافاصله می‌دانید اول پرانتز ($4 + 2 = 6$)، بعد ضرب و تقسیم از چپ ($5 \times 6 = 30$ و $30/3 = 10$) و در آخر تفریق ($8 - 10 = -2$). اما برنامه فقط یک رشته از نویسه‌ها می‌بیند: `8-5*(4+2)/3`. اگر از چپ به راست حساب کند، $8 - 5 = 3$، $3 \times 6 = 18$، $18/3 = 6$ به دست می‌آید که **غلط** است.

الگوریتم · روش این فصل

1. **ساخت درخت عبارت** از روی رشته؛ درخت تمام اطلاعات «چه چیزی اول حساب شود» را در شکل خود نگه می‌دارد.
2. **حل بازگشتی** روی درخت؛ دیگر هیچ قاعدهٔ اولویتی لازم نیست.

۷.۲ عبارت، عملگر و عملوند

تعریف · عبارت

عبارت (expression) مجموعه‌ای از **عملگرها** (operator) و **عملوندها** (operand) است که با نظمی خاص یک مقدار را نشان می‌دهند.

- **عملگر:** نمادی که یک عمل خاص را انجام می‌دهد (مثل $+$، $\times$، $\sin$).
- **عملوند:** مقداری که عمل روی آن انجام می‌شود (مثل a، 5).

در عبارت $(a + 5)$، نماد $+$ عملگر است و a و 5 عملوندهای آن.

عبارت‌ها می‌توانند خیلی پیچیده باشند؛ مثلاً این فرمول (بسط تیلور با باقی‌ماندهٔ انتگرالی):

$$f(a) + \sum_{k=1}^{n} \frac{1}{k!} \frac{d^k}{dt^k}\Big|_{t=0} f(u(t)) + \int_0^1 \frac{(1-t)^n}{n!} \frac{d^{n+1}}{dt^{n+1}} f(u(t))\, dt.$$

با این همه، همهٔ عبارت‌ها از همان دو جزء ساخته شده‌اند: عملگرهایی که روی عملوندها (یا روی عبارت‌های کوچک‌تر) اعمال می‌شوند. همین «ساخته شدن از عبارت‌های کوچک‌تر» است که عبارت را به یک ساختار **بازگشتی** و در نتیجه به درخت تبدیل می‌کند.

۷.۳ عملیات جبری: یگانی و دودویی

تعریف · عملیات جبری

نوعی عمل ریاضی که روی **یک** عملوند (عمل **یگانی**، unary) یا روی **دو** عملوند (عمل **دودویی**، binary) انجام می‌شود.

عملیات دودویی	عملیات یگانی
$2 + a$	-2
$b / 7$	$\sin \theta$
$\mathit{false} \vee S$	$\log n$
$2 \hat{} m$	$\neg\ \mathit{true}$

چرا؟ · پرسش اسلاید: مقدار $5 - 2$ چقدر است؟

-3. اما نکتهٔ پرسش این است: نماد «$-$» در $5 - 2$ یک عملگر **دودویی** است (تفریق: دو عملوند ۲ و ۵) و در -3 یا -2 یک عملگر **یگانی** (قرینه: یک عملوند). یک نماد، دو نقش! برنامه باید از روی **جای** نماد تشخیص دهد کدام است: اگر «$-$» در ابتدای عبارت یا بلافاصله پس از یک عملگر یا پرانتز باز بیاید، یگانی است؛ وگرنه دودویی.

۷.۴ ترتیب عملیات

یک عبارت جبری می‌تواند شامل چندین عمل جبری باشد. مقدار این عبارت چقدر است؟

$$6 \times 3 + 2\hat{}4$$

جواب به اینکه **کدام عمل اول** انجام شود بستگی دارد:

ترتیب	محاسبه	مقدار
اولویت‌های ریاضی (توان، ضرب، جمع)	$(6 \times 3) + (2\hat{}4) = 18 + 16$	**۳۴**
جمع اول	$6 \times (3 + 2)\hat{}4 = 6 \times 625$	۳۷۵۰
کاملاً از چپ به راست	$((6 \times 3) + 2)\hat{}4 = 20^4$	۱۶۰۰۰۰

تعریف · ترتیب عملیات جبری

در ریاضیات هر عمل **اولویت** متفاوتی دارد: اول توان، سپس ضرب و تقسیم، و در آخر جمع و تفریق. اگر اولویت دو عمل یکسان باشد، از **چپ** محاسبه می‌کنیم. می‌توان این اولویت را با **پرانتز** صریح نمایش داد:

$$((6 \times 3) + (2\hat{}4)) = 34.$$

شهود · پرانتز = ساختار درخت

پرانتزبندی کامل دقیقاً می‌گوید کدام عملگر روی کدام دو عبارت اعمال می‌شود. وقتی این اطلاعات را داشته باشیم، دیگر به قاعدهٔ اولویت نیازی نیست. درخت عبارت همین پرانتزبندی است که به شکل شاخه‌ها کشیده شده.

۷.۵ گرامر مستقل از متن (به زبان ساده)

برای اینکه دقیق بگوییم «عبارت معتبر» چیست، از **گرامر مستقل از متن** (Context-free Grammar یا CFG) استفاده می‌کنیم. جزئیات رسمی آن در درس «نظریهٔ زبان‌ها و ماشین‌ها» می‌آید؛ این‌جا به‌طور غیررسمی: **گرامر روشی است برای تولید کلمات (جمله‌های) یک زبان با قواعد مشخص.**

مثال · در زبان فارسی

«من به دانشگاه می‌روم.» یک جملهٔ معتبر است، چون از قواعد دستور زبان (نهاد + متمم + فعل) پیروی می‌کند. «از دانشگاه را در فردا.» کلمه‌هایش فارسی‌اند ولی جمله **معتبر نیست**، چون هیچ قاعده‌ای چنین ترکیبی را تولید نمی‌کند.

گرامر عبارت‌های جبری (با پرانتزبندی کامل) چند قاعده دارد. علامت «$\to$» یعنی «می‌تواند جایگزین شود با» و «$|$» یعنی «یا»:

$$
\begin{aligned}
&E \to \textit{operand} \\
&E \to (\alpha E), \qquad && \alpha \to - \mid ! \mid \sin \mid \log \mid \dots \\
&E \to (E \beta E), && \beta \to + \mid \times \mid \hat{} \mid \wedge \mid \dots
\end{aligned}
$$

E «یک عبارت». قاعده‌ها می‌گویند یک عبارت از چه چیزهایی ساخته می‌شود.

قاعدهٔ ۱ **ساده‌ترین عبارت:** یک عملوند تنها، مثل a یا 2.

قاعدهٔ ۲ **عبارت یگانی:** یک عملگر یگانی α و پس از آن یک عبارت، داخل پرانتز. مثل (-2)، $((!\,a)!)$، $(\sin\theta)$.

قاعدهٔ ۳ **عبارت دودویی:** عبارت، عملگر دودویی β، عبارت، داخل پرانتز. مثل $(2+b)$، $(c\hat{}(8-4))$، $(\mathit{True} \wedge S)$.

چرا؟ · چرا این گرامر «بازگشتی» است؟

در سمت راست قاعده‌های ۲ و ۳ دوباره خود E آمده است: «یک عبارت می‌تواند از عبارت‌های کوچک‌تر ساخته شود». به همین دلیل از چند قاعدهٔ کوتاه، بی‌نهایت عبارت با هر عمقی تولید می‌شود؛ درست مثل تعریف بازگشتی درخت در فصل ۶.

مثال · مثال ۱: تولید $((8-5)\times((4+2)/3))$

در هر گام، یک E را با سمت راست یکی از قاعده‌ها جایگزین می‌کنیم:

$$
\begin{aligned}
E &\Rightarrow (E \times E) && \text{قاعدهٔ ۳} \\
&\Rightarrow ((E - E) \times E) && \text{قاعدهٔ ۳} \\
&\Rightarrow ((8 - 5) \times E) && \text{قاعدهٔ ۱ دو بار} \\
&\Rightarrow ((8 - 5) \times (E / E)) && \text{قاعدهٔ ۳} \\
&\Rightarrow ((8 - 5) \times ((E + E) / E)) && \text{قاعدهٔ ۳} \\
&\Rightarrow ((8 - 5) \times ((4 + 2) / 3)) && \text{قاعدهٔ ۱ سه بار}
\end{aligned}
$$

پس این رشته یک عبارت معتبر است. مقدارش: $3 \times 2 = 6$. (دقت کنید این عبارتِ **پرانتزبندی‌شده** با عبارت مقدمه، $8 - 5 \times (4+2)/3 = -2$، فرق دارد.)

اشتباه رایج · مثال ۲: رشته‌هایی که عبارت نیستند

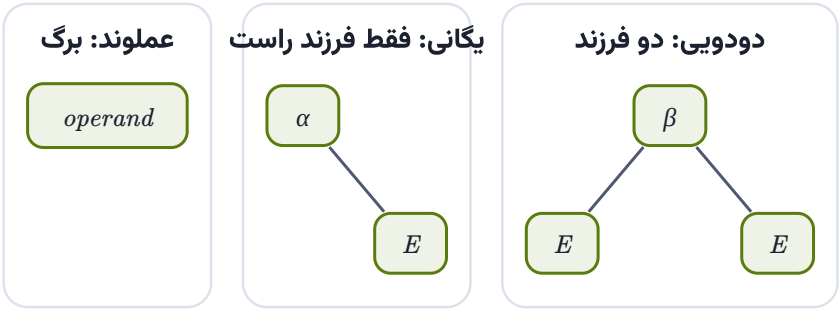
- $(\times + a)$: پس از $+$ باید یک E بیاید، ولی $\times$ با هیچ قاعده‌ای یک E نمی‌سازد.
- $a)$: پرانتزها نامتوازن‌اند؛ هر قاعده‌ای که پرانتز باز تولید می‌کند، بسته‌اش را هم تولید می‌کند.
- $+$: یک عملگر تنها؛ قاعدهٔ ۳ می‌گوید $+$ باید بین دو عبارت باشد.

۷.۶ درخت عبارت

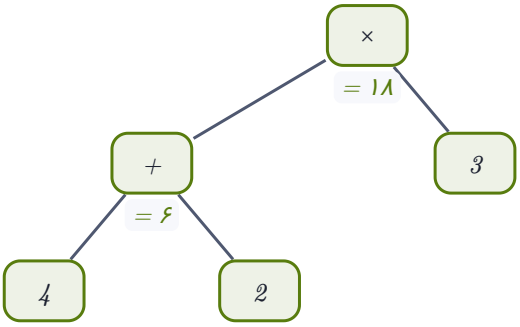
تعریف · درخت عبارت

درخت عبارت (expression tree) نوعی درخت **دودویی** برای نمایش یک عبارت است. چرا دودویی؟ چون هر عملگر یا یگانی است یا دودویی؛ پس هیچ گره‌ای به بیش از دو فرزند نیاز ندارد.

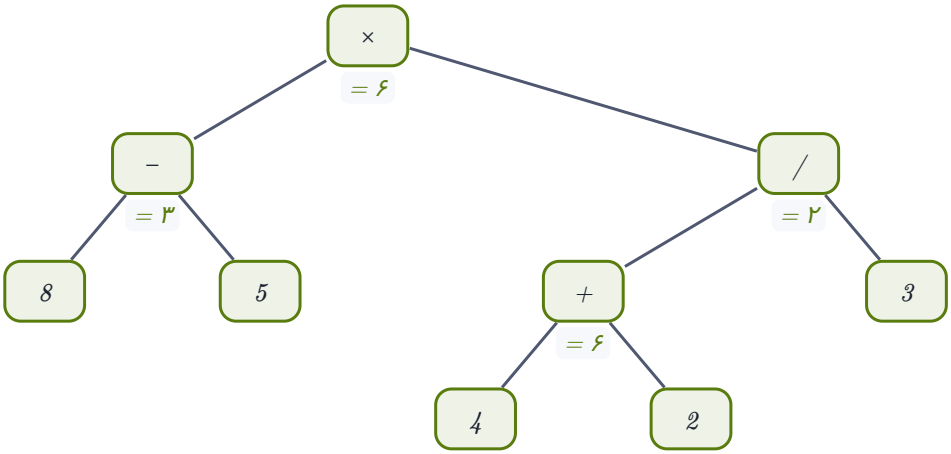
هر قاعدهٔ گرامر دقیقاً به یک نوع گره تبدیل می‌شود:



شکل ۷-۱ $E \to operand$: یک **برگ** با مقدار عملوند. $E \to (\alpha E)$: گرهٔ میانی با **فقط یک فرزند راست** (زیردرخت). $E \to (E \beta E)$: گرهٔ میانی با **دو فرزند** (زیردرخت).



شکل ۷-۲ درخت عبارت $((4 + 2) \times 3)$. عملگرها در گره‌های داخلی و عملوندها در برگ‌ها هستند. زیر هر گره مقدار زیردرختش نوشته شده است.



شکل ۷-۳ درخت عبارت $((8-5) \times ((4+2)/3))$. ریشه آخرین عملی است که انجام می‌شود ($\times$) و هر زیردرخت یک زیرعبارت پرانتزدار است.

چرا؟ · پرانتزها کجا رفتند؟

در درخت هیچ پرانتزی نیست، چون **شکل درخت** خودش گروه‌بندی را نشان می‌دهد: هر زیردرخت یک واحد مستقل است که باید قبل از عملگر پدرش حساب شود. عمل‌های عمیق‌تر زودتر انجام می‌شوند و ریشه آخر از همه.

۷.۷ مقداریابی بازگشتی

وقتی درخت را داریم، پیدا کردن مقدار نهایی یک تابع بازگشتی ساده است:

EVAL(v)

```
1  if v is a leaf
2      return value(v)                          // E → operand
3  if v has only a right child                  // E → (αE)
4      return APPLY(v.op, EVAL(v.right))
5  return APPLY(v.op, EVAL(v.left), EVAL(v.right))   // E → (EβE)
```

مثال · اجرای EVAL روی درخت $((8-5) \times ((4+2)/3))$

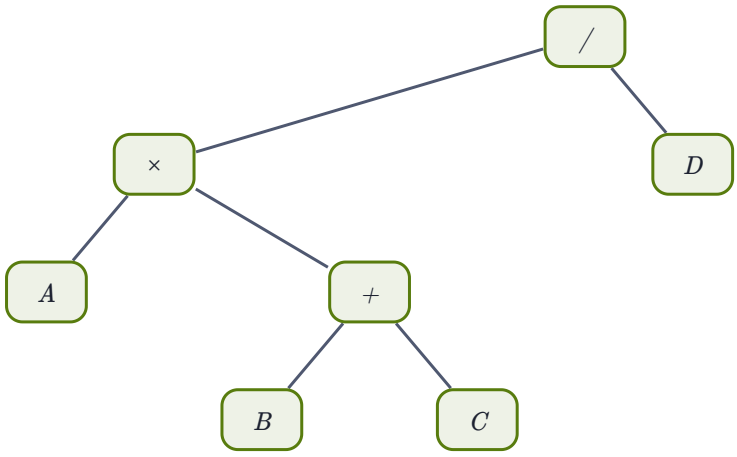
1. EVAL$(\times)$ اول باید EVAL$(-)$ را حساب کند: $8-5=3$.
2. سپس EVAL$(/)$، که خودش اول EVAL$(+)$ $= 4+2=6$ و بعد $6/3=2$ را حساب می‌کند.
3. در آخر $3 \times 2 = 6$.

نکته · EVAL یک پیمایش پس‌ترتیب است

هر گره **پس از** فرزندانش ارزیابی می‌شود (چون عملگر به مقدار عملوندهایش نیاز دارد). این دقیقاً پیمایش postorder فصل ۶ است. هر گره یک بار ملاقات می‌شود، پس زمان $\Theta(n)$ است.

۷.۸ سه نگارش: میانوندی، پسوندی و پیشوندی

یک درخت عبارت را می‌توان به سه شکل به رشته تبدیل کرد، متناظر با سه پیمایش:



شکل ۷-۴ درخت عبارت $((A \times (B + C))/D)$ که در هر سه نگارش زیر یکی است.

نگارش	جای عملگر	مثال	گرامر	پرانتز	پیمایش
میانوندی (infix)	میان دو عملوند	$2 + a$ $((A \times (B + C))/D)$	$E \to (E \beta E)$	**لازم است**	inorder
پسوندی (postfix)	پس از عملوندها	$2\,a +$ $A\,B\,C + \times D /$	$E \to E E \beta$	**لازم نیست**	postorder
پیشوندی (prefix)	پیش از عملوندها	$+\,2\,a$ $/ \times A + B\,C\,D$	$E \to \beta E E$	**لازم نیست**	preorder

(برای عملگر یگانی هم به ترتیب $E \to (\alpha E)$، $E \to E\alpha$ و $E \to \alpha E$.)

چرا؟ · چرا میانوندی پرانتز لازم دارد ولی پسوندی نه؟

رشتهٔ میانوندی $A \times B + C$ بدون پرانتز دو درخت متفاوت می‌تواند داشته باشد: $((A \times B) + C)$ یا $(A \times (B + C))$. پیمایش inorder این دو درخت، بدون پرانتز، یکسان است. پس یا باید پرانتز بگذاریم یا قاعدهٔ اولویت را بدانیم.

اما در پسوندی این دو درخت رشته‌های **متفاوتی** می‌دهند: $AB\times C+$ و $ABC+\times$. در پسوندی، هر عملگر روی **دو عبارت کاملی که دقیقاً پیش از آن آمده‌اند** اعمال می‌شود؛ این قاعده هیچ ابهامی ندارد، پس ترتیب نمادها به‌تنهایی کل ساختار درخت را مشخص می‌کند. پیشوندی هم به همین دلیل (به‌صورت قرینه) بدون پرانتز است.

تکمیلی (فراتر از اسلاید) · ارزیابی پسوندی با پشته (ماشین‌حساب‌های RPN)

```
EVAL-POSTFIX(tokens)
```

```
create empty stack S
for each token t (left to right)
    if t is an operand
        S.push(t)
    else                          // binary operator
        y = S.pop()               // the right operand is on top!
        x = S.pop()
        S.push(APPLY(t, x, y))
return S.pop()
```

برای `8 5 - 4 2 + 3 / ×` : پشته $[8,5]\leftarrow[3]\leftarrow-\leftarrow[3,4,2]\leftarrow+\leftarrow[3,6]\leftarrow[3,6,3]\leftarrow/\leftarrow[3,2]\leftarrow\times\leftarrow[6]$. جواب ۶ بدون هیچ پرانتز یا قاعدهٔ اولویتی.

۷.۹ مسئلهٔ ۱: ساخت درخت از نگارش میانوندی با پرانتزبندی کامل

ورودی: $((A\times(B+C))/D)$. هدف: ساختن درخت عبارت. (همین درخت را در نگارش پسوندی `ABC+×D/` و پیشوندی `/×A+BCD` هم داریم و در بخش بعد از آن‌ها هم می‌سازیم.)

روش بازگشتی: پیدا کردن ریشه

۱ **کدام قاعده؟** رشته با «(» شروع می‌شود، پس قاعدهٔ ۲ یا ۳ است. اگر پس از پرانتز اول یک عملگر یگانی بیاید، قاعدهٔ ۲؛ وگرنه قاعدهٔ ۳: $E\to(E\beta E)$.

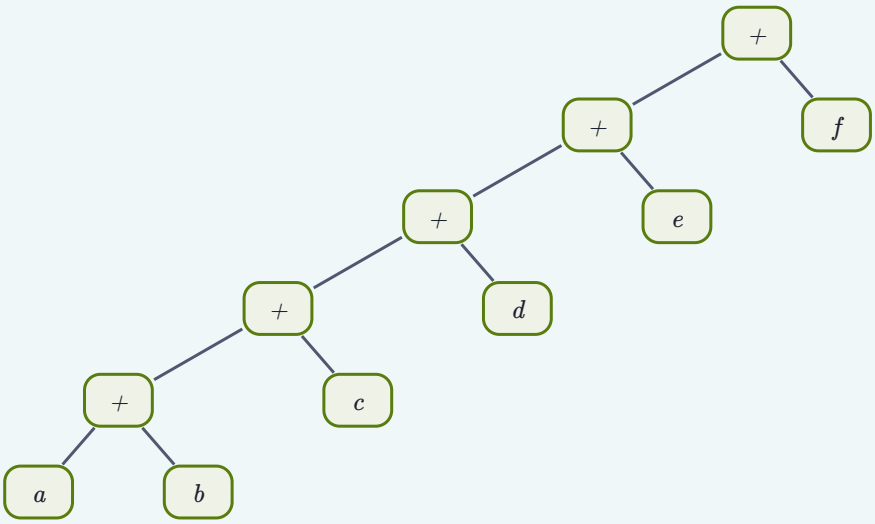
۲ **پیدا کردن هر بخش با شمردن پرانتزها:** داخل پرانتز بیرونی، از چپ پیش می‌رویم و عمق پرانتز را می‌شماریم (هر «(» یکی اضافه، هر «)» یکی کم). **عملگری که در عمق صفر** (بیرون از همهٔ پرانتزهای داخلی) قرار دارد، β و ریشهٔ درخت است. برای $((A\times(B+C))/D)$: داخل پرانتز بیرونی، $(A\times(B+C))$ تماماً داخل پرانتز است و اولین عملگر عمق صفر «/» است. پس ریشه «/»، عبارت چپ $(A\times(B+C))$ و عبارت راست D.

۳ **حل بازگشتی برای هر زیربخش:** همین کار را روی $(A\times(B+C))$ (ریشه $\times$) و D (برگ) تکرار می‌کنیم.

چرا؟ · زمان اجرای این روش

در هر سطح بازگشت، برای پیدا کردن ریشه کل زیررشته را پیمایش می‌کنیم. عبارت «چپ‌سنگین» زیر را در نظر بگیرید:

$$(((((a+b)+c)+d)+e)+f)$$



شکل ۷-۵ درخت این عبارت یک زنجیر رو به چپ است.

ریشه (آخرین $+$) نزدیک انتهای رشته است، پس برای پیدا کردنش تقریباً کل رشته (n نویسه) خوانده می‌شود. در سطح بعد تقریباً $n-4$ نویسه، بعد $n-8$، ... . جمع این‌ها $\Theta(n^2)$ است. **پس زمان اجرا** $O(n^2)$.

پیاده‌سازی در زمان خطی

ایده: **به‌جای اینکه اول هر بخش را پیدا کنیم و بعد زیردرختش را بسازیم، همزمان با پیدا کردن هر بخش، زیردرخت مربوطه را می‌سازیم.** یعنی از چپ به راست می‌خوانیم و گرامر به ما می‌گوید نویسهٔ بعدی چه نقشی دارد؛ هیچ‌وقت لازم نیست جلوتر را برای پیدا کردن پرانتز بسته جست‌وجو کنیم.

```
PARSE() ▷ reads the next expression from the input, returns its tree
```

```
t = next token
if t is an operand
    return Leaf(t)                          // E → operand
// otherwise t = "("
if the next token is a unary operator α
    α = next token
    sub = PARSE()
    read ")"
    return Node(α, null, sub)               // E → (αE)
left = PARSE()                              // E → (EβE)
β = next token
right = PARSE()
read ")"
return Node(β, left, right)
```

چرا؟ · چرا خطی است؟

هر نویسهٔ ورودی دقیقاً یک بار توسط next token خوانده می‌شود و برای هر نویسه کار ثابتی انجام می‌شود. بازگشت فقط «جای کار» را در پشتهٔ فراخوانی نگه می‌دارد. پس زمان کل $\Theta(n)$ است. به این روش «تجزیهٔ نزولی بازگشتی» (recursive descent) می‌گویند و قلب بسیاری از کامپایلرهاست.

۷.۱۰ ساخت درخت از نگارش پیشوندی و پسوندی

ورودی: پسوندی `ABC+×D/` و پیشوندی `/×A+BCD` (هر کدام جداگانه کافی است).

روش اسلاید: پیدا کردن ریشه و دو عبارت

مثال · پیشوندی: $E \to \beta EE$

ریشه = اولین نماد: «/». **عبارت اول** بلافاصله پس از آن شروع می‌شود و جایی تمام می‌شود که یک عبارت کامل شده باشد: $\times A + BC$. **عبارت دوم** = بقیه: D.

مثال · پسوندی: $E \to EE\beta$

ریشه = آخرین نماد: «/». **عبارت دوم** درست پیش از ریشه تمام می‌شود: D. **عبارت اول** = بقیه: $ABC + \times$.

چرا؟ · از کجا بفهمیم یک عبارت کجا تمام می‌شود؟

یک «شمارندهٔ نیاز» نگه می‌داریم: تعداد عبارت‌هایی که هنوز کم داریم. در پیشوندی از ۱ شروع می‌کنیم و از چپ می‌خوانیم: هر **عملگر دودویی** یک جای خالی را پر می‌کند ولی دو جای خالی جدید می‌سازد $(+1)$؛ هر **عملوند** یک جای خالی را پر می‌کند (-1). وقتی شمارنده صفر شد، عبارت تمام است.

$$\times\,(2)\quad A\,(1)\quad +\,(2)\quad B\,(1)\quad C\,(0)\leftarrow \text{first expression} = \times A + BC$$

در پسوندی همین کار را از راست به چپ انجام می‌دهیم.

روش خطی با پشته (پسوندی)

BUILD-FROM-POSTFIX(tokens)

```
create empty stack S                    // stack of subtrees
for each token t (left to right)
    if t is an operand
        S.push(Leaf(t))
    else if t is a unary operator
        S.push(Node(t, null, S.pop()))
    else                                // binary operator
        right = S.pop()
        left = S.pop()
        S.push(Node(t, left, right))
return S.pop()                          // exactly one tree remains
```

token	action	stack of subtrees (bottom → top)
A	push leaf	A
B	push leaf	A, B
C	push leaf	A, B, C
+	pop C, B → (B+C)	A, (B+C)
×	pop (B+C), A → (A×(B+C))	(A×(B+C))
D	push leaf	(A×(B+C)), D
/	pop D, (A×(B+C)) → ((A×(B+C))/D)	**((A×(B+C))/D)**

هر نماد یک بار خوانده و حداکثر یک بار push و pop می‌شود: $\Theta(n)$. برای پیشوندی هم تابع بازگشتی `build(): t = next; if operand return Leaf(t); left = build(); right = build(); return Node(t, left, right)` در $\Theta(n)$ کار می‌کند.

۷.۱۱ مسئلهٔ ۲: ساخت درخت از نگارش میانوندی بدون پرانتز

حالا سخت‌ترین حالت: عبارتی بدون پرانتز که فقط با قواعد اولویت معنا دارد. یک مدل ساده‌شده (فقط جمع، تفریق، ضرب، تقسیم و توان):

$$E \to \textit{operand} \qquad E \to \alpha E,\ \alpha \to - \qquad E \to E \beta E,\ \beta \to + \mid - \mid \times \mid \div \mid \hat{}$$

- اولویت عملگرها: اول **توان**، بعد **ضرب و تقسیم** و در نهایت **جمع و تفریق**.
- اگر اولویت‌ها یکسان باشد، از **سمت چپ** محاسبه می‌کنیم.

شهود · «تکلیف» یک عملگر کِی معلوم می‌شود؟

وقتی در $a + b \ldots$ به $+$ می‌رسیم، نمی‌دانیم فرزند راستش فقط b است یا چیزی بزرگ‌تر مثل $b \times c$. **تکلیف** **مشخص نیست**، پس آن را کنار می‌گذاریم (در یک پشته). تکلیف یک عملگر وقتی معلوم می‌شود که به عملگری با اولویت **کمتر یا مساوی** برسیم (یا عبارت تمام شود): در آن لحظه مطمئنیم عملوند راستش کامل شده است.

الگوریتم دو پشته دارد: پشتهٔ **عملگرهای معلق** و پشتهٔ **زیردرخت‌های ساخته‌شده**.

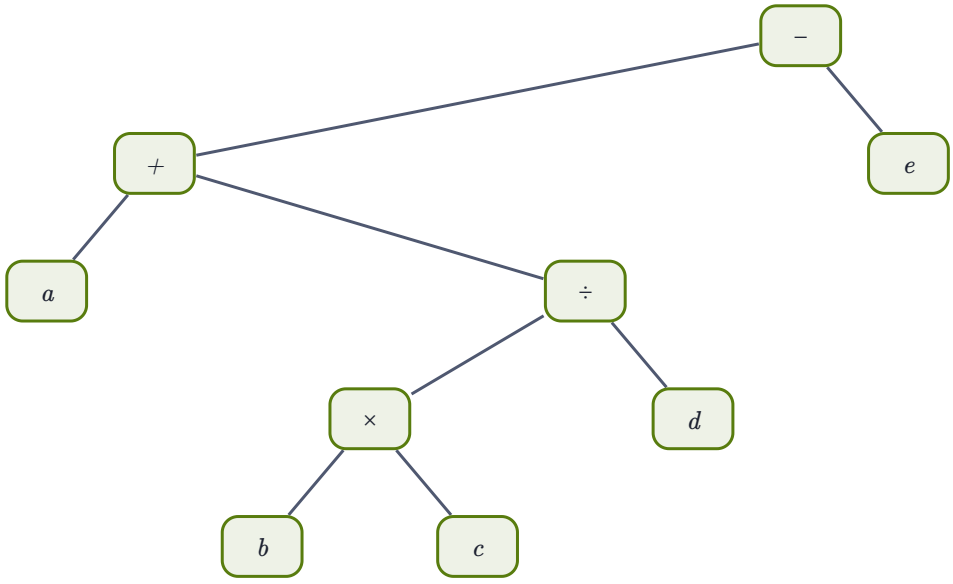
```
BUILD-FROM-INFIX(tokens) ▷ no parentheses, left-associative operators
```

```
create empty stacks OPS (operators) and TREES (subtrees)
for each token t (left to right)
    if t is an operand
        TREES.push(Leaf(t))
    else                                        // t is a binary operator
        while OPS not empty and prec(OPS.top) ≥ prec(t)
            COMBINE()                           // OPS.top is now settled
        OPS.push(t)
while OPS not empty                             // end of input settles everythi
    COMBINE()
return TREES.pop()

COMBINE():
    op = OPS.pop()
    right = TREES.pop()
    left = TREES.pop()
    TREES.push(Node(op, left, right))
```

مثال · ردگیری $a + b \times c \div d - e$

token	چه اتفاقی می‌افتد	OPS	TREES
a	عملوند است: یک برگ می‌سازیم.	–	a
+	جمع دودویی است، ولی فرزند راستش معلوم نیست: به پشته.	+	a
b	برگ.	+	a, b
×	اولویت ضرب از جمع بیشتر است، پس تکلیف جمع هنوز معلوم نیست: × به پشته.	+, ×	a, b
c	برگ.	+, ×	a, b, c
÷	اولویت تقسیم برابر ضرب است و اولویت با چپ‌ترین است: **تکلیف ضرب مشخص شد**. b و c فرزندان × می‌شوند. اولویت ÷ از + بیشتر است، پس + می‌ماند و ÷ به پشته.	+, ÷	a, (b×c)
d	برگ.	+, ÷	a, (b×c), d
-	اولویت تقسیم از تفریق بیشتر است: **تکلیف تقسیم مشخص شد**؛ (b×c) و d فرزندان ÷. اولویت تفریق برابر جمع است و چپ‌ترین مقدم است: **تکلیف جمع هم مشخص شد**؛ a و ((b×c)÷d) فرزندان +. پشته خالی شد: - به پشته.	-	(a+((b×c)÷d))
e	برگ.	-	(a+…), e
end	عبارت تمام شد: تکلیف پشته معلوم است؛ دو زیردرخت آخر فرزندان - می‌شوند.	–	((a+((b×c)÷d))-e)



شکل ۷-۶ درخت نهایی $a + b \times c \div d - e$: تفریق آخرین عمل است (ریشه)، و $b \times c$ زودتر از همه (عمیق‌ترین).

چرا؟ · چرا $\geq$ و نه $>$؟

«اولویت یکسان از چپ» یعنی در $a - b - c$ اول $a - b$ حساب شود: $((a - b) - c)$. وقتی به دومین $-$ می‌رسیم، اولین $-$ (با اولویت **مساوی**) باید همان لحظه بسته شود؛ پس شرط $\geq$ است. اگر $>$ بگذاریم، اولی منتظر می‌ماند و درخت $(a - (b - c))$ ساخته می‌شود که مقدار دیگری دارد (مثلاً $1 - 3 - 5$ به‌جای 1، عدد 3 می‌شد).

نکته · زمان اجرا

هر عملوند یک بار push می‌شود و هر عملگر یک بار push و یک بار pop. حلقهٔ `while` داخلی ممکن است گاهی چند بار اجرا شود (مثل خواندن «-» در مثال)، اما با همان استدلال سرشکنی فصل ۴ و ۵، کل کار $\Theta(n)$ است.

تکمیلی (فراتر از اسلاید) · توان و پرانتز

- **توان از راست شرکت‌پذیر است:** $2\text{^}3\text{^}2 = 2^{(3^2)} = 512$، نه $(2^3)^2 = 64$. برای ^ شرط حلقه را $>$ بگذارید تا توانِ قبلی منتظر بماند.
- **پرانتز:** «(» را مستقیماً push کنید؛ با رسیدن به «)»، تا رسیدن به «(» COMBINE کنید و «(» را دور بریزید. حلقهٔ اولویت هم هرگز از «(» رد نمی‌شود. این الگوریتم کامل «حیاط شنتینگ» (shunting-yard) دایکسترا است.
- **منفی یگانی:** اگر «-» در ابتدا یا پس از عملگر دیگری بیاید، یگانی است؛ آن را با بالاترین اولویت و فقط یک فرزند (راست) ترکیب کنید.

جمع‌بندی · فصل ۷ در یک نگاه

- عبارت = عملگرها + عملوندها؛ عملگرها یگانی یا دودویی‌اند، پس درخت عبارت دودویی است.
- گرامر: $E \to \textit{operand} \mid (\alpha E) \mid (E \beta E)$؛ هر قاعده یک نوع گره.
- مقداریابی = پیمایش پس‌ترتیب، $\Theta(n)$.
- inorder ↔ infix (پرانتز لازم)، preorder ↔ prefix، postorder ↔ postfix (بی‌پرانتز).
- ساخت درخت: از infix پرانتزدار با یافتن ریشه $O(n^2)$، با تجزیهٔ همزمان $O(n)$؛ از postfix با پشته $O(n)$؛ از infix بی‌پرانتز با دو پشته و قاعدهٔ اولویت $O(n)$.

۷.۱۲ خودآزمایی فصل ۷

خودآزمایی

1. نگارش پیشوندی و پسوندی $((8-5)\times((4+2)/3))$ را بنویسید.

پاسخ: پیشوندی: `× - 8 5 / + 4 2 3` . پسوندی: `8 5 - 4 2 + 3 / ×` .

2. مقدار عبارت پسوندی `2 3 4 × + 5 -` چیست؟

پاسخ: $3\times4=12$، $2+12=14$، $14-5=9$.

3. عبارت پیشوندی `+ × a b c` را به میانوندی با پرانتزبندی کامل تبدیل کنید.

پاسخ: ریشه +، عبارت اول $a\times b$ و عبارت دوم c: $((a\times b)+c)$.

4. آیا $(a+b)\,c$ طبق گرامر عبارت معتبر است؟

پاسخ: خیر. پس از یک E فقط می‌تواند «)» یا یک β بیاید؛ کنار هم گذاشتن دو عبارت بدون عملگر در هیچ قاعده‌ای نیست.

5. با الگوریتم مسئلهٔ ۲ درخت $a\times b+c\times d$ را بسازید.

پاسخ: a؛ × به پشته؛ b؛ با رسیدن + تکلیف × معلوم می‌شود: (a×b)؛ + به پشته؛ c؛ × اولویت بیشتری دارد، به پشته؛ d؛ پایان: (c×d) و سپس ((a×b)+(c×d)).

فصل ۸

هرم و صف اولویت

صف معمولی به ترتیب ورود سرویس می‌دهد؛ اورژانس بیمارستان به ترتیب **اهمیت**. هرم داده‌ساختاری است که همیشه مهم‌ترین عنصر را در دسترس نگه می‌دارد و هر تغییر را فقط در $O(\log n)$ انجام می‌دهد.

صف اولویت | هرم بیشینه و کمینه | نمایش آرایه‌ای | درج و حذف | ساخت خطی هرم | مرتب‌سازی هرمی

۸.۱ از صف تا صف اولویت

یادآوری از فصل ۵: **صف** نوعی لیست است که فقط امکان افزودن به انتها و حذف از ابتدا را می‌دهد (مدل First-In-First-Out). عملیات روی صف Q:

$\text{Enqueue}(Q, x)$	درج عنصر x در انتهای صف
$\text{Dequeue}(Q)$	حذف و برگرداندن عنصر ابتدای صف
$\text{Front}(Q)$	برگرداندن عنصر ابتدای صف بدون حذف
$\text{isEmpty}(Q)$	آیا Q خالی است؟
$\text{Size}(Q)$	اندازهٔ Q

اگر عنصر بیشینه (یا کمینه) را بخواهیم چه؟ صف معمولی فقط ترتیب ورود را می‌شناسد. اما در خیلی از کاربردها باید اول مهم‌ترین کار انجام شود:

- اورژانس: بیمار بدحال‌تر زودتر ویزیت می‌شود، حتی اگر دیرتر آمده باشد.
- سیستم‌عامل: پردازه‌ای با اولویت بالاتر زودتر پردازنده را می‌گیرد.
- الگوریتم‌های گراف (مثل دایکسترا): همیشه نزدیک‌ترین رأس بعدی پردازش می‌شود.

تعریف · صف اولویت (Priority Queue)

داده‌ساختاری شبیه صف (یا پشته) که به هر عنصر x_i یک مقدار p_i به‌عنوان **اولویت** نسبت داده شده و پردازش روی عنصری با **بالاترین اولویت** انجام می‌شود. عملیات:

- $\text{insert}(S, x, p)$: درج عنصر x با اولویت p در S.
- $\max(S)$: برگرداندن عنصر با بالاترین اولویت در S.
- $\text{extract_max}(S)$: برگرداندن و **حذف** عنصر با بالاترین اولویت در S.

اگر «کمترین عدد = بالاترین اولویت» باشد (مثل نوبت‌دهی بر اساس ساعت)، صف اولویت **کمینه** داریم با عملیات min و extract_min.

۸.۲ پیاده‌سازی‌های ساده و مشکلشان

صف اولویت کمینه با لیست مرتب (از کم به زیاد)

- **هزینهٔ خواندن کمینه:** $O(1)$. کمینه همیشه اولین عنصر لیست است.
- **هزینهٔ حذف کمینه:** $O(1)$. حذف از ابتدای لیست پیوندی (فصل ۵).
- **هزینهٔ اضافه کردن:** $O(n)$. باید جای درست عنصر جدید را پیدا کنیم تا لیست مرتب بماند؛ در بدترین حالت کل لیست پیمایش می‌شود.

پیاده‌سازی	insert	(min یا) max	extract
آرایه / لیست نامرتب	$O(1)$	$O(n)$	$O(n)$
لیست مرتب	$O(n)$	$O(1)$	$O(1)$
هرم	$O(\log n)$	$O(1)$	$O(\log n)$

شهود · چرا هرم بهتر است؟ کار کمتر، دقیقاً به اندازهٔ نیاز

لیست مرتب «ترتیب کامل» همهٔ عناصر را حفظ می‌کند، در حالی که ما فقط می‌خواهیم **بزرگ‌ترین** را بدانیم. حفظ ترتیب کامل گران است. لیست نامرتب هیچ ترتیبی حفظ نمی‌کند، پس پیدا کردن بزرگ‌ترین گران است. هرم یک «ترتیب جزئی» نگه می‌دارد: **فقط هر پدر از فرزندانش بزرگ‌تر است.** این دقیقاً کافی است تا بزرگ‌ترین در ریشه بماند، و آن‌قدر ضعیف است که حفظ کردنش ارزان باشد.

۸.۳ هرم

هرم (heap) یکی از بهینه‌ترین پیاده‌سازی‌های صف اولویت است، به‌طوری که معمولاً دو کلمهٔ «هرم» و «صف اولویت» به جای هم استفاده می‌شوند. بهترین پیاده‌سازی هرم با یک **درخت دودویی کامل** است.

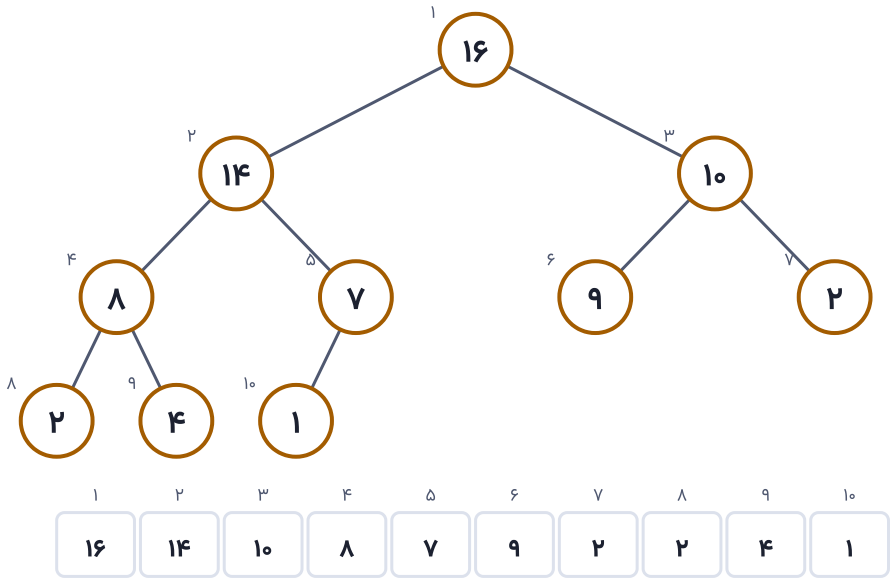
تعریف · یادآوری: درخت دودویی کامل

درختی که همهٔ سطح‌ها به‌جز سطح آخر کاملاً پرند و سطح آخر از چپ به راست پر شده است (فصل ۶).

تعریف · هرم بیشینه (کمینه)

یک درخت دودویی **کامل** که مقدار هر گره **بیشتر (کمتر)** یا مساوی مقدار فرزندانش باشد.

پس هرم دو شرط دارد: یکی دربارهٔ **شکل** (کامل بودن) و یکی دربارهٔ **مقادیر** (ویژگی هرم). هر عملی که روی هرم انجام می‌دهیم باید هر دو شرط را حفظ کند.



شکل ۸-۱ یک هرم بیشینه با ۱۰ عنصر و نمایش آرایه‌ای آن (اعداد کوچک کنار گره‌ها اندیس آرایه‌اند). هر گره از فرزندانش بزرگ‌تر یا مساوی است.

قضیه · بزرگ‌ترین عنصر در ریشه است

در هرم بیشینه، ریشه بزرگ‌ترین عنصر است.

هر گرهٔ v با یک مسیر از ریشه $r = a_0, a_1, \ldots, a_k = v$ قابل دسترسی است. ویژگی هرم می‌گوید $a_0 \geq a_1 \geq \cdots \geq a_k$، پس با تعدی $r \geq v$. چون این برای همهٔ گره‌ها درست است، ریشه بیشینه است. پس $\max(S)$ فقط خواندن ریشه است: $O(1)$.

■

اشتباه رایج · هرم مرتب نیست!

بین برادرها یا بین گره‌های دو زیردرخت مختلف هیچ رابطه‌ای تضمین نمی‌شود. در شکل بالا، ۸ در زیردرخت چپ از ۹ در زیردرخت راست کوچک‌تر است، با اینکه عمق کمتری ندارد. پیمایش آرایه از چپ به راست ترتیب مرتب نمی‌دهد.

چرا؟ · ارتفاع هرم چقدر است؟

هرم یک درخت دودویی کامل است، پس ارتفاعش $\lfloor \log_2 n \rfloor$ است (اثبات در فصل ۶). همهٔ عملیات تغییردهندهٔ هرم در یک مسیر از ریشه تا برگ حرکت می‌کنند، پس هزینه‌شان $O(\log n)$ است.

۸.۴ پیاده‌سازی هرم با آرایه

در این بخش فقط **ساختار** هرم را بحث می‌کنیم؛ توابع روی هرم در بخش‌های بعد می‌آیند. با استفاده از **اشاره‌گر** هرم همان پیاده‌سازی درخت دودویی (فصل ۶) است. اما چون هرم **کامل** است، راه بهتری داریم: **آرایهٔ** $A[1..n]$ بدون هیچ اشاره‌گری.

تعریف · نمایش آرایه‌ای

- ریشه در $A[1]$.
- فرزند چپ گرهٔ i در $A[2i]$.
- فرزند راست گرهٔ i در $A[2i+1]$.
- پدر گرهٔ i در $A[\lfloor i/2 \rfloor]$.

مثال · بررسی روی شکل

گرهٔ ۴ (مقدار ۸): فرزند چپ $A[8] = 2$ و فرزند راست $A[9] = 4$. پدر گرهٔ ۹: $\lfloor 9/2 \rfloor = 4$. پدر گرهٔ ۱۰: $\lfloor 10/2 \rfloor = 5$ (مقدار ۷). ✓

چرا؟ · چرا این فرمول‌ها درست‌اند؟

گره‌ها را سطح‌به‌سطح و در هر سطح از چپ به راست شماره‌گذاری کرده‌ایم. سطح L شامل اندیس‌های 2^L تا $2^{L+1}-1$ است (سطح ۰: فقط ۱؛ سطح ۱: ۲ و ۳؛ سطح ۲: ۴ تا ۷). گرهٔ i در سطح L، گرهٔ شمارهٔ $j = i - 2^L$ (از صفر) در سطح خودش است. همهٔ j گرهٔ سمت چپش هر کدام دو فرزند دارند که پیش از فرزندان i در سطح $L+1$ می‌آیند، پس فرزند چپ i در جایگاه $2j$ سطح بعد است، یعنی اندیس $2^{L+1} + 2j = 2^{L+1} + 2i - 2^{L+1} = 2i$. ✓

این فقط به این دلیل کار می‌کند که درخت **کامل** است: هیچ «حفره»‌ای بین گره‌ها نیست، پس شماره‌گذاری فشرده است و هیچ خانه‌ای هدر نمی‌رود.

۸.۵ افزایش مقدار یک گره

مسئله: فرض کنیم مقدار گرهٔ ۹ (عدد ۴) را ۱۱ واحد افزایش دهیم: $4 + 11 = 15$. حالا ۱۵ از پدرش (۸) بزرگ‌تر است و ویژگی هرم به هم خورده. چطور درستش کنیم؟

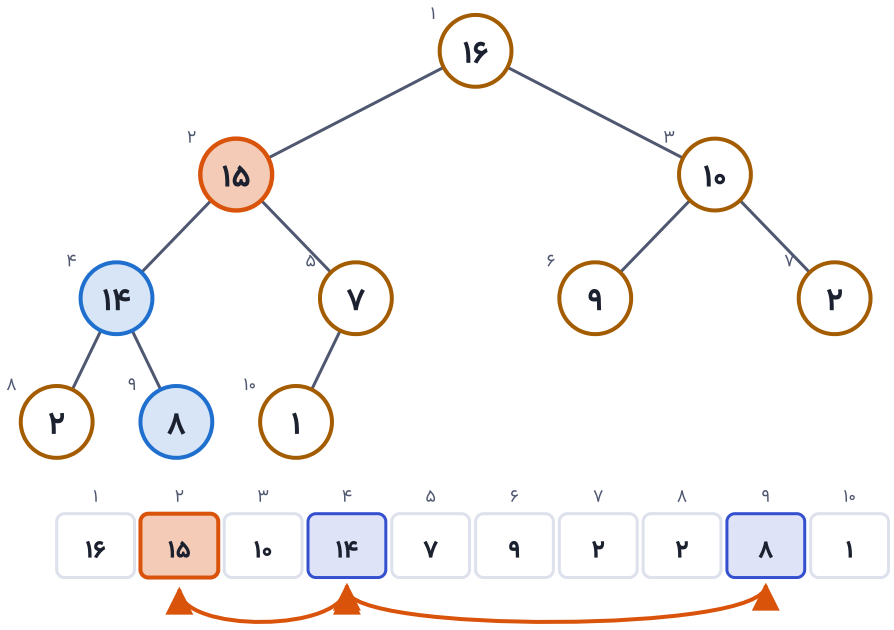
الگوریتم · بالا بردن (sift-up)

گرهٔ تغییریافته در هر مرحله: **اگر از پدرش بزرگ‌تر باشد، با آن جابه‌جا می‌شود.** این کار را تا جایی ادامه می‌دهیم که از پدرش بزرگ‌تر نباشد یا به ریشه برسد.

۱. ۱۵ در خانهٔ ۹ و پدرش ۸ در خانهٔ ۴: $15 > 8$، جابه‌جا.

۲. ۱۵ در خانهٔ ۴ و پدرش ۱۴ در خانهٔ ۲: $15 > 14$، جابه‌جا.

۳. ۱۵ در خانهٔ ۲ و پدرش ۱۶ در ریشه: $15 < 16$، توقف.



شکل ۸-۲ نتیجه پس از افزایش. گره‌های رنگی مسیر حرکت ۱۵ هستند و پیکان‌ها جابه‌جایی‌ها را در آرایه نشان می‌دهند (۹ ↔ ۴ ↔ ۲).

```
1  = A[i]">
2  A[i] = key
3  while i > 1 and A[floor(i/2)] < A[i]
4      swap A[i] and A[floor(i/2)]
5      i = floor(i/2)
```

اثبات · چرا این روش درست است؟ (پاسخ پرسش‌های اسلاید)

فرض کنید x (مقدار جدید) با پدرش p جابه‌جا شود، چون $x > p$.

- **آیا x از فرزند دیگرِ p (برادرش) هم بزرگ‌تر است؟** بله: پیش از تغییر، $p \geq$ برادر (ویژگی هرم)، و $x > p$. پس $x >$ برادر.
- **آیا p که پایین آمده، از فرزندان جدیدش بزرگ‌تر است؟** بله: فرزندان جدید p همان فرزندان قدیمی x هستند. پیش از افزایش، آن‌ها $\leq$ مقدار قدیمی x بودند و مقدار قدیمی x هم $p \leq$ بود.
- **آیا پدر از نوه‌های خودش نیز بزرگ‌تر است؟** با تعدی از دو مورد بالا، همه چیز زیر x درست است.

پس پس از هر جابه‌جایی، تنها جای ممکن برای خرابی، رابطهٔ x با پدر **جدیدش** است؛ یعنی مشکل یک سطح بالا رفته. وقتی x از پدرش بزرگ‌تر نباشد (یا به ریشه برسد)، هرم درست است.

∎

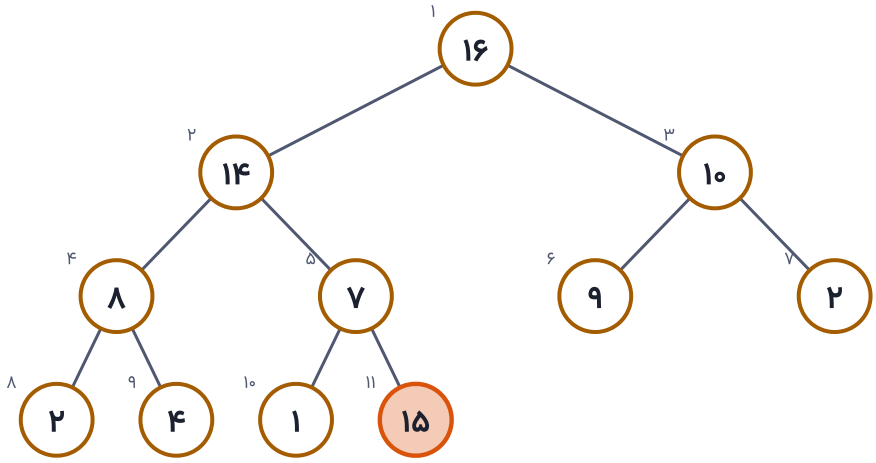
زمان: در هر جابه‌جایی یک سطح بالا می‌رویم، پس حداکثر به اندازهٔ ارتفاع: $O(\log n)$.

۸.۶ افزودن یک گرهٔ جدید

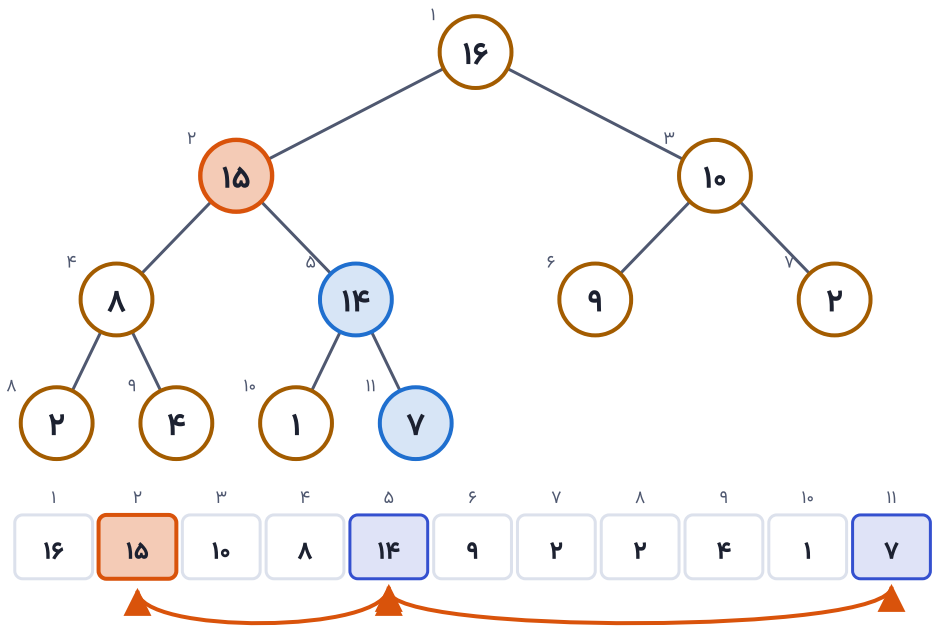
مسئله: می‌خواهیم گرهٔ جدیدی با مقدار ۱۵ را به هرم شکل اول اضافه کنیم.

۱ **گره را به ته درخت اضافه می‌کنیم:** یعنی خانهٔ $n + 1 = 11$ آرایه. این کار شکل درخت را **کامل** نگه می‌دارد (سطح آخر از چپ پر می‌شود).

۲ **گره را بالا می‌بریم:** مثل بخش قبل؛ اگر از پدرش بزرگ‌تر باشد با آن جابه‌جا می‌شود. ۱۵ و ۷ (خانهٔ ۵): جابه‌جا. ۱۵ و ۱۴ (خانهٔ ۲): جابه‌جا. ۱۵ و ۱۶: توقف.



شکل ۸-۳ گام ۱: ۱۵ در خانهٔ ۱۱، فرزند راست ۷.



شکل ۸-۴ گام ۲: پس از دو جابه‌جایی. ۱۵ در خانهٔ ۲ است.

شهود · درج = افزایش کلید

انگار گره‌ای با مقدار $-\infty$ به ته آرایه اضافه کرده‌ایم (که هیچ ویژگی‌ای را خراب نمی‌کند، چون از همه کوچک‌تر است) و حالا مقدارش را به ۱۵ افزایش می‌دهیم. پس درج همان HEAP-INCREASE-KEY است:

```
HEAP-INSERT(A, n, key)
```

```
1  n = n + 1
2  A[n] = -infinity
3  HEAP-INCREASE-KEY(A, n, key)
```

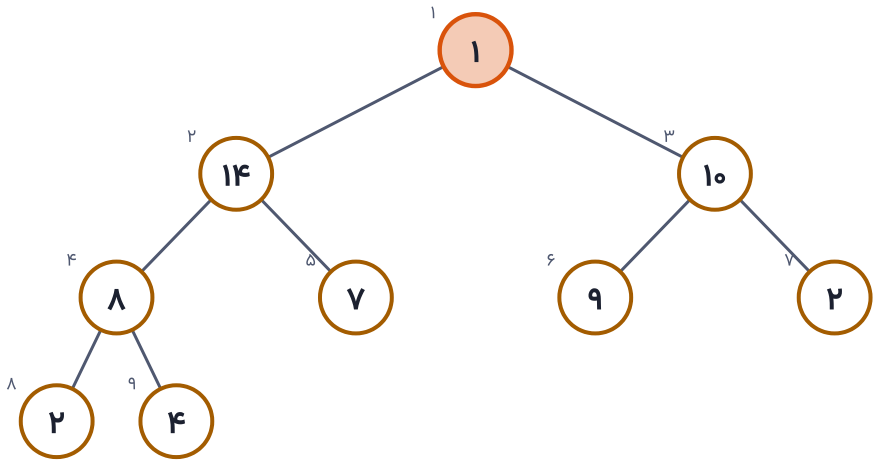
زمان: $O(\log n)$.

۸.۷ حذف بزرگ‌ترین عنصر

مسئله: فرض کنیم بزرگ‌ترین گره (۱۶، ریشه) را می‌خواهیم حذف کنیم. اگر ریشه را برداریم، درخت دو تکه می‌شود. روش؟

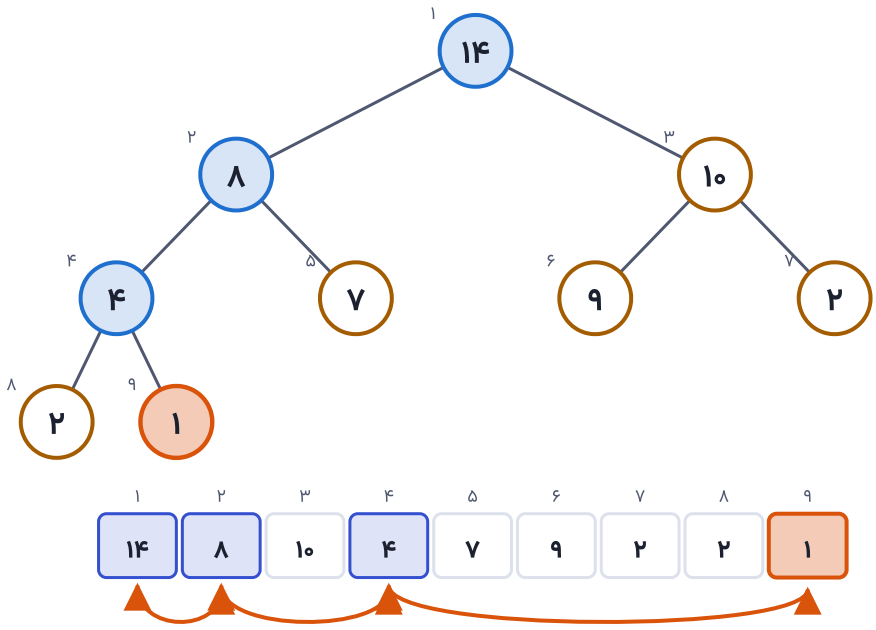
۱ **عنصر انتهای درخت را به محل ریشه می‌آوریم:** عنصر خانهٔ n (عدد ۱) به خانهٔ ۱ می‌رود و n یکی کم می‌شود. شکل درخت کامل می‌ماند (آخرین برگ حذف شد).

۲ **گره را پایین می‌بریم:** گرهٔ تغییریافته در هر مرحله، اگر از فرزندانش کوچک‌تر باشد، با **بزرگ‌ترین فرزندش** جابه‌جا می‌شود.



شکل ۸-۵ گام ۱: ۱۶ حذف شد و ۱ (آخرین عنصر) به ریشه آمد. ویژگی هرم در ریشه خراب است.

- ۱ با فرزندان ۱۴ و ۱۰: کوچک‌تر است؛ با بزرگ‌ترین (۱۴) جابه‌جا.
- ۱ با فرزندان ۸ و ۷: با ۸ جابه‌جا.
- ۱ با فرزندان ۲ و ۴: با ۴ جابه‌جا. حالا ۱ برگ است: توقف.



شکل ۸-۶ گام ۲: پس از پایین بردن. ۱۴ بیشینهٔ جدید است.

چرا؟ · چرا با بزرگ‌ترین فرزند؟

فرزندی که بالا می‌آید، پدرِ فرزند دیگر می‌شود، پس باید از آن بزرگ‌تر یا مساوی باشد. اگر ۱ را با ۱۰ جابه‌جا می‌کردیم، ۱۰ پدر ۱۴ می‌شد و ویژگی هرم خراب می‌شد. فقط بزرگ‌ترین فرزند می‌تواند پدر خوبی برای دیگری باشد.

```
MAX-HEAPIFY(A, i, n) ▷ sift A[i] down; subtrees of i are heaps
l = 2i, r = 2i + 1
largest = i
if l ⩽ n and A[l] > A[largest] then largest = l
if r ⩽ n and A[r] > A[largest] then largest = r
if largest ≠ i
    swap A[i] and A[largest]
    MAX-HEAPIFY(A, largest, n)
```

```
EXTRACT-MAX(A, n)
if n < 1 then error "heap underflow"
max = A[1]
A[1] = A[n]
n = n - 1
MAX-HEAPIFY(A, 1, n)
return max
```

زمان: در هر سطح دو مقایسه و حداکثر یک جابه‌جایی، و حداکثر $\lfloor \log_2 n \rfloor$ سطح: $O(\log n)$.

۸.۸ ساخت هرم روی ورودی با اندازهٔ n

ورودی: $12, 5, 11, 3, 10, 2, 9, 4, 8, 15, 7, 6$. **هدف:** ساخت یک هرم **کمینه** (هر پدر کوچک‌تر یا مساوی فرزندانش). روش؟

روش اول: مرتب‌سازی

با یک الگوریتم مرتب‌سازی (مثلاً مرتب‌سازی سریع) آرایه را مرتب کن. **زمان اجرا:** $O(n \log n)$.

چرا؟ · چرا آرایهٔ مرتب یک هرم کمینه است؟

در آرایهٔ صعودی، برای هر i: $A[i] \le A[2i]$ و $A[i] \le A[2i+1]$، چون $2i > i$ و $2i+1 > i$. پس هر پدر از فرزندانش کوچک‌تر است. اما ما خیلی بیش از نیاز کار کرده‌ایم: ترتیب کامل ساخته‌ایم، در حالی که فقط ترتیب جزئی لازم بود.

روش دوم: درج یکی‌یکی

یک هرم خالی در نظر بگیر و در هر مرحله یکی از داده‌ها را به آن اضافه کن (HEAP-INSERT). **زمان اجرا:** $O(n \log n)$، چون درج iام حداکثر $\log i$ جابه‌جایی دارد و $\sum_{i=1}^{n} \log i = \log(n!) = \Theta(n \log n)$. (این کران دقیق است: اگر داده‌ها نزولی وارد هرم کمینه شوند، هر عنصر تا ریشه بالا می‌رود.)

روش سوم: آیا می‌توان در زمان خطی این کار را کرد؟

بله! این الگوریتم Floyd's buildHeap است:

۱. همهٔ مقادیر را به همان ترتیب در آرایه بگذار. آرایه حالا یک درخت دودویی کامل است، ولی ویژگی هرم را ندارد.

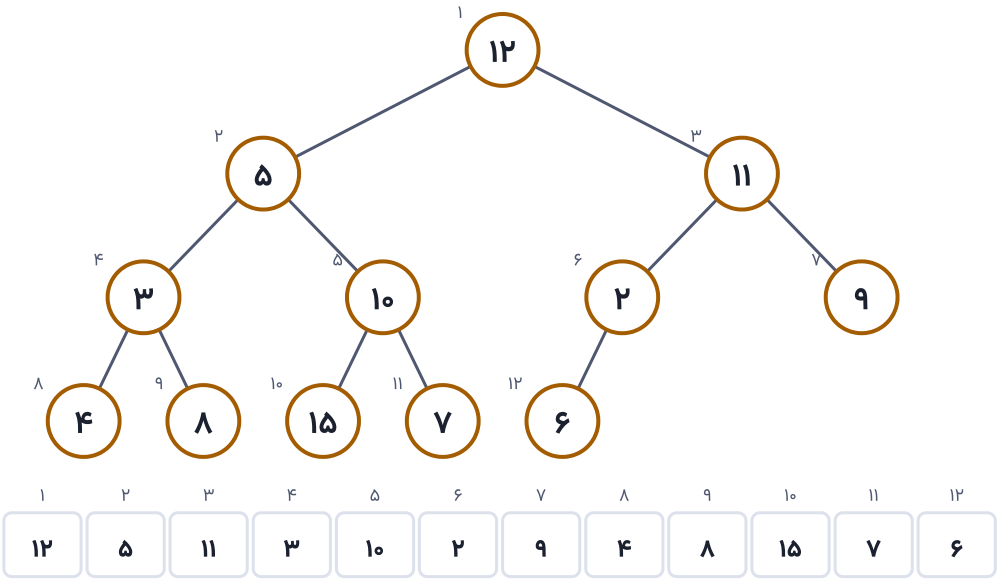
۲. از آخرین گرهٔ داخلی (اندیس $\lfloor n/2 \rfloor$) شروع کن و به عقب تا ریشه برو؛ هر گره را **پایین ببر** (در هرم کمینه: با **کوچک‌ترین** فرزند جابه‌جا کن).

```
BUILD-MIN-HEAP(A, n)
```

```
1  for i = floor(n/2) downto 1
2      MIN-HEAPIFY(A, i, n)        // like MAX-HEAPIFY, with the smallest child
```

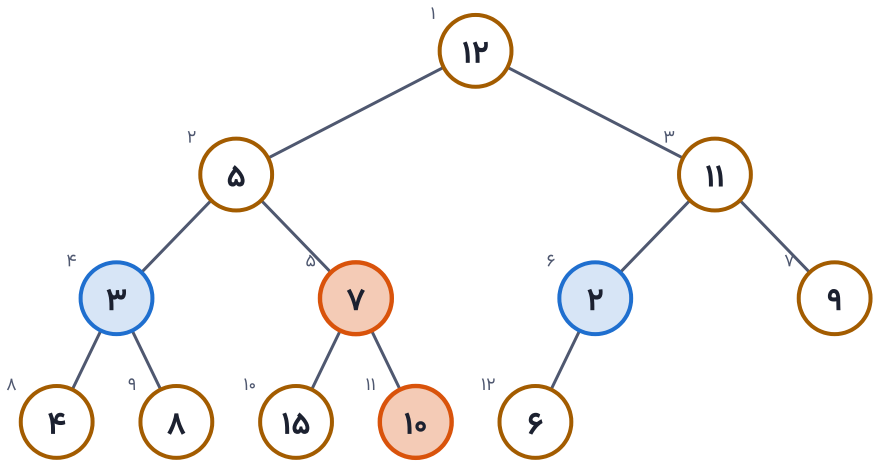
شهود · ناوردا: «همهٔ زیردرخت‌های این مرحله یک هرم هستند»

برگ‌ها (اندیس‌های $\lfloor n/2 \rfloor + 1$ تا n) به‌تنهایی هرم‌اند؛ درختی با یک گره همیشه هرم است. پس وقتی به گرهٔ i می‌رسیم، دو زیردرخت آن از قبل هرم شده‌اند (چون اندیس‌هایشان بزرگ‌تر است و زودتر پردازش شده‌اند). MIN-HEAPIFY دقیقاً همین شرط را لازم دارد: «زیردرخت‌های i هرم‌اند، فقط خود i شاید سر جایش نباشد». پس از آن، کل زیردرخت i هرم است. وقتی به ریشه برسیم، کل درخت هرم است.

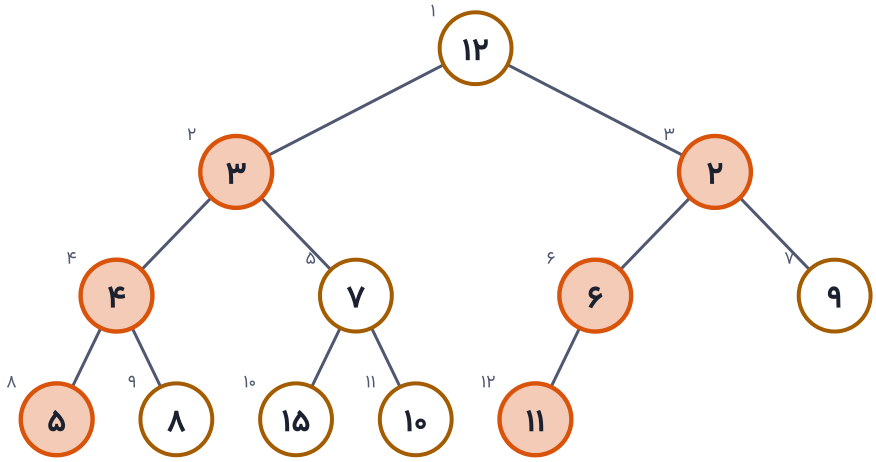


شکل ۸-۷ مرحلهٔ ۰: ورودی به شکل درخت کامل. برگ‌ها (۴، ۸، ۱۵، ۷، ۶ و ۹) هر کدام به‌تنهایی هرم‌اند.

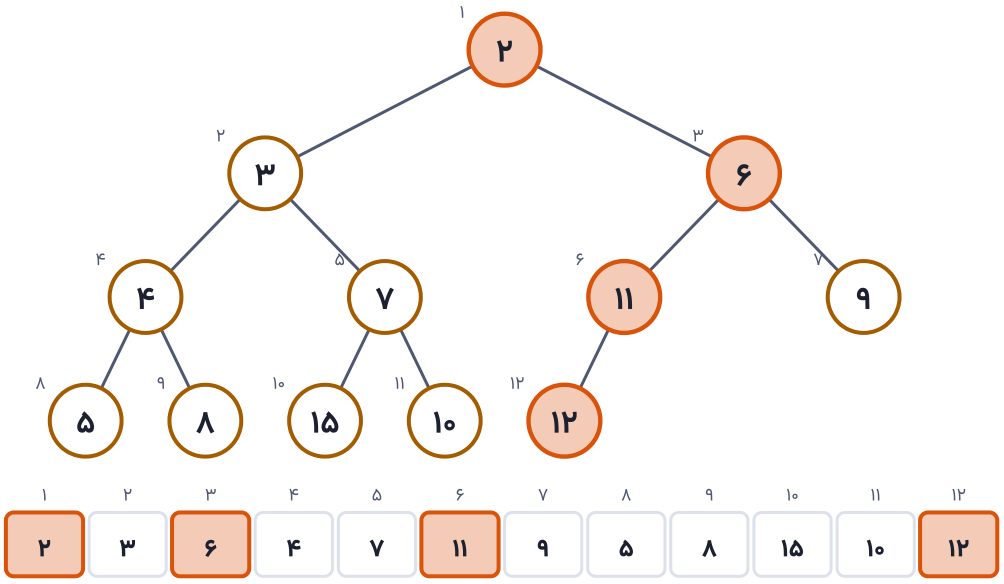
i	A[i]	چه می‌شود	array after (1..12)
6	2	فرزندش ۶ است؛ ۲ کوچک‌تر است. بدون تغییر.	12 5 11 3 10 2 9 4 8 15 7 6
5	10	فرزندان ۱۵ و ۷؛ با کوچک‌ترین (۷) جابه‌جا.	12 5 11 3 **7** 2 9 4 8 15 **10** 6
4	3	فرزندان ۴ و ۸؛ ۳ کوچک‌تر است. بدون تغییر.	12 5 11 3 7 2 9 4 8 15 10 6
3	11	فرزندان ۲ و ۹؛ با ۲ جابه‌جا. ادامه: ۱۱ در خانهٔ ۶ با فرزند ۶؛ جابه‌جا.	12 5 **2** 3 7 **6** 9 4 8 15 10 **11**
2	5	فرزندان ۳ و ۷؛ با ۳ جابه‌جا. ادامه: ۵ در خانهٔ ۴ با فرزندان ۴ و ۸؛ با ۴ جابه‌جا.	12 **3** 2 **4** 7 6 9 **5** 8 15 10 11
1	12	فرزندان ۳ و ۲؛ با ۲ جابه‌جا. ادامه: با ۶، سپس با ۱۱.	**2** 3 **6** 4 7 **11** 9 5 8 15 10 **12**



شکل ۸-۸ پس از پردازش سطح یکی‌مانده‌به‌آخر (اندیس‌های ۶ تا ۴): فقط ۱۰ و ۷ جابه‌جا شدند.



شکل ۸-۹ پس از پردازش سطح ۱ (اندیس‌های ۳ و ۲): زیردرخت‌های ریشه هر دو هرم کمینه‌اند.



شکل ۸-۱۰ پس از پردازش ریشه: هرم کمینهٔ نهایی $2, 3, 6, 4, 7, 11, 9, 5, 8, 15, 10, 12$.

تحلیل: چرا خطی است؟

هزینهٔ پایین بردن یک گره متناسب با **ارتفاع** آن است (حداکثر تا برگ پایین می‌رود). نکتهٔ کلیدی این است که بیشتر گره‌ها ارتفاع کمی دارند: نیمی از گره‌ها برگ‌اند (هزینهٔ صفر)، یک‌چهارم ارتفاع ۱ دارند و ... . جدول زیر (با شمارش ارتفاع از ۱ برای پایین‌ترین گره‌های داخلی، مثل اسلاید) هزینه را نشان می‌دهد:

ارتفاع گره‌ها	تعداد گره‌ها در این ارتفاع	هزینهٔ هر گره در این ارتفاع
1	$n/2$	1
2	$n/4$	2
3	$n/8$	3
$\dots$	$\dots$	$\dots$
$\log n$	$n/n = 1$	$\log n$

$$\text{هزینهٔ کل} = 1\left(\frac{n}{2}\right) + 2\left(\frac{n}{4}\right) + 3\left(\frac{n}{8}\right) + \cdots + \log n\left(\frac{n}{n}\right) = n\left(\frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \dots\right).$$

اثبات · چرا $\frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \cdots = 2$؟

جملهٔ $\frac{h}{2^h}$ را به‌صورت h تا $\frac{1}{2^h}$ بنویسید و جمع را «ستونی» مرتب کنید:

$$\begin{aligned} \tfrac{1}{2} + \tfrac{1}{4} + \tfrac{1}{8} + \tfrac{1}{16} + \dots &= 1 \\ \tfrac{1}{4} + \tfrac{1}{8} + \tfrac{1}{16} + \dots &= \tfrac{1}{2} \\ \tfrac{1}{8} + \tfrac{1}{16} + \dots &= \tfrac{1}{4} \\ \tfrac{1}{16} + \dots &= \tfrac{1}{8} \end{aligned}$$

ستون hام دقیقاً h بار $\frac{1}{2^h}$ دارد، پس جمع کل جدول همان سری ماست. جمع هر سطر یک سری هندسی است و جمع سطرها: $1 + \frac{1}{2} + \frac{1}{4} + \cdots = 2$. بنابراین

$$\text{هزینهٔ کل} \le n\left(1 + \frac{1}{2} + \frac{1}{4} + \dots\right) = 2n \in O(n).$$

■

شهود · چرا روش فلوید از درج یکی‌یکی سریع‌تر است؟

در هر دو روش گره‌ها جابه‌جا می‌شوند، ولی در **جهت‌های مخالف**. در درج یکی‌یکی، هر گره ممکن است تا **ریشه** بالا برود؛ و بیشتر گره‌ها پایین درخت‌اند، یعنی فاصله‌شان تا ریشه زیاد ($\log n$) است. در روش فلوید، هر گره فقط تا **برگ‌ها** پایین می‌رود؛ و بیشتر گره‌ها نزدیک برگ‌اند. کار زیاد برای گره‌های کم، کار کم برای گره‌های زیاد.

۸.۹ مسئلهٔ ۱: شمارش اعداد بزرگ‌تر از k

تعریف · صورت مسئله

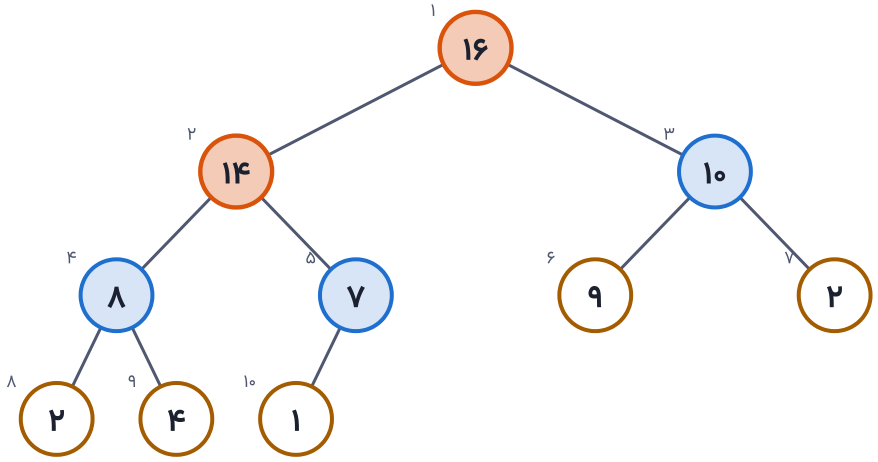
در لیستی به اندازهٔ n، تعداد اعداد بزرگ‌تر از k را گزارش کنید. (فرض: پرسش‌های زیادی با kهای مختلف خواهیم داشت؛ t = تعداد جواب‌های یک پرسش.)

روش	ایده	پیش‌پردازش	هر پرس‌وجو
۱	بررسی کل اعداد	–	$O(n)$
۲	مرتب‌سازی نزولی، سپس از ابتدا خواندن تا رسیدن به عددی $\le k$	$O(n \log n)$	$O(t)$
۳	ساخت هرم بیشینه، سپس شمردن اعداد بزرگ‌تر از k با پیمایش هرس‌شده	$O(n)$	$O(t)$

روش ۳ با جزئیات: از ریشه شروع می‌کنیم. اگر گره‌ای بزرگ‌تر از k بود، آن را می‌شماریم و به فرزندانش می‌رویم؛ اگر نبود، **کل زیردرختش را کنار می‌گذاریم**.

```
COUNT-GREATER(A, i, n, k) ▷ call with i = 1
```

```
1  if i > n or A[i] ⩽ k
2      return 0                      // prune: nothing below can be > k
3  return 1 + COUNT-GREATER(A, 2i, n, k) + COUNT-GREATER(A, 2i+1, n, k)
```



شکل ۸-۱۱ مثال $k = 11$: گره‌های نارنجی (۱۶ و ۱۴) شمرده می‌شوند، پس $t = 2$. گره‌های آبی (۱۰، ۸ و ۷) بررسی و رد می‌شوند و زیردرخت‌هایشان هرگز دیده نمی‌شود.

چرا؟ · چرا هرس درست است و چرا $O(t)$؟

درستی: اگر گرهٔ v حداکثر k باشد، همهٔ اولادش $k \le v \le$ هستند (ویژگی هرم با تعدی)، پس هیچ جوابی در زیردرختش نیست.

زمان: فقط گره‌هایی بررسی می‌شوند که یا جواب‌اند (t تا) یا فرزند یک جواب‌اند (حداکثر $2t$ تا) یا ریشه‌اند. پس حداکثر $2t+1$ گره بررسی می‌شود: $O(t)$. در مثال: $2 \cdot 2 + 1 = 5$ بررسی (۱۶، ۱۴، ۱۰، ۸ و ۷). چون گزارش t جواب به‌هرحال $\Omega(t)$ زمان می‌برد، این بهینه است.

۸.۱۰ مسئلهٔ ۲: مرتب‌سازی با هرم

تعریف · صورت مسئله

ورودی: n عدد با نظم دلخواه. **خروجی:** همان n عدد به‌صورت مرتب.

الگوریتم · مرتب‌سازی هرمی (Heapsort)

1. **ساخت هرم بیشینه** با روش فلوید: هزینهٔ $O(n)$.
2. **n بار حذف بزرگ‌ترین عنصر:** هر بار $O(\log n)$، در کل $O(n \log n)$.

جمع: $O(n) + O(n \log n) = O(n \log n)$.

ترفند زیبا: عنصری که حذف می‌شود را در خانه‌ای می‌گذاریم که هرم تازه از دست داده (انتهای آرایه). پس مرتب‌سازی **درجا** انجام می‌شود و حافظهٔ اضافه نمی‌خواهد:

```
HEAPSORT(A, n)
```

```
1  BUILD-MAX-HEAP(A, n)
2  for last = n downto 2
3      swap A[1] and A[last]           // the maximum goes to its final place
4      MAX-HEAPIFY(A, 1, last - 1)     // restore the heap on A[1..last-1]
```

مثال · اجرا روی $4, 10, 3, 5, 1$

step	heap part | sorted part
build max-heap	10 5 3 4 1 |
swap 10, heapify	5 4 3 1 | 10
swap 5, heapify	4 1 3 | 5 10
swap 4, heapify	3 1 | 4 5 10
swap 3	1 | 3 4 5 10
done	**1 3 4 5 10**

نکته · جایگاه مرتب‌سازی هرمی

مرتب‌سازی هرمی در **بدترین حالت** $O(n \log n)$ است (برخلاف کوئیک‌سورت) و **درجا** است (برخلاف مرتب‌سازی ادغامی). اما **پایدار نیست** (فصل ۹) و در عمل به‌خاطر دسترسی‌های پراکنده به حافظه معمولاً از کوئیک‌سورت کندتر است. در فصل ۹ می‌بینیم $O(n \log n)$ برای مرتب‌سازی مقایسه‌ای بهینه است.

جمع‌بندی · فصل ۸ در یک نگاه

- صف اولویت: insert، max و extract_max. هرم همه را در $O(1)$ یا $O(\log n)$ انجام می‌دهد.
- هرم = درخت دودویی کامل + ویژگی هرم (پدر $\geq$ فرزندان). بیشینه در ریشه، ارتفاع $\lfloor \log n \rfloor$.
- آرایه: ریشه $A[1]$، فرزندان $2i$ و $2i+1$، پدر $\lfloor i/2 \rfloor$.
- افزایش کلید و درج: بالا بردن. حذف بیشینه: آخرین عنصر به ریشه و پایین بردن با بزرگ‌ترین فرزند.
- ساخت هرم با روش فلوید $O(n)$، چون $\sum h/2^h = 2$.
- شمارش اعداد $k >$ در $O(t)$؛ مرتب‌سازی هرمی $O(n \log n)$ و درجا.

۸.۱۱ خودآزمایی فصل ۸

خودآزمایی

۱. آیا آرایهٔ $20, 15, 18, 7, 9, 18, 2$ یک هرم بیشینه است؟

پاسخ: بله. فرزندان ۲۰: ۱۵ و ۱۸؛ فرزندان ۱۵: ۷ و ۹؛ فرزندان ۱۸ (خانهٔ ۳): ۱۸ و ۲. همه $\leq$ پدرشان‌اند (برابری مجاز است).

2. در یک هرم بیشینه، کوچک‌ترین عنصر کجا می‌تواند باشد؟

پاسخ: فقط در یک برگ (اندیس‌های $\lfloor n/2 \rfloor + 1$ تا n)، چون هر گرهٔ داخلی از فرزندش بزرگ‌تر یا مساوی است. پس پیدا کردن کمینه در هرم بیشینه $O(n)$ است.

3. عدد ۲۰ را به هرم شکل اول $(16, 14, 10, 8, 7, 9, 2, 2, 4, 1)$ اضافه کنید.

پاسخ: خانهٔ ۱۱؛ جابه‌جایی با ۷ (خانهٔ ۵)، با ۱۴ (خانهٔ ۲) و با ۱۶ (ریشه): $20, 16, 10, 8, 14, 9, 2, 2, 4, 1, 7$.

4. ارتفاع هرمی با ۱۰۰۰ عنصر چقدر است؟

پاسخ: $\lfloor \log_2 1000 \rfloor = 9$، چون $2^9 = 512 \le 1000 < 1024 = 2^{10}$.

5. چطور kامین بزرگ‌ترین عنصر را با هرم پیدا کنیم و با چه زمانی؟

پاسخ: ساخت هرم بیشینه $O(n)$، سپس k بار extract_max، هر بار $O(\log n)$: در کل $O(n + k \log n)$.

فصل ۹

کران پایین مرتب‌سازی و مرتب‌سازی در زمان خطی

ثابت می‌کنیم هیچ الگوریتمی که فقط با **مقایسه** کار کند نمی‌تواند سریع‌تر از $n \log n$ مرتب کند. بعد با کنار گذاشتن مقایسه و استفاده از خود مقدارها، در زمان خطی مرتب می‌کنیم.

۹

مرور مرتب‌سازی‌ها | درخت تصمیم | کران پایین | مرتب‌سازی شمارشی | پایداری | مرتب‌سازی مبنایی

۹.۱ مرور: تا این‌جا چطور مرتب کردیم؟

الگوریتم	ایده	بهترین	میانگین / امید	بدترین	فصل
درجی	هر عنصر را در بخش مرتب‌شدهٔ قبلی درج کن	$O(n)$	$O(n^2)$	$O(n^2)$	۱
ادغامی	تقسیم و حل: دو نیمه را بازگشتی مرتب کن و در زمان خطی ادغام کن	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	۲
سریع (تصادفی)	یک جداکنندهٔ تصادفی انتخاب کن، به دو زیرآرایه تقسیم کن و بازگشتی مرتب کن	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	۳
هرمی	هرم بساز و n بار بیشینه را حذف کن	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	۸

بهترین بدترین‌حالتی که تا حالا دیده‌ایم $O(n \log n)$ است. آیا می‌توان بهتر عمل کرد؟

۹.۲ چقدر سریع می‌توان مرتب کرد؟

تعریف · مرتب‌سازی مقایسه‌ای

همهٔ الگوریتم‌های جدول بالا **مرتب‌سازی مقایسه‌ای** (comparison sort) هستند: تنها عملی که برای کسب اطلاعات دربارهٔ ترتیب عناصر به کار می‌برند، **مقایسهٔ دوبه‌دوی دو عنصر** است ($a_i < a_j$؟). آن‌ها مقدار عددی عنصر را هیچ‌جای دیگری استفاده نمی‌کنند؛ برای همین روی هر چیزِ قابل مقایسه (اعداد، رشته‌ها، تاریخ‌ها) کار می‌کنند.

چرا؟ · چرا دست‌کم $n-1$ مقایسه لازم است؟

یک گراف بسازید که رأس‌هایش عناصرند و برای هر مقایسه بین a_i و a_j یک یال بکشید. اگر کمتر از $n-1$ مقایسه انجام شده باشد، این گراف **ناهمبند** است (گراف همبند با n رأس حداقل $n-1$ یال دارد؛ فصل ۶). پس دو گروه از عناصر هستند که هیچ‌کدام از اعضای یکی با دیگری مقایسه نشده. می‌توان همهٔ عناصر یک گروه را به مقدار بزرگی اضافه کرد بدون اینکه نتیجهٔ هیچ مقایسه‌ای عوض شود؛ الگوریتم همان خروجی را می‌دهد، ولی ترتیب درست عوض شده است. پس دست‌کم برای یکی از دو ورودی اشتباه کرده است.

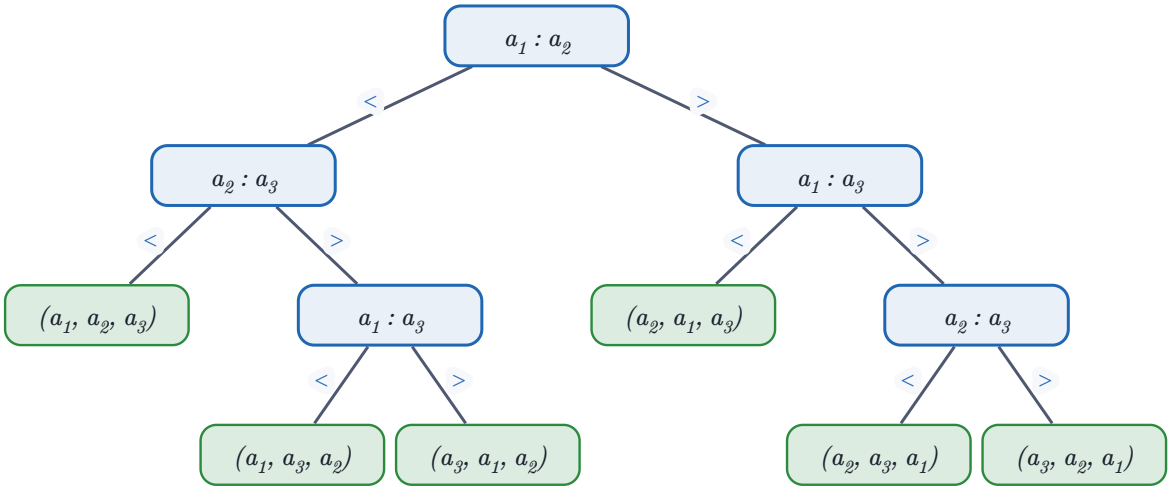
پس کران پایین $\Omega(n)$ بدیهی است. اما فاصلهٔ n تا $n \log n$ زیاد است. **بهترین زمان ممکن برای مرتب‌سازی مقایسه‌ای چیست؟** برای پاسخ، ابزار تازه‌ای لازم داریم.

۹.۳ درخت تصمیم

تعریف · درخت تصمیم

هر فرایند مرتب‌سازی مبتنی بر مقایسه را می‌توان با یک **درخت تصمیم دودویی** (decision tree) نمایش داد:

- هر **گرهٔ داخلی** یک مقایسه است (مثلاً $a_1 : a_2$) و مجموعه‌ای از ترتیب‌های ممکن را نشان می‌دهد که با همهٔ مقایسه‌های انجام‌شده تا آن‌جا سازگارند.
- **یال‌ها** نتیجهٔ مقایسه‌اند ($<$ یا $>$).
- هر **برگ** یک ترتیب نهایی است: الگوریتم به آن‌جا که برسد، می‌داند جایگشت مرتب کدام است.



شکل ۹-۱ درخت تصمیم مرتب‌سازی درجی روی سه عنصر a_1, a_2, a_3. مثلاً برگ (a_1, a_3, a_2) یعنی ترتیب مرتب $a_1 < a_3 < a_2$ است.

مثال · خواندن درخت: یک اجرا = یک مسیر

ورودی $a_1 = 5, a_2 = 9, a_3 = 7$: ریشه $a_1 : a_2$ ← $5 < 9$ چپ. سپس $a_2 : a_3$ ← $9 > 7$ راست. سپس $a_1 : a_3$ ← $5 < 7$ چپ. برگ (a_1, a_3, a_2) یعنی $5, 7, 9$. ✓ سه مقایسه انجام شد.

ورودی مرتب $1, 2, 3$ فقط دو مقایسه لازم دارد (چپ، چپ)؛ این همان بهترین حالت $O(n)$ مرتب‌سازی درجی است.

نکته · هر الگوریتم، درخت خودش

الگوریتم دیگر درخت دیگری دارد. اسلاید درخت «الگوریتم X» را هم نشان می‌دهد که ریشه‌اش هر ۶ ترتیب $a < b < c, \ldots, c < b < a$ را دارد؛ مقایسهٔ $a : b$ آن‌ها را به دو گروه سه‌تایی تقسیم می‌کند، مقایسهٔ بعدی هر گروه را به یک ترتیب تنها و یک گروه دوتایی، و مقایسهٔ سوم آن دوتایی را جدا می‌کند. هر چه باشد، همهٔ درخت‌ها یک ویژگی مشترک دارند که در بخش بعد از آن استفاده می‌کنیم.

۹.۴ کران پایین $\Omega(n \log n)$ برای مرتب‌سازی مقایسه‌ای

- **تعداد مقایسه‌ها در بدترین حالت = عمق عمیق‌ترین برگ** (ارتفاع درخت)، چون هر اجرا یک مسیر از ریشه تا یک برگ است و روی هر یال یک مقایسه انجام می‌شود.
- تعداد **میانگین** مقایسه‌ها = میانگین عمق برگ‌ها (اگر همهٔ ترتیب‌های ورودی هم‌احتمال باشند).

قضیه · تعداد برگ‌ها

درخت تصمیمی که N عنصر را درست مرتب کند، دست‌کم $N!$ برگ دارد.

N عنصر متمایز $N!$ ترتیب ممکن دارند و هر کدام پاسخ متفاوتی می‌خواهند (جایگشتی که آن‌ها را مرتب می‌کند فرق دارد). اگر دو ترتیب ورودی متفاوت به **یک** برگ برسند، الگوریتم برای هر دو همان جایگشت را اعمال می‌کند و دست‌کم برای یکی اشتباه است. پس هر ترتیب ورودی باید به برگ خودش برسد: دست‌کم $N!$ برگ.

∎

قضیه · کران پایین

هر الگوریتم مرتب‌سازی که فقط بر پایهٔ مقایسهٔ عناصر باشد، در بدترین حالت دست‌کم $\lceil \log_2(N!) \rceil$ مقایسه انجام می‌دهد، و $\log_2(N!) = \Omega(N \log N)$.

گام ۱: درخت دودویی به عمق h حداکثر 2^h برگ دارد (در هر سطح تعداد گره‌ها حداکثر دو برابر می‌شود). پس

$$2^h \geq \#\text{برگ‌ها} \geq N! \quad \Longrightarrow \quad h \geq \log_2(N!) \quad \Longrightarrow \quad h \geq \lceil \log_2(N!) \rceil .$$

گام ۲: $\log_2(N!)$ را از پایین کران می‌زنیم:

$$\begin{aligned}
\log_2(N!) &= \log\big(N(N-1)(N-2)\cdots(2)(1)\big) \\
&= \log N + \log(N-1) + \log(N-2) + \cdots + \log 2 + \log 1 && \text{لگاریتم ضرب = جمع لگاریتم‌ها} \\
&\geq \log N + \log(N-1) + \cdots + \log(N/2) && \text{فقط نیمهٔ بزرگ‌تر جملات} \\
&\geq \frac{N}{2} \log \frac{N}{2} && \text{هر کدام } \geq \log(N/2) \\
&= \frac{N}{2}\log N - \frac{N}{2} = \Omega(N \log N).
\end{aligned}$$

∎

چرا؟ · چرا این کران قوی است؟

این استدلال دربارهٔ **هر** الگوریتم مقایسه‌ای است، حتی الگوریتم‌هایی که هنوز کشف نشده‌اند. فرقی نمی‌کند الگوریتم چقدر زیرکانه باشد؛ تا وقتی فقط مقایسه انجام می‌دهد، باید $N!$ حالت را از هم جدا کند و هر مقایسه حداکثر تعداد حالت‌های ممکن را نصف می‌کند. پس به دست‌کم $\log_2(N!)$ «نصف کردن» نیاز دارد. (مثل بازی «۲۰ سؤال بله/خیر» برای حدس یکی از $N!$ جایگشت.)

مثال · عدد

- $N = 3$: $\log_2 6 \approx 2.58$، پس دست‌کم ۳ مقایسه در بدترین حالت. درخت مرتب‌سازی درجی ارتفاع ۳ دارد: برای ۳ عنصر **بهینه** است.
- $N = 4$: $\log_2 24 \approx 4.58$، پس دست‌کم ۵ مقایسه.
- $N = 5$: $\lceil \log_2 120 \rceil = 7$ مقایسه.

جمع‌بندی · نتیجه

زمان هر مرتب‌سازی مقایسه‌ای روی n عنصر $\Omega(n \log n)$ است. **نتیجهٔ فرعی:** مرتب‌سازی ادغامی (و هرمی) از نظر مجانبی **بهینه**‌اند. با میانگین عمق برگ‌ها هم می‌توان نشان داد امید زمان کوئیک‌سورت تصادفی، $O(n \log n)$، بهینه است.

اما عنوان این فصل «مرتب‌سازی در زمان خطی» است! **چطور می‌توان از** $\Omega(n \log n)$ **بهتر عمل کرد؟** تنها راه: **مقایسه نکنیم**. یعنی فرضی دربارهٔ ورودی بگذاریم و از خود مقدارها استفاده کنیم.

۹.۵ مرتب‌سازی شمارشی

تعریف · چرا مرتب‌سازی مقایسه‌ای نیست؟

- **فرض:** ورودی اعداد صحیح در بازهٔ $0..k$ است.
- **هیچ مقایسه‌ای انجام نمی‌شود!** از مقدار هر عنصر به‌عنوان **اندیس آرایه** استفاده می‌کنیم؛ این عملی است که درخت تصمیم نمی‌تواند مدلش کند (یک «سؤال» با $k+1$ جواب).

شهود · ایدهٔ اصلی: رتبه

برای هر عنصر ورودی x، **رتبه**‌اش را پیدا کن: تعداد عناصر کوچک‌تر از x به‌علاوهٔ ۱. وقتی رتبهٔ r را بدانیم، x را مستقیماً در خانهٔ r خروجی می‌گذاریم. مثلاً اگر ۱۷ عنصر از x کوچک‌تر باشند، جای x خانهٔ ۱۸ است. اگر مقدار تکراری داریم، «تعداد عناصر کوچک‌تر یا مساوی x» آخرین خانهٔ گروهِ مقدار x را می‌دهد.

```
COUNTING-SORT(A, B, n, k) ▷ A[1..n] has values in 0..k; output in B[1..n]
```

```
1  for i = 0 to k                  // 1) clear the counters
2      C[i] = 0
3  for j = 1 to n                  // 2) C[v] = number of elements equal to v
4      C[A[j]] = C[A[j]] + 1
5  for i = 1 to k                  // 3) C[v] = number of elements ⩽ v
6      C[i] = C[i] + C[i-1]
7  for j = n downto 1              // 4) place each element at its final position
8      B[C[A[j]]] = A[j]
9      C[A[j]] = C[A[j]] - 1
```

نکته · اندیس‌ها

آرایهٔ C باید اندیس ۰ تا k داشته باشد، چون مقدار ۰ هم ممکن است. در اسلاید حلقهٔ اول از ۱ و حلقهٔ سوم از ۲ شروع شده؛ این‌جا آن‌ها را از ۰ و ۱ نوشته‌ایم تا $C[0]$ هم درست مقداردهی و جمع زده شود.

مثال گام‌به‌گام

$A = [2, 5, 3, 0, 2, 3, 0, 3]$ با $n = 8$ و $k = 5$.

value v	0	1	2	3	4	5
C after step 2 (count of v)	2	0	2	3	0	1
C after step 3 (count of ≤ v)	2	2	4	7	7	8

معنای سطر دوم: مثلاً $C[3] = 7$ یعنی ۷ عنصر ≤ 3 هستند؛ پس **آخرین** ۳ باید در خانهٔ ۷ خروجی برود. $C[2] = 4$ یعنی آخرین ۲ در خانهٔ ۴. حالا گام ۴ از آخر آرایه شروع می‌کند:

j	A[j]	goes to B[…]	B[1..8] after	C[0..5] after
8	3	B[7]	_ _ _ _ _ _ 3 _	2 2 4 **6** 7 8
7	0	B[2]	_ 0 _ _ _ _ 3 _	**1** 2 4 6 7 8
6	3	B[6]	_ 0 _ _ _ 3 3 _	1 2 4 **5** 7 8
5	2	B[4]	_ 0 _ 2 _ 3 3 _	1 2 **3** 5 7 8
4	0	B[1]	0 0 _ 2 _ 3 3 _	**0** 2 3 5 7 8
3	3	B[5]	0 0 _ 2 3 3 3 _	0 2 3 **4** 7 8
2	5	B[8]	0 0 _ 2 3 3 3 5	0 2 3 4 7 **7**
1	2	B[3]	**0 0 2 2 3 3 3 5**	0 2 **2** 4 7 7

چرا؟ · چرا $C[A[j]]$ را پس از هر گذاشتن یکی کم می‌کنیم؟

پس از گذاشتن یک ۳ در خانهٔ ۷، ۳ بعدی (اگر باشد) باید در خانهٔ ۶ برود؛ یعنی «آخرین جای خالیِ گروه ۳» یکی عقب رفته است. $C[v]$ همیشه نشان می‌دهد عنصر بعدی با مقدار v کجا برود.

زمان اجرا

حلقهٔ ۱ $O(k)$

حلقهٔ ۲ $O(n)$

حلقهٔ ۳ $O(k)$

حلقهٔ ۴ $O(n)$

زمان کل: $O(n+k)$. معمولاً $k = O(n)$ است، پس مرتب‌سازی شمارشی در $O(n)$ اجرا می‌شود. حافظهٔ اضافه: $O(n+k)$ (آرایه‌های B و C).

اشتباه رایج · ولی مرتب‌سازی $\Omega(n \lg n)$ بود!

تناقضی نیست. کران پایین فقط برای مرتب‌سازی **مقایسه‌ای** است. مرتب‌سازی شمارشی اصلاً مقایسه نمی‌کند (در واقع هیچ مقایسه‌ای بین عناصر در آن نیست)؛ بیرون از مدل درخت تصمیم است.

۹.۶ پایداری

تعریف · مرتب‌سازی پایدار

یک الگوریتم مرتب‌سازی **پایدار** (stable) است اگر برای هر دو اندیس $i < j$ با $a_i = a_j$، عنصر a_i در خروجی پیش از a_j بیاید. یعنی عناصر مساوی ترتیب اولیه‌شان را حفظ کنند.

مثال · مثال

زیرنویس‌ها فقط برای تشخیص عناصر مساوی‌اند:

Input: $2_1\ 7_1\ 4_1\ 4_2\ 2_2\ 5_1\ 2_3\ 6_1$ → Output: $2_1\ 2_2\ 2_3\ 4_1\ 4_2\ 5_1\ 6_1\ 7_1$

سه ۲ به همان ترتیب ورودی (۱، ۲، ۳) و دو ۴ هم به ترتیب ورودی آمده‌اند: خروجی پایدار است.

قضیه · پایداری مرتب‌سازی شمارشی

مرتب‌سازی شمارشی (با حلقهٔ آخر از $j = n$ به ۱) پایدار است.

حلقهٔ ۴ از $j = n$ به عقب حرکت می‌کند. از میان عناصری با مقدار v، **آخرین** عنصر ورودی اول دیده می‌شود و به **آخرین** خانهٔ گروه v (یعنی $C[v]$) می‌رود؛ عنصر قبلی با مقدار v به خانهٔ قبلی، و ... . پس عناصر مساوی در خروجی به همان ترتیب ورودی چیده می‌شوند. (در جدول بالا: سه ۳ در خانه‌های ۵، ۶ و ۷ به ترتیب $A[3], A[6], A[8]$ قرار گرفتند.)

اگر حلقهٔ ۴ از $j = 1$ تا n می‌رفت، خروجی هنوز مرتب بود، اما عناصر مساوی **برعکس** می‌شدند: ناپایدار.

■

چرا؟ · پایداری به چه دردی می‌خورد؟

در عمل، معمولاً «رکوردها» را بر اساس یک «کلید» مرتب می‌کنیم (مثلاً دانشجویان بر اساس معدل). اگر فهرست قبلاً بر اساس نام مرتب بوده، مرتب‌سازی پایدار بر اساس معدل باعث می‌شود دانشجویان هم‌معدل همچنان به ترتیب الفبا بمانند. مهم‌تر از آن، **مرتب‌سازی مبنایی بدون پایداری کار نمی‌کند** (بخش بعد).

چرا همیشه از مرتب‌سازی شمارشی استفاده نکنیم؟

مرتب‌سازی خطی! عالی است! اما زمانش به **بازهٔ** k بستگی دارد، نه فقط به n. آیا می‌توان با آن اعداد صحیح ۳۲ بیتی را مرتب کرد؟ **نه**: $k = 2^{32} = 4{,}294{,}967{,}296$. آرایهٔ C چند گیگابایت حافظه می‌خواهد و حتی برای مرتب کردن ۱۰ عدد، باید ۴ میلیارد شمارنده را صفر و جمع کنیم: $O(n + k)$ با k غالب.

۹.۷ مرتب‌سازی مبنایی

تعریف · چرا مرتب‌سازی مقایسه‌ای نیست؟

- **فرض:** هر عنصر ورودی d رقم دارد و هر رقم بین 0 و k است. مثال: مرتب کردن تعدادی عدد چهاررقمی که هر رقم بین ۰ و ۹ است.
- **ایدهٔ اصلی:** عناصر را رقم‌به‌رقم مرتب کن، **از کم‌ارزش‌ترین رقم شروع کن**، و برای هر مرحله از یک مرتب‌سازی **پایدار** (مثل شمارشی) استفاده کن.

RADIX-SORT(A, d)

```
1  for i = 1 to d
2      sort the numbers in A by their i-th digit from the right,
3          using a stable sorting algorithm (e.g. counting sort)
```

مثال · مرتب‌سازی هفت عدد سه‌رقمی

input	after pass 1 (ones)	after pass 2 (tens)	after pass 3 (hundreds)
329	72**0**	7**2**0	**3**29
457	35**5**	3**2**9	**3**55
657	43**6**	4**3**6	**4**36
839	45**7**	8**3**9	**4**57
436	65**7**	3**5**5	**6**57
720	32**9**	4**5**7	**7**20
355	83**9**	6**5**7	**839**

در گذر ۱ دقت کنید ۴۵۷ پیش از ۶۵۷ و ۳۲۹ پیش از ۸۳۹ مانده است (رقم یکان مساوی؛ ترتیب ورودی حفظ شد). در گذر ۲، ۷۲۰ پیش از ۳۲۹ است چون هر دو رقم دهگان ۲ دارند و ۷۲۰ از گذر قبل جلوتر بود.

اثبات · چرا از کم‌ارزش‌ترین رقم، و چرا پایدار؟

ادعا: پس از گذر i، اعداد بر اساس i **رقم آخرشان** مرتب‌اند. استقرا روی i:

- **پایه ($i = 1$):** گذر ۱ بر اساس رقم آخر مرتب می‌کند. ✓
- **گام:** گذر $i+1$ بر اساس رقم $(i+1)$ام مرتب می‌کند. دو عدد x و y را در نظر بگیرید. اگر رقم $(i+1)$امشان فرق کند، گذر $i+1$ آن‌ها را درست می‌چیند. اگر برابر باشد، چون مرتب‌سازی **پایدار** است، ترتیب قبلی‌شان حفظ می‌شود؛ و طبق فرض استقرا آن ترتیب بر اساس i رقم آخر درست بود. در هر دو حالت x و y بر اساس $i+1$ رقم آخر درست چیده شده‌اند. ✓

پس از d گذر، اعداد بر اساس همهٔ رقم‌ها، یعنی کاملاً، مرتب‌اند.

■

اشتباه رایج · اگر از پرارزش‌ترین رقم شروع کنیم یا مرتب‌سازی ناپایدار باشد؟

- **از رقم پرارزش:** پس از مرتب کردن بر اساس صدگان، مرتب کردن کل آرایه بر اساس دهگان ترتیب صدگان را به هم می‌ریزد (مگر اینکه هر گروه صدگان را جداگانه و بازگشتی مرتب کنیم که پیچیده‌تر است).
- **ناپایدار:** برای ۱۳ و ۱۲، گذر ۱ می‌دهد ۱۲، ۱۳. در گذر ۲ هر دو رقم دهگان ۱ دارند؛ یک مرتب‌سازی ناپایدار ممکن است آن‌ها را جابه‌جا کند و ۱۳، ۱۲ بدهد: غلط.

زمان اجرا

هر گذر روی یک رقم با مرتب‌سازی شمارشی $O(n+k)$ زمان می‌برد. d گذر داریم، پس **زمان کل** $O(d\,n + d\,k) = O(d(n+k))$.
وقتی d ثابت باشد و $k = O(n)$، این همان $O(n)$ است. مرتب‌سازی مبنایی **پایدار و سریع** است.

تکمیلی (فراتر از اسلاید) · انتخاب مبنا: اعداد ۳۲ بیتی در زمان خطی

«رقم» لازم نیست دهدهی باشد. یک عدد ۳۲ بیتی را به ۴ «رقم» ۸ بیتی بشکنید: $d = 4$ و $k = 2^8 - 1 = 255$. زمان $O(4(n+256)) = O(n)$ با فقط ۲۵۶ شمارنده؛ این همان مشکلی است که مرتب‌سازی شمارشی خالص با $k = 2^{32}$ داشت. با «رقم»های ۱۶ بیتی، $d = 2$ ولی $k = 65535$: گذرهای کمتر، آرایهٔ شمارندهٔ بزرگ‌تر. انتخاب بهینه به n بستگی دارد.

۹.۸ جمع‌بندی مرتب‌سازی‌ها

الگوریتم	بدترین	میانگین / امید	درجا	پایدار	مقایسه‌ای
درجی	$O(n^2)$	$O(n^2)$	بله	بله	بله
ادغامی	$O(n \log n)$	$O(n \log n)$	خیر ($O(n)$ حافظه)	بله	بله
سریع تصادفی	$O(n^2)$	$O(n \log n)$	بله	خیر	بله
هرمی	$O(n \log n)$	$O(n \log n)$	بله	خیر	بله
شمارشی	$O(n+k)$	$O(n+k)$	خیر	بله	خیر
مبنایی	$O(d(n+k))$	$O(d(n+k))$	خیر	بله	خیر

جمع‌بندی · فصل ۹ در یک نگاه

- هر الگوریتم مرتب‌سازی مبتنی بر مقایسه باید $\Omega(n \log n)$ زمان بگیرد (درخت تصمیم با $n!$ برگ و ارتفاع $\geq \log_2 n!$).
- مرتب‌سازی ادغامی و سریع مقایسه‌ای‌اند و $O(n \log n)$ زمان می‌برند (سریع در امید)؛ پس بهینه‌اند.
- الگوریتم‌های دیگر با استفاده از فرض‌هایی دربارهٔ ورودی می‌توانند سریع‌تر باشند.
- مرتب‌سازی شمارشی و مبنایی برای اعداد صحیح در یک بازهٔ محدود، زمان خطی می‌گیرند. شمارشی پایدار است و مبنایی به همین پایداری متکی است.

۹.۹ خودآزمایی فصل ۹

خودآزمایی

1. حداقل تعداد مقایسه در بدترین حالت برای مرتب کردن ۵ عنصر با هر الگوریتم مقایسه‌ای چقدر است؟

پاسخ: $\lceil \log_2 5! \rceil = \lceil \log_2 120 \rceil = 7$.

2. مرتب‌سازی شمارشی را روی $A = [1, 4, 1, 2, 4, 0]$ با $k = 4$ اجرا کنید؛ C پس از گام ۲ و ۳ چیست؟

پاسخ: پس از گام ۲: $C = [1, 2, 1, 0, 2]$. پس از گام ۳: $C = [1, 3, 4, 4, 6]$. خروجی: $[0, 1, 1, 2, 4, 4]$.

3. مرتب‌سازی مبنایی را روی $170, 45, 75, 90, 802, 24, 2, 66$ اجرا کنید (اعداد را سه‌رقمی فرض کنید).

پاسخ: گذر ۱: $170, 90, 802, 2, 24, 45, 75, 66$. گذر ۲: $802, 2, 24, 45, 66, 170, 75, 90$. گذر ۳: $2, 24, 45, 66, 75, 90, 170, 802$.

4. اگر حلقهٔ آخر مرتب‌سازی شمارشی از $j = 1$ تا n برود، آیا خروجی هنوز مرتب است؟ پایدار چطور؟

پاسخ: مرتب است، چون هر عنصر هنوز در گروه درست خودش قرار می‌گیرد؛ ولی ترتیب عناصر مساوی برعکس می‌شود، پس ناپایدار است و دیگر نمی‌توان آن را در مرتب‌سازی مبنایی به کار برد.

5. چطور n عدد صحیح در بازهٔ 0 تا $n^2 - 1$ را در زمان $O(n)$ مرتب کنیم؟

پاسخ: هر عدد را در مبنای n با دو رقم بنویسید ($d = 2$، هر رقم بین 0 و $n - 1$). مرتب‌سازی مبنایی با شمارشی: $O(2(n + n)) = O(n)$. مرتب‌سازی شمارشی مستقیم $O(n^2)$ می‌شد.

6. آیا کران پایین $\Omega(n \log n)$ شامل الگوریتمی می‌شود که فقط از عمل «$a_i \le a_j$؟» و جابه‌جایی استفاده کند؟

پاسخ: بله؛ هر اطلاعاتی که الگوریتم دربارهٔ ترتیب به دست می‌آورد از مقایسه است، پس رفتارش با یک درخت تصمیم مدل می‌شود و کران پایین برقرار است.

پیوست

برگهٔ خلاصه، واژه‌نامه و منابع

همهٔ فرمول‌ها و زمان‌های اجرایی که در طول جزوه به آن‌ها نیاز داشتیم، در یک جا؛ برای مرور شب امتحان.

پ.۱ فرمول‌های پرکاربرد

جمع‌ها

نام	فرمول	کجا به کار رفت
جمع حسابی	$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$	مرتب‌سازی درجی، حلقه‌های تودرتو
جمع مربعات	$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$	تحلیل‌های مکعبی
سری هندسی متناهی	$\sum_{i=0}^{n} r^i = \frac{r^{n+1}-1}{r-1} \quad (r \neq 1)$	قضیهٔ اصلی، آرایهٔ پویا
سری هندسی نامتناهی	$\sum_{i=0}^{\infty} r^i = \frac{1}{1-r} \quad (\lvert r\rvert < 1)$	شمارنده $(\sum 2^{-i} = 2)$، انتخاب تصادفی
عدد هارمونیک	$H_n = \sum_{i=1}^{n} \frac{1}{i} = \ln n + O(1)$	مسئلهٔ استخدام، کوئیک‌سورت
لگاریتم فاکتوریل	$\log(n!) = \Theta(n \log n)$	خودآزمایی فصل ۱

لگاریتم و توان

$\log(ab) = \log a + \log b$	$\log(a/b) = \log a - \log b$
$\log(a^k) = k \log a$	$\log_b a = \dfrac{\log_c a}{\log_c b}$
$a^{\log_b n} = n^{\log_b a}$	$2^{\log_2 n} = n$
$\log_2 3 \approx 1.585$	$\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618, \quad \log_2 \varphi \approx 0.694$

احتمال

امید ریاضی	$E[X] = \sum_\omega P[\omega] X(\omega) = \sum_r r\, P[X = r]$
خطی بودن (بدون نیاز به استقلال)	$E[\sum X_i] = \sum E[X_i], \quad E[aX + b] = aE[X] + b$
متغیر نشانگر	$E[I_A] = P[A]$
تعداد تلاش تا اولین موفقیت (احتمال p)	$E[Y] = 1/p$
شکست c تلاش مستقل	$(1-p)^c$

قضیهٔ اصلی و رابطه‌های بازگشتی

رابطه	جواب	مثال
$aT(n/b) + O(n^d),\ \log_b a < d$	$O(n^d)$	$T(n/2) + n$
$aT(n/b) + O(n^d),\ \log_b a = d$	$O(n^d \log n)$	مرتب‌سازی ادغامی
$aT(n/b) + O(n^d),\ \log_b a > d$	$O(n^{\log_b a})$	کاراتسوبا
$T(an) + T(bn) + O(n),\ a + b < 1$	$O(n)$	انتخاب قطعی
$T(an) + T(bn) + O(n),\ a + b = 1$	$O(n \log n)$	کوئیک‌سورت با محور خوب
$T(n-1) + O(1)$	$O(n)$	–
$T(n-1) + O(n)$	$O(n^2)$	بدترین حالت کوئیک‌سورت
$T(n-1) + T(n-2) + O(1)$	$\Theta(\varphi^n)$	فیبوناچی بازگشتی

داده‌ساختارها

نشانی خانهٔ آرایه	$\text{base} + i \times \text{element_size}$
صف حلقوی	$\text{next}(i) = (i+1) \bmod C$
درخت با n رأس	$n-1$ یال
درخت دودویی به ارتفاع h	حداکثر $2^{h+1}-1$ گره؛ کامل با n گره: $h = \lfloor \log_2 n \rfloor$
هرم در آرایهٔ $A[1..n]$	فرزندان $2i$ و $2i+1$، پدر $\lfloor i/2 \rfloor$
تحلیل ساخت هرم	$\sum_{h \geq 1} h/2^h = 2$
کران پایین مرتب‌سازی	$\log_2(N!) \geq \frac{N}{2}\log\frac{N}{2} = \Omega(N \log N)$

پ.۲ جدول الگوریتم‌های جزوه

مسئله	الگوریتم	زمان	فصل
مرتب‌سازی	درجی	$\Theta(n^2)$ بدترین، $\Theta(n)$ بهترین	۱
	ادغامی	$\Theta(n \log n)$	۲
	کوئیک‌سورت تصادفی	امید $O(n \log n)$، بدترین $\Theta(n^2)$	۳
فیبوناچی F_n	بازگشتی	$\Theta(\varphi^n)$	۱
	حلقه	$O(n)$	۱
	توان ماتریس	$O(\log n)$	۱
زیرآرایهٔ بیشینه	جامع	$O(n^3)$	۱
	مجموع پیشوندی	$O(n^2)$	۱
	تقسیم و حل	$O(n \log n)$	۲
	کادان	$O(n)$	۱ (تکمیلی)
ضرب دو عدد n بیتی	دبستانی	$\Theta(n^2)$	۲
	تقسیم و حل با ۴ ضرب	$\Theta(n^2)$	۲
	کاراتسوبا	$O(n^{1.585})$	۲
جمع دو عدد n بیتی	دبستانی (بهینه)	$\Theta(n)$	۲
میانهٔ δ-تقریبی	APPROX-MEDIAN-1 (مونت‌کارلو)	$O(n)$، موفقیت $\approx 2\delta$	۳
	APPROX-MEDIAN-2	$O(cn)$، موفقیت $1-(1-2\delta)^c$	۳
	APPROX-MEDIAN-3 (لاس‌وگاس)	امید $O(n/\delta)$	۳
انتخاب (رتبهٔ i)	تصادفی	امید $O(n)$، بدترین $\Theta(n^2)$	۳
	میانهٔ میانه‌ها	بدترین $O(n)$	۳
استخدام	با جایگشت تصادفی	امید هزینه $f\,H_n \approx f \ln n$	۳

مسئله	الگوریتم	زمان	فصل
جایگشت تصادفی	فیشر-ییتس	$O(n)$	۳
پشته با MULTIPOP	—	سرشکن $O(1)$	۴
شمارندهٔ دودویی	INCREMENT	سرشکن $O(1)$، بدترین $O(k)$	۴
آرایهٔ پویا	دو برابر کردن	سرشکن $O(1)$ برای هر درج	۴ (تکمیلی)
آرایه	دسترسی به $A[i]$	$\Theta(1)$	۵
	درج / حذف در اندیس i	$\Theta(n)$	۵
	جست‌وجو (نامرتب)	$\Theta(n)$	۵
لیست پیوندی	درج در ابتدا / بعد از گرهٔ معلوم	$\Theta(1)$	۵
	جست‌وجو / رسیدن به موقعیت i	$\Theta(n)$	۵
	معکوس کردن درجا	$\Theta(n)$ زمان، $\Theta(1)$ حافظه	۵
	شمارش گره‌ها با دور (فلوید)	$O(n)$ زمان، $O(1)$ حافظه	۵
ژوزفوس	لیست حلقوی	$O(nk)$	۵
پشته / صف حلقوی	push، pop، enqueue، dequeue	$\Theta(1)$	۵
بزرگ‌تر بعدی	پشتهٔ یکنوا	$\Theta(n)$	۵
جایگشت قطار	شبیه‌سازی حریصانه با پشته	$\Theta(n)$	۵
پیمایش درخت	پیش‌ترتیب / میان‌ترتیب / پس‌ترتیب	$\Theta(n)$	۶
بازسازی درخت	از پیش‌ترتیب و پس‌ترتیب	ساده $O(n^2)$	۶
درخت عبارت	مقداریابی (پس‌ترتیب)	$\Theta(n)$	۷
	ساخت از میانوندی پرانتزدار (یافتن ریشه)	$O(n^2)$	۷
	ساخت با تجزیهٔ همزمان / پسوندی با پشته	$\Theta(n)$	۷
	ساخت از میانوندی بی‌پرانتز (دو پشته)	$\Theta(n)$	۷
هرم	max	$O(1)$	۸

مسئله	الگوریتم	زمان	فصل
	insert / increase-key / extract-max	$O(\log n)$	۸
	ساخت (فلوید)	$O(n)$	۸
	شمارش اعداد بزرگ‌تر از k	$O(t)$	۸
مرتب‌سازی	هرمی	$O(n \log n)$، درجا	۸
مرتب‌سازی مقایسه‌ای	کران پایین	$\Omega(n \log n)$	۹
مرتب‌سازی	شمارشی	$O(n + k)$، پایدار	۹
مرتب‌سازی	مبنایی	$O(d(n + k))$	۹

پ۳۰ واژه‌نامهٔ فارسی–انگلیسی

فارسی	English
تحلیل مجانبی	asymptotic analysis
کران بالا / پایین / محکم	upper / lower / tight bound
بدترین حالت / حالت میانگین / بهترین حالت	worst / average / best case
ناوردای حلقه	loop invariant
جست‌وجوی جامع	brute force
زمان چندجمله‌ای	polynomial time
تقسیم و حل	divide and conquer
رابطهٔ بازگشتی	recurrence relation
درخت بازگشت	recursion tree
قضیهٔ اصلی	master theorem
روش حدس و استقرا (جای‌گذاری)	guess and induction (substitution method)
مرتب‌سازی ادغامی / درجی / سریع	mergesort / insertion sort / quicksort
زیرآرایه با بیشترین مجموع	maximum sum subarray
فضای نمونه / پیشامد	sample space / event
متغیر تصادفی / متغیر نشانگر	random variable / indicator random variable
امید ریاضی / خطی بودن امید	expected value / linearity of expectation
به‌طور یکنواخت تصادفی	uniformly at random (u.a.r.)
جایگشت تصادفی	random permutation
میانهٔ تقریبی / رتبه	approximate median / rank
جداکننده / محور	splitter / pivot
میانهٔ میانه‌ها	median of medians
تحلیل سرشکنی / هزینهٔ سرشکن‌شده	amortized analysis / amortized cost

فارسی	English
روش انبوهه / حساب‌داری / تابع پتانسیل	aggregate / accounting / potential method
داده‌ساختار / نوع دادهٔ انتزاعی	data structure / abstract data type (ADT)
لیست پیوندی (یک‌طرفه / دوپیوندی / حلقوی)	(singly / doubly / circular) linked list
اشاره‌گر / گره	pointer / node
پشته / صف / صف حلقوی	stack (LIFO) / queue (FIFO) / circular queue
پشتهٔ یکنوا / بزرگ‌تر بعدی	monotonic stack / next greater element
درخت / جنگل / ریشه / برگ	tree / forest / root / leaf
پدر / فرزند / برادر / جد / فرزندزاده	parent / child / sibling / ancestor / descendant
عمق / ارتفاع / سطح	depth / height / level
درخت دودویی کامل / پر / متوازن	complete / full / balanced binary tree
پیمایش پیش‌ترتیب / میان‌ترتیب / پس‌ترتیب	preorder / inorder / postorder traversal
فرزند چپ، برادر راست	left-child, right-sibling
عبارت / عملگر / عملوند	expression / operator / operand
یگانی / دودویی	unary / binary
نگارش میانوندی / پیشوندی / پسوندی	infix / prefix / postfix notation
گرامر مستقل از متن	context-free grammar
صف اولویت / هرم	priority queue / heap
بالا بردن / پایین بردن	sift-up / sift-down (heapify)
درخت تصمیم	decision tree
مرتب‌سازی مقایسه‌ای / پایدار	comparison sort / stable sort
مرتب‌سازی شمارشی / مبنایی / هرمی	counting sort / radix sort / heapsort

پ۴۰ منابع

1. Jon Kleinberg, Éva Tardos. *Algorithm Design*. Pearson Addison-Wesley؛ بخش‌های ۲٫۱ تا ۲٫۴ (تحلیل و زمان‌های رایج)، ۵٫۱، ۵٫۲، ۵٫۴ و ۵٫۵ (تقسیم و حل) و فصل ۱۳ (الگوریتم‌های تصادفی).
2. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press؛ بخش ۲ (مرتب‌سازی درجی)، فصل ۴ (رابطه‌های بازگشتی و قضیهٔ اصلی)، فصل ۵ (مسئلهٔ استخدام) و فصل ۱۷ (تحلیل سرشکنی).
3. Udi Manber. *Introduction to Algorithms: A Creative Approach*. Addison-Wesley, 1989؛ بخش ۵٫۸ (زیرآرایهٔ بیشینه).
4. Mark de Berg. *Lecture notes on Advanced Algorithms* (میانهٔ تقریبی و الگوریتم‌های مونت‌کارلو و لاس‌وگاس).
5. اسلایدهای انگلیسی درس بر پایهٔ اسلایدهای Kevin Wayne (Pearson Addison-Wesley) تهیه شده‌اند.
6. اسلایدهای تحلیل سرشکنی: درس داده‌ساختارها و مبانی الگوریتم‌ها، دانشکدهٔ مهندسی کامپیوتر، محمد قدسی.
7. CLRS، فصل ۶ (هرم و مرتب‌سازی هرمی)، فصل ۸ (کران پایین و مرتب‌سازی در زمان خطی)، فصل ۱۰ (پشته، صف و لیست پیوندی) و بخش ۱۰٫۴ (نمایش درخت‌های ریشه‌دار).
8. نمونهٔ الگوریتم ساخت هرم فلوید بر پایهٔ اسلایدهای درس CSE 373 (Kasey Champion) در اسلایدهای هرم آمده است.

نکته · دربارهٔ این جزوه

ترتیب و محتوای اصلی این جزوه از اسلایدهای درس گرفته شده و توضیحات، اثبات‌های کامل، مثال‌های حل‌شده، شکل‌ها و خودآزمایی‌ها به آن افزوده شده است. هر جا اسلاید خطای کوچکی داشت (مثل اندیس‌های شبه‌کد ادغام، زیرآرایهٔ بیشینه و مرتب‌سازی شمارشی) در متن تذکر داده شده است. مطالب کادرهای «تکمیلی» فراتر از اسلایدهاست.